

---

# **pypdf**

***Release 6.10.1***

**Mathieu Fenniak**

**Apr 14, 2026**



# USER GUIDE

|           |                                               |           |
|-----------|-----------------------------------------------|-----------|
| <b>1</b>  | <b>Installation</b>                           | <b>3</b>  |
| <b>2</b>  | <b>Robustness and strict=False</b>            | <b>5</b>  |
| <b>3</b>  | <b>Security</b>                               | <b>7</b>  |
| <b>4</b>  | <b>Exceptions, Warnings, and Log messages</b> | <b>9</b>  |
| <b>5</b>  | <b>Metadata</b>                               | <b>11</b> |
| <b>6</b>  | <b>Extract Text from a PDF</b>                | <b>17</b> |
| <b>7</b>  | <b>Post-Processing of Text Extraction</b>     | <b>23</b> |
| <b>8</b>  | <b>Extract Images</b>                         | <b>27</b> |
| <b>9</b>  | <b>Handle Attachments</b>                     | <b>29</b> |
| <b>10</b> | <b>Encryption and Decryption of PDFs</b>      | <b>33</b> |
| <b>11</b> | <b>Merging PDF files</b>                      | <b>35</b> |
| <b>12</b> | <b>Cropping and Transforming PDFs</b>         | <b>39</b> |
| <b>13</b> | <b>Reading PDF Annotations</b>                | <b>55</b> |
| <b>14</b> | <b>Adding PDF Annotations</b>                 | <b>59</b> |
| <b>15</b> | <b>Adding a Stamp or Watermark to a PDF</b>   | <b>71</b> |
| <b>16</b> | <b>Adding JavaScript to a PDF</b>             | <b>75</b> |
| <b>17</b> | <b>Adding Viewer Preferences</b>              | <b>77</b> |
| <b>18</b> | <b>Interactions with PDF Forms</b>            | <b>79</b> |
| <b>19</b> | <b>Handling Outlines</b>                      | <b>83</b> |
| <b>20</b> | <b>Streaming Data with pypdf</b>              | <b>89</b> |
| <b>21</b> | <b>Reduce PDF File Size</b>                   | <b>91</b> |
| <b>22</b> | <b>PDF Version Support</b>                    | <b>95</b> |

|                                         |            |
|-----------------------------------------|------------|
| <b>23 PDF/A Compliance</b>              | <b>97</b>  |
| <b>24 The PdfReader Class</b>           | <b>99</b>  |
| <b>25 The PdfWriter Class</b>           | <b>105</b> |
| <b>26 The Destination Class</b>         | <b>121</b> |
| <b>27 The DocumentInformation Class</b> | <b>123</b> |
| <b>28 The Field Class</b>               | <b>125</b> |
| <b>29 The Fit Class</b>                 | <b>127</b> |
| <b>30 The PageObject Class</b>          | <b>129</b> |
| <b>31 The PageRange Class</b>           | <b>137</b> |
| <b>32 The PaperSize Class</b>           | <b>139</b> |
| <b>33 The RectangleObject Class</b>     | <b>141</b> |
| <b>34 The Transformation Class</b>      | <b>143</b> |
| <b>35 The XmpInformation Class</b>      | <b>147</b> |
| <b>36 The annotations module</b>        | <b>149</b> |
| <b>37 Constants</b>                     | <b>151</b> |
| <b>38 Errors</b>                        | <b>155</b> |
| <b>39 Generic PDF objects</b>           | <b>157</b> |
| <b>40 The PdfDocCommon Class</b>        | <b>169</b> |
| <b>41 Developer Intro</b>               | <b>175</b> |
| <b>42 The PDF Format</b>                | <b>179</b> |
| <b>43 How pypdf parses PDF files</b>    | <b>183</b> |
| <b>44 How pypdf writes PDF files</b>    | <b>185</b> |
| <b>45 CMaps</b>                         | <b>187</b> |
| <b>46 The Deprecation Process</b>       | <b>189</b> |
| <b>47 Documentation</b>                 | <b>191</b> |
| <b>48 Testing</b>                       | <b>193</b> |
| <b>49 Releasing</b>                     | <b>195</b> |
| <b>50 CHANGELOG</b>                     | <b>199</b> |
| <b>51 Changelog of PyPDF2 1.X</b>       | <b>255</b> |
| <b>52 Migration Guide: 1.x to 2.x</b>   | <b>275</b> |

|                                      |            |
|--------------------------------------|------------|
| <b>53 Project Governance</b>         | <b>281</b> |
| <b>54 Taking Ownership of pypdf</b>  | <b>285</b> |
| <b>55 History of pypdf</b>           | <b>287</b> |
| <b>56 Contributors</b>               | <b>289</b> |
| <b>57 Scope of pypdf</b>             | <b>293</b> |
| <b>58 pypdf vs X</b>                 | <b>295</b> |
| <b>59 Frequently Asked Questions</b> | <b>297</b> |
| <b>60 Indices and tables</b>         | <b>299</b> |
| <b>Python Module Index</b>           | <b>301</b> |
| <b>Index</b>                         | <b>303</b> |



pypdf is a [free](#) and open source pure-python PDF library capable of splitting, merging, cropping, and transforming the pages of PDF files. It can also add custom data, viewing options, and passwords to PDF files. pypdf can retrieve text and metadata from PDFs as well.

See [pdfly](#) for a CLI application that uses pypdf to interact with PDFs.

You can contribute to [pypdf on GitHub](#).





## INSTALLATION

There are several ways to install pypdf. The most common option is to use pip.

### 1.1 pip

pypdf requires Python 3.9+ to run.

Typically, Python comes with `pip`, a package installer. Using it, you can install pypdf:

```
pip install pypdf
```

If you are not a superuser (a system administrator / root), you can also just install pypdf for your current user:

```
pip install --user pypdf
```

#### 1.1.1 Optional dependencies

pypdf tries to be as self-contained as possible, but for some tasks, the amount of work to properly maintain the code would be too high. This is especially the case for cryptography and image formats.

If you simply want to install all optional dependencies, run:

```
pip install pypdf[full]
```

Alternatively, you can install just some:

If you plan to use pypdf for encrypting or decrypting PDFs that use AES, you will need to install some extra dependencies. Encryption using RC4 is supported using the regular installation.

```
pip install pypdf[crypto]
```

If you plan to use image extraction, you need Pillow:

```
pip install pypdf[image]
```

For JBIG2 support, you need to install a global OS-level package as well: `jbig2dec`. The installation procedure depends on our operating system. For Ubuntu, use the following, for example:

```
sudo apt-get install jbig2dec
```

## 1.2 Python Version Support

Since pypdf 4.0, every release, including point releases, should work with all supported versions of [Python](#). Thus, every point release is designed to work with all existing Python versions, excluding end-of-life versions.

Previous versions of pypdf support the following versions of Python:

| Python                 | 3.11 | 3.10 | 3.9 | 3.8 | 3.7 | 3.6 | 2.7 |
|------------------------|------|------|-----|-----|-----|-----|-----|
| pypdf 3.x              |      |      |     |     |     |     |     |
| PyPDF2 >= 2.0          |      |      |     |     |     |     |     |
| PyPDF2 1.20.0 - 1.28.4 |      |      |     |     |     |     |     |
| PyPDF2 1.15.0 - 1.20.0 |      |      |     |     |     |     |     |

## 1.3 Anaconda

Anaconda users can [install pypdf via conda-forge](#).

## 1.4 Development Version

In case you want to use the current version under development:

```
pip install git+https://github.com/py-pdf/pypdf.git
```

## ROBUSTNESS AND STRICT=False

PDF is *specified in various versions*. The specification of PDF 2.0 has 1003 pages. This length makes it hard to get everything right. As a consequence, a lot of PDF files are not strictly following the specification.

If a PDF file does not follow the specification, it is not always possible to be certain what the intended effect would be. Think of the following broken Python code as an example:

```
# Broken
function (foo, bar):

# Potentially intended:
def function(foo, bar):
    ...

# Also possible:
function = (foo, bar)
```

Writing a parser, you can go two paths: Either you try to be forgiving and try to figure out what the user intended, or you are strict and just tell the user that they should fix their stuff.

pypdf gives you the option to be strict or not.

pypdf has two core objects:

- *PdfReader*
- *PdfWriter*

PdfReader and PdfWriter both have a `strict` parameter.

Choosing `strict=True` means that pypdf will raise an exception if a PDF does not follow the specification.

Choosing `strict=False` means that pypdf will try to be forgiving and do something reasonable, but it will log a warning message. It is a best-effort approach.



We strive to provide a library with secure defaults.

## 3.1 Configuration

### 3.1.1 Filters

*pypdf* currently employs output size limits for some filters which are known to possibly have large compression ratios. The usual limit is at 75 MB of uncompressed data during decompression. If this is too low for your use case, and you are aware of the possible side effects, you can modify the following constants which define the desired maximal output size in bytes:

- `pypdf.filters.ZLIB_MAX_OUTPUT_LENGTH` for the *FlateDecode* filter (zlib compression)
- `pypdf.filters.LZW_MAX_OUTPUT_LENGTH` for the *LZWDecode* filter (LZW compression)
- `pypdf.filters.RUN_LENGTH_MAX_OUTPUT_LENGTH` for the *RunLengthDecode* filter (run-length compression)

For JBIG2 images, there is a similar parameter to limit the memory usage during decoding: `pypdf.filters.JBIG2_MAX_OUTPUT_LENGTH`. It defaults to 75 MB as well.

For all streams, the maximum allowed value for the `/Length` field is limited to `pypdf.filters.MAX_DECLARED_STREAM_LENGTH`, which defaults to 75 MB as well.

For all array-based streams, the maximum allowed output length is limited to `pypdf.filters.MAX_ARRAY_BASED_STREAM_OUTPUT_LENGTH`, which defaults to 75 MB as well.

For the *FlateDecode* filter, the number of bytes to attempt recovery with can be set by `pypdf.filters.ZLIB_MAX_RECOVERY_INPUT_LENGTH`. It defaults to 5 MB due to the much more complex recovery approach.

For the *JBIG2Decode* filter, calling the external *jb2dec* tool can be disabled by setting `pypdf.filters.JBIG2DEC_BINARY = None`.

### 3.1.2 Reading

*pypdf* currently employs the following reading limits on *PdfReader* instances:

- `root_object_recovery_limit` limits the number of objects to read before stopping with Root object recovery in non-strict mode. It defaults to 10 000. Setting it to `None` will fully disable this limit.

If you want to employ custom limits for the *PdfWriter* as well, the currently preferred way is to initialize it from the reader, id est something like `PdfWriter(clone_from=PdfReader("file.pdf", root_object_recovery_limit=42))`.

## 3.2 Reporting possible vulnerabilities

Please refer to our [security policy](#).

## 3.3 Invalid reports

### 3.3.1 Exceptions

Most exceptions raised by our code are considered bugs or robustness issues and can be reported publicly. We consider it the task of the library user to catch exceptions which could cause their service to crash, although we try to only raise a known set of exception types.

### 3.3.2 Cryptographic functions

We receive reports about possibly insecure cryptography from time to time. This includes the following aspects:

- Using the ARC4 cipher
- Using the AES cipher in ECB mode
- Using MD5 for hashing

These are requirements of the PDF standard, which we need to achieve the greatest compatibility with. Although some of them might be deprecated in PDF 2.0, the PDF 2.0 adoption rate is very low and legacy documents need to be supported.

### 3.3.3 XML parsing

We use `xml.minidom` for parsing XMP information. Given recent Python versions built against recent Expat versions, the usual attacks (exponential entity expansion and external entity expansion) should not be possible. We have corresponding tests in place to ensure this for the platforms our tests run against.

For some details, see [the official documentation](#) and the [README for defusedxml](#).

Please note that automated scanners tend to still flag any direct imports of XML modules from the Python standard library as unsafe. There have been discussions about this being outdated already, but they are still being flagged.

## EXCEPTIONS, WARNINGS, AND LOG MESSAGES

pypdf makes use of three mechanisms to show if something went wrong:

- **Exceptions** are error cases that pypdf users should explicitly handle. In the `strict=True` mode, most log messages with the warning level will become exceptions. This can be useful in applications where you can require a user to fix the broken PDF.
- **Warnings** are avoidable issues, such as using deprecated classes / functions / parameters. Another example is missing capabilities of pypdf. In those cases, pypdf users should adjust their code. Warnings are issued by the `warnings` module - those are different from the log-level “warning.”
- **Log messages** are informative messages that can be used for post-mortem analysis. Most of the time, users can ignore them. They come in different *levels*, such as `info` / `warning` / `error` indicating the severity. Examples are non-standard compliant PDF files which pypdf can deal with or a missing implementation that leads to a part of the text not being extracted.

### 4.1 Exceptions

Exceptions need to be caught if you want to handle them. For example, you could want to read the text from a PDF as a part of a search function.

Most PDF files do not follow the specification. In this case, pypdf needs to guess which kinds of mistakes were potentially done when the PDF file was created. See [the robustness page](#) for the related issues.

As a user, you likely do not care about it. If it is readable in any way, you want the text. You might use `pdfminer.six` as a fallback and do this:

```
from pypdf import PdfReader
from pdfminer.high_level import extract_text as fallback_text_extraction

text = ""
try:
    reader = PdfReader("example.pdf")
    for page in reader.pages:
        text += page.extract_text()
except Exception as exc:
    text = fallback_text_extraction("example.pdf")
```

You could also capture `pypdf.errors.PyPdfError` if you prefer something more specific.

## 4.2 Warnings

The `warnings` module allows you to ignore warnings:

```
import warnings

warnings.filterwarnings("ignore")
```

In many cases, you actually want to start Python with the `-W` flag so that you see all warnings. This is especially true for Continuous Integration (CI).

## 4.3 Log messages

Log messages can be noisy in some cases. pypdf hopefully has a reasonable level of log messages, but you can reduce which types of messages you want to see:

```
import logging

logger = logging.getLogger("pypdf")
logger.setLevel(logging.ERROR)
```

The `logging` module defines six log levels:

- CRITICAL
- ERROR
- WARNING
- INFO
- DEBUG
- NOTSET



## METADATA

PDF files can have two types of metadata: “Regular” and XMP ones. They can both exist at the same time.

### 5.1 Reading metadata

```
from pypdf import PdfReader

reader = PdfReader("example.pdf")

meta = reader.metadata

# All the following could be None!
print(meta.title)
print(meta.author)
print(meta.subject)
print(meta.creator)
print(meta.producer)
print(meta.creation_date)
print(meta.modification_date)
```

### 5.2 Writing metadata

```
from datetime import datetime
from pypdf import PdfReader, PdfWriter

reader = PdfReader("example.pdf")
writer = PdfWriter()

# Add all pages to the writer
for page in reader.pages:
    writer.add_page(page)

# If you want to add the old metadata, include these two lines
if reader.metadata is not None:
    writer.add_metadata(reader.metadata)

# Format the current date and time for the metadata
utc_time = "-05'00'" # UTC time optional
time = datetime.now().strftime(f"D\072%Y%m%d%H%M%S{utc_time}")
```

(continues on next page)

(continued from previous page)

```
# Add the new metadata
writer.add_metadata(
    {
        "/Author": "Martin",
        "/Producer": "Libre Writer",
        "/Title": "Title",
        "/Subject": "Subject",
        "/Keywords": "Keywords",
        "/CreationDate": time,
        "/ModDate": time,
        "/Creator": "Creator",
        "/CustomField": "CustomField",
    }
)

# Save the new PDF to a file
writer.write("out-meta-create.pdf")
```

## 5.3 Updating metadata

```
from pypdf import PdfWriter

writer = PdfWriter(clone_from="example.pdf")

# Change some values
writer.add_metadata(
    {
        "/Author": "Martin",
        "/Producer": "Libre Writer",
        "/Title": "Title",
    }
)

# Clear all data but keep the entry in PDF
writer.metadata = {}

# Replace all entries with new set of entries
writer.metadata = {
    "/Author": "Martin",
    "/Producer": "Libre Writer",
}

# Save the new PDF to a file
writer.write("out-meta-update.pdf")
```

## 5.4 Removing metadata entry

```
from pypdf import PdfWriter

writer = PdfWriter("example.pdf")

# Remove Metadata (/Info entry)
writer.metadata = None

# Save the new PDF to a file
writer.write("out-meta-remove.pdf")
```

## 5.5 Reading XMP metadata

```
from pypdf import PdfReader

reader = PdfReader("example.pdf")

meta = reader.xmp_metadata
if meta:
    print(meta.dc_title)
    print(meta.dc_description)
    print(meta.xmp_create_date)
```

## 5.6 Creating XMP metadata

You can create XMP metadata easily using the `XmpInformation.create()` method:

```
from pypdf import PdfWriter
from pypdf.xmp import XmpInformation

# Create a new XMP metadata object
xmp = XmpInformation.create()

# Set metadata fields
xmp.dc_title = {"x-default": "My Document Title"}
xmp.dc_creator = ["Author One", "Author Two"]
xmp.dc_description = {"x-default": "Document description"}
xmp.dc_subject = ["keyword1", "keyword2", "keyword3"]
xmp.pdf_producer = "pypdf"

# Create a writer and add the metadata
writer = PdfWriter()
writer.add_blank_page(612, 792) # Add a page
writer.xmp_metadata = xmp
writer.write("out-xmp-create.pdf")
```

## 5.7 Setting XMP metadata fields

The `XmpInformation` class provides property-based access for all supported metadata fields:

### 5.7.1 Dublin Core fields

```
from datetime import datetime
from pypdf.xmp import XmpInformation

xmp = XmpInformation.create()

# Single value fields
xmp.dc_coverage = "Global coverage"
xmp.dc_format = "application/pdf"
xmp.dc_identifier = "unique-id-123"
xmp.dc_source = "Original Source"

# Array fields (bags - unordered)
xmp.dc_contributor = ["Contributor One", "Contributor Two"]
xmp.dc_language = ["en", "fr", "de"]
xmp.dc_publisher = ["Publisher One"]
xmp.dc_relation = ["Related Doc 1", "Related Doc 2"]
xmp.dc_subject = ["keyword1", "keyword2"]
xmp.dc_type = ["Document", "Text"]

# Sequence fields (ordered arrays)
xmp.dc_creator = ["Primary Author", "Secondary Author"]
xmp.dc_date = [datetime.now()]

# Language alternative fields
xmp.dc_title = {"x-default": "Title", "en": "English Title", "fr": "Titre français"}
xmp.dc_description = {"x-default": "Description", "en": "English Description"}
xmp.dc_rights = {"x-default": "All rights reserved"}
```

### 5.7.2 XMP fields

```
from datetime import datetime

# Date fields accept both datetime objects and strings
xmp.xmp_create_date = datetime.now()
xmp.xmp_modify_date = datetime.fromisoformat("2023-12-25T10:30:45Z")
xmp.xmp_metadata_date = datetime.now()

# Text field
xmp.xmp_creator_tool = "pypdf"
```

### 5.7.3 PDF fields

```
xmp.pdf_keywords = "keyword1, keyword2, keyword3"
xmp.pdf_pdfversion = "1.4"
xmp.pdf_producer = "pypdf"
```

### 5.7.4 XMP Media Management fields

```
xmp.xmpmm_document_id = "uuid:12345678-1234-1234-1234-123456789abc"
xmp.xmpmm_instance_id = "uuid:87654321-4321-4321-4321-cba987654321"
```

### 5.7.5 PDF/A fields

```
xmp.pdfaid_part = "1"
xmp.pdfaid_conformance = "B"
```

### 5.7.6 Clearing metadata fields

You can clear any field by assigning None:

```
xmp.dc_title = None
xmp.dc_creator = None
xmp.pdf_producer = None
```

### 5.7.7 Incrementally updating XMP metadata fields

When modifying existing XMP metadata, it is often necessary to add or update individual entries while preserving existing values. The XMP properties return standard Python data structures that can be manipulated directly:

```
from pypdf.xmp import XmpInformation

xmp = XmpInformation.create()

# Language alternative fields return dictionaries
title = xmp.dc_title or {}
title["en"] = "English Title"
title["fr"] = "Titre français"
xmp.dc_title = title

# Bag fields (unordered collections) return lists
subjects = xmp.dc_subject or []
subjects.append("new_keyword")
xmp.dc_subject = subjects

# Sequence fields (ordered collections) return lists
creators = xmp.dc_creator or []
creators.append("New Author")
xmp.dc_creator = creators
```

This approach provides direct control over the data structures while maintaining the property-based interface.

## 5.8 Modifying XMP metadata

Modifying XMP metadata is a bit more complicated.

As an example, we want to add the following PDF/UA identifier section to the XMP metadata:

```
<rdf:Description rdf:about="" xmlns:pdfuaid="http://www.aiim.org/pdfua/ns/id/">
  <pdfuaid:part>1</pdfuaid:part>
</rdf:Description>
```

This could be written like this:

```
from pypdf import PdfWriter

writer = PdfWriter(clone_from="commented-xmp.pdf")

metadata = writer.xmp_metadata
assert metadata # Ensure that it is not `None`.
rdf_root = metadata.rdf_root
xmp_meta = rdf_root.parentNode
xmp_document = xmp_meta.parentNode

# Please note that without a text node, the corresponding elements might
# be omitted completely.
pdfuaid_description = xmp_document.createElement("rdf:Description")
pdfuaid_description.setAttribute("rdf:about", "")
pdfuaid_description.setAttribute("xmlns:pdfuaid", "http://www.aiim.org/pdfua/ns/id/")
pdfuaid_part = xmp_document.createElement("pdfuaid:part")
pdfuaid_part_text = xmp_document.createTextNode("1")
pdfuaid_part.appendChild(pdfuaid_part_text)
pdfuaid_description.appendChild(pdfuaid_part)
rdf_root.appendChild(pdfuaid_description)

metadata.stream.set_data(xmp_document.toxml().encode("utf-8"))

writer.write("out-xmp-update.pdf")
```

For further details on modifying the structure, please refer to `xml.dom.minidom`.

## EXTRACT TEXT FROM A PDF

You can extract text from a PDF:

```
from pypdf import PdfReader

reader = PdfReader("test Orient.pdf")
page = reader.pages[0]
print(page.extract_text())

# extract only text oriented up
print(page.extract_text(0))

# extract text oriented up and turned left
print(page.extract_text((0, 90)))

# extract text in a fixed width format that closely adheres to the rendered
# layout in the source pdf
print(page.extract_text(extraction_mode="layout"))

# extract text preserving horizontal positioning without excess vertical
# whitespace (removes blank and "whitespace only" lines)
print(page.extract_text(extraction_mode="layout", layout_mode_space_vertically=False))

# adjust horizontal spacing
print(page.extract_text(extraction_mode="layout", layout_mode_scale_weight=1.0))

# exclude (default) or include (as shown below) text rotated w.r.t. the page
print(page.extract_text(extraction_mode="layout", layout_mode_strip_rotated=False))
```

Refer to `extract_text()` for more details.

### Note

Extracting the text of a page requires parsing its whole content stream. This can require quite a lot of memory - we have seen 10 GB RAM being required for an uncompressed content stream of about 300 MB (which should not occur very often).

To limit the size of the content streams to process (and avoid OOM errors in your application), consider checking `len(page.get_contents().get_data())` beforehand.

**Note**

If a PDF page appears to contain only an image (e.g., a scanned document), the extracted text may be minimal or visually empty. In such cases, consider using OCR software such as [Tesseract OCR](#) to extract text from images.

## 6.1 Using a visitor

You can use visitor functions to control which part of a page you want to process and extract. The visitor functions you provide will get called for each operator or for each text fragment.

The function provided in argument `visitor_text` of function `extract_text` has five arguments:

- `text`: the current text (as long as possible, can be up to a full line)
- `user_matrix`: current matrix to move from user coordinate space (also known as CTM)
- `tm_matrix`: current matrix from text coordinate space
- `font_dictionary`: full font dictionary
- `font_size`: the size (in text coordinate space)

The matrix stores six parameters. The first four provide the rotation/scaling matrix, and the last two provide the translation (horizontal/vertical). It is recommended to use the `user_matrix` as it takes into account all transformations.

Notes :

- As indicated in §8.3.3 of the PDF 1.7 or PDF 2.0 specification, the user matrix applies to text space/image space/form space/pattern space.
- If you want to get the full transformation from text to user space, you can use the `mult()` function as follows:  
`txt2user = mult(tm, cm)`. The font size is the raw text size and affected by the `user_matrix`.

The `font_dictionary` may be `None` in case of unknown fonts. If not `None`, it could contain something like the key `"/BaseFont"` with the value `"/Arial,Bold"`.

**Caveat:** In complicated documents, the calculated positions may be difficult to determine (if you move from multiple forms to page user space, for example).

The function provided in argument `visitor_operand_before` has four arguments: operator, operand-arguments, current transformation matrix, and text matrix.

### 6.1.1 Example 1: Ignore header and footer

The following example reads the text of page four of [this PDF document](#), but ignores the header (`y > 720`) and footer (`y < 50`). In this file we also need to include new line characters (`y == 0`).

```
from pypdf import PdfReader

reader = PdfReader("GeoBase_NHNC1_Data_Model_UML_EN.pdf")
page = reader.pages[3]

parts = []

def visitor_body(text, cm, tm, font_dict, font_size):
    y = tm[5]
    if 50 < y < 720 or y == 0:
```

(continues on next page)



(continued from previous page)

```

        parts.append(text)

page.extract_text(visitor_text=visitor_body)
text_body = "".join(parts)

print(text_body)

```

## 6.1.2 Example 2: Extract rectangles and texts into an SVG file

The following example converts page three of [this PDF document](#) into an SVG file.

Such an SVG export may help to understand what is going on in a page.

```

from pypdf import PdfReader
import svgwrite

reader = PdfReader("GeoBase_NHNC1_Data_Model_UML_EN.pdf")
page = reader.pages[2]

dwg = svgwrite.Drawing("GeoBase_test.svg", profile="tiny")

def visitor_svg_rect(op, args, cm, tm):
    if op == b"re":
        (x, y, w, h) = (args[i].as_numeric() for i in range(4))
        dwg.add(dwg.rect((x, y), (w, h), stroke="red", fill_opacity=0.05))

def visitor_svg_text(text, cm, tm, font_dict, font_size):
    (x, y) = (cm[4], cm[5])
    dwg.add(dwg.text(text, insert=(x, y), fill="blue"))

page.extract_text(
    visitor_operand_before=visitor_svg_rect, visitor_text=visitor_svg_text
)
dwg.save()

```

The SVG generated here is bottom-up because the coordinate systems of PDF and SVG differ.

Unfortunately, in complicated PDF documents the coordinates given to the visitor functions may be wrong.

## 6.2 Why Text Extraction is hard

### 6.2.1 Unclear Objective

Extracting text from a PDF can be tricky. In several cases, there is no clear answer to what the expected result should look like:

1. **Paragraphs:** Should the text of a paragraph have line breaks at the same places where the original PDF had them or should it rather be one block of text?
2. **Page numbers:** Should they be included in the extract?

3. **Headers and Footers:** Similar to page numbers - should they be extracted?
4. **Outlines:** Should outlines be extracted at all?
5. **Formatting:** If the text is **bold** or *italic*, should it be included in the output?
6. **Tables:** Should the text extraction skip tables? Should it extract just the text? Should the borders be shown in some Markdown-like way or should the structure be present e.g. as an HTML table? How would you deal with merged cells?
7. **Captions:** Should image and table captions be included?
8. **Ligatures:** The Unicode symbol `U+FB00` is a single symbol `ff` for two lowercase letters 'f'. Should that be parsed as the Unicode symbol 'ff' or as two ASCII symbols 'ff'?
9. **SVG images:** Should the text parts be extracted?
10. **Mathematical Formulas:** Should they be extracted? Formulas have indices and nested fractions.
11. **Whitespace characters:** How many new lines should be extracted for 3 cm of vertical whitespace? How many spaces should be extracted if there is 3 cm of horizontal whitespace? When would you extract tabs and when spaces?
12. **Footnotes:** When the text of multiple pages is extracted, where should footnotes be shown?
13. **Hyperlinks and Metadata:** Should it be extracted at all? Where should it be placed in which format?
14. **Linearization:** Assume you have a floating figure in between a paragraph. Do you first finish the paragraph, or do you put the figure text in between?

Then there are issues where most people would agree on the correct output, but the way PDF stores information just makes it hard to achieve that:

1. **Tables:** Typically, tables are just absolutely positioned text. In the worst case, every single letter could be absolutely positioned. That makes it hard to tell where columns / rows are.
2. **Images:** Sometimes PDFs do not contain the text as it is displayed, but instead an image. You notice that when you cannot copy the text. Then there are PDF files that contain an image and a text layer in the background. That typically happens when a document was scanned. Although the scanning software (OCR) is pretty good today, it still fails once in a while. pypdf is no OCR software; it will not be able to detect those failures. pypdf will also never be able to extract text from images.

Finally, there are issues that pypdf will deal with. If you find such a text extraction bug, please share the PDF with us so we can work on it!

## 6.2.2 Missing Semantic Layer

The PDF file format is all about producing the desired visual result for printing. It was not created for parsing the content. PDF files don't contain a semantic layer.

Specifically, there is no information what the header, footer, page numbers, tables, and paragraphs are. The visual appearance is there, and people might find heuristics to make educated guesses, but there is no way of being certain.

This is a shortcoming of the PDF file format, not of pypdf.

It is possible to apply machine learning on PDF documents to make good heuristics, but that will not be part of pypdf. However, pypdf could be used to feed such a machine learning system with the relevant information.

### 6.2.3 Whitespaces

The PDF format is meant for printing. It is not designed to be read by machines. The text within a PDF document is absolutely positioned, meaning that every single character could be positioned on the page.

The text

This is a test document by Ethan Nelson.

can be represented as

```
[(This is a )9(te)-3(st)9( do)-4(cu)13(m)-4(en)12(t )-3(b)3(y)-3( )9(Et)-2(h)3(an)4( Nels)13(o)-5(n)3(.)]
TJ
```

Where the numbers are adjustments of vertical space. This representation used within the PDF file makes it very hard to guarantee correct whitespaces.

More information:

- [issue #1507](#)
- [Negative numbers in PDF content stream text object](#)
- Mark Stephens: [Understanding PDF text objects](#), 2010.

## 6.3 OCR vs. Text Extraction

Optical Character Recognition (OCR) is the process of extracting text from images. Software which does this is called *OCR software*. The [tesseract OCR engine](#) is the most commonly known open source OCR software.

pypdf is **not** OCR software.

### 6.3.1 Digitally-born vs. Scanned PDF files

PDF documents can contain images and text. PDF files don't store text in a semantically meaningful way, but in a way that makes it easy to show the text on screen or print it. For this reason, text extraction from PDFs is hard.

If you scan a document, the resulting PDF typically shows the image of the scan. Scanners then also run OCR software and put the recognized text in the background of the image. pypdf can extract this result of the scanners OCR software. However, in such cases, it's recommended to directly use OCR software as errors can accumulate: The OCR software is not perfect in recognizing the text. Then it stores the text in a format that is not meant for text extraction and pypdf might make mistakes parsing that.

Hence, I would distinguish three types of PDF documents:

- **Digitally born PDF files:** The file was created digitally on the computer. It can contain images, texts, links, outline items (a.k.a., bookmarks), JavaScript, ... If you Zoom in a lot, the text still looks sharp.
- **Scanned PDF files:** Any number of pages was scanned. The images were then stored in a PDF file. Hence, the file is just a container for those images. You cannot copy the text, you don't have links, outline items, JavaScript.
- **OCRed PDF files:** The scanner ran OCR software and put the recognized text in the background of the image. Hence, you can copy the text, but it still looks like a scan. If you zoom in enough, you can recognize pixels.

### 6.3.2 Can we just always use OCR?

You might now wonder if it makes sense to just always use OCR software. If the PDF file is digitally-born, you can render it to an image.

I would recommend not to do that.

Text extraction software like pypdf can use more information from the PDF than just the image. It can know about fonts, encodings, typical character distances and similar topics.

That means pypdf has a clear advantage when it comes to characters which are easy to confuse such as o00ö. **pypdf will never confuse characters.** It just reads what is in the file.

pypdf also has an edge when it comes to characters which are rare, e.g. . OCR software will not be able to recognize smileys correctly.

## **6.4 Attempts to prevent text extraction**

If people who share PDF documents want to prevent text extraction, they have multiple ways to do so:

1. Store the contents of the PDF as an image
2. Use a scrambled font

However, text extraction cannot be completely prevented if people should still be able to read the document. In the worst case, people can make a screenshot, print it, scan it, and run OCR over it.

## POST-PROCESSING OF TEXT EXTRACTION

Post-processing can recognizably improve the results of text extraction. It is, however, outside the scope of pypdf itself. Hence, the library will not give any direct support for it. It is a natural language processing (NLP) task.

This page lists a few examples of what can be done as well as a community recipe that can be used as a general purpose post-processing step. If you know more about the specific domain of your documents, e.g., the language, it is likely that you can find custom solutions that work better in your context.

### 7.1 Ligature Replacement

```
def replace_ligatures(text: str) -> str:
    ligatures = {
        "ff": "ff",
        "fi": "fi",
        "fl": "fl",
        "ffi": "ffi",
        "ffl": "ffl",
        "st": "ft",
        "st": "st",
        # "": "AA",
        # "Æ": "AE",
        "": "aa",
    }
    for search, replace in ligatures.items():
        text = text.replace(search, replace)
    return text
```

### 7.2 Dehyphenation

Hyphens are used to break words up so that the appearance of the page is nicer.

```
from typing import List

def remove_hyphens(text: str) -> str:
    """
    This fails for:
    * Natural dashes: well-known, self-replication, use-cases, non-semantic,
      Post-processing, Window-wise, viewpoint-dependent
```

(continues on next page)

(continued from previous page)

```

* Trailing math operands: 2 - 4
* Names: Lopez-Ferreras, VGG-19, CIFAR-100
"""
lines = [line.rstrip() for line in text.split("\n")]

# Find dashes
line_numbers = []
for line_no, line in enumerate(lines[:-1]):
    if line.endswith("-"):
        line_numbers.append(line_no)

# Replace
for line_no in line_numbers:
    lines = dehyphenate(lines, line_no)

return "\n".join(lines)

def dehyphenate(lines: List[str], line_no: int) -> List[str]:
    next_line = lines[line_no + 1]
    word_suffix = next_line.split(" ")[0]

    lines[line_no] = lines[line_no][:-1] + word_suffix
    lines[line_no + 1] = lines[line_no + 1][len(word_suffix) :]
    return lines

```

## 7.3 Header/Footer Removal

The following header/footer removal has several drawbacks:

- False-positives, e.g., for the first page when there is a date like 2024.
- False-negatives in many cases:
  - Dynamic part, e.g., page label is in the header.
  - Even/odd pages have different headers.
  - Some pages, e.g., the first one or chapter pages, do not have a header.

```

def remove_footer(extracted_texts: list[str], page_labels: list[str]):
    def remove_page_labels(extracted_texts, page_labels):
        processed = []
        for text, label in zip(extracted_texts, page_labels):
            text_left = text.lstrip()
            if text_left.startswith(label):
                text = text_left[len(label) :]

            text_right = text.rstrip()
            if text_right.endswith(label):
                text = text_right[: -len(label)]

            processed.append(text)

```

(continues on next page)

(continued from previous page)

```
    return processed

    extracted_texts = remove_page_labels(extracted_texts, page_labels)
    return extracted_texts
```

## 7.4 Other ideas

- Whitespaces in units: Between a number and its unit should be a space. ([source](#)). That means: 42 ms, 42 GHz, 42 GB.
- Percent: English style guides prescribe writing the percent sign following the number without any space between (e.g., 50%).
- Whitespaces before dots: Should typically be removed.
- Whitespaces after dots: Should typically be added.





## EXTRACT IMAGES

**Note**

In order to use the following code you need to install optional dependencies, see *installation guide*.

Every page of a PDF document can contain an arbitrary number of images. The names of the files may not be unique.

```
from pypdf import PdfReader

reader = PdfReader("example.pdf")

page = reader.pages[0]

for i, image_file_object in enumerate(page.images):
    file_name = "out-image-" + str(i) + "-" + image_file_object.name
    image_file_object.image.save(file_name)
```

### 8.1 Other images

Some other objects can contain images, such as stamp annotations.

You can extract the image from the annotation with the following code:

```
from pypdf import PdfReader

reader = PdfReader("example.pdf")
im = (
    reader.pages[0]["/Annots"][4]["/Parent"]
    .get_object()["/AP"]["/N"]["/Resources"]["/XObject"]["/Im4"]
    .decode_as_image()
)

im.save("out-annotation-image.png")
```

### 8.2 Error handling

Iterating over `page.images` directly will raise an exception on the first issue. If you expect some more or less broken PDF files, but still want to retrieve as many images as possible, consider making this a multistep process:

```
from pypdf import PdfReader

reader = PdfReader("example.pdf")

for page in reader.pages:
    for name in page.images.keys():
        try:
            # Try to retrieve actual image.
            image = page.images[name]
        except Exception as exception:
            # Handle exceptions.
            pass
```

## HANDLE ATTACHMENTS

PDF documents can contain attachments, from time to time named embedded file as well.

### 9.1 Retrieve Attachments

Attachments have a name, but it might not be unique. For this reason, the value of `reader.attachments["attachment_name"]` is a list.

You can extract all attachments like this:

```
from pypdf import PdfReader

reader = PdfReader("example.pdf")

for name, content_list in reader.attachments.items():
    for i, content in enumerate(content_list):
        with open(f"out-attachment-{i}-{name}", "wb") as fp:
            fp.write(content)
```

Alternatively, you can retrieve them in an object-oriented fashion if you need further details for these files:

```
from pypdf import PdfReader

reader = PdfReader("example.pdf")

for attachment in reader.attachment_list:
    print(attachment.name, attachment.alternative_name, attachment.content)
```

### 9.2 Add Attachments

To add a new attachment, use the following code:

```
from pypdf import PdfWriter

writer = PdfWriter(clone_from="example.pdf")
writer.add_attachment(filename="test.txt", data=b"Hello World!")
```

As you can see, the basic attachment properties are its name and content. If you want to modify further properties of it, the returned object provides corresponding setters:

```

import datetime
import hashlib

from pypdf import PdfWriter
from pypdf.generic import create_string_object, ByteStringObject, NameObject, \
    NumberObject

writer = PdfWriter(clone_from="example.pdf")
embedded_file = writer.add_attachment(filename="test.txt", data=b"Hello World!")

embedded_file.size = NumberObject(len(b"Hello World!"))
embedded_file.alternative_name = create_string_object("test1.txt")
embedded_file.description = create_string_object("My test file")
embedded_file.subtype = NameObject("/text/plain")
embedded_file.checksum = ByteStringObject(hashlib.md5(b"Hello World!").digest())
embedded_file.modification_date = datetime.datetime.now(tz=datetime.timezone.utc)
# embedded_file.content = "My new content."

writer.write("out-add-attachment.pdf")

```

The same functionality is available if you iterate over the attachments of a writer using `writer.attachment_list`.

## 9.3 Delete Attachments

To delete an existing attachment, use the following code:

```

from pypdf import PdfWriter

writer = PdfWriter(clone_from="example.pdf")
attachment = writer.add_attachment(filename="test.txt", data=b"Hello World!")
attachment.delete()
assert list(writer.attachment_list) == []

```

Please note that this will not delete the associated file relationship if it exists. Deleting them as well would require us to know where this has been defined, which requires more complexity. For now, please consider looking for the corresponding definition yourself and delete it from the array.

## 9.4 PDF/A compliance

The following example shows how to add an attachment to a PDF/A-3B compliant document without breaking compliance:

```

from pypdf import PdfWriter
from pypdf.constants import AFRelationship
from pypdf.generic import create_string_object, ArrayObject, NameObject

writer = PdfWriter(clone_from="example.pdf")
attachment = writer.add_attachment(filename="test.txt", data="Hello World!")
attachment.subtype = NameObject("/text/plain")
attachment.associated_file_relationship = NameObject(AFRelationship.SUPPLEMENT)
attachment.alternative_name = create_string_object(attachment.name)

```

(continues on next page)

(continued from previous page)

```
if "/AF" in writer.root_object:
    af = writer.root_object["/AF"].get_object()
else:
    af = ArrayObject()
    writer.root_object[NameObject("/AF")] = af
af.append(attachment.pdf_object.indirect_reference)

writer.write("out-a3b.pdf")
```

This example marks a relationship of the attachment to the whole document. Alternatively, it can be added to most of the other PDF objects as well. For details, see the corresponding PDF specification, like section 14.13 of the PDF 2.0 specification.



## ENCRYPTION AND DECRYPTION OF PDFS

PDF encryption makes use of [RC4](#) and [AES](#) algorithms with different key length. `pypdf` supports all of them until PDF-2.0, which is the latest PDF standard.

`pypdf` use an extra dependency to do encryption or decryption for AES algorithms. We recommend [pyca/cryptography](#). Alternatively, you can use [pycryptodome](#).

### Note

Please see the note in the *installation guide* for installing the extra dependencies if interacting with PDFs that use AES.

## 10.1 Encrypt

You can encrypt a PDF by using a password:

```
from pypdf import PdfReader, PdfWriter

reader = PdfReader("example.pdf")
writer = PdfWriter(clone_from=reader)

# Add a password to the new PDF
writer.encrypt("my-secret-password", algorithm="AES-256")

# Save the new PDF to a file
writer.write("out-encrypt.pdf")
```

The algorithm can be one of RC4-40, RC4-128, AES-128, AES-256-R5, AES-256. We recommend using AES-256-R5.

### Warning

`pypdf` uses RC4 by default for compatibility if you omit the “algorithm” parameter. Since RC4 is insecure, you should use AES algorithms.

## 10.2 Decrypt

You can decrypt a PDF using the appropriate password:

```
from pypdf import PdfReader, PdfWriter

reader = PdfReader("encrypted-file.pdf")

if reader.is_encrypted:
    reader.decrypt("test") # secret password

writer = PdfWriter(clone_from=reader)

# Save the new PDF to a file
writer.write("out-decrypt.pdf")
```



## MERGING PDF FILES

### 11.1 Basic Example

```
from pypdf import PdfWriter

merger = PdfWriter()

for pdf in ["example.pdf", "hello-world.pdf", "jpeg.pdf"]:
    merger.append(pdf)

merger.write("out-basic.pdf")
```

For more details, see an excellent answer on [StackOverflow](#) by Paul Rooney.

#### Note

Dealing with large PDF files might reach the recursion limit of the current Python interpreter. In these cases, increasing the limit might help:

```
import sys

# Example: Increase the current limit by factor 5.
sys.setrecursionlimit(sys.getrecursionlimit() * 5)
```

### 11.2 Showing more merging options

```
from pypdf import PdfWriter

merger = PdfWriter()

with (
    open("Seige_of_Vicksburg_Sample_OCR.pdf", "rb") as input1,
    open("two-different-pages.pdf", "rb") as input2,
    open("example.pdf", "rb") as input3
):
    # Add the first 3 pages of input1 document to output
    merger.append(fileobj=input1, pages=(0, 3))

    # Insert the first page of input2 into the output beginning after the second page
```

(continues on next page)

(continued from previous page)

```
merger.merge(position=2, fileobj=input2, pages=(0, 1))

# Append entire input3 document to the end of the output document
merger.append(input3)

# Write to an output PDF document
merger.write("out-advanced.pdf")
```

## 11.3 append

append has been slightly extended in PdfWriter. See [append\(\)](#) for more details.

### 11.3.1 Examples

```
from pypdf import PdfWriter, PdfReader

writer = PdfWriter()

source_file_name = "GeoBase_NHNC1_Data_Model_UML_EN.pdf"

# Append the first 10 pages from pdf file
writer.append(source_file_name, (0, 10))

reader = PdfReader(source_file_name)

# Append the first and 10th page from reader and create an outline
writer.append(reader, "page 1 and 10", [0, 9])
```

During merging, the relevant named destination will also be imported.

If you want to insert pages in the middle of the destination, use `merge` (which provides an insertion position). You can insert the same page multiple times, if necessary, even using a list-based syntax:

```
# Insert pages 2 and 3, with page 1 before, between, and after
writer.append(reader, [0, 1, 0, 2, 0])
```

## 11.4 add\_page / insert\_page

It is recommended to use `append` or `merge` instead.

## 11.5 Merging forms

When merging forms, some form fields may have the same names, preventing access to some data.

A grouping field should be added before adding the source PDF to prevent that. The original fields will be identified by adding the group name.

For example, after calling `reader.add_form_topname("form1")`, the field previously named `field1` is now identified as `form1.field1` when calling `reader.get_form_text_fields(True)` or `reader.get_fields()`.

After that, you can append the input PDF completely or partially using `writer.append` or `writer.merge`. If you insert a set of pages, only those fields will be listed.

## 11.6 reset\_translation

During cloning, if an object has been already cloned, it will not be cloned again, and a pointer to this previously cloned object is returned instead. Because of that, if you add/merge a page that has already been added, the same object will be added the second time. If you modify any of these two pages later, both pages can be modified independently.

To reset, call `writer.reset_translation(reader)`.

## 11.7 Advanced cloning

To prevent side effects between pages/objects and all objects linked cloning is done during the merge.

This process will be automatically applied if you use `PdfWriter.append/merge/add_page/insert_page`. If you want to clone an object before attaching it “manually”, use the `clone` method of any *PdfObject*:

```
from pypdf.generic import NameObject, NumberObject, StreamObject

stream_object = StreamObject()

cloned_object = stream_object.clone(writer)
```

If you try to clone an object already belonging to the writer, it will return the same object:

```
assert cloned_object == stream_object.clone(writer)
```

The same holds true if you try to clone an object twice. It will return the previously cloned object:

```
assert stream_object.clone(writer) == stream_object.clone(writer)
```

Please note that if you clone an object, you will clone all the objects below as well, including the objects pointed by *IndirectObject*. Due to this, if you clone a page that includes some articles (”/B”), not only the first article, but also all the chained articles and the pages where those articles can be read will be copied. This means that you may copy lots of objects which will be saved in the output PDF as well.

To prevent this, you can provide the list of fields in the dictionaries to be ignored:

```
new_page = writer.add_page(reader.pages[0], excluded_keys=[/B])
```

### 11.7.1 Merging rotated pages

If you are working with rotated pages, you might want to call `transfer_rotation_to_content()` on the page before merging to avoid wrongly rotated results:

```
background = PdfReader("jpeg.pdf").pages[0]

for page in writer.pages:
    if page.rotation != 0:
        page.transfer_rotation_to_content()
        page.merge_page(background, over=False)
```



## CROPPING AND TRANSFORMING PDFS

### Note

Just because content is no longer visible, it is not gone. Cropping works by adjusting the viewBox. That means content that was cropped away can still be restored.

```
from pypdf import PdfReader, PdfWriter

reader = PdfReader("Seige_of_Vicksburg_Sample_OCR.pdf")
writer = PdfWriter()

# Add page 1 from reader to output document, unchanged.
writer.add_page(reader.pages[0])

# Add page 2 from reader, but rotated clockwise 90 degrees.
writer.add_page(reader.pages[1].rotate(90))

# Add page 3 from reader, but crop it to half size.
page3 = writer.add_page(reader.pages[2])
page3.mediabox.upper_right = (
    page3.mediabox.right / 2,
    page3.mediabox.top / 2,
)

writer.write("out-all-in-one.pdf")
```

### 12.1 Page rotation

The most typical rotation is a clockwise rotation of the page by multiples of 90 degrees. That is done when the orientation of the page is wrong. You can do that with the `rotate()` method:

```
from pypdf import PdfReader, PdfWriter

reader = PdfReader("example.pdf")
writer = PdfWriter()

writer.add_page(reader.pages[0])
writer.pages[0].rotate(90)
```

(continues on next page)

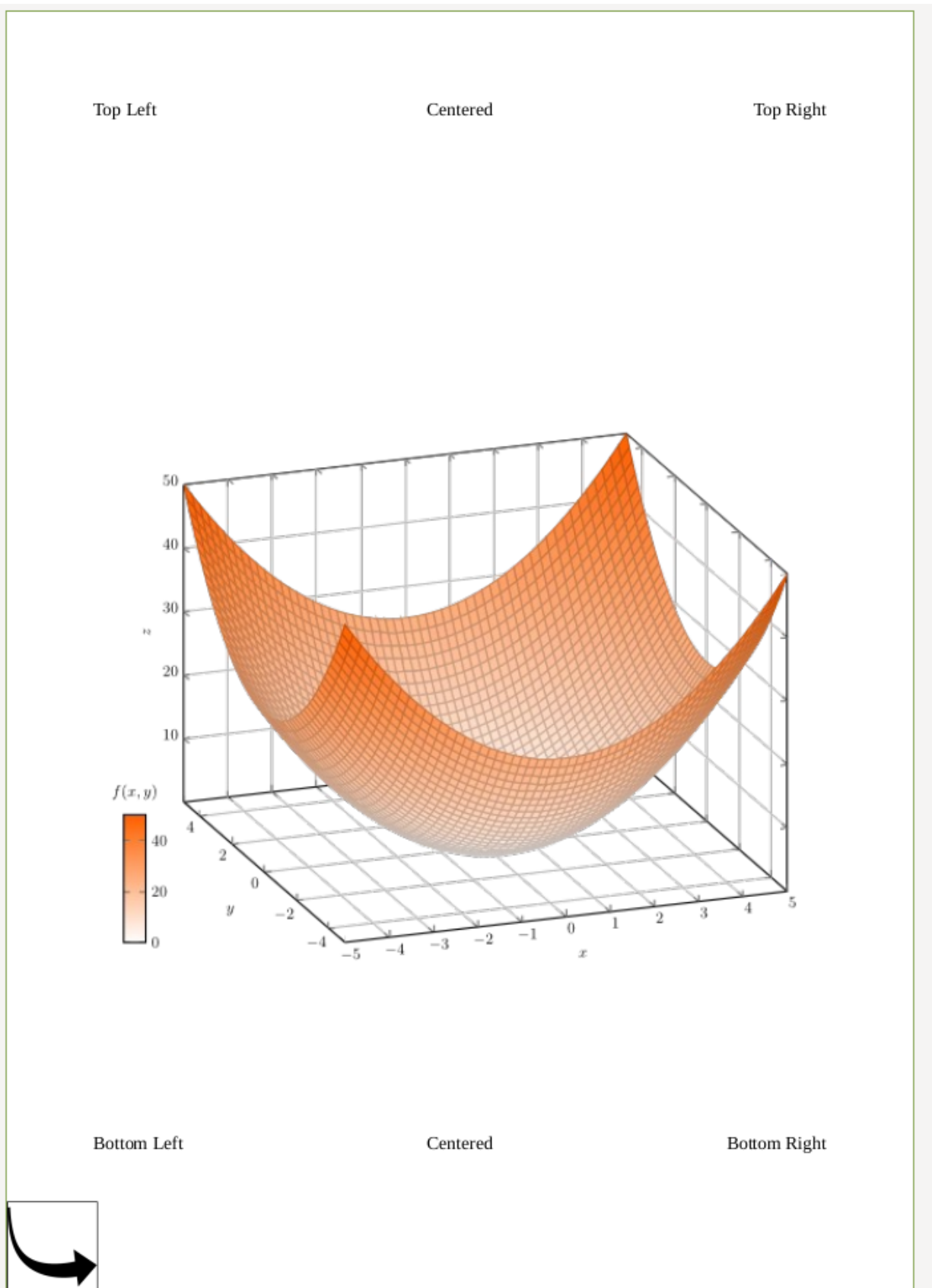
(continued from previous page)

```
writer.write("out-page-rotation.pdf")
```

The `rotate` method is typically preferred over the `page.add_transformation(Transformation().rotate())` method, because `rotate` will ensure that the page is still in the mediabox/cropbox. The transformation object operates on the coordinates of the page contents and does not change the mediabox or cropbox.



## 12.2 Plain Merge





is the result of

```
from pypdf import PdfReader, PdfWriter, Transformation

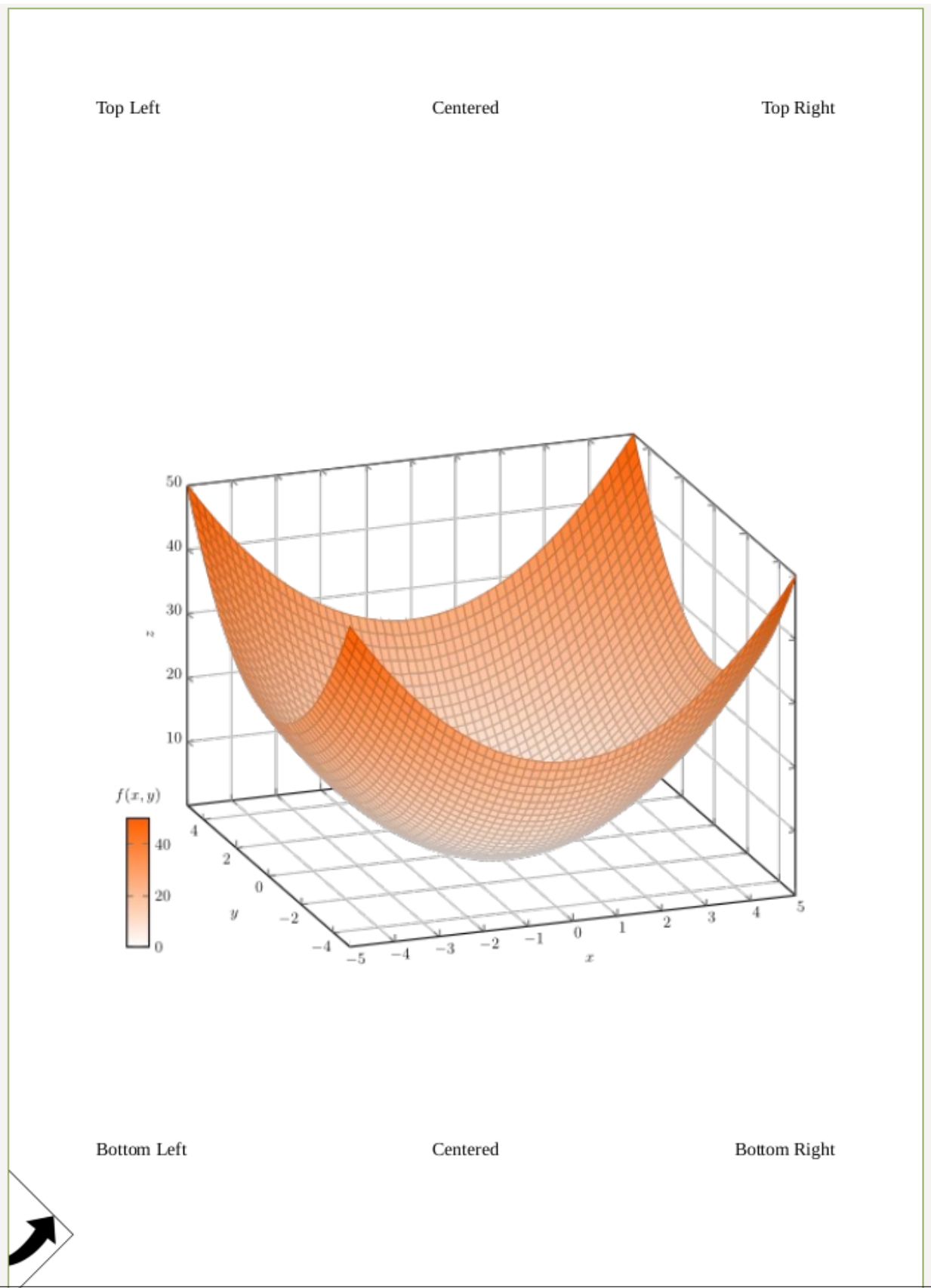
# Get the data
reader_base = PdfReader("labeled-edges-center-image.pdf")
page_base = reader_base.pages[0]

reader = PdfReader("box.pdf")
page_box = reader.pages[0]

# Write the result back
writer = PdfWriter()
page = writer.add_page(page_base)
page.merge_page(page_box)
writer.write("out-plain-merge.pdf")
```



## 12.3 Merge with Rotation



```
from pypdf import PdfReader, PdfWriter, Transformation

# Get the data
reader_base = PdfReader("labeled-edges-center-image.pdf")
page_base = reader_base.pages[0]

reader = PdfReader("box.pdf")
page_box = reader.pages[0]

# Prepare writer
writer = PdfWriter()

# Add base page.
writer_page = writer.add_page(page_base)

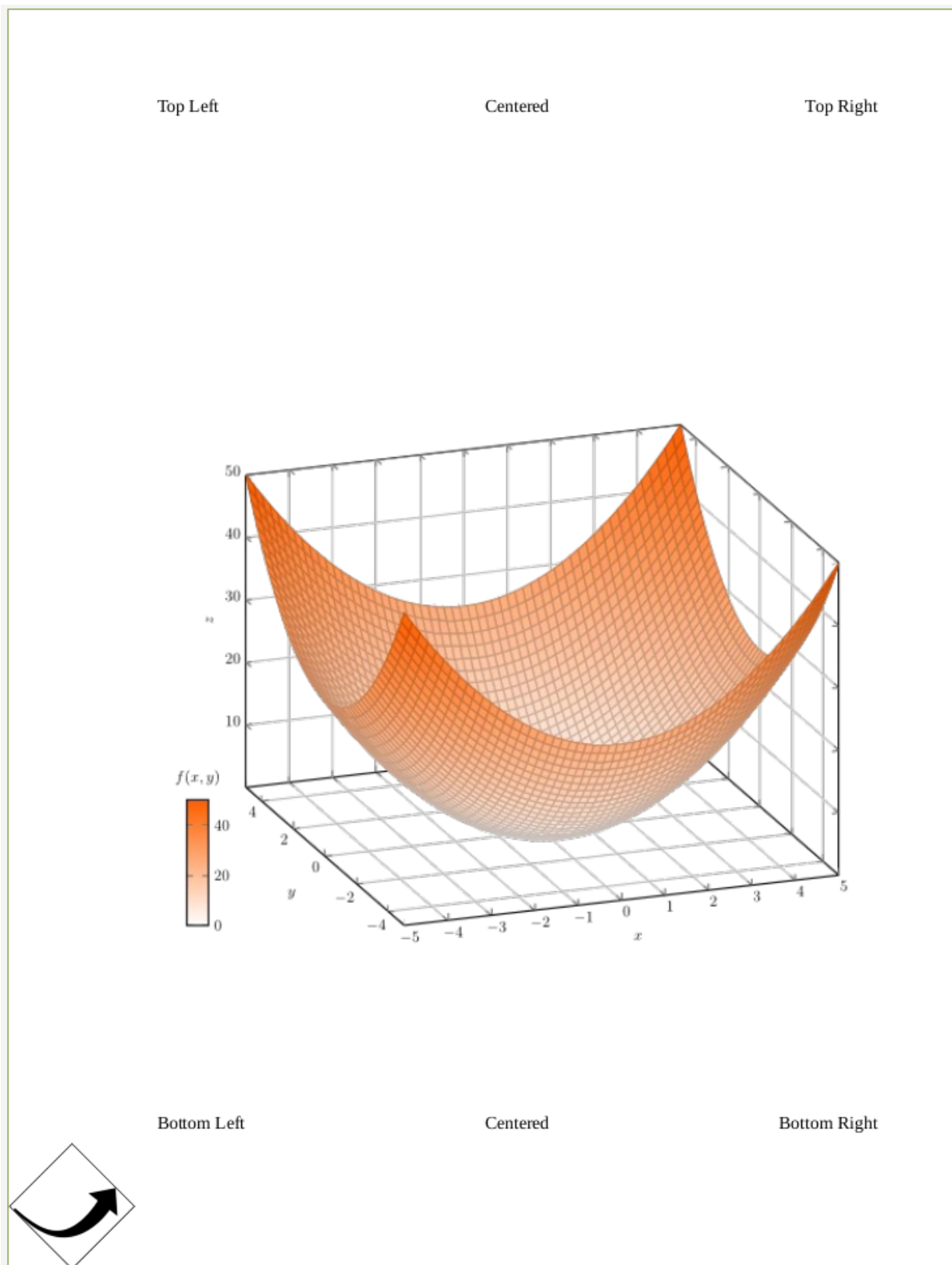
# Apply the transformation and merge the pages.
transformation = Transformation().rotate(45)
writer_page.merge_transformed_page(page_box, transformation)

# Write the result back
writer.write("out-merge-with-rotation.pdf")
```

If you add the `expand` parameter:

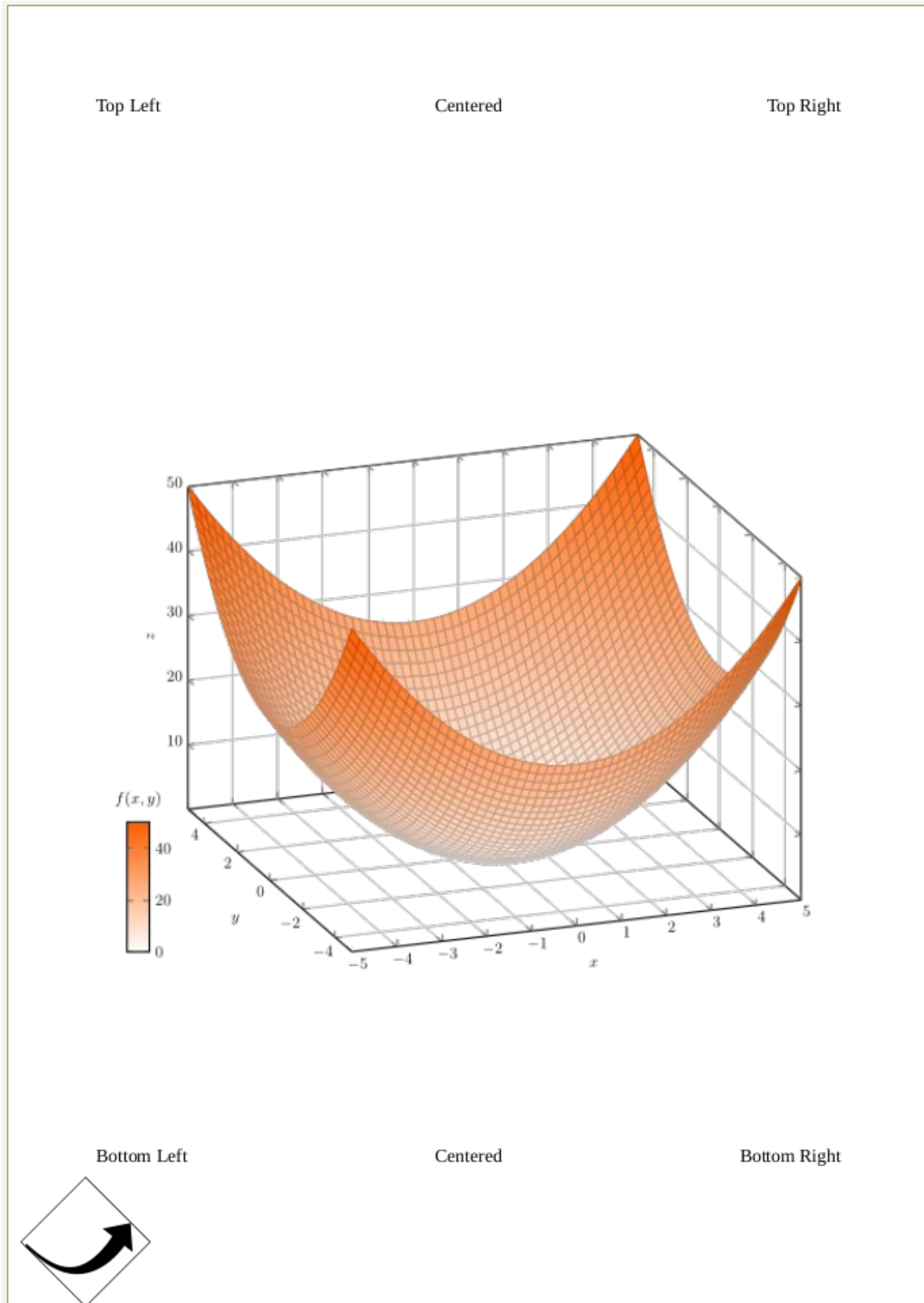
```
transformation = Transformation().rotate(45)
writer_page.merge_transformed_page(page_box, transformation, expand=True)
```

you get:



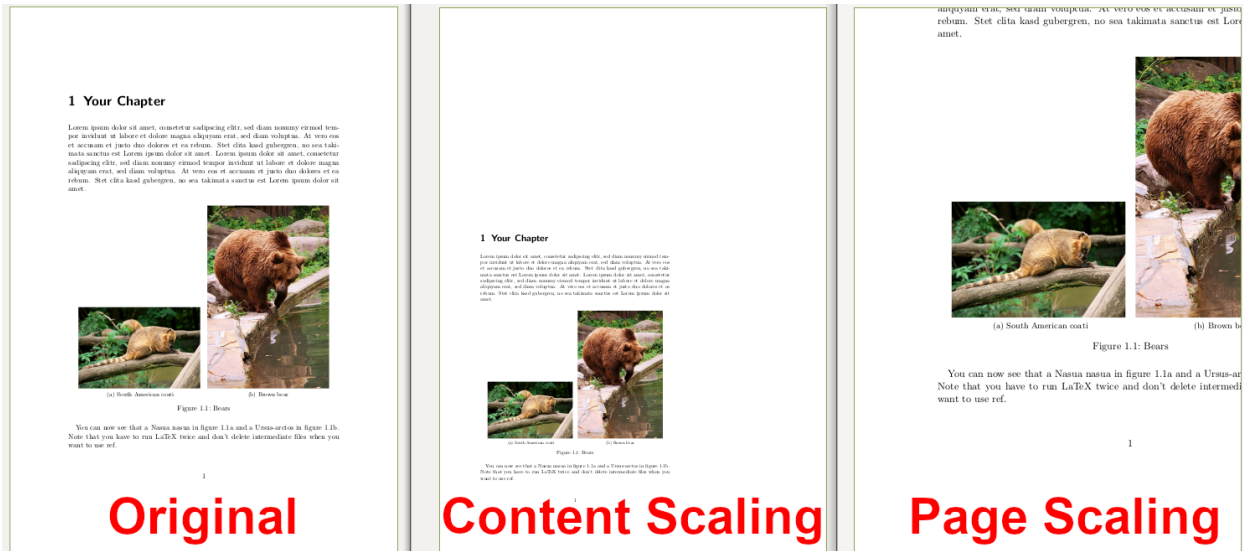
Alternatively, you can move the merged image a bit to the right by using

```
op = Transformation().rotate(45).translate(tx=50)
```



## 12.4 Scaling

In pypdf, the content and the page can either be scaled together or separately. Content scaling scales the contents on a page, and page scaling scales just the page size (the canvas). Typically, you want to combine both.



### 12.4.1 Scaling both the Page and contents together

```
from pypdf import PdfReader, PdfWriter

# Read the input
reader = PdfReader("side-by-side-subfig.pdf")
page = reader.pages[0]

# Add to the writer
writer = PdfWriter()
writer_page = writer.add_page(page)

# Scale
writer_page.scale_by(0.5)

# Write the result to a file
writer.write("out-scale-all.pdf")
```

### 12.4.2 Scaling the content only

The content is scaled around the origin of the coordinate system. Typically, that is the lower-left corner.

```
from pypdf import PdfReader, PdfWriter, Transformation

# Read the input
reader = PdfReader("side-by-side-subfig.pdf")
page = reader.pages[0]

# Prepare the writer
writer = PdfWriter()
```

(continues on next page)



(continued from previous page)

```

writer_page = writer.add_page(page)

# Scale
op = Transformation().scale(sx=0.7, sy=0.7)
writer_page.add_transformation(op)

# Write the result to a file
writer.write("out-scale-content.pdf")

```

### 12.4.3 Scaling the page only

To scale the page by `sx` in the X direction and `sy` in the Y direction:

```
page.mediabox = page.mediabox.scale(sx=0.7, sy=0.7)
```

If you wish to have more control, you can adjust the various page boxes directly:

```

from pypdf.generic import RectangleObject

mb = page.mediabox

page.mediabox = RectangleObject((mb.left, mb.bottom, mb.right, mb.top))
page.croptbox = RectangleObject((mb.left, mb.bottom, mb.right, mb.top))
page.trimbox = RectangleObject((mb.left, mb.bottom, mb.right, mb.top))
page.bleedbox = RectangleObject((mb.left, mb.bottom, mb.right, mb.top))
page.artbox = RectangleObject((mb.left, mb.bottom, mb.right, mb.top))

```

### 12.4.4 pypdf.\_page.MERGE\_CROP\_BOX

pypdf<=3.4.0 used to merge the other page with `trimbox`. pypdf>3.4.0 changes this behavior to `croptbox`.

In case anybody has good reasons to use/expect `trimbox`, you can add the following code to get the old behavior:

```

import pypdf

pypdf._page.MERGE_CROP_BOX = "trimbox"

```

## 12.5 Transforming several copies of the same page

We have designed the following business card (A8 format) to advertise our new startup.



We would like to copy this card sixteen times on an A4 page, to print it, cut it, and give it to all our friends. Having learned about the `merge_page()` method and the `Transformation` class, we run the following code. Notice that we had to tweak the media box of the source page to extend it, which is already a dirty hack (in this case).

```

from pypdf import PaperSize, PdfReader, PdfWriter, Transformation

# Read source file
reader = PdfReader("nup-source.pdf")
sourcepage = reader.pages[0]

# Create a destination file, and add a blank page to it
writer = PdfWriter()
destpage = writer.add_blank_page(width=PaperSize.A4.height, height=PaperSize.A4.width)

# Copy source page to destination page, several times
for x in range(4):
    for y in range(4):
        # Translate page
        transformation = Transformation().translate(
            x * PaperSize.A8.height,
            y * PaperSize.A8.width,
        )
        # Merge translated page
        destpage.merge_transformed_page(sourcepage, transformation)

# Write file
writer.write("out-nup-dest1.pdf")

```



There is still some work to do, for instance, to insert margins between and around cards, but this is left as an exercise for the reader...

## 12.6 Possible issues

Especially when combining `merge_page()` with transformations, you might end up with a cropped PDF file. In these cases, consider setting `expand=True` to re-calculate the corresponding media box.



## READING PDF ANNOTATIONS

PDF 2.0 defines the following annotation types:

- Text
- Link
- FreeText
- Line
- Square
- Circle
- Polygon
- PolyLine
- Highlight
- Underline
- Squiggly
- StrikeOut
- Caret
- Stamp
- Ink
- Popup
- FileAttachment
- Sound
- Movie
- Screen
- Widget
- PrinterMark
- TrapNet
- Watermark
- 3D
- Redact

- Projection
- RichMedia

In general, annotations can be read like this:

```
from pypdf import PdfReader

reader = PdfReader("example.pdf")

for page in reader.pages:
    if "/Annots" in page:
        for annotation in page["/Annots"]:
            obj = annotation.get_object()
            print({"subtype": obj["/Subtype"], "location": obj["/Rect"]})
```

Examples of reading three of the most common annotations:

## 13.1 Text

```
from pypdf import PdfReader

reader = PdfReader("example.pdf")

for page in reader.pages:
    if "/Annots" in page:
        for annotation in page["/Annots"]:
            subtype = annotation.get_object()["/Subtype"]
            if subtype == "/Text":
                print(annotation.get_object()["/Contents"])
```

## 13.2 Highlights

```
from pypdf import PdfReader

reader = PdfReader("example.pdf")

for page in reader.pages:
    if "/Annots" in page:
        for annotation in page["/Annots"]:
            subtype = annotation.get_object()["/Subtype"]
            if subtype == "/Highlight":
                coords = annotation.get_object()["/QuadPoints"]
                x1, y1, x2, y2, x3, y3, x4, y4 = coords
```

## 13.3 Attachments

```
from pypdf import PdfReader

reader = PdfReader("example.pdf")
```

(continues on next page)

(continued from previous page)

```
attachments = {}
for page in reader.pages:
    if "/Annots" in page:
        for annotation in page["/Annots"]:
            subtype = annotation.get_object()["/Subtype"]
            if subtype == "/FileAttachment":
                fileobj = annotation.get_object()["/FS"]
                attachments[fileobj["/F"]] = fileobj["/EF"]["/F"].get_data()
```





## ADDING PDF ANNOTATIONS

### Note

By default, some annotations might be invisible, for example polylines, as the default color is “transparent”.

To circumvent this, make sure to add the /C entry to the annotation, being an array and each array value being in the range 0.0 to 1.0:

- With one element, a grayscale value.
- With three elements, a RGB definition.
- With four elements, a CMYK definition.

### 14.1 Attachments

```
from pypdf import PdfWriter

writer = PdfWriter()
writer.add_blank_page(width=200, height=200)

data = b"any bytes - typically read from a file"
writer.add_attachment("smile.png", data)

writer.write("out-attachment.pdf")
```

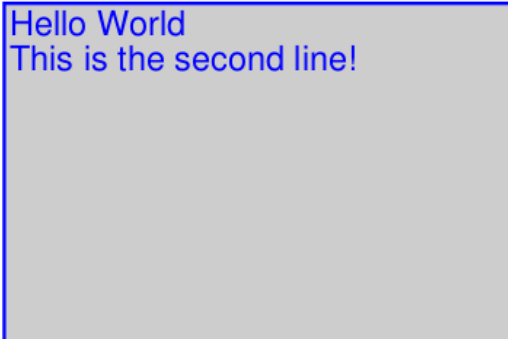
### 14.2 Free Text

If you want to add text in a box like this

## *The Crazy Ones*

*October 14, 1998*

Heres to the crazy ones. The misfits. The rebels. The troublemakers.  
The round pegs in the square holes.



erently. Theyre not fond of rules. And  
he status quo. You can quote them,  
y or vilify them.  
nt do is ignore them. Because they change  
y imagine. They heal. They explore. They  
y push the human race forward.  
.

How else can you stare at an empty canvas and see a work of art? Or  
sit in silence and hear a song thats never been written? Or gaze at  
a red planet and see a laboratory on wheels?

We make tools for these kinds of people.

While some see them as the crazy ones, we see genius. Because the  
people who are crazy enough to think they can change the world,  
are the ones who do.

you can use *FreeText*:

```
from pypdf import PdfReader, PdfWriter
from pypdf.annotations import FreeText

# Fill the writer with the pages you want
reader = PdfReader("crazyones.pdf")
page = reader.pages[0]
writer = PdfWriter()
writer.add_page(page)

# Create the annotation and add it
annotation = FreeText(
    text="Hello World\nThis is the second line!",
    rect=(50, 550, 200, 650),
    font="Arial",
    bold=True,
    italic=True,
    font_size="20pt",
    font_color="00ff00",
    border_color="0000ff",
```

(continues on next page)

(continued from previous page)

```
        background_color="cdcdcd",
    )

    # Set annotation flags to 4 for printable annotations.
    # See "AnnotationFlag" for other options, e.g. hidden etc.
    annotation.flags = 4

    writer.add_annotation(page_number=0, annotation=annotation)

    # Write the annotated file to disk
    writer.write("out-free-text.pdf")
```

### 14.3 Text

A text annotation looks like this:

|                                                                                     |                 |   |
|-------------------------------------------------------------------------------------|-----------------|---|
|                                                                                     | <b>Contents</b> |   |
|                                                                                     | 1 Foo           | 2 |
|                                                                                     | 2 Bar           | 2 |
|                                                                                     | 3 Baz           | 2 |
|                                                                                     | 4 Foo           | 2 |
|                                                                                     | 5 Bar           | 3 |
|                                                                                     | 6 Baz           | 3 |
|  | 7 Foo           | 3 |
|                                                                                     | 8 Bar           | 4 |
|                                                                                     | 9 Baz           | 4 |

### 14.4 Line

If you want to add a line like this:

## *The Crazy Ones*

*October 14, 1998*

Heres to the crazy ones. The misfits. The rebels. The troublemakers.  
The round pegs in the square holes.

The ones who see things differently. Theyre not fond of rules. And  
they have no respect for the status quo. You can quote them,  
disagree with them, glorify or vilify them.

About the only thing you cant do is ignore them. Because they change  
things. They invent. They imagine. They heal. They explore. They  
create. They inspire. They push the human race forward.

Maybe they have to be crazy.

How else can you stare at an empty canvas and see a work of art? Or  
sit in silence and hear a song thats never been written? Or gaze at  
a red planet and see a laboratory on wheels?

We make tools for these kinds of people.

While some see them as the crazy ones, we see genius. Because the  
people who are crazy enough to think they can change the world,  
are the ones who do.

you can use *Line*:

```
from pypdf import PdfReader, PdfWriter
from pypdf.annotations import Line

reader = PdfReader("crazyones.pdf")
page = reader.pages[0]
writer = PdfWriter()
writer.add_page(page)

# Add the line
annotation = Line(
    text="Hello World\nLine2",
    rect=(50, 550, 200, 650),
    p1=(50, 550),
    p2=(200, 650),
)
writer.add_annotation(page_number=0, annotation=annotation)
```

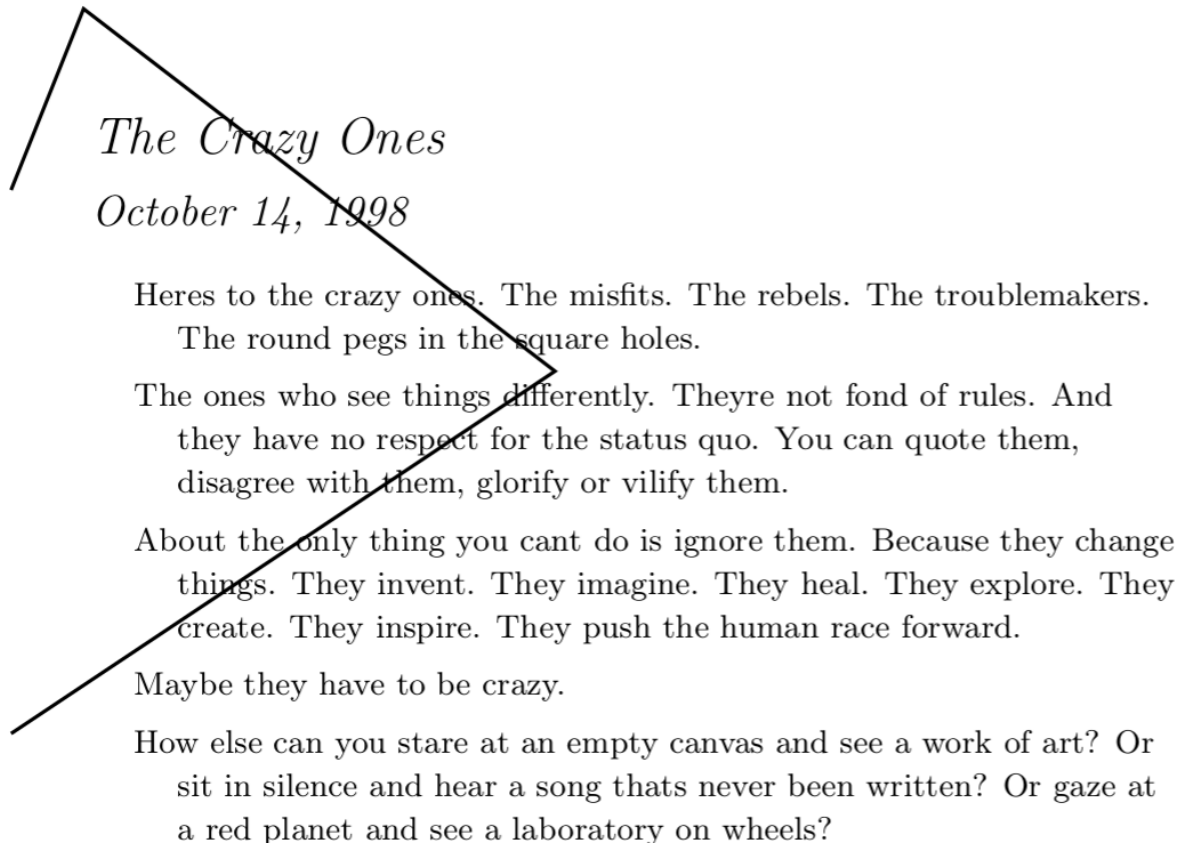
(continues on next page)

(continued from previous page)

```
# Write the annotated file to disk
writer.write("out-line.pdf")
```

## 14.5 PolyLine

If you want to add a line like this:



you can use *PolyLine*:

```
from pypdf import PdfReader, PdfWriter
from pypdf.annotations import PolyLine
from pypdf.generic import ArrayObject, FloatObject, NameObject

reader = PdfReader("crazyones.pdf")
page = reader.pages[0]
writer = PdfWriter()
writer.add_page(page)

# Add the polyline
# By default, the line will be transparent. Set an explicit color.
annotation = PolyLine(
    vertices=[(50, 550), (200, 650), (70, 750), (50, 700)],
```

(continues on next page)

(continued from previous page)

```

)
annotation[NameObject("/C")] = ArrayObject(
    [FloatObject(0.9), FloatObject(0.1), FloatObject(0)]
)
writer.add_annotation(page_number=0, annotation=annotation)

# Write the annotated file to disk
writer.write("out-polyline.pdf")

```

## 14.6 Rectangle

If you want to add a rectangle like this:

*The Crazy Ones*

*October 14, 1998*

Heres to the crazy ones. The misfits. The rebels. The troublemakers.

The round pegs in the square holes.

The ones who see things differently. Theyre not fond of rules. And they have no respect for the status quo. You can quote them, disagree with them, glorify or vilify them.

About the only thing you cant do is ignore them. Because they change things. They invent. They imagine. They heal. They explore. They create. They inspire. They push the human race forward.

Maybe they have to be crazy.

How else can you stare at an empty canvas and see a work of art? Or sit in silence and hear a song thats never been written? Or gaze at a red planet and see a laboratory on wheels?

We make tools for these kinds of people.

you can use *Rectangle*:

```

from pypdf import PdfReader, PdfWriter
from pypdf.annotations import Rectangle

reader = PdfReader("crazyones.pdf")
page = reader.pages[0]
writer = PdfWriter()
writer.add_page(page)

# Add the rectangle
annotation = Rectangle(
    rect=(50, 550, 200, 650),
)
writer.add_annotation(page_number=0, annotation=annotation)

```

(continues on next page)

(continued from previous page)

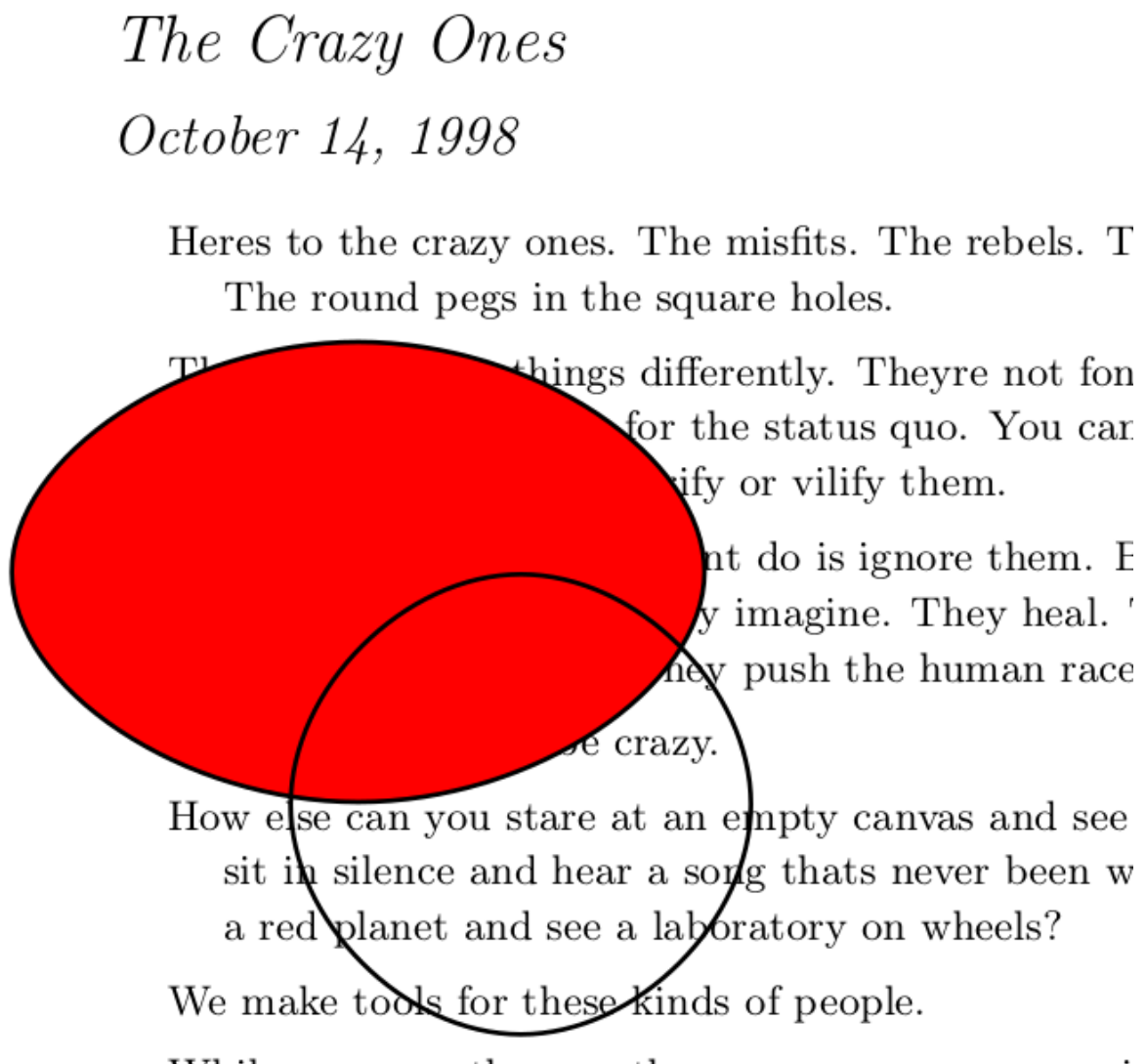
```
# Write the annotated file to disk
writer.write("out-rectangle.pdf")
```

If you want the rectangle to be filled, use the `interiour_color="ff0000"` parameter.

This method uses the “square” annotation type of the PDF format.

## 14.7 Ellipse

If you want to add a circle like this:



you can use *Ellipse*:

```
from pypdf import PdfReader, PdfWriter
from pypdf.annotations import Ellipse
```

(continues on next page)

(continued from previous page)

```

reader = PdfReader("crazyones.pdf")
page = reader.pages[0]
writer = PdfWriter()
writer.add_page(page)

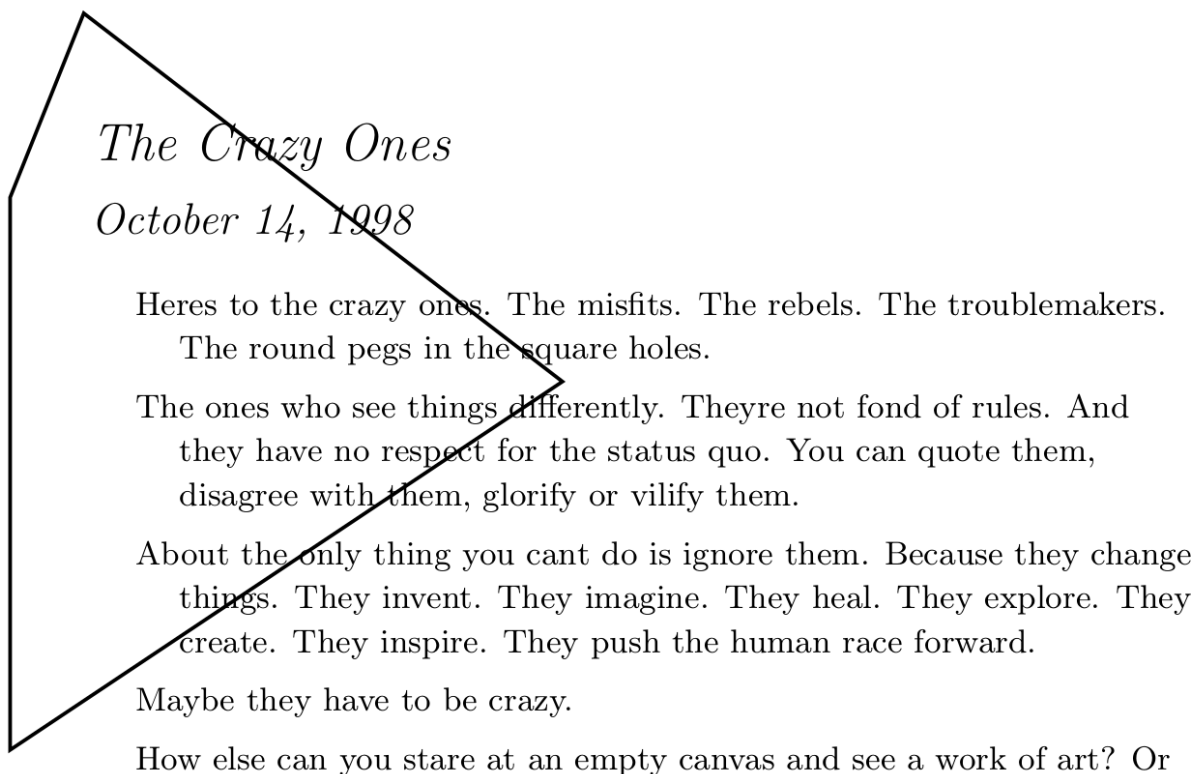
# Add the rectangle
annotation = Ellipse(
    rect=(50, 550, 200, 650),
)
writer.add_annotation(page_number=0, annotation=annotation)

# Write the annotated file to disk
writer.write("out-ellipse.pdf")

```

## 14.8 Polygon

If you want to add a polygon like this:



you can use *Polygon*:

```

from pypdf import PdfReader, PdfWriter
from pypdf.annotations import Polygon

reader = PdfReader("crazyones.pdf")
page = reader.pages[0]

```

(continues on next page)



(continued from previous page)

```

writer = PdfWriter()
writer.add_page(page)

# Add the line
annotation = Polygon(
    vertices=[(50, 550), (200, 650), (70, 750), (50, 700)],
)
writer.add_annotation(page_number=0, annotation=annotation)

# Write the annotated file to disk
writer.write("out-polygon.pdf")

```

## 14.9 Popup

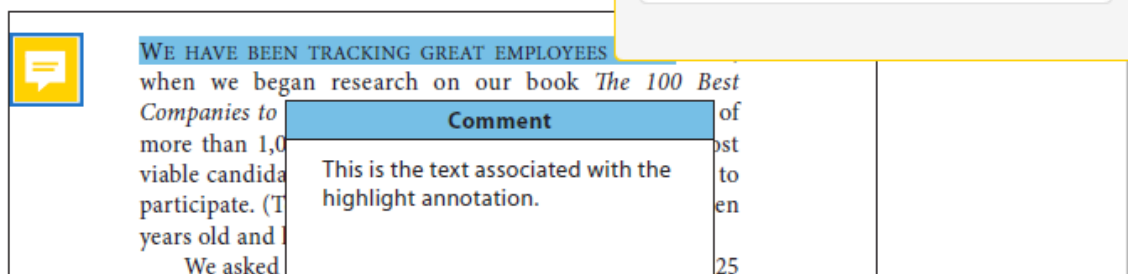
Manage the Popup windows for markups, looks like this:

|           |     |                      |
|-----------|-----|----------------------|
| CHAPTER 8 | 604 | Interactive Features |
|-----------|-----|----------------------|

### 8.4 Annotations

An *annotation* associates an object such as a note, sound, or movie with a location on a page of a PDF document, or provides a way to interact with the user by means of the mouse and keyboard. PDF includes a wide variety of standard annotation types, described in detail in Section 8.4.5, “Annotation Types.”

Many of the standard annotation types may be displayed in either the open or the closed state. When closed, they appear on the page in some form, such as an icon, a box, or a rubber stamp, depending on the type. When the user *activates* the annotation by clicking it, it can be opened, such as by opening a pop-up window displaying a text field, playing a sound or a movie.



you can use *Popup*:

```

from pypdf import PdfWriter
from pypdf.annotations import Popup, Text

# Arrange
writer = PdfWriter()

```

(continues on next page)

(continued from previous page)

```

writer.append("crazyones.pdf", [0])

# Act
text_annotation = writer.add_annotation(
    0,
    Text(
        text="Hello World\nThis is the second line!",
        rect=(50, 550, 200, 650),
        open=True,
    ),
)

popup_annotation = Popup(
    rect=(50, 550, 200, 650),
    open=True,
    parent=text_annotation, # use the output of add_annotation
)

writer.write("out-popup.pdf")

```

You have to use the returned result from `add_annotation()` as it is the parent annotation with which this popup annotation shall be associated.

## 14.10 Link

If you want to add a link, you can use *Link*:

```

from pypdf import PdfReader, PdfWriter
from pypdf.annotations import Link

reader = PdfReader("crazyones.pdf")
page = reader.pages[0]
writer = PdfWriter()
writer.add_page(page)

# Add the link
annotation = Link(
    rect=(50, 550, 200, 650),
    url="https://martin-thoma.com/",
)
writer.add_annotation(page_number=0, annotation=annotation)

# Write the annotated file to disk
writer.write("out-link.pdf")

```

You can also add internal links:

```

from pypdf import PdfReader, PdfWriter
from pypdf.annotations import Link
from pypdf.generic import Fit

reader = PdfReader("crazyones.pdf")

```

(continues on next page)

(continued from previous page)

```

page = reader.pages[0]
writer = PdfWriter()
writer.add_page(page)

# Add the link
annotation = Link(
    rect=(50, 550, 200, 650),
    target_page_index=3,
    fit=Fit(fit_type="/FitH", fit_args=(123,)),
)
writer.add_annotation(page_number=0, annotation=annotation)

# Write the annotated file to disk
writer.write("out-internal-link.pdf")

```

## 14.11 Text Markup Annotations

Text markup annotations refer to a specific piece of text within the document.

These are a bit more complicated as you need to know exactly where the text is, the so-called “Quad points”.

### 14.11.1 Highlighting

If you want to highlight text like this:

*The **Crazy** Ones*  
*October 14, 1998*

Heres to the crazy ones. The misfits. The rebels. The troublemakers.  
 The round pegs in the square holes.

you can use *Highlight*:

```

from pypdf import PdfReader, PdfWriter
from pypdf.annotations import Highlight
from pypdf.generic import ArrayObject, FloatObject

reader = PdfReader("crazyones.pdf")
page = reader.pages[0]
writer = PdfWriter()
writer.add_page(page)

rect = (50, 550, 200, 650)
quad_points = [rect[0], rect[1], rect[2], rect[1], rect[0], rect[3], rect[2], rect[3]]

# Add the highlight

```

(continues on next page)

(continued from previous page)

```
annotation = Highlight(  
    rect=rect,  
    quad_points=ArrayObject([FloatObject(quad_point) for quad_point in quad_points]),  
)  
writer.add_annotation(page_number=0, annotation=annotation)  
  
# Write the annotated file to disk  
writer.write("out-highlight.pdf")
```

## ADDING A STAMP OR WATERMARK TO A PDF

Adding stamps or watermarks are two common ways to manipulate PDF files. A stamp is adding something on top of the document, a watermark is in the background of the document.

### 15.1 Stamp (Overlay) / Watermark (Underlay)

The process of stamping and watermarking is the same, you just need to set `over` parameter to `True` for stamping and `False` for watermarking.

You can use `merge_page()` if you don't need to transform the stamp:

```
from pypdf import PdfReader, PdfWriter

stamp = PdfReader("jpeg.pdf").pages[0]
writer = PdfWriter(clone_from="crazyones.pdf")
for page in writer.pages:
    page.merge_page(stamp, over=False) # here set to False for watermarking

writer.write("out-watermark.pdf")
```

Otherwise use `merge_transformed_page()` with `Transformation` if you need to translate, rotate, scale, etc. the stamp before merging it to the content page.

```
from pathlib import Path
from typing import List, Union

from pypdf import PdfReader, PdfWriter, Transformation

def stamp(
    content_pdf: Union[Path, str],
    stamp_pdf: Union[Path, str],
    pdf_result: Union[Path, str],
    page_indices: Union[None, List[int]] = None,
):
    stamp_page = PdfReader(stamp_pdf).pages[0]

    writer = PdfWriter()
    # page_indices can be a List(array) of page, tuples are for range definition
    reader = PdfReader(content_pdf)
    writer.append(reader, pages=page_indices)
```

(continues on next page)

(continued from previous page)

```

for content_page in writer.pages:
    content_page.merge_transformed_page(
        stamp_page,
        Transformation().scale(0.5),
    )

writer.write(pdf_result)

stamp("crazyones.pdf", "jpeg.pdf", "out-scale.pdf")

```

If you are experiencing wrongly rotated watermarks/stamps, try to use `transfer_rotation_to_content()` on the corresponding pages beforehand to fix the page boxes.

## *The Crazy Ones*

*October 14, 1998*

Heres to the crazy ones. The misfits. The rebels. The troublemakers.  
The round pegs in the square holes.

The ones who see things differently. Theyre not fond of rules. And  
they have no respect for the status quo. You can quote them,  
disagree with them, glorify or vilify them.

About the only thing you cant do is ignore them. Because they see  
things. They invent. They imagine. They heal. They explore. They  
create. They inspire. They push the human race forward.

Maybe they have to be crazy.

How else can you stare at an empty canvas and see a work of art? Or  
sit in silence and hear a song thats never been written? Or gaze at  
a red planet and see a laboratory on wheels?

We need tools for these kinds of people.

Whatsomeone calls them as the crazy ones, we call them genius. Because the  
people who are crazy enough to think they can change the world,  
are the ones who do.

Example of stamp:

# *The Crazy Ones*

*October 14, 1998*

Heres to the crazy ones. The misfits. The rebels. The troublemakers.  
The round pegs in the square holes.

The ones who see things differently. Theyre not fond of rules. And  
they have no respect for the status quo. You can quote them,  
disagree with them, glorify or vilify them.

About the only thing you cant do is ignore them. Because they change  
things. They invent. They imagine. They heal. They explore. They  
create. They inspire. They push the human race forward.

Maybe they have to be crazy.

How else can you stare at an empty canvas and see a work of art? Or  
sit in silence and hear a song thats never been written? Or gaze at  
a red planet and see a laboratory on wheels?

We make tools for these kinds of people.

While some see them as the crazy ones, we see genius. Because the  
people who are crazy enough to think they can change the world,  
are the ones who do.

Example of watermark:

## 15.2 Stamping images directly

The above code only works for stamps that are already in PDF format. However, you can easily convert an image to PDF image using [Pillow](#).

```
from io import BytesIO
from pathlib import Path
from typing import List, Union

from PIL import Image
from pypdf import PageRange, PdfReader, PdfWriter, Transformation

def image_to_pdf(stamp_img: Union[Path, str]) -> PdfReader:
    img = Image.open(stamp_img)
    img_as_pdf = BytesIO()
    img.save(img_as_pdf, "pdf")
```

(continues on next page)

(continued from previous page)

```
    return PdfReader(img_as_pdf)

def stamp_img(
    content_pdf: Union[Path, str],
    stamp_img: Union[Path, str],
    pdf_result: Union[Path, str],
    page_indices: Union[PageRange, List[int], None] = None,
):
    # Convert the image to a PDF
    stamp_pdf = image_to_pdf(stamp_img)

    # Then use the same stamp code from above
    stamp_page = stamp_pdf.pages[0]

    writer = PdfWriter()

    reader = PdfReader(content_pdf)
    writer.append(reader, pages=page_indices)
    for content_page in writer.pages:
        content_page.merge_transformed_page(
            stamp_page,
            Transformation(),
        )

    writer.write(pdf_result)

stamp_img("crazyones.pdf", "nup-source.png", "out-image.pdf")
```



## ADDING JAVASCRIPT TO A PDF

PDF readers vary in the extent they support JavaScript, with some not supporting it at all.

Adobe has documentation on its support here: <https://opensource.adobe.com/dc-acrobat-sdk-docs/library/jsapi-ref/index.html>

### 16.1 Launch print window on opening

```
from pypdf import PdfWriter

writer = PdfWriter(clone_from="example.pdf")

# Add JavaScript to launch the print window on opening this PDF.
writer.add_js("this.print({bUI:true,bSilent:false,bShrinkToFit:true});")

writer.write("out-print-window.pdf")
```



## ADDING VIEWER PREFERENCES

It is possible to set viewer preferences of a PDF file. §12.2 of the [PDF 1.7 specification](#).

Note that the `/ViewerPreferences` dictionary does not exist by default. If it is not already present, it must be created by calling the `create_viewer_preferences()` method.

If viewer preferences exist in a PDF file being read with `PdfReader`, you can access them as properties of `viewer_preferences`. Otherwise, the `viewer_preferences` property will be set to `None`.

### 17.1 Example

```
from pypdf import PdfWriter
from pypdf.generic import ArrayObject, NumberObject

writer = PdfWriter()

writer.create_viewer_preferences()

# /HideToolbar
writer.viewer_preferences.hide_toolbar = True
# /HideMenubar
writer.viewer_preferences.hide_menubar = True
# /HideWindowUI
writer.viewer_preferences.hide_windowui = True
# /FitWindow
writer.viewer_preferences.fit_window = True
# /CenterWindow
writer.viewer_preferences.center_window = True
# /DisplayDocTitle
writer.viewer_preferences.display_doctitle = True

# /NonFullScreenPageMode
writer.viewer_preferences.non_fullscreen_pagemode = "/UseNone" # default
writer.viewer_preferences.non_fullscreen_pagemode = "/UseOutlines"
writer.viewer_preferences.non_fullscreen_pagemode = "/UseThumbs"
writer.viewer_preferences.non_fullscreen_pagemode = "/UseOC"

# /Direction
writer.viewer_preferences.direction = "/L2R" # default
writer.viewer_preferences.direction = "/R2L"
```

(continues on next page)

(continued from previous page)

```
# /ViewArea
writer.viewer_preferences.view_area = "/CropBox"
# /ViewClip
writer.viewer_preferences.view_clip = "/CropBox"
# /PrintArea
writer.viewer_preferences.print_area = "/CropBox"
# /PrintClip
writer.viewer_preferences.print_clip = "/CropBox"

# /PrintScaling
writer.viewer_preferences.print_scaling = "/None"
writer.viewer_preferences.print_scaling = "/AppDefault" # default according to PDF spec

# /Duplex
writer.viewer_preferences.duplex = "/Simplex"
writer.viewer_preferences.duplex = "/DuplexFlipShortEdge"
writer.viewer_preferences.duplex = "/DuplexFlipLongEdge"

# /PickTrayByPDFSize
writer.viewer_preferences.pick_tray_by_pdfsize = True
# /PrintPageRange
writer.viewer_preferences.print_pagerange = ArrayObject(
    [NumberObject("1"), NumberObject("10"), NumberObject("20"), NumberObject("30")]
)
# /NumCopies
writer.viewer_preferences.num_copies = 2

for i in range(40):
    writer.add_blank_page(10, 10)

writer.write("out.pdf")
```

The names beginning with a slash character are part of the PDF file format. They are included here to ease searching the pypdf documentation for these names from the PDF specification.

## INTERACTIONS WITH PDF FORMS

### 18.1 Reading form fields

```
from pypdf import PdfReader

reader = PdfReader("form.pdf")
fields = reader.get_form_text_fields()
fields == {"key": "value", "key2": "value2"}

# You can also get all fields:
fields = reader.get_fields()
```

### 18.2 Filling out forms

```
from pypdf import PdfReader, PdfWriter

reader = PdfReader("form.pdf")
writer = PdfWriter()

page = reader.pages[0]
fields = reader.get_fields()

writer.append(reader)

writer.update_page_form_field_values(
    writer.pages[0],
    {"fieldname": "some filled in text"},
    auto_regenerate=False,
)

writer.write("out-filled-form.pdf")
```

Generally speaking, you will always want to use `auto_regenerate=False`. The parameter is `True` by default for legacy compatibility, but this flags the PDF processor to recompute the field's rendering, and may trigger a “save changes” dialog for users who open the generated PDF.

If you want to flatten your form, that is, keeping all form field contents while removing the form fields themselves, you can set the `flatten` parameter in `update_page_form_field_values()` to `True`. This will convert form field contents to regular PDF content. Afterwards, use `remove_annotations()` with `subtypes="/Widget"` to remove all form fields to get an actual flattened PDF.

## 18.3 Some notes about form fields and annotations

PDF forms have a dual-nature approach to the fields:

- Within the root object, an `/AcroForm` structure exists. Inside it, you could find (optional):
  - some global elements (Fonts, Resources,...)
  - some global flags (like `/NeedAppearances` (set/cleared with `auto_regenerate` parameter in `update_page_form_field_values()`) that indicates if the reading program should re-render the visual fields upon document launch)
  - `/XFA` that houses a form in XDP format (very specific XML that describes the form rendered by some viewers); the `/XFA` form overrides the page content
  - `/Fields` that houses an array of indirect references that reference the upper *Field* Objects (roots)
- Within the page `/Annots`, you will spot `/Widget` annotations that define the visual rendering.

To flesh out this overview:

- The core-specific properties of a field are:
  - `/FT`: The field type (Button, Text, Choice, or Signature).
  - `/T`: The partial field name.
  - `/V`: The field's value, whose format varies depending on the field type.
  - `/DV`: The default value to which the field reverts when a reset-form action is executed.
- To streamline readability, *Field* Objects and *Widget* Objects can be fused housing all properties.
- Fields can be organized hierarchically, id est one field can be placed under another. In such instances, the `/Parent` will have an `IndirectObject` providing Bottom-Up links and `/Kids` is an array carrying `IndirectObjects` for Top-Down navigation; *Widget* Objects are still required for visual rendering. To call upon them, use the *fully qualified field name* (where all the individual names of the parent objects are separated by `.`)  
 For instance, take two (visual) fields both called *city*, but attached below *sender* and *receiver*; the corresponding full names will be *sender.city* and *receiver.city*.
- When a field is repeated on multiple pages, the Field Object will have many *Widget* Objects in `/Kids`. These objects are pure *widgets*, containing no *field* specific data.
- If `Fields` stores only hidden values, no *Widgets* are required.

In *pypdf* fields are extracted from the `/Fields` array:

```
from pypdf import PdfReader

reader = PdfReader("form.pdf")
fields = reader.get_fields()
```

```
from pypdf import PdfReader
from pypdf.constants import AnnotationDictionaryAttributes

reader = PdfReader("form.pdf")
fields = []
for page in reader.pages:
    for annot in page.annotations:
        annot = annot.get_object()
```

(continues on next page)

(continued from previous page)

```
if annot[AnnotationDictionaryAttributes.Subtype] == "/Widget":  
    fields.append(annot)
```

However, while similar, there are some relevant differences between the two above blocks of code. Most importantly, the first block will return a list of Field objects, whereas the second will return more generic dictionary-like objects. The objects lists will *mostly* reference the same object in the underlying PDF, meaning you'll find that `obj_taken_fom_first_list.indirect_reference == obj_taken_from_second_list.indirect_reference`. Field objects are generally more ergonomic, as the exposed data can be accessed via clearly named properties. However, the more generic dictionary-like objects will contain data that the Field object does not expose, such as the Rect (the widget's position on the page). Therefore, the correct approach depends on your use case.

However, it is also important to note that the two lists do not *always* refer to the same underlying PDF object. For example, if the form contains radio buttons, you will find that `reader.get_fields()` will get the parent object (the group of radio buttons) whereas `page.annotations` will return all the child objects (the individual radio buttons).

#### Note

Remember that fields are not stored in pages; if you use `add_page()` the field structure is not copied. It is recommended to use `.append()` with the proper parameters instead.

In case of missing *field* objects in `/Fields`, `writer.reattach_fields()` will parse page(s) annotations and will reattach them. This fix cannot guess intermediate fields and will not report fields using the same *name*.

## 18.4 Identify pages where fields are used

To ease locating page fields you can use `get_pages_showing_field` of PdfReader or PdfWriter. This method accepts a field object, a *PdfObject* that represents a field (as extracted from `_root_object["/AcroForm"]["/Fields"]`). The method returns a list of pages, because a field can have multiple widgets as mentioned previously (e.g., radio buttons or text displayed on multiple pages).

The page numbers can then be retrieved as usual by using `page.page_number`.





## HANDLING OUTLINES

PDF outlines - also known as bookmarks - provide a structured navigation panel in PDF readers. `pypdf` allows you to read, create, and modify both simple and deeply nested outlines.

### 19.1 Writing PDF Outlines

To add outlines, use the `add_outline_item()` method. This method returns a reference to the created outline, which you can use as a parent to create nested (hierarchical) bookmarks.

#### 19.1.1 Adding a Simple Outline

The following example shows how to add a single top-level bookmark. We add an outline item pointing to the first page (index 0) and save the result.

```
from pypdf import PdfWriter

writer = PdfWriter(clone_from="crazyones.pdf")

# Add a top-level bookmark
writer.add_outline_item(
    title="Introduction",
    page_number=0
)

writer.write("simple-example.pdf")
```

#### 19.1.2 Adding Nested Outlines

You can build hierarchies (like Chapter → Section) by passing the parent outline item to the `parent` parameter of a new item.

In the example below, we create a root item “Introduction” and nest two sections under it.

```
from pypdf import PdfWriter

writer = PdfWriter(clone_from="crazyones.pdf")

# Add parent (Chapter)
introduction = writer.add_outline_item(
    title="Chapter 1",
    page_number=0
```

(continues on next page)

(continued from previous page)

```

)

# Add children (sections) nested under the introduction
writer.add_outline_item(
    title="Section 1.1",
    page_number=0,
    parent=introduction
)

writer.add_outline_item(
    title="Section 1.2",
    page_number=0,
    parent=introduction
)

writer.write("nested-example.pdf")

```

### 19.1.3 Advanced Styling and View Modes (Fit Options)

You can customize the appearance and behavior of bookmarks using optional parameters, such as changing the text color or applying bold and italic styles.

For detailed information on all available parameters and their formats, please refer to the [add\\_outline\\_item\(\)](#) API documentation.

The fit parameter determines how the page is displayed when the user clicks the bookmark. You can use the [Fit](#) helper to specify modes like [fit\(\)](#), [fit\\_horizontally\(\)](#), or [xyz\(\)](#).

```

from pypdf import PdfWriter
from pypdf.generic import Fit

writer = PdfWriter(clone_from="crazyones.pdf")

# Top-level chapter (Points to Page 3, Index 2)
chapter2 = writer.add_outline_item(
    title="Chapter 2",
    page_number=0,
    color=(0, 0, 1),
    bold=True,
    italic=False,
    is_open=True,
    fit=Fit.fit()
)

# Section under Chapter 2 (Points to Page 3, Index 2)
section2_1 = writer.add_outline_item(
    title="Section 2.1",
    page_number=0,
    parent=chapter2,
    color=(0, 0.5, 0),
    bold=False,
    italic=True,
    is_open=False,

```

(continues on next page)

(continued from previous page)

```

fit=Fit.fit_horizontally(top=800)
)

# Section with custom zoom (Points to Page 3, Index 2)
section2_2 = writer.add_outline_item(
    title="Section 2.2",
    page_number=0,
    parent=chapter2,
    color=(1, 0, 0),
    bold=True,
    italic=True,
    is_open=True,
    fit=Fit.xyz(left=0, top=800, zoom=1.25)
)

writer.write("advanced-example.pdf")

```

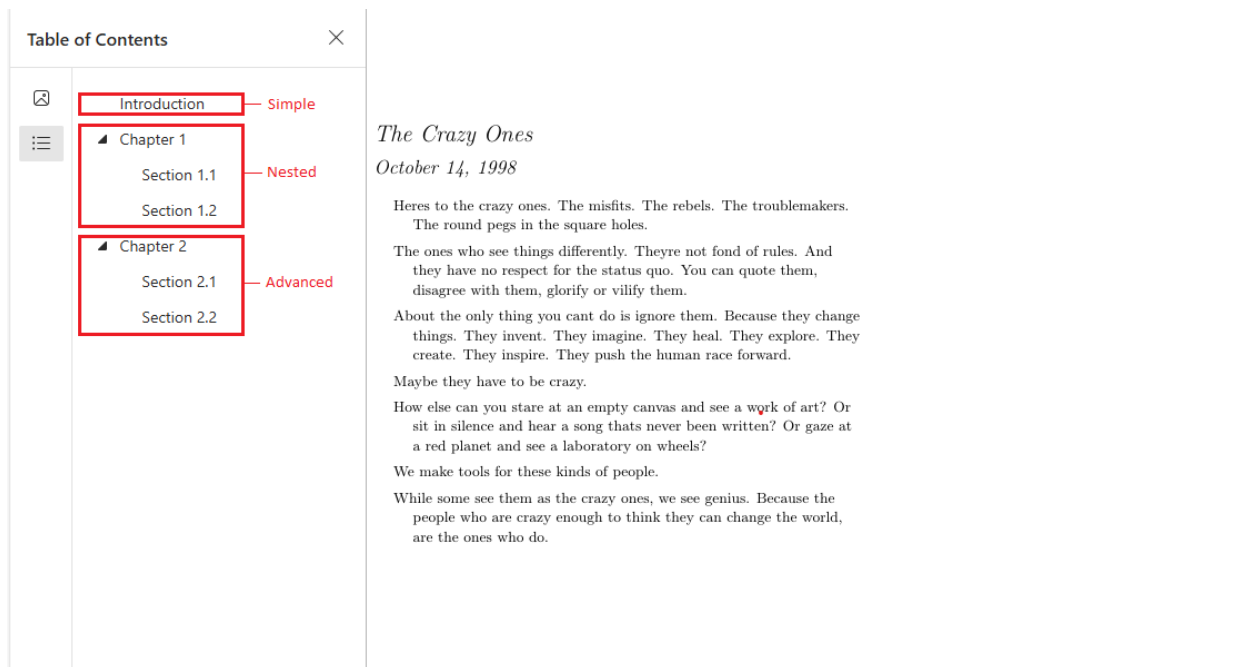


Fig. 1: An annotated screenshot illustrating simple, nested, and advanced PDF bookmarks in a Table of Contents.

## 19.2 Reading PDF Outlines

pypdf represents outlines as a list of *Destination* objects. If an outline has children, they appear as a nested list directly following their parent.

To retrieve the page number a bookmark points to, use the *get\_destination\_page\_number()* method, which returns a zero-based page index.

## 19.2.1 Reading Simple Outlines

To extract only the top-level bookmarks (ignoring nested sections), you can iterate over the `outline` property. Since nested children appear as lists within the outline structure, you must explicitly check for and skip them (`isinstance(outline, list)`) to avoid errors. The example below reads the file created in the previous section.

```
from pypdf import PdfReader

reader = PdfReader("simple-example.pdf")

print("Simple Outline (Top-Level Only):")
print("-" * 32)

for outline in reader.outline:
    # Check if the item is a list (which represents nested children)
    if isinstance(outline, list):
        continue # Skip the nested parts completely

    page_number = reader.get_destination_page_number(outline)

    if page_number is None:
        print(f"{outline.title} -> No page destination")
    else:
        print(f"{outline.title} -> page {page_number + 1}")
```

```
Simple Outline (Top-Level Only):
-----
Introduction -> page 1
```

## 19.2.2 Reading Nested Outlines

When dealing with hierarchical bookmarks, the `outline` property may contain lists inside lists. You should use a recursive function to traverse the tree.

The following example defines a `print_outline` function that handles indentation and nested lists to display the structure of the document we created earlier.

```
from typing import List, Union

from pypdf import PdfReader
from pypdf.generic import Destination

def print_outline(
    outlines: List[Union[Destination, List[Destination]]],
    reader: PdfReader,
    level: int = 0
) -> None:
    """Recursively print all outline items with indentation."""
    for item in outlines:
        if isinstance(item, list):
            # Recursively handle the nested list of children
            print_outline(item, reader, level + 1)
        else:
```

(continues on next page)

(continued from previous page)

```
page_number = reader.get_destination_page_number(item)

indent = "  " * level

if page_number is None:
    print(f"{indent}- {item.title} (No page destination)")
else:
    print(f"{indent}- {item.title} (Page {page_number + 1})")

reader = PdfReader("nested-example.pdf")

print("Nested Outline Hierarchy:")
print("-" * 25)

print_outline(reader.outline, reader)
```

Nested Outline Hierarchy:

```
-----
- Chapter 1 (Page 1)
  - Section 1.1 (Page 1)
  - Section 1.2 (Page 1)
```



## STREAMING DATA WITH PYPDF

In some cases, you might want to avoid saving things explicitly as a file to disk, e.g. when you want to store the PDF in a database or AWS S3.

pypdf supports streaming data to a file-like object:

```
from io import BytesIO
from pypdf import PdfReader, PdfWriter

# Prepare example
with open("example.pdf", "rb") as fh:
    bytes_stream = BytesIO(fh.read())

# Read from bytes_stream
reader = PdfReader(bytes_stream)

# Write to bytes_stream
writer = PdfWriter()
with BytesIO() as bytes_stream:
    writer.write(bytes_stream)
```

### 20.1 Writing a PDF directly to AWS S3

Suppose you want to manipulate a PDF and write it directly to AWS S3 without having to write the document to a file first. We have the original PDF in `raw_bytes_data` as bytes and want to set `my-secret-password`:

```
from io import BytesIO

import boto3
from pypdf import PdfReader, PdfWriter

reader = PdfReader(BytesIO(raw_bytes_data))
writer = PdfWriter()

# Add all pages to the writer
for page in reader.pages:
    writer.add_page(page)

# Add a password to the new PDF
writer.encrypt("my-secret-password")
```

(continues on next page)

(continued from previous page)

```
# Save the new PDF to a file
with BytesIO() as bytes_stream:
    writer.write(bytes_stream)
    bytes_stream.seek(0)
    s3 = boto3.client("s3")
    s3.write_get_object_response(
        Body=bytes_stream, RequestRoute=request_route, RequestToken=request_token
    )
```

## 20.2 Reading PDFs directly from cloud services

One option is to first download the file and then pass the local file path to PdfReader. Another option is to get a byte stream.

For AWS S3 it works like this:

```
from io import BytesIO

import boto3
from pypdf import PdfReader

s3 = boto3.client("s3")
obj = s3.get_object(Body=csv_buffer.getvalue(), Bucket="my-bucket", Key="my/doc.pdf")
reader = PdfReader(BytesIO(obj["Body"].read()))
```

To use with Google Cloud storage:

```
from io import BytesIO

from google.cloud import storage

# os.environ["GOOGLE_APPLICATION_CREDENTIALS"] must be set
storage_client = storage.Client()
blob = storage_client.bucket("my-bucket").blob("mydoc.pdf")
file_stream = BytesIO()
blob.download_to_file(file_stream)
reader = PdfReader(file_stream)
```



## REDUCE PDF FILE SIZE

There are multiple ways to reduce the size of a given PDF file. The easiest one is to remove content (e.g., images) or pages.

### 21.1 Removing duplication

Some PDF documents contain the same object multiple times. For example, if an image appears three times in a PDF, it could be embedded three times. Or it can be embedded once and referenced twice.

When adding data to a PdfWriter, the data is copied while respecting the original format. For example, if two pages include the same image which is duplicated in the source document, the object will be duplicated in the PdfWriter object.

Additionally, when you delete objects in a document, pypdf cannot easily identify whether the objects are used elsewhere or not or if the user wants to keep them in. When writing the PDF file, these objects will be hidden within (part of the file, but not displayed).

To reduce the file size, use a compression call: `writer.compress_identical_objects(remove_duplicates=True, remove_unreferenced=True)`

- `remove_duplicates` enables/disables compression merging identical objects.
- `remove_unreferenced` enables/disables suppression of unused objects.

It is recommended to apply this process just before writing to the file/stream.

It depends on the PDF how well this works, but we have seen an 86% file reduction (from 5.7 MB to 0.8 MB) within a real PDF.

### 21.2 Removing Images

```
from pypdf import PdfWriter

writer = PdfWriter(clone_from="example.pdf")

writer.remove_images()

writer.write("out-no-images.pdf")
```

## 21.3 Reducing Image Quality

If we reduce the quality of the images within the PDF, we can **sometimes** reduce the file size of the PDF overall. That depends on how well the reduced quality image can be compressed.

```
from pypdf import PdfWriter

writer = PdfWriter(clone_from="example.pdf")

for page in writer.pages:
    for img in page.images:
        img.replace(img.image, quality=80)

writer.write("out-low-quality.pdf")
```

## 21.4 Lossless Compression

pypdf supports the FlateDecode filter which uses the zlib/deflate compression method. It is a lossless compression, meaning the resulting PDF looks exactly the same.

Deflate compression can be applied to a page via `page.compress_content_streams`:

```
from pypdf import PdfWriter

writer = PdfWriter(clone_from="example.pdf")

for page in writer.pages:
    page.compress_content_streams() # This is CPU intensive!

writer.write("out-lossless.pdf")
```

`page.compress_content_streams` uses `zlib.compress` and supports the `level` parameter: `level=0` means no compression, `level=9` refers to the highest compression.

Using this method, we have seen a reduction by 70% (from 11.8 MB to 3.5 MB) with a real PDF.

## 21.5 Removing Sources

When a page is removed from the page list, its content will still be present in the PDF file. This means that the data may still be used elsewhere.

Simply removing a page from the page list will reduce the page count but not the file size. To exclude the content completely, the pages should not be added to the PDF using the `PdfWriter.append()` function. Instead, only the desired pages should be selected for inclusion (note: [PR #1843](#) will add a page deletion feature).

There can be issues with poor PDF formatting, such as when all pages are linked to the same resource. In such cases, dropping references to specific pages becomes useless because there is only one source for all pages.

Cropping is an ineffective method for reducing the file size because it only adjusts the viewboxes and not the external parts of the source image. Therefore, the content that is no longer visible will still be present in the PDF.

## 21.6 Going Further

The presentation [Putting a Squeeze on Your PDF](#) has other suggestions. One takeaway is that most of the significant size optimizations usually come from image and font modification. However, font optimization, such as replacing, merging, and subsetting, is not within the functionality of pypdf at the moment.



## PDF VERSION SUPPORT

PDF comes in the following versions:

- 1993: 1.0
- 1994: 1.1
- 1996: 1.2
- 1999: 1.3
- 2001: 1.4
- 2003: 1.5
- 2004: 1.6
- 2008: 1.7, ISO 32000-1:2008
- 2017: 2.0, ISO 32000-2:2017

The general format didn't change, but new features got added. It can be that pypdf can do the operations you want on PDF 2.0 files without fully supporting all features of PDF 2.0.

### 22.1 PDF Feature Support by pypdf

| Feature                        | PDF Version | pypdf Support |
|--------------------------------|-------------|---------------|
| CMaps                          | 1.4         |               |
| Transparent Graphics           | 1.4         |               |
| Content Stream Compression     | 1.5         |               |
| Cross-reference Streams        | 1.5         |               |
| Object Streams                 | 1.5         |               |
| Optional Content Groups (OCGs) | 1.5         |               |
| AES Encryption                 | 1.6         |               |

This table is not complete - if in doubt, consider having a look at the API documentation or inside the issues or try with a corresponding PDF file. In general, we are open to add support for missing features. Please open a new issue if it does not exist yet, and keep in mind that we rely on external contributors to support us with the implementation.

One commonly requested feature is proper support reading/handling incremental PDF files, see [issue #3304](#).

See [History of PDF](#) for more features.

Some PDF features are not supported by pypdf, but other libraries can be used for them:

- [pyHanko](#): Cryptographically sign a PDF ([#302](#))

- [camelot-py: Table Extraction \(#231\)](#)

## PDF/A COMPLIANCE

PDF/A is a specialized, ISO-standardized version of the Portable Document Format (PDF) specifically designed for the long-term preservation and archiving of electronic documents. It ensures that files remain accessible, readable, and true to their original appearance by embedding all necessary fonts, images, and metadata within the document itself. By adhering to strict guidelines and minimizing dependencies on external resources or proprietary software, PDF/A ensures the consistent and reliable reproduction of content, safeguarding it against future technological changes and obsolescence.

### 23.1 PDF/A Versions

- **PDF/A-1:** Based on PDF 1.4, PDF/A-1 is the first version of the standard and is divided into two levels: PDF/A-1a (Level A, ensuring accessibility) and PDF/A-1b (Level B, ensuring visual preservation).
  - **Level B** (Basic): Ensures visual preservation and basic requirements for archiving.
  - **Level A** (Accessible): Everything from level B, but includes additional requirements for accessibility, such as tagging, Unicode character mapping, and logical structure.
- **PDF/A-2:** Based on PDF 1.7 (ISO 32000-1), PDF/A-2 adds features and improvements over PDF/A-1, while maintaining compatibility with PDF/A-1b (Level B) documents.
  - **Level B** (Basic): Like PDF/A-1b, but support for PDF 1.7 features such as transparency layers.
  - **Level U** (Unicode): Ensures Unicode mapping without the full accessibility requirements of PDF/A-1a (Level A).
  - **Level A** (Accessible): Similar to PDF/A-1a
- **PDF/A-3:** Based on PDF 1.7 (ISO 32000-1), PDF/A-3 is similar to PDF/A-2 but allows the embedding of non-PDF/A files as attachments, enabling the archiving of source or supplementary data alongside the PDF/A document. This is interesting for invoices which can add XML files.
- **PDF/A-4:** Based on PDF 2.0 (ISO 32000-2), PDF/A-4 introduces new features and improvements for better archiving and accessibility. The previous levels are replaced by PDF/A-4f (ensuring visual preservation and allowing attachments) and PDF/A-4e (Engineering, allows 3D content).

### 23.2 PDF/A-1b

In contrast to other PDF documents, PDF/A-1b documents must fulfill those requirements:

- **MarkInfo Object:** The MarkInfo object is a dictionary object within a PDF/A file that provides information about the logical structure and tagging of the document. The MarkInfo object indicates whether the document is tagged, contains optional content, or has a structure tree that describes the logical arrangement of content such as headings, paragraphs, lists, and tables. By including the MarkInfo object, PDF/A ensures that electronic documents are accessible to users with disabilities, such as those using screen readers or other assistive technologies.

- **Embedded fonts:** All fonts used in the document must be embedded to ensure consistent text rendering across different devices and systems.
- **Color Spaces:** DeviceRGB is a device-dependent color space that relies on the specific characteristics of the output device, which can lead to inconsistent color rendering across various devices. To achieve accurate and consistent color representation, PDF/A requires the use of device-independent color spaces, such as ICC-based color profiles.
- **XMP (Extensible Metadata Platform) metadata:** XMP metadata provides a standardized and extensible way to store essential information about a document and its properties. XMP metadata is an XML-based format embedded directly within a PDF/A file. It contains various types of information, such as document title, author, creation and modification dates, keywords, and copyright information, as well as PDF/A-specific details like conformance level and OutputIntent.

## 23.3 Validation

VeraPDF is the go-to PDF/A validator.

There are several online validators that allow you to simply upload the document:

- [pdfen.com](https://pdfen.com)
- [avepdf.com](https://avepdf.com) : Gives an error report
- [pdfa.org](https://pdfa.org)
- [visual-paradigm.com](https://visual-paradigm.com) - can convert the PDF to a PDF/A
- [pdf2go.com](https://pdf2go.com)
- [slub-dresden.de](https://slub-dresden.de) links to relevant parts in the specification.

## 23.4 pypdf and PDF/A

At the moment, pypdf does not make any guarantees regarding PDF/A. Support is very welcome.



## THE PDFREADER CLASS

```
class pypdf.PdfReader(stream: str | IO[Any] | Path, strict: bool = False, password: None | str | bytes = None, *,
                      root_object_recovery_limit: int | None = 10000)
```

Bases: *PdfDocCommon*

Initialize a PdfReader object.

This operation can take some time, as the PDF stream's cross-reference tables are read into memory.

### Parameters

- **stream** – A File object or an object that supports the standard read and seek methods similar to a File object. Could also be a string representing a path to a PDF file.
- **strict** – Determines whether user should be warned of all problems and also causes some correctable problems to be fatal. Defaults to False.
- **password** – Decrypt PDF file at initialization. If the password is None, the file will not be decrypted. Defaults to None.
- **root\_object\_recovery\_limit** – The maximum number of objects to query for recovering the Root object in non-strict mode. To disable this security measure, pass None.

**strict:** `bool` = False

**flattened\_pages:** `list[PageObject] | None` = None

**resolved\_objects:** `dict[tuple[Any, Any], PdfObject | None]`

Storage of parsed PDF objects.

**close()** → `None`

Close the stream if opened in `__init__` and clear memory.

**property root\_object:** *DictionaryObject*

Provide access to “/Root”. Standardized with PdfWriter.

**property pdf\_header:** `str`

The first 8 bytes of the file.

This is typically something like '%PDF-1.6' and can be used to detect if the file is actually a PDF file and which version it is.

**property xmp\_metadata:** *XmpInformation* | `None`

XMP (Extensible Metadata Platform) data.

**get\_object(indirect\_reference: int | IndirectObject)** → *PdfObject* | `None`

**read\_object\_header**(*stream*: *IO[Any]*) → tuple[int, int]

**cache\_get\_indirect\_object**(*generation*: int, *idnum*: int) → PdfObject | None

**cache\_indirect\_object**(*generation*: int, *idnum*: int, *obj*: PdfObject | None) → PdfObject | None

**read**(*stream*: *IO[Any]*) → None

Read and process the PDF stream, extracting necessary data.

**Parameters**

**stream** – The PDF file stream.

**decrypt**(*password*: str | bytes) → PasswordType

When using an encrypted / secured PDF file with the PDF Standard encryption handler, this function will allow the file to be decrypted. It checks the given password against the document's user password and owner password, and then stores the resulting decryption key if either password is correct.

It does not matter which password was matched. Both passwords provide the correct decryption key that will allow the document to be used with this library.

**Parameters**

**password** – The password to match.

**Returns**

An indicator if the document was decrypted and whether it was the owner password or the user password.

**property is\_encrypted:** bool

Read-only boolean property showing whether this PDF file is encrypted.

Note that this property, if true, will remain true even after the `decrypt()` method is called.

**add\_form\_topname**(*name*: str) → DictionaryObject | None

Add a top level form that groups all form fields below it.

**Parameters**

**name** – text string of the “/T” Attribute of the created object

**Returns**

The created object. None means no object was created.

**rename\_form\_topname**(*name*: str) → DictionaryObject | None

Rename top level form field that all form fields below it.

**Parameters**

**name** – text string of the “/T” field of the created object

**Returns**

The modified object. None means no object was modified.

**property are\_permissions\_valid:** bool | None

Whether the /Perms integrity check passed for this document.

For AES-256 encrypted documents (R=5/R=6), the /Perms field is an encrypted copy of the permissions that can be verified independently. Returns False if this check fails (the /P permissions may have been tampered with).

Returns None if the document is not encrypted or has not yet been decrypted via `decrypt()`. Returns True for non-AES-256 encryption (no /Perms to check).

**property attachment\_list:** `Generator[EmbeddedFile, None, None]`

Iterable of attachment objects.

**property attachments:** `Mapping[str, list[bytes]]`

Mapping of attachment filenames to their content.

**decode\_permissions**(*permissions\_code*: `int`) → `dict[str, bool]`

Take the permissions as an integer, return the allowed access.

**get\_destination\_page\_number**(*destination*: `Destination`) → `int | None`

Retrieve page number of a given Destination object.

#### Parameters

**destination** – The destination to get page number.

#### Returns

The page number or None if page is not found

**get\_fields**(*tree*: `TreeObject | None = None`, *retval*: `dict[Any, Any] | None = None`, *fileobj*: `Any | None = None`, *stack*: `list[PdfObject] | None = None`) → `dict[str, Any] | None`

Extract field data if this PDF contains interactive form fields.

The *tree*, *retval*, *stack* parameters are for recursive use.

#### Parameters

- **tree** – Current object to parse.
- **retval** – In-progress list of fields.
- **fileobj** – A file object (usually a text file) to write a report to on all interactive form fields found.
- **stack** – List of already parsed objects.

#### Returns

A dictionary where each key is a field name, and each value is a *Field* object. By default, the mapping name is used for keys. None if form data could not be located.

**get\_form\_text\_fields**(*full\_qualified\_name*: `bool = False`) → `dict[str, Any]`

Retrieve form fields from the document with textual data.

#### Parameters

**full\_qualified\_name** – to get full name

#### Returns

A dictionary. The key is the name of the form field, the value is the content of the field.

If the document contains multiple form fields with the same name, the second and following will get the suffix .2, .3, ...

**get\_named\_dest\_root**() → *ArrayObject*

**get\_num\_pages**() → `int`

Calculate the number of pages in this PDF file.

#### Returns

The number of pages of the parsed PDF file.

#### Raises

*PdfReadError* – If restrictions prevent this action.

**get\_page**(*page\_number: int*) → *PageObject*

Retrieve a page by number from this PDF file. Most of the time `.pages[page_number]` is preferred.

**Parameters**

**page\_number** – The page number to retrieve (pages begin at zero)

**Returns**

A *PageObject* instance.

**get\_page\_number**(*page: PageObject*) → *int* | *None*

Retrieve page number of a given *PageObject*.

**Parameters**

**page** – The page to get page number. Should be an instance of *PageObject*

**Returns**

The page number or *None* if page is not found

**get\_pages\_showing\_field**(*field: Field | PdfObject | IndirectObject*) → *list[PageObject]*

Provides list of pages where the field is called.

**Parameters**

**field** – Field Object, PdfObject or IndirectObject referencing a Field

**Returns**

*List of pages* –

- **Empty list:**

The field has no widgets attached (either hidden field or ancestor field).

- **Single page list:**

Page where the widget is present (most common).

- **Multi-page list:**

Field with multiple kids widgets (example: radio buttons, field repeated on multiple pages).

**property metadata:** *DocumentInformation* | *None*

Retrieve the PDF file's document information dictionary, if it exists.

Note that some PDF files use metadata streams instead of document information dictionaries, and these metadata streams will not be accessed by this function.

**property named\_destinations:** *dict[str, Destination]*

A read-only dictionary which maps names to destinations.

**property open\_destination:** *None* | *Destination* | *TextStringObject* | *ByteStringObject*

Property to access the opening destination (`/OpenAction` entry in the PDF catalog). It returns *None* if the entry does not exist or is not set.

**Raises**

*Exception* – If a destination is invalid.

**property outline:** *list[Destination]* | *list[Destination]* | *list[Destination]*

Read-only property for the outline present in the document (i.e., a collection of 'outline items' which are also known as 'bookmarks').

**property page\_labels:** *list[str]*

A list of labels for the pages in this document.

This property is read-only. The labels are in the order that the pages appear in the document.

**property** `page_layout`: `str` | `None`

Get the page layout currently being used.

Table 1: Valid layout values

|                               |                                                            |
|-------------------------------|------------------------------------------------------------|
| <code>/NoLayout</code>        | Layout explicitly not specified                            |
| <code>/SinglePage</code>      | Show one page at a time                                    |
| <code>/OneColumn</code>       | Show one column at a time                                  |
| <code>/TwoColumnLeft</code>   | Show pages in two columns, odd-numbered pages on the left  |
| <code>/TwoColumn-Right</code> | Show pages in two columns, odd-numbered pages on the right |
| <code>/TwoPageLeft</code>     | Show two pages at a time, odd-numbered pages on the left   |
| <code>/TwoPageRight</code>    | Show two pages at a time, odd-numbered pages on the right  |

**property** `page_mode`: `Literal['/UseNone', '/UseOutlines', '/UseThumbs', '/FullScreen', '/UseOC', '/UseAttachments']` | `None`

Get the page mode currently being used.

Table 2: Valid mode values

|                              |                                          |
|------------------------------|------------------------------------------|
| <code>/UseNone</code>        | Do not show outline or thumbnails panels |
| <code>/UseOutlines</code>    | Show outline (aka bookmarks) panel       |
| <code>/UseThumbs</code>      | Show page thumbnails panel               |
| <code>/FullScreen</code>     | Fullscreen view                          |
| <code>/UseOC</code>          | Show Optional Content Group (OCG) panel  |
| <code>/UseAttachments</code> | Show attachments panel                   |

**property** `pages`: `list[PageObject]`

Property that emulates a list of `PageObject`. This property allows to get a page or a range of pages.

#### Note

For PdfWriter only: Provides the capability to remove a page/range of page from the list (using the del operator). Remember: Only the page entry is removed, as the objects beneath can be used elsewhere. A solution to completely remove them - if they are not used anywhere - is to write to a buffer/temporary file and then load it into a new PdfWriter.

**remove\_page**(*page*: `int` | `PageObject` | `IndirectObject`, *clean*: `bool` = `False`) → `None`

Remove page from pages list.

#### Parameters

- **page** –
  - `int`: Page number to be removed.
  - `PageObject`: page to be removed. If the page appears many times only the first one will be removed.
  - `IndirectObject`: Reference to page to be removed.
- **clean** – replace `PageObject` with `NullObject` to prevent annotations or destinations to reference a detached page.

**property threads:** [\*ArrayObject\*](#) | **None**

Read-only property for the list of threads.

See §12.4.3 from the PDF 1.7 or 2.0 specification.

It is an array of dictionaries with “/F” (the first bead in the thread) and “/I” (a thread information dictionary containing information about the thread, such as its title, author, and creation date) properties or None if there are no articles.

Since PDF 2.0 it can also contain an indirect reference to a metadata stream containing information about the thread, such as its title, author, and creation date.

**property user\_access\_permissions:** [\*UserAccessPermissions\*](#) | **None**

Get the user access permissions for encrypted documents. Returns None if not encrypted.

 **Warning**

For AES-256 encrypted documents (R=5/R=6), the returned permissions are derived from the /P field, which is only trustworthy if the /Perms integrity check passed. Check [\*are\\_permissions\\_valid\*](#) to verify.

**property viewer\_preferences:** [\*ViewerPreferences\*](#) | **None**

Returns the existing ViewerPreferences as an overloaded dictionary.

**property xfa:** [\*dict\[str, Any\]\*](#) | **None**

**class** pypdf.PasswordType(\*values)

Bases: [\*IntEnum\*](#)

NOT\_DECRYPTED = 0

USER\_PASSWORD = 1

OWNER\_PASSWORD = 2

## THE PDFWRITER CLASS

```
class pypdf.PdfWriter(fileobj: None | PdfReader | str | IO[Any] | Path = "", clone_from: None | PdfReader | str |
                      IO[Any] | Path = None, incremental: bool = False, full: bool = False, strict: bool =
                      False)
```

Bases: *PdfDocCommon*

Write a PDF file out, given pages produced by another class or through cloning a PDF file during initialization.

Typically data is added from a *PdfReader*.

### Parameters

- **clone\_from** – identical to fileobj (for compatibility)
- **incremental** – If true, loads the document and set the PdfWriter in incremental mode.  
When writing incrementally, the original document is written first and new/modified content is appended. To be used for signed document/forms to keep signature valid.
- **full** – If true, loads all the objects (always full if incremental = True). This parameter may allow loading large PDFs.
- **strict** – If true, pypdf will raise an exception if a PDF does not follow the specification. If false, pypdf will try to be forgiving and do something reasonable, but it will log a warning message. It is a best-effort approach.

**strict:** `bool = False`

If true, pypdf will raise an exception if a PDF does not follow the specification. If false, pypdf will try to be forgiving and do something reasonable, but it will log a warning message. It is a best-effort approach.

**flattened\_pages:** `list[PageObject] | None = None`

**incremental**

Returns if the PdfWriter object has been started in incremental mode.

**property is\_encrypted:** `bool`

Read-only boolean property showing whether this PDF file is encrypted.

Note that this property, if true, will remain true even after the *decrypt()* method is called.

**property root\_object:** *DictionaryObject*

Provide direct access to PDF Structure.

### Note

Recommended only for read access.

**property** `xmp_metadata`: [XmpInformation](#) | `None`

XMP (Extensible Metadata Platform) data.

**property** `with_as_usage`: `bool`

**property** `pdf_header`: `str`

Read/Write property of the PDF header that is written.

This should be something like '%PDF-1.5'. It is recommended to set the lowest version that supports all features which are used within the PDF file.

Note: `pdf_header` returns a string but accepts bytes or str for writing

**get\_object**(*indirect\_reference*: `int` | [IndirectObject](#)) → [PdfObject](#)

**set\_need\_appearances\_writer**(*state*: `bool` = `True`) → `None`

Sets the “NeedAppearances” flag in the PDF writer.

The “NeedAppearances” flag indicates whether the appearance dictionary for form fields should be automatically generated by the PDF viewer or if the embedded appearance should be used.

**Parameters**

**state** – The actual value of the NeedAppearances flag.

**Returns**

`None`

**create\_viewer\_preferences**() → [ViewerPreferences](#)

**add\_page**(*page*: [PageObject](#), *excluded\_keys*: [Iterable](#)[`str`] = ()) → [PageObject](#)

Add a page to this PDF file.

Recommended for advanced usage including the adequate `excluded_keys`.

The page is usually acquired from a [PdfReader](#) instance.

**Parameters**

- **page** – The page to add to the document. Should be an instance of [PageObject](#)
- **excluded\_keys**

**Returns**

The added [PageObject](#).

**insert\_page**(*page*: [PageObject](#), *index*: `int` = 0, *excluded\_keys*: [Iterable](#)[`str`] = ()) → [PageObject](#)

Insert a page in this PDF file. The page is usually acquired from a [PdfReader](#) instance.

**Parameters**

- **page** – The page to add to the document.
- **index** – Position at which the page will be inserted.
- **excluded\_keys**

**Returns**

The added [PageObject](#).

**add\_blank\_page**(*width*: `float` | `None` = `None`, *height*: `float` | `None` = `None`) → [PageObject](#)

Append a blank page to this PDF file and return it.

If no page size is specified, use the size of the last page.

**Parameters**



- **width** – The width of the new page expressed in default user space units.
- **height** – The height of the new page expressed in default user space units.

**Returns**

The newly appended page.

**Raises**

**PageSizeNotDefinedError** – if width and height are not defined and previous page does not exist.

**insert\_blank\_page**(width: *float* | *Decimal* | *None* = *None*, height: *float* | *Decimal* | *None* = *None*, index: *int* = 0) → *PageObject*

Insert a blank page to this PDF file and return it.

If no page size is specified for a dimension, use the size of the last page.

**Parameters**

- **width** – The width of the new page in default user space units.
- **height** – The height of the new page in default user space units.
- **index** – Position to add the page.

**Returns**

The newly inserted page.

**Raises**

- **PageSizeNotDefinedError** – if width and height are not defined and previous page does not exist.
- **IndexError** – Index is outside of [-self.get\_num\_pages(), self.get\_num\_pages()]

**property open\_destination:** *None* | *Destination* | *TextStringObject* | *ByteStringObject*

**add\_js**(javascript: *str*) → *None*

Add JavaScript which will launch upon opening this PDF.

**Parameters**

**javascript** – Your JavaScript.

**Example**

This will launch the print window when the PDF is opened.

```
>>> from pypdf import PdfWriter
>>> output = PdfWriter()
>>> output.add_js("this.print({bUI:true,bSilent:false,bShrinkToFit:true});")
```

**add\_attachment**(filename: *str*, data: *str* | *bytes*) → *EmbeddedFile*

Embed a file inside the PDF.

Reference: [https://opensource.adobe.com/dc-acrobat-sdk-docs/pdfstandards/PDF32000\\_2008.pdf](https://opensource.adobe.com/dc-acrobat-sdk-docs/pdfstandards/PDF32000_2008.pdf) Section 7.11.3

**Parameters**

- **filename** – The filename to display.
- **data** – The data in the file.

**Returns**

EmbeddedFile instance for the newly created embedded file.

**append\_pages\_from\_reader**(*reader*: PdfReader, *after\_page\_append*: Callable[[PageObject], None] | None = None) → None

Copy pages from reader to writer. Includes an optional callback parameter which is invoked after pages are appended to the writer.

append should be preferred.

**Parameters**

- **reader** – a PdfReader object from which to copy page annotations to this writer object. The writer's annots will then be updated.
- **after\_page\_append** – Callback function that is invoked after each page is appended to the writer. Signature includes a reference to the appended page (delegates to `append_pages_from_reader`). The single parameter of the callback is a reference to the page just appended to the document.

**FFBITS\_NUL = 0**

**update\_page\_form\_field\_values**(*page*: ~pypdf.\_page.PageObject | list[~pypdf.\_page.PageObject] | None, *fields*: ~collections.abc.Mapping[str, str | list[str] | tuple[str, str, float]], *flags*: ~pypdf.constants.FieldDictionaryAttributes.FfBits = <FfBits: 0>, *auto\_regenerate*: bool | None = True, *flatten*: bool = False) → None

Update the form field values for a given page from a fields dictionary.

Copy field texts and values from fields to page. If the field links to a parent object, add the information to the parent.

**Parameters**

- **page** – *PageObject* - references **PDF writer's page** where the annotations and field data will be updated. *List[Pageobject]* - provides list of pages to be processed. *None* - all pages.
- **fields** – a Python dictionary of:
  - field names (/T) as keys and text values (/V) as value
  - field names (/T) as keys and list of text values (/V) for multiple choice list
  - **field names (/T) as keys and tuple of:**
    - \* text values (/V)
    - \* font id (e.g. /F1, the font id must exist)
    - \* font size (0 for autosize)
- **flags** – A set of flags from *FfBits*.
- **auto\_regenerate** – Set/unset the need\_appearances flag; the flag is unchanged if auto\_regenerate is None.
- **flatten** – Whether or not to flatten the annotation. If True, this adds the annotation's appearance stream to the page contents. Note that this option does not remove the annotation itself.

**reattach\_fields**(*page*: PageObject | None = None) → list[DictionaryObject]

Parse annotations within the page looking for orphan fields and reattach then into the Fields Structure.

**Parameters**

**page** – page to analyze. If none is provided, all pages will be analyzed.

**Returns**

list of reattached fields.

**clone\_reader\_document\_root**(*reader*: PdfReader) → None

Copy the reader document root to the writer and all sub-elements, including pages, threads, outlines, ... For partial insertion, append should be considered.

**Parameters**

**reader** – PdfReader from which the document root should be copied.

**clone\_document\_from\_reader**(*reader*: PdfReader, *after\_page\_append*: Callable[[PageObject], None] | None = None) → None

Create a copy (clone) of a document from a PDF file reader cloning section ‘/Root’ and ‘/Info’ and ‘/ID’ of the pdf.

**Parameters**

- **reader** – PDF file reader instance from which the clone should be created.
- **after\_page\_append** – Callback function that is invoked after each page is appended to the writer. Signature includes a reference to the appended page (delegates to `append_pages_from_reader`). The single parameter of the callback is a reference to the page just appended to the document.

**generate\_file\_identifiers**() → None

Generate an identifier for the PDF that will be written.

The only point of this is ensuring uniqueness. Reproducibility is not required. When a file is first written, both identifiers shall be set to the same value. If both identifiers match when a file reference is resolved, it is very likely that the correct and unchanged file has been found. If only the first identifier matches, a different version of the correct file has been found. see §14.4 “File Identifiers”.

**encrypt**(*user\_password*: str, *owner\_password*: str | None = None, *use\_128bit*: bool = True, *permissions\_flag*: ~pypdf.constants.UserAccessPermissions = <UserAccessPermissions.PRINT|MODIFY|EXTRACT|ADD\_OR\_MODIFY|R7|R8|FILL\_FORM\_FIELDS|EXTRACT\_TEXT\_AND\_GRAPHICS>, \*, *algorithm*: str | None = None) → None

Encrypt this PDF file with the PDF Standard encryption handler.

**Parameters**

- **user\_password** – The password which allows for opening and reading the PDF file with the restrictions provided.
- **owner\_password** – The password which allows for opening the PDF files without any restrictions. By default, the owner password is the same as the user password.
- **use\_128bit** – flag as to whether to use 128bit encryption. When false, 40bit encryption will be used. By default, this flag is on.
- **permissions\_flag** – permissions as described in Table 3.20 of the PDF 1.7 specification. A bit value of 1 means the permission is granted. Hence an integer value of -1 will set all flags. Bit position 3 is for printing, 4 is for modifying content, 5 and 6 control annotations, 9 for form fields, 10 for extraction of text and graphics.
- **algorithm** – encrypt algorithm. Values may be one of “RC4-40”, “RC4-128”, “AES-128”, “AES-256-R5”, “AES-256”. If it is valid, *use\_128bit* will be ignored.

**write\_stream**(*stream*: *IO[Any]*) → *None*

**write**(*stream*: *Path* | *str* | *IO[Any]*) → *tuple*[*bool*, *IO[Any]*]

Write the collection of pages added to this object out as a PDF file.

**Parameters**

**stream** – An object to write the file to. The object can support the write method and the tell method, similar to a file object, or be a file path, just like the fileobj, just named it stream to keep existing workflow.

**Returns**

A tuple (bool, IO).

**list\_objects\_in\_increment**() → *list*[*IndirectObject*]

For analysis or debugging. Provides the list of new or modified objects that will be written in the increment. Deleted objects will not be freed but will become orphans.

**Returns**

List of new or modified IndirectObjects

**property metadata**: *DocumentInformation* | *None*

Retrieve/set the PDF file's document information dictionary, if it exists.

**Parameters**

**value** – dict with the entries to be set. if None : remove the /Info entry from the pdf.

Note that some PDF files use (XMP) metadata streams instead of document information dictionaries, and these metadata streams will not be accessed by this function, but by *xmp\_metadata()*.

**add\_metadata**(*infos*: *dict*[*str*, *Any*]) → *None*

Add custom metadata to the output.

**Parameters**

**infos** – a Python dictionary where each key is a field and each value is your new meta-data.

**compress\_identical\_objects**(*remove\_identicals*: *~typing.Any* = <object object>, *remove\_orphans*: *~typing.Any* = <object object>, \*, *remove\_duplicates*: *bool* = *True*, *remove\_unreferenced*: *bool* = *True*) → *None*

Parse the PDF file and merge objects that have the same hash. This will make objects common to multiple pages. Recommended to be used just before writing output.

**Parameters**

- **remove\_identicals** – Deprecated.
- **remove\_orphans** – Deprecated.
- **remove\_duplicates** – Remove duplicate objects.
- **remove\_unreferenced** – Remove unreferenced objects.

**get\_reference**(*obj*: *PdfObject*) → *IndirectObject*

**get\_outline\_root**() → *TreeObject*

**get\_threads\_root**() → *ArrayObject*

The list of threads.

See §12.4.3 of the PDF 1.7 or PDF 2.0 specification.

**Returns**

An array (possibly empty) of Dictionaries with an `/F` key, and optionally information about the thread in `/I` or `/Metadata` keys.

**property threads:** *ArrayObject*

Read-only property for the list of threads.

See §12.4.3 of the PDF 1.7 or PDF 2.0 specification.

Each element is a dictionary with an `/F` key, and optionally information about the thread in `/I` or `/Metadata` keys.

**add\_outline\_item\_destination**(*page\_destination: IndirectObject | PageObject | TreeObject, parent: None | TreeObject | IndirectObject = None, before: None | TreeObject | IndirectObject = None, is\_open: bool = True*) → *IndirectObject*

**add\_outline\_item\_dict**(*outline\_item: OutlineItem | Destination, parent: None | TreeObject | IndirectObject = None, before: None | TreeObject | IndirectObject = None, is\_open: bool = True*) → *IndirectObject*

**add\_outline\_item**(*title: str, page\_number: None | ~pypdf.\_page.PageObject | ~pypdf.generic.\_base.IndirectObject | int, parent: None | ~pypdf.generic.\_data\_structures.TreeObject | ~pypdf.generic.\_base.IndirectObject = None, before: None | ~pypdf.generic.\_data\_structures.TreeObject | ~pypdf.generic.\_base.IndirectObject = None, color: tuple[float, float, float] | str | None = None, bold: bool = False, italic: bool = False, fit: ~pypdf.generic.\_fit.Fit = <pypdf.generic.\_fit.Fit object>, is\_open: bool = True*) → *IndirectObject*

Add an outline item (commonly referred to as a “Bookmark”) to the PDF file.

**Parameters**

- **title** – Title to use for this outline item.
- **page\_number** – Page number this outline item will point to.
- **parent** – A reference to a parent outline item to create nested outline items.
- **before**
- **color** – Color of the outline item’s font as a red, green, blue tuple from 0.0 to 1.0 or as a Hex String (#RRGGBB)
- **bold** – Outline item font is bold
- **italic** – Outline item font is italic
- **fit** – The fit of the destination page.

**Returns**

The added outline item as an indirect object.

**add\_outline()** → *None*

**add\_named\_destination\_array**(*title: TextStringObject, destination: IndirectObject | ArrayObject*) → *None*

**add\_named\_destination\_object**(*page\_destination: PdfObject*) → *IndirectObject*

**add\_named\_destination**(*title: str, page\_number: int*) → *IndirectObject*

**remove\_links()** → *None*

Remove links and annotations from this output.

```
remove_annotations(subtypes: Literal['Text', '/Link', '/FreeText', '/Line', '/Square', '/Circle', '/Polygon',
'/PolyLine', '/Highlight', '/Underline', '/Squiggly', '/StrikeOut', '/Caret', '/Stamp', '/Ink',
'/Popup', '/FileAttachment', '/Sound', '/Movie', '/Screen', '/Widget', '/PrinterMark',
'/TrapNet', '/Watermark', '/3D', '/Redact', '/Projection', '/RichMedia'] |
Iterable[Literal['Text', '/Link', '/FreeText', '/Line', '/Square', '/Circle', '/Polygon',
'/PolyLine', '/Highlight', '/Underline', '/Squiggly', '/StrikeOut', '/Caret', '/Stamp', '/Ink',
'/Popup', '/FileAttachment', '/Sound', '/Movie', '/Screen', '/Widget', '/PrinterMark',
'/TrapNet', '/Watermark', '/3D', '/Redact', '/Projection', '/RichMedia']] | None) → None
```

Remove annotations by annotation subtype.

#### Parameters

**subtypes** – subtype or list of subtypes to be removed. Examples are: “/Link”, “/FileAttachment”, “/Sound”, “/Movie”, “/Screen”, ... If you want to remove all annotations, use subtypes=None.

```
remove_objects_from_page(page: PageObject | DictionaryObject, to_delete: ObjectDeletionFlag |
Iterable[ObjectDeletionFlag], text_filters: dict[str, Any] | None = None) →
None
```

Remove objects specified by to\_delete from the given page.

#### Parameters

- **page** – Page object to clean up.
- **to\_delete** – Objects to be deleted; can be a *ObjectDeletionFlag* or a list of *ObjectDeletionFlag*
- **text\_filters** – Properties of text to be deleted, if applicable. Optional. This is a Python dictionary with the following properties:
  - **font\_ids**: List of font resource IDs (such as /F1 or /T1\_0) to be deleted.

```
remove_images(to_delete: ~pypdf.constants.ImageType = <ImageType.ALL: 7>) → None
```

Remove images from this output.

#### Parameters

**to\_delete** – The type of images to be deleted (default = all images types)

```
remove_text(font_names: list[str] | None = None) → None
```

Remove text from the PDF.

#### Parameters

**font\_names** – List of font names to remove, such as “Helvetica-Bold”. Optional. If not specified, all text will be removed.

```
add_uri(page_number: int, uri: str, rect: RectangleObject, border: ArrayObject | None = None) → None
```

Add an URI from a rectangular area to the specified page.

#### Parameters

- **page\_number** – index of the page on which to place the URI action.
- **uri** – URI of resource to link to.
- **rect** – *RectangleObject* or array of four integers specifying the clickable rectangular area [xLL, yLL, xUR, yUR], or string in the form "[ xLL yLL xUR yUR ]".
- **border** – if provided, an array describing border-drawing properties. See the PDF spec for details. No border will be drawn if this argument is omitted.

**set\_page\_layout**(*layout*: *Literal*['/NoLayout', '/SinglePage', '/OneColumn', '/TwoColumnLeft', '/TwoColumnRight', '/TwoPageLeft', '/TwoPageRight']) → *None*

Set the page layout.

#### Parameters

**layout** – The page layout to be used

Table 1: Valid layout arguments

|                  |                                                            |
|------------------|------------------------------------------------------------|
| /NoLayout        | Layout explicitly not specified                            |
| /SinglePage      | Show one page at a time                                    |
| /OneColumn       | Show one column at a time                                  |
| /TwoColumnLeft   | Show pages in two columns, odd-numbered pages on the left  |
| /TwoColumn-Right | Show pages in two columns, odd-numbered pages on the right |
| /TwoPageLeft     | Show two pages at a time, odd-numbered pages on the left   |
| /TwoPageRight    | Show two pages at a time, odd-numbered pages on the right  |

**property page\_layout**: *Literal*['/NoLayout', '/SinglePage', '/OneColumn', '/TwoColumnLeft', '/TwoColumnRight', '/TwoPageLeft', '/TwoPageRight'] | *None*

Page layout property.

Table 2: Valid layout values

|                  |                                                            |
|------------------|------------------------------------------------------------|
| /NoLayout        | Layout explicitly not specified                            |
| /SinglePage      | Show one page at a time                                    |
| /OneColumn       | Show one column at a time                                  |
| /TwoColumnLeft   | Show pages in two columns, odd-numbered pages on the left  |
| /TwoColumn-Right | Show pages in two columns, odd-numbered pages on the right |
| /TwoPageLeft     | Show two pages at a time, odd-numbered pages on the left   |
| /TwoPageRight    | Show two pages at a time, odd-numbered pages on the right  |

**property are\_permissions\_valid**: *bool* | *None*

Whether the /Perms integrity check passed for this document.

For AES-256 encrypted documents (R=5/R=6), the /Perms field is an encrypted copy of the permissions that can be verified independently. Returns *False* if this check fails (the /P permissions may have been tampered with).

Returns *None* if the document is not encrypted or has not yet been decrypted via *decrypt()*. Returns *True* for non-AES-256 encryption (no /Perms to check).

**property attachment\_list**: *Generator*[*EmbeddedFile*, *None*, *None*]

Iterable of attachment objects.

**property attachments**: *Mapping*[*str*, *list*[*bytes*]]

Mapping of attachment filenames to their content.

**decode\_permissions**(*permissions\_code*: *int*) → *dict*[*str*, *bool*]

Take the permissions as an integer, return the allowed access.

**get\_destination\_page\_number**(*destination*: *Destination*) → *int* | *None*

Retrieve page number of a given *Destination* object.

**Parameters**

**destination** – The destination to get page number.

**Returns**

The page number or None if page is not found

**get\_fields**(*tree*: *TreeObject* | *None* = *None*, *retval*: *dict*[*Any*, *Any*] | *None* = *None*, *fileobj*: *Any* | *None* = *None*, *stack*: *list*[*PdfObject*] | *None* = *None*) → *dict*[*str*, *Any*] | *None*

Extract field data if this PDF contains interactive form fields.

The *tree*, *retval*, *stack* parameters are for recursive use.

**Parameters**

- **tree** – Current object to parse.
- **retval** – In-progress list of fields.
- **fileobj** – A file object (usually a text file) to write a report to on all interactive form fields found.
- **stack** – List of already parsed objects.

**Returns**

A dictionary where each key is a field name, and each value is a *Field* object. By default, the mapping name is used for keys. None if form data could not be located.

**get\_form\_text\_fields**(*full\_qualified\_name*: *bool* = *False*) → *dict*[*str*, *Any*]

Retrieve form fields from the document with textual data.

**Parameters**

**full\_qualified\_name** – to get full name

**Returns**

A dictionary. The key is the name of the form field, the value is the content of the field.

If the document contains multiple form fields with the same name, the second and following will get the suffix .2, .3, ...

**get\_named\_dest\_root**() → *ArrayObject*

**get\_num\_pages**() → *int*

Calculate the number of pages in this PDF file.

**Returns**

The number of pages of the parsed PDF file.

**Raises**

*PdfReadError* – If restrictions prevent this action.

**get\_page**(*page\_number*: *int*) → *PageObject*

Retrieve a page by number from this PDF file. Most of the time *.pages*[*page\_number*] is preferred.

**Parameters**

**page\_number** – The page number to retrieve (pages begin at zero)

**Returns**

A *PageObject* instance.

**get\_page\_number**(*page*: *PageObject*) → *int* | *None*

Retrieve page number of a given *PageObject*.



**Parameters**

**page** – The page to get page number. Should be an instance of *PageObject*

**Returns**

The page number or None if page is not found

**get\_pages\_showing\_field**(*field*: *Field* | *PdfObject* | *IndirectObject*) → *list*[*PageObject*]

Provides list of pages where the field is called.

**Parameters**

**field** – Field Object, PdfObject or IndirectObject referencing a Field

**Returns**

*List of pages* –

- **Empty list:**

The field has no widgets attached (either hidden field or ancestor field).

- **Single page list:**

Page where the widget is present (most common).

- **Multi-page list:**

Field with multiple kids widgets (example: radio buttons, field repeated on multiple pages).

**property named\_destinations:** *dict*[*str*, *Destination*]

A read-only dictionary which maps names to destinations.

**property outline:** *list*[*Destination* | *list*[*Destination* | *list*[*Destination*]]]

Read-only property for the outline present in the document (i.e., a collection of ‘outline items’ which are also known as ‘bookmarks’).

**property page\_labels:** *list*[*str*]

A list of labels for the pages in this document.

This property is read-only. The labels are in the order that the pages appear in the document.

**property pages:** *list*[*PageObject*]

Property that emulates a list of *PageObject*. This property allows to get a page or a range of pages.

**Note**

For PdfWriter only: Provides the capability to remove a page/range of page from the list (using the del operator). Remember: Only the page entry is removed, as the objects beneath can be used elsewhere. A solution to completely remove them - if they are not used anywhere - is to write to a buffer/temporary file and then load it into a new PdfWriter.

**remove\_page**(*page*: *int* | *PageObject* | *IndirectObject*, *clean*: *bool* = *False*) → *None*

Remove page from pages list.

**Parameters**

- **page** –

- *int*: Page number to be removed.

- *PageObject*: page to be removed. If the page appears many times only the first one will be removed.

- *IndirectObject*: Reference to page to be removed.

- **clean** – replace PageObject with NullObject to prevent annotations or destinations to reference a detached page.

**property user\_access\_permissions:** *UserAccessPermissions* | *None*

Get the user access permissions for encrypted documents. Returns None if not encrypted.

#### **Warning**

For AES-256 encrypted documents (R=5/R=6), the returned permissions are derived from the /P field, which is only trustworthy if the /Perms integrity check passed. Check *are\_permissions\_valid* to verify.

**property viewer\_preferences:** *ViewerPreferences* | *None*

Returns the existing ViewerPreferences as an overloaded dictionary.

**property xfa:** *dict[str, Any]* | *None*

**property page\_mode:** *Literal['/UseNone', '/UseOutlines', '/UseThumbs', '/FullScreen', '/UseOC', '/UseAttachments']* | *None*

Page mode property.

Table 3: Valid mode values

|                 |                                          |
|-----------------|------------------------------------------|
| /UseNone        | Do not show outline or thumbnails panels |
| /UseOutlines    | Show outline (aka bookmarks) panel       |
| /UseThumbs      | Show page thumbnails panel               |
| /FullScreen     | Fullscreen view                          |
| /UseOC          | Show Optional Content Group (OCG) panel  |
| /UseAttachments | Show attachments panel                   |

**add\_annotation**(*page\_number: int* | *PageObject*, *annotation: dict[str, Any]*) → *DictionaryObject*

Add a single annotation to the page. The added annotation must be a new annotation. It cannot be recycled.

#### **Parameters**

- **page\_number** – PageObject or page index.
- **annotation** – Annotation to be added (created with annotation).

#### **Returns**

The inserted object. This can be used for popup creation, for example.

**clean\_page**(*page: PageObject* | *IndirectObject*) → *PageObject*

Perform some clean up in the page. Currently: convert NameObject named destination to TextStringObject (required for names/destds list)

#### **Parameters**

**page**

#### **Returns**

The cleaned PageObject

**append**(*fileobj: str* | *IO[Any]* | *PdfReader* | *Path*, *outline\_item: str* | *None* | *PageRange* | *tuple[int, int]* | *tuple[int, int, int]* | *list[int] = None*, *pages: None* | *PageRange* | *tuple[int, int]* | *tuple[int, int, int]* | *list[int]* | *list[PageObject] = None*, *import\_outline: bool = True*, *excluded\_fields: list[str]* | *tuple[str, ...]* | *None = None*) → *None*

Identical to the `merge()` method, but assumes you want to concatenate all pages onto the end of the file instead of specifying a position.

#### Parameters

- **fileobj** – A File Object or an object that supports the standard read and seek methods similar to a File Object. Could also be a string representing a path to a PDF file.
- **outline\_item** – Optionally, you may specify a string to build an outline (aka ‘bookmark’) to identify the beginning of the included file.
- **pages** – Can be a `PageRange` or a `(start, stop[, step])` tuple or a list of pages to be processed to merge only the specified range of pages from the source document into the output document.
- **import\_outline** – You may prevent the source document’s outline (collection of outline items, previously referred to as ‘bookmarks’) from being imported by specifying this as `False`.
- **excluded\_fields** – Provide the list of fields/keys to be ignored if `/Annots` is part of the list, the annotation will be ignored if `/B` is part of the list, the articles will be ignored

`merge(position: int | None, fileobj: Path | str | IO[Any] | PdfReader, outline_item: str | None = None, pages: str | PageRange | tuple[int, int] | tuple[int, int, int] | list[int] | list[PageObject] | None = None, import_outline: bool = True, excluded_fields: list[str] | tuple[str, ...] | None = ()) → None`

Merge the pages from the given file into the output file at the specified page number.

#### Parameters

- **position** – The *page number* to insert this file. File will be inserted after the given number.
- **fileobj** – A File Object or an object that supports the standard read and seek methods similar to a File Object. Could also be a string representing a path to a PDF file.
- **outline\_item** – Optionally, you may specify a string to build an outline (aka ‘bookmark’) to identify the beginning of the included file.
- **pages** – can be a `PageRange` or a `(start, stop[, step])` tuple or a list of pages to be processed to merge only the specified range of pages from the source document into the output document.
- **import\_outline** – You may prevent the source document’s outline (collection of outline items, previously referred to as ‘bookmarks’) from being imported by specifying this as `False`.
- **excluded\_fields** – provide the list of fields/keys to be ignored if `/Annots` is part of the list, the annotation will be ignored if `/B` is part of the list, the articles will be ignored

#### Raises

**TypeError** – The pages attribute is not configured properly

`add_filtered_articles(ftr: Pattern[Any] | str, pages: dict[int, PageObject], reader: PdfReader) → None`

Add articles matching the defined criteria.

#### Parameters

- **fltr**
- **pages**
- **reader**

**close()** → *None*

Implemented for API harmonization.

**find\_outline\_item**(*outline\_item: dict[str, Any]*, *root: list[Destination | list[Destination | list[Destination]]] | None = None*) → *list[int] | None*

**reset\_translation**(*reader: None | PdfReader | IndirectObject = None*) → *None*

Reset the translation table between reader and the writer object.

Late cloning will create new independent objects.

#### Parameters

**reader** – PdfReader or IndirectObject referencing a PdfReader object. if set to None or omitted, all tables will be reset.

**set\_page\_label**(*page\_index\_from: int*, *page\_index\_to: int*, *style: PageLabelStyle | None = None*, *prefix: str | None = None*, *start: int | None = 0*) → *None*

Set a page label to a range of pages.

Page indexes must be given starting from 0. Labels must have a style, a prefix or both. If a range is not assigned any page label, a decimal label starting from 1 is applied.

#### Parameters

- **page\_index\_from** – page index of the beginning of the range starting from 0
- **page\_index\_to** – page index of the beginning of the range starting from 0
- **style** – The numbering style to be used for the numeric portion of each page label:
  - /D Decimal Arabic numerals
  - /R Uppercase Roman numerals
  - /r Lowercase Roman numerals
  - /A Uppercase letters (A to Z for the first 26 pages, AA to ZZ for the next 26, and so on)
  - /a Lowercase letters (a to z for the first 26 pages, aa to zz for the next 26, and so on)
- **prefix** – The label prefix for page labels in this range.
- **start** – The value of the numeric portion for the first page label in the range. Subsequent pages are numbered sequentially from this value, which must be greater than or equal to 1. Default value: 1.

**class** pypdf.ObjectDeletionFlag(\**values*)

Bases: *IntFlag*

**NONE** = 0

**TEXT** = 1

**LINKS** = 2

ATTACHMENTS = 4  
OBJECTS\_3D = 8  
ALL\_ANNOTATIONS = 16  
XOBJECT\_IMAGES = 32  
INLINE\_IMAGES = 64  
DRAWING\_IMAGES = 128  
IMAGES = 224



## THE DESTINATION CLASS

```
class pypdf.generic.Destination(title: str | bytes, page: NumberObject | IndirectObject | NullObject |  
                                DictionaryObject, fit: Fit)
```

Bases: *TreeObject*

A class representing a destination within a PDF file.

See section 12.3.2 of the PDF 2.0 reference.

### Parameters

- **title** – Title of this destination.
- **page** – Reference to the page of this destination. Should be an instance of *IndirectObject*.
- **fit** – How the destination is displayed.

### Raises

*PdfReadError* – If destination type is invalid.

**node:** *DictionaryObject* | **None** = **None**

**property dest\_array:** *ArrayObject*

**write\_to\_stream**(stream: *IO[Any]*, encryption\_key: *None* | *str* | *bytes* = *None*) → *None*

**property title:** *str* | **None**

Read-only property accessing the destination title.

**property page:** *IndirectObject* | **None**

Read-only property accessing the IndirectObject of the destination page.

**property typ:** *str* | **None**

Read-only property accessing the destination type.

**property zoom:** *int* | **None**

Read-only property accessing the zoom factor.

**property left:** *FloatObject* | **None**

Read-only property accessing the left horizontal coordinate.

**property right:** *FloatObject* | **None**

Read-only property accessing the right horizontal coordinate.

**property top:** *FloatObject* | **None**

Read-only property accessing the top vertical coordinate.

**property bottom:** *FloatObject* | **None**

Read-only property accessing the bottom vertical coordinate.

**property color:** *ArrayObject* | **None**

Read-only property accessing the color in (R, G, B) with values 0.0-1.0.

**property font\_format:** *OutlineFontFlag* | **None**

Read-only property accessing the font type.

1=italic, 2=bold, 3=both

**property outline\_count:** *int* | **None**

Read-only property accessing the outline count.

positive = expanded negative = collapsed absolute value = number of visible descendants at all levels



## THE DOCUMENTINFORMATION CLASS

**class** pypdf.DocumentInformation

Bases: *DictionaryObject*

A class representing the basic document metadata provided in a PDF File. This class is accessible through *PdfReader.metadata*.

All text properties of the document metadata have *two* properties, e.g. `author` and `author_raw`. The non-raw property will always return a `TextStringObject`, making it ideal for a case where the metadata is being displayed. The raw property can sometimes return a `ByteStringObject`, if pypdf was unable to decode the string's text encoding; this requires additional safety in the caller and therefore is not as commonly accessed.

**property** `title`: `str` | `None`

Read-only property accessing the document's title.

Returns a `TextStringObject` or `None` if the title is not specified.

**property** `title_raw`: `str` | `None`

The "raw" version of title; can return a `ByteStringObject`.

**property** `author`: `str` | `None`

Read-only property accessing the document's author.

Returns a `TextStringObject` or `None` if the author is not specified.

**property** `author_raw`: `str` | `None`

The "raw" version of author; can return a `ByteStringObject`.

**property** `subject`: `str` | `None`

Read-only property accessing the document's subject.

Returns a `TextStringObject` or `None` if the subject is not specified.

**property** `subject_raw`: `str` | `None`

The "raw" version of subject; can return a `ByteStringObject`.

**property** `creator`: `str` | `None`

Read-only property accessing the document's creator.

If the document was converted to PDF from another format, this is the name of the application (e.g. OpenOffice) that created the original document from which it was converted. Returns a `TextStringObject` or `None` if the creator is not specified.

**property** `creator_raw`: `str` | `None`

The "raw" version of creator; can return a `ByteStringObject`.

**property producer: str | None**

Read-only property accessing the document's producer.

If the document was converted to PDF from another format, this is the name of the application (for example, macOS Quartz) that converted it to PDF. Returns a `TextStringObject` or `None` if the producer is not specified.

**property producer\_raw: str | None**

The “raw” version of producer; can return a `ByteStringObject`.

**property creation\_date: datetime | None**

Read-only property accessing the document's creation date.

**property creation\_date\_raw: str | None**

The “raw” version of creation date; can return a `ByteStringObject`.

Typically in the format `D:YYYYMMDDhhmmss[+Z-]hh'mm` where the suffix is the offset from UTC.

**property modification\_date: datetime | None**

Read-only property accessing the document's modification date.

The date and time the document was most recently modified.

**property modification\_date\_raw: str | None**

The “raw” version of modification date; can return a `ByteStringObject`.

Typically in the format `D:YYYYMMDDhhmmss[+Z-]hh'mm` where the suffix is the offset from UTC.

**property keywords: str | None**

Read-only property accessing the document's keywords.

Returns a `TextStringObject` or `None` if keywords are not specified.

**property keywords\_raw: str | None**

The “raw” version of keywords; can return a `ByteStringObject`.

## THE FIELD CLASS

**class** pypdf.generic.Field(*data*: DictionaryObject)

Bases: *TreeObject*

A class representing a field dictionary.

This class is accessed through *get\_fields()*

**property** field\_type: *NameObject* | *None*

Read-only property accessing the type of this field.

**property** parent: *DictionaryObject* | *None*

Read-only property accessing the parent of this field.

**property** kids: *ArrayObject* | *None*

Read-only property accessing the kids of this field.

**property** name: *str* | *None*

Read-only property accessing the name of this field.

**property** alternate\_name: *str* | *None*

Read-only property accessing the alternate name of this field.

**property** mapping\_name: *str* | *None*

Read-only property accessing the mapping name of this field.

This name is used by pypdf as a key in the dictionary returned by *get\_fields()*

**property** flags: *int* | *None*

Read-only property accessing the field flags, specifying various characteristics of the field (see Table 8.70 of the PDF 1.7 reference).

**property** value: *Any* | *None*

Read-only property accessing the value of this field.

Format varies based on field type.

**property** default\_value: *Any* | *None*

Read-only property accessing the default value of this field.

**property** additional\_actions: *DictionaryObject* | *None*

Read-only property accessing the additional actions dictionary.

This dictionary defines the field's behavior in response to trigger events. See Section 8.5.2 of the PDF 1.7 reference.



## THE FIT CLASS

```
class pypdf.generic.Fit(fit_type: str, fit_args: tuple[None | float | Any, ...] = ())
```

Bases: `object`

```
classmethod xyz(left: float | None = None, top: float | None = None, zoom: float | None = None) → Fit
```

Display the page designated by page, with the coordinates (left, top) positioned at the upper-left corner of the window and the contents of the page magnified by the factor zoom.

A null value for any of the parameters left, top, or zoom specifies that the current value of that parameter is to be retained unchanged.

A zoom value of 0 has the same meaning as a null value.

### Parameters

- **left**
- **top**
- **zoom**

### Returns

The created fit object.

```
classmethod fit() → Fit
```

Display the page designated by page, with its contents magnified just enough to fit the entire page within the window both horizontally and vertically.

If the required horizontal and vertical magnification factors are different, use the smaller of the two, centering the page within the window in the other dimension.

```
classmethod fit_horizontally(top: float | None = None) → Fit
```

Display the page designated by page, with the vertical coordinate top positioned at the top edge of the window and the contents of the page magnified just enough to fit the entire width of the page within the window.

A null value for top specifies that the current value of that parameter is to be retained unchanged.

### Parameters

**top**

### Returns

The created fit object.

```
classmethod fit_vertically(left: float | None = None) → Fit
```

**classmethod** `fit_rectangle(left: float | None = None, bottom: float | None = None, right: float | None = None, top: float | None = None) → Fit`

Display the page designated by page, with its contents magnified just enough to fit the rectangle specified by the coordinates left, bottom, right, and top entirely within the window both horizontally and vertically.

If the required horizontal and vertical magnification factors are different, use the smaller of the two, centering the rectangle within the window in the other dimension.

A null value for any of the parameters may result in unpredictable behavior.

**Parameters**

- **left**
- **bottom**
- **right**
- **top**

**Returns**

The created fit object.

**classmethod** `fit_box() → Fit`

Display the page designated by page, with its contents magnified just enough to fit its bounding box entirely within the window both horizontally and vertically.

If the required horizontal and vertical magnification factors are different, use the smaller of the two, centering the bounding box within the window in the other dimension.

**classmethod** `fit_box_horizontally(top: float | None = None) → Fit`

Display the page designated by page, with the vertical coordinate top positioned at the top edge of the window and the contents of the page magnified just enough to fit the entire width of its bounding box within the window.

A null value for top specifies that the current value of that parameter is to be retained unchanged.

**Parameters**

**top**

**Returns**

The created fit object.

**classmethod** `fit_box_vertically(left: float | None = None) → Fit`

Display the page designated by page, with the horizontal coordinate left positioned at the left edge of the window and the contents of the page magnified just enough to fit the entire height of its bounding box within the window.

A null value for left specifies that the current value of that parameter is to be retained unchanged.

**Parameters**

**left**

**Returns**

The created fit object.

## THE PAGEOBJECT CLASS

```
class pypdf._page.PageObject(pdf: PdfCommonDocProtocol | None = None, indirect_reference:
                               IndirectObject | None = None)
```

Bases: *DictionaryObject*

PageObject represents a single page within a PDF file.

Typically these objects will be created by accessing the *pages* property of the *PdfReader* class, but it is also possible to create an empty page with the *create\_blank\_page()* static method.

### Parameters

- **pdf** – PDF file the page belongs to.
- **indirect\_reference** – Stores the original indirect reference to this object in its source PDF

**original\_page:** *PageObject*

**hash\_bin()** → *int*

Used to detect modified object.

Note: this function is overloaded to return the same results as a DictionaryObject.

### Returns

Hash considering type and value.

**hash\_value\_data()** → *bytes*

**property user\_unit:** *float*

A read-only positive number giving the size of user space units.

It is in multiples of 1/72 inch. Hence a value of 1 means a user space unit is 1/72 inch, and a value of 3 means that a user space unit is 3/72 inch.

```
static create_blank_page(pdf: PdfCommonDocProtocol | None = None, width: float | Decimal | None =
                          None, height: float | Decimal | None = None) → PageObject
```

Return a new blank page.

If width or height is None, try to get the page size from the last page of *pdf*.

### Parameters

- **pdf** – PDF file the page is within.
- **width** – The width of the new page expressed in default user space units.
- **height** – The height of the new page expressed in default user space units.

### Returns

The new blank page

**Raises**

*PageSizeNotDefinedError* – if pdf is None or contains no page

**property images:** *VirtualListImages*

Read-only property emulating a list of images on a page.

Get a list of all images on the page. The key can be: - A string (for the top object) - A tuple (for images within XObject forms) - An integer

**Examples**

- *reader.pages[0].images[0]* # return first image
- *reader.pages[0].images['/I0']* # return image '/I0'
- *reader.pages[0].images['/TP1', '/Image1']* # return image '/Image1' within '/TP1' XObject form
- *for img in reader.pages[0].images:* # loops through all objects

*images.keys()* and *images.items()* can be used.

The ImageFile has the following properties:

- *.name* : name of the object
- *.data* : bytes of the object
- *.image* : PIL Image Object
- *.indirect\_reference* : object reference

**and the following methods:**

*.replace(new\_image: PIL.Image.Image, \*\*kwargs) :*

replace the image in the pdf with the new image applying the saving parameters indicated (such as quality)

Example usage:

```
reader.pages[0].images[0].replace(Image.open("new_image.jpg"), quality=20)
```

Inline images are extracted and named ~0~, ~1~, ..., with the *indirect\_reference* set to None.

**property rotation:** *int*

The visual rotation of the page.

This number has to be a multiple of 90 degrees: 0, 90, 180, or 270 are valid values. This property does not affect /Contents.

**transfer\_rotation\_to\_content()** → *None*

Apply the rotation of the page to the content and the media/crop/... boxes.

It is recommended to apply this function before page merging.

**rotate(*angle: int*)** → *PageObject*

Rotate a page clockwise by increments of 90 degrees.

**Parameters**

**angle** – Angle to rotate the page. Must be an increment of 90 deg.

**Returns**

The rotated PageObject



**get\_contents()** → *ContentStream* | *None*

Access the page contents.

#### Returns

The `/Contents` object, or `None` if it does not exist. `/Contents` is optional, as described in §7.7.3.3 of the PDF Reference.

**replace\_contents**(*content*: *None* | *ContentStream* | *EncodedStreamObject* | *ArrayObject*) → *None*

Replace the page contents with the new content and nullify old objects :param *content*: new content; if `None` delete the content field.

**merge\_page**(*page2*: *PageObject*, *expand*: *bool* = *False*, *over*: *bool* = *True*) → *None*

Merge the content streams of two pages into one.

Resource references (e.g. fonts) are maintained from both pages. The mediabox, cropbox, etc of this page are not altered. The parameter *page*'s content stream will be added to the end of this page's content stream, meaning that it will be drawn after, or "on top" of this page.

#### Parameters

- **page2** – The page to be merged into this one. Should be an instance of *PageObject*.
- **over** – set the *page2* content over *page1* if *True* (default) else under
- **expand** – If *True*, the current page dimensions will be expanded to accommodate the dimensions of the page to be merged.

**merge\_transformed\_page**(*page2*: *PageObject*, *ctm*: *tuple*[*float*, *float*, *float*, *float*, *float*, *float*] | *Transformation*, *over*: *bool* = *True*, *expand*: *bool* = *False*) → *None*

Similar to *merge\_page()*, but a transformation matrix is applied to the merged stream.

#### Parameters

- **page2** – The page to be merged into this one.
- **ctm** – a 6-element tuple containing the operands of the transformation matrix
- **over** – set the *page2* content over *page1* if *True* (default) else under
- **expand** – Whether the page should be expanded to fit the dimensions of the page to be merged.

**merge\_scaled\_page**(*page2*: *PageObject*, *scale*: *float*, *over*: *bool* = *True*, *expand*: *bool* = *False*) → *None*

Similar to *merge\_page()*, but the stream to be merged is scaled by applying a transformation matrix.

#### Parameters

- **page2** – The page to be merged into this one.
- **scale** – The scaling factor
- **over** – set the *page2* content over *page1* if *True* (default) else under
- **expand** – Whether the page should be expanded to fit the dimensions of the page to be merged.

**merge\_rotated\_page**(*page2*: *PageObject*, *rotation*: *float*, *over*: *bool* = *True*, *expand*: *bool* = *False*) → *None*

Similar to *merge\_page()*, but the stream to be merged is rotated by applying a transformation matrix.

#### Parameters

- **page2** – The page to be merged into this one.

- **rotation** – The angle of the rotation, in degrees
- **over** – set the page2 content over page1 if True (default) else under
- **expand** – Whether the page should be expanded to fit the dimensions of the page to be merged.

**merge\_translated\_page**(page2: [PageObject](#), tx: *float*, ty: *float*, over: *bool* = *True*, expand: *bool* = *False*)  
→ *None*

Similar to [merge\\_page\(\)](#), but the stream to be merged is translated by applying a transformation matrix.

#### Parameters

- **page2** – the page to be merged into this one.
- **tx** – The translation on X axis
- **ty** – The translation on Y axis
- **over** – set the page2 content over page1 if True (default) else under
- **expand** – Whether the page should be expanded to fit the dimensions of the page to be merged.

**add\_transformation**(ctm: [Transformation](#) | *tuple*[*float*, *float*, *float*, *float*, *float*, *float*], expand: *bool* = *False*) → *None*

Apply a transformation matrix to the page.

#### Parameters

**ctm** – A 6-element tuple containing the operands of the transformation matrix. Alternatively, a [Transformation](#) object can be passed.

See [Cropping and Transforming PDFs](#).

**scale**(sx: *float*, sy: *float*) → *None*

Scale a page by the given factors by applying a transformation matrix to its content and updating the page size.

This updates the various page boundaries (bleedbox, trimbox, etc.) and the contents of the page.

#### Parameters

- **sx** – The scaling factor on horizontal axis.
- **sy** – The scaling factor on vertical axis.

**scale\_by**(factor: *float*) → *None*

Scale a page by the given factor by applying a transformation matrix to its content and updating the page size.

#### Parameters

**factor** – The scaling factor (for both X and Y axis).

**scale\_to**(width: *float*, height: *float*) → *None*

Scale a page to the specified dimensions by applying a transformation matrix to its content and updating the page size.

#### Parameters

- **width** – The new width.
- **height** – The new height.

**compress\_content\_streams**(*level: int = -1*) → None

Compress the size of this page by joining all content streams and applying a FlateDecode filter.

However, it is possible that this function will perform no action if content stream compression becomes “automatic”.

**property page\_number: int | None**

Read-only property which returns the page number within the PDF file.

#### Returns

Page number; None if the page is not attached to a PDF.

**extract\_text**(\*args: Any, orientations: int | tuple[int, ...] = (0, 90, 180, 270), space\_width: float = 200.0, visitor\_operand\_before: Callable[[Any, Any, Any, Any], None] | None = None, visitor\_operand\_after: Callable[[Any, Any, Any, Any], None] | None = None, visitor\_text: Callable[[Any, Any, Any, Any, Any], None] | None = None, extraction\_mode: Literal['plain', 'layout'] = 'plain', \*\*kwargs: Any) → str

Locate all text drawing commands, in the order they are provided in the content stream, and extract the text.

This works well for some PDF files, but poorly for others, depending on the generator used. This will be refined in the future.

Do not rely on the order of text coming out of this function, as it will change if this function is made more sophisticated.

Arabic and Hebrew are extracted in the correct order. If required a custom RTL range of characters can be defined; see function `set_custom_rtl`.

Additionally you can provide visitor methods to get informed on all operations and all text objects. For example in some PDF files this can be useful to parse tables.

#### Parameters

- **orientations** – list of orientations `extract_text` will look for default = (0, 90, 180, 270) note: currently only 0 (up), 90 (turned left), 180 (upside down), 270 (turned right) Silently ignored in “layout” mode.
- **space\_width** – force default space width if not extracted from font (default: 200) Silently ignored in “layout” mode.
- **visitor\_operand\_before** – function to be called before processing an operation. It has four arguments: operator, operand-arguments, current transformation matrix and text matrix. Ignored with a warning in “layout” mode.
- **visitor\_operand\_after** – function to be called after processing an operation. It has four arguments: operator, operand-arguments, current transformation matrix and text matrix. Ignored with a warning in “layout” mode.
- **visitor\_text** – function to be called when extracting some text at some position. It has five arguments: text, current transformation matrix, text matrix, font-dictionary and font-size. The font-dictionary may be None in case of unknown fonts. If not None it may e.g. contain key “/BaseFont” with value “/Arial,Bold”. Ignored with a warning in “layout” mode.
- **extraction\_mode** (*Literal["plain", "layout"]*) – “plain” for legacy functionality, “layout” for experimental layout mode functionality. NOTE: orientations, space\_width, and visitor\_\* parameters are NOT respected in “layout” mode.

**kwargs:**

**layout\_mode\_space\_vertically (bool): include blank lines inferred from**  
y distance + font height. Defaults to True.

**layout\_mode\_scale\_weight (float): multiplier for string length when calculating**  
weighted average character width. Defaults to 1.25.

**layout\_mode\_strip\_rotated (bool): layout mode does not support rotated text.**  
Set to False to include rotated text anyway. If rotated text is discovered, layout will be degraded  
and a warning will result. Defaults to True.

**layout\_mode\_debug\_path (Path | None): if supplied, must target a directory.**  
creates the following files with debug information for layout mode functions if supplied:

- fonts.json: output of self.\_layout\_mode\_fonts
- tjs.json: individual text render ops with corresponding transform matrices
- bts.json: text render ops left justified and grouped by BT/ET operators
- bt\_groups.json: BT/ET operations grouped by rendered y-coord (aka lines)

**layout\_mode\_font\_height\_weight (float): multiplier for font height when calculating**  
blank lines. Defaults to 1.

#### Returns

The extracted text

**extract\_xform\_text**(xform: [EncodedStreamObject](#), orientations: *tuple[int, ...] = (0, 90, 270, 360)*,  
space\_width: *float = 200.0*, visitor\_operand\_before: *Callable[[Any, Any, Any, Any],  
None] | None = None*, visitor\_operand\_after: *Callable[[Any, Any, Any, Any], None] |  
None = None*, visitor\_text: *Callable[[Any, Any, Any, Any], None] | None =  
None*) → str

Extract text from an XObject.

#### Parameters

- **xform**
- **orientations**
- **space\_width** – force default space width (if not extracted from font (default 200))
- **visitor\_operand\_before**
- **visitor\_operand\_after**
- **visitor\_text**

#### Returns

The extracted text

#### property mediabox

A [RectangleObject](#), expressed in default user space units, defining the boundaries of the physical  
medium on which the page is intended to be displayed or printed.

#### property cropbox

A [RectangleObject](#), expressed in default user space units, defining the visible region of default user  
space.

When the page is displayed or printed, its contents are to be clipped (cropped) to this rectangle and then im-  
posed on the output medium in some implementation-defined manner. Default value: same as [mediabox](#).

**property bleedbox**

A [RectangleObject](#), expressed in default user space units, defining the region to which the contents of the page should be clipped when output in a production environment.

**property trimbox**

A [RectangleObject](#), expressed in default user space units, defining the intended dimensions of the finished page after trimming.

**property artbox**

A [RectangleObject](#), expressed in default user space units, defining the extent of the page's meaningful content as intended by the page's creator.

**property annotations:** [ArrayObject](#) | **None**

```
class pypdf._page.VirtualListImages(ids_function: Callable[[], list[str | list[str]]], get_function:
    Callable[[str | list[str] | tuple[str]], ImageFile])
```

Bases: [Sequence](#)[[ImageFile](#)]

Provides access to images referenced within a page. Only one copy will be returned if the usage is used on the same page multiple times. See [PageObject.images\(\)](#) for more details.

**keys()** → list[str | list[str]]

**items()** → list[tuple[str | list[str], [ImageFile](#)]]

```
class pypdf._page.ImageFile(name: str = "", data: bytes = b'', image: Image | None = None,
    indirect_reference: IndirectObject | None = None)
```

Bases: [object](#)

Image within the PDF file. *This object is not designed to be built.*

This object should not be modified except using [ImageFile.replace\(\)](#) to replace the image with a new one.

**name:** [str](#) = ''

Filename as identified within the PDF file.

**data:** [bytes](#) = b''

Data as bytes.

**image:** [Image](#) | **None** = **None**

Data as PIL image.

**indirect\_reference:** [IndirectObject](#) | **None** = **None**

Reference to the object storing the stream.

**replace**(new\_image: [Image](#), \*\*kwargs: [Any](#)) → **None**

Replace the image with a new PIL image.

**Parameters**

- **new\_image** ([Image](#)) – The new PIL image to replace the existing image.
- **\*\*kwargs** – Additional keyword arguments to pass to [Image.save\(\)](#).

**Raises**

- **TypeError** – If the image is inline or in a PdfReader.
- **TypeError** – If the image does not belong to a PdfWriter.
- **TypeError** – If *new\_image* is not a PIL Image.

**Note**

This method replaces the existing image with a new image. It is not allowed for inline images or images within a PdfReader. The *kwargs* parameter allows passing additional parameters to *Image.save()*, such as quality.

`pypdf.mult(m: list[float], n: list[float]) → list[float]`

## THE PAGERANGE CLASS

**class** pypdf.**PageRange**(arg: *slice* | *PageRange* | *str*)

Bases: *object*

A slice-like representation of a range of page indices.

For example, page numbers, only starting at zero.

The syntax is like what you would put between brackets [ ]. The slice is one of the few Python types that can't be subclassed, but this class converts to and from slices, and allows similar use.

- *PageRange*(str) parses a string representing a page range.
- *PageRange*(slice) directly “imports” a slice.
- *to\_slice()* gives the equivalent slice.
- *str()* and *repr()* allow printing.
- *indices*(n) is like *slice.indices*(n).

**static valid**(input: *Any*) → *bool*

True if input is a valid initializer for a *PageRange*.

### Parameters

**input** – A possible *PageRange* string or a *PageRange* object.

### Returns

True, if the input is a valid *PageRange*.

**to\_slice**() → *slice*

Return the slice equivalent of this page range.

**indices**(n: *int*) → *tuple*[*int*, *int*, *int*]

Assuming a sequence of length n, calculate the start and stop indices, and the stride length of the *PageRange*.

See *help*(*slice.indices*).

### Parameters

**n** – the length of the list of pages to choose from.

### Returns

Arguments for *range*().





## THE PAPERSIZE CLASS

```
class pypdf.PaperSize
    Bases: object
    (width, height) of the paper in portrait mode in pixels at 72 ppi.
    A0 = (2384, 3370)
    A1 = (1684, 2384)
    A2 = (1191, 1684)
    A3 = (842, 1191)
    A4 = (595, 842)
    A5 = (420, 595)
    A6 = (298, 420)
    A7 = (210, 298)
    A8 = (147, 210)
    C4 = (649, 918)
```

### 32.1 Add blank page with PaperSize

```
from pypdf import PaperSize, PdfWriter

writer = PdfWriter(clone_from="example.pdf")
writer.add_blank_page(PaperSize.A8.width, PaperSize.A8.height)
writer.write("out-add-page.pdf")
```

### 32.2 Insert blank page with PaperSize

```
from pypdf import PaperSize, PdfWriter

writer = PdfWriter(clone_from="example.pdf")
writer.insert_blank_page(PaperSize.A8.width, PaperSize.A8.height, 1)
writer.write("out-insert-page.pdf")
```



## THE RECTANGLEOBJECT CLASS

```
class pypdf.generic.RectangleObject(arr: RectangleObject | tuple[float, float, float, float])
```

Bases: *ArrayObject*

This class is used to represent *page boxes* in pypdf.

These boxes include:

- *artbox*
- *bleedbox*
- *cropbox*
- *mediabox*
- *trimbox*

```
scale(sx: float, sy: float) → RectangleObject
```

```
property left: FloatObject
```

```
property bottom: FloatObject
```

```
property right: FloatObject
```

```
property top: FloatObject
```

```
property lower_left: tuple[float, float]
```

Property to read and modify the lower left coordinate of this box in (x,y) form.

```
property lower_right: tuple[float, float]
```

Property to read and modify the lower right coordinate of this box in (x,y) form.

```
property upper_left: tuple[float, float]
```

Property to read and modify the upper left coordinate of this box in (x,y) form.

```
property upper_right: tuple[float, float]
```

Property to read and modify the upper right coordinate of this box in (x,y) form.

```
property width: float
```

```
property height: float
```



## THE TRANSFORMATION CLASS

```
class pypdf.Transformation(ctm: tuple[float, float, float, float, float, float] = (1, 0, 0, 1, 0, 0))
```

Bases: `object`

Represent a 2D transformation.

The transformation between two coordinate systems is represented by a 3-by-3 transformation matrix with the following form:

```
a b 0
c d 0
e f 1
```

Because a transformation matrix has only six elements that can be changed, it is usually specified in PDF as the six-element array [ a b c d e f ].

Coordinate transformations are expressed as matrix multiplications:

```
[ x y 1 ] = [ x y 1 ] ×   a b 0
                        c d 0
                        e f 1
```

### Example

```
>>> from pypdf import PdfWriter, Transformation
>>> page = PdfWriter().add_blank_page(800, 600)
>>> op = Transformation().scale(sx=2, sy=3).translate(tx=10, ty=20)
>>> page.add_transformation(op)
```

**property matrix:** `tuple[tuple[float, float, float], tuple[float, float, float], tuple[float, float, float]]`

Return the transformation matrix as a tuple of tuples in the form:

`((a, b, 0), (c, d, 0), (e, f, 1))`

**static compress**(*matrix*: `tuple[tuple[float, float, float], tuple[float, float, float], tuple[float, float, float]]`) → `tuple[float, float, float, float, float, float]`

Compresses the transformation matrix into a tuple of (a, b, c, d, e, f).

### Parameters

**matrix** – The transformation matrix as a tuple of tuples.

### Returns

A tuple representing the transformation matrix as (a, b, c, d, e, f)

**transform**(*m*: *Transformation*) → *Transformation*

Apply one transformation to another.

**Parameters**

**m** – a Transformation to apply.

**Returns**

A new Transformation instance

**Example**

```
>>> from pypdf import PdfWriter, Transformation
>>> height, width = 40, 50
>>> page = PdfWriter().add_blank_page(800, 600)
>>> op = Transformation((1, 0, 0, -1, 0, height)) # vertical mirror
>>> op = Transformation().transform(Transformation((-1, 0, 0, 1, width, 0)))
→ # horizontal mirror
>>> page.add_transformation(op)
```

**translate**(*tx*: *float* = 0, *ty*: *float* = 0) → *Transformation*

Translate the contents of a page.

**Parameters**

- **tx** – The translation along the x-axis.
- **ty** – The translation along the y-axis.

**Returns**

A new Transformation instance

**scale**(*sx*: *float* | *None* = *None*, *sy*: *float* | *None* = *None*) → *Transformation*

Scale the contents of a page towards the origin of the coordinate system.

Typically, that is the lower-left corner of the page. That can be changed by translating the contents / the page boxes.

**Parameters**

- **sx** – The scale factor along the x-axis.
- **sy** – The scale factor along the y-axis.

**Returns**

A new Transformation instance with the scaled matrix.

**rotate**(*rotation*: *float*) → *Transformation*

Rotate the contents of a page.

**Parameters**

**rotation** – The angle of rotation in degrees.

**Returns**

A new Transformation instance with the rotated matrix.

**apply\_on**(*pt*: *list*[*float*], *as\_object*: *bool* = *False*) → *list*[*float*]

**apply\_on**(*pt*: *tuple*[*float*, *float*], *as\_object*: *bool* = *False*) → *tuple*[*float*, *float*]

Apply the transformation matrix on the given point.

**Parameters**

- **pt** – A tuple or list representing the point in the form (x, y).

- **as\_object** – If True, return items as FloatObject, otherwise as plain floats.

**Returns**

A tuple or list representing the transformed point in the form (x', y')





## THE XMPINFORMATION CLASS

**class** pypdf.xmp.XmpInformation(*stream: ContentStream*)

Bases: *XmpInformationProtocol*, *PdfObject*

An object that represents Extensible Metadata Platform (XMP) metadata. Usually accessed by *xmp\_metadata()*.

**Raises**

*PdfReadError* – if XML is invalid

**classmethod** create() → *XmpInformation*

Create a new XmpInformation object with minimal structure.

**Returns**

A new XmpInformation instance with empty metadata fields.

**write\_to\_stream**(*stream: IO[Any]*, *encryption\_key: None | str | bytes = None*) → *None*

**get\_element**(*about\_uri: str*, *namespace: str*, *name: str*) → *Iterator[Any]*

**get\_nodes\_in\_namespace**(*about\_uri: str*, *namespace: str*) → *Iterator[Any]*

**property** dc\_contributor: *list[str] | None*

Contributors to the resource (other than the authors).

**property** dc\_coverage: *str | None*

Text describing the extent or scope of the resource.

**property** dc\_creator: *list[str] | None*

A sorted array of names of the authors of the resource, listed in order of precedence.

**property** dc\_date: *list[datetime] | None*

A sorted array of dates of significance to the resource. The dates and times are in UTC.

**property** dc\_description: *dict[str, str] | None*

A language-keyed dictionary of textual descriptions of the content of the resource.

**property** dc\_format: *str | None*

The mime-type of the resource.

**property** dc\_identifier: *str | None*

Unique identifier of the resource.

**property** dc\_language: *list[str] | None*

An unordered array specifying the languages used in the resource.

**property dc\_publisher:** `list[str] | None`

An unordered array of publisher names.

**property dc\_relation:** `list[str] | None`

An unordered array of text descriptions of relationships to other documents.

**property dc\_rights:** `dict[str, str] | None`

A language-keyed dictionary of textual descriptions of the rights the user has to this resource.

**property dc\_source:** `str | None`

Unique identifier of the work from which this resource was derived.

**property dc\_subject:** `list[str] | None`

An unordered array of descriptive phrases or keywords that specify the topic of the content.

**property dc\_title:** `dict[str, str] | None`

A language-keyed dictionary of the title of the resource.

**property dc\_type:** `list[str] | None`

An unordered array of textual descriptions of the document type.

**property pdf\_keywords:** `str | None`

An unformatted text string representing document keywords.

**property pdf\_pdfversion:** `str | None`

The PDF file version, for example 1.0 or 1.3.

**property pdf\_producer:** `str | None`

The name of the tool that saved the document as a PDF.

**property xmp\_create\_date:** `datetime | None`

The date and time the resource was originally created. Returned as a UTC datetime object.

**property xmp\_modify\_date:** `datetime | None`

The date and time the resource was last modified. Returned as a UTC datetime object.

**property xmp\_metadata\_date:** `datetime | None`

The date and time that any metadata for this resource was last changed. Returned as a UTC datetime object.

**property xmp\_creator\_tool:** `str | None`

The name of the first known tool used to create the resource.

**property xmpmm\_document\_id:** `str | None`

The common identifier for all versions and renditions of this resource.

**property xmpmm\_instance\_id:** `str | None`

An identifier for a specific incarnation of a document, updated each time a file is saved.

**property pdfaid\_part:** `str | None`

The part of the PDF/A standard that the document conforms to (e.g., 1, 2, 3).

**property pdfaid\_conformance:** `str | None`

The conformance level within the PDF/A standard (e.g., 'A', 'B', 'U').

**property custom\_properties:** `dict[Any, Any]`

Retrieve custom metadata properties defined in the undocumented pdfx metadata schema.

#### Returns

A dictionary of key/value items for custom metadata properties.

## THE ANNOTATIONS MODULE

PDF specifies several annotation types which pypdf makes available here.

The names of the annotations and their attributes do not reflect the names in the specification in all cases. For example, the PDF standard defines a ‘Square’ annotation that does not actually need to be square. For this reason, pypdf calls it ‘Rectangle’.

At their core, all annotation types are DictionaryObjects. That means if pypdf does not implement a feature, users can easily extend the given functionality.

**class** pypdf.annotations.**AnnotationDictionary**

Bases: *DictionaryObject*, *ABC*

**property flags:** *AnnotationFlag*

**class** pypdf.annotations.**Ellipse**(*rect: RectangleObject | tuple[float, float, float, float], \*, interior\_color: str | None = None, \*\*kwargs: Any*)

Bases: *MarkupAnnotation*

**class** pypdf.annotations.**FreeText**(*\*, text: str, rect: RectangleObject | tuple[float, float, float, float], font: str = 'Helvetica', bold: bool = False, italic: bool = False, font\_size: str = '14pt', font\_color: str = '000000', border\_color: str | None = '000000', background\_color: str | None = 'ffff', \*\*kwargs: Any*)

Bases: *MarkupAnnotation*

A FreeText annotation

**class** pypdf.annotations.**Highlight**(*\*, rect: RectangleObject | tuple[float, float, float, float], quad\_points: ArrayObject, highlight\_color: str = 'ff0000', printing: bool = False, \*\*kwargs: Any*)

Bases: *MarkupAnnotation*

**class** pypdf.annotations.**Line**(*p1: tuple[float, float], p2: tuple[float, float], rect: RectangleObject | tuple[float, float, float, float], text: str = "", \*\*kwargs: Any*)

Bases: *MarkupAnnotation*

**class** pypdf.annotations.**Link**(*\*, rect: ~pypdf.generic.\_rectangle.RectangleObject | tuple[float, float, float, float], border: ~pypdf.generic.\_data\_structures.ArrayObject | None = None, url: str | None = None, target\_page\_index: int | None = None, fit: ~pypdf.generic.\_fit.Fit = <pypdf.generic.\_fit.Fit object>, \*\*kwargs: ~typing.Any*)

Bases: *AnnotationDictionary*

**class** pypdf.annotations.**MarkupAnnotation**(*\*, title\_bar: str | None = None, in\_reply\_to: DictionaryObject | IndirectObject | None = None, reply\_type: Literal['R', 'Group'] = 'R', annotation\_name: str | None = None*)

Bases: [AnnotationDictionary](#), [ABC](#)

Base class for all markup annotations.

#### Parameters

- **title\_bar** – Text to be displayed in the title bar of the annotation; by convention this is the name of the author
- **in\_reply\_to** – The annotation that this annotation is “in reply to” (PDF 1.5). Can be either an annotation (previously added using [add\\_annotation\(\)](#)) or a reference to the target annotation.
- **reply\_type** – The relationship between this annotation and the one specified by `in_reply_to`. Either "R" (a reply, default) or "Group" (grouped with the parent annotation). Raises `ValueError` if a non-default value is provided without `in_reply_to`.
- **annotation\_name** – A text string uniquely identifying this annotation among all annotations on its page. Automatically generated when `in_reply_to` is set and no name is provided. Raises `ValueError` if provided without `in_reply_to`.

**class** pypdf.annotations.**PolyLine**(vertices: *list[tuple[float, float]]*, \*\*kwargs: *Any*)

Bases: [MarkupAnnotation](#)

**class** pypdf.annotations.**Polygon**(vertices: *list[tuple[float, float]]*, \*\*kwargs: *Any*)

Bases: [MarkupAnnotation](#)

**class** pypdf.annotations.**Popup**(\**rect*: [RectangleObject](#) | *tuple[float, float, float, float]*, parent: [DictionaryObject](#) | *None* = *None*, open: *bool* = *False*, \*\*kwargs: *Any*)

Bases: [AnnotationDictionary](#)

**class** pypdf.annotations.**Rectangle**(rect: [RectangleObject](#) | *tuple[float, float, float, float]*, \*, interior\_color: *str* | *None* = *None*, \*\*kwargs: *Any*)

Bases: [MarkupAnnotation](#)

**class** pypdf.annotations.**Text**(\**rect*: *~pypdf.generic.\_rectangle.RectangleObject* | *tuple[float, float, float, float]*, text: *str*, open: *bool* = *False*, flags: *int* = *<AnnotationFlag: 0>*, \*\*kwargs: *~typing.Any*)

Bases: [MarkupAnnotation](#)

A text annotation.

#### Parameters

- **rect** – array of four integers [xLL, yLL, xUR, yUR] specifying the clickable rectangular area
- **text** – The text that is added to the document
- **open**
- **flags**

## CONSTANTS

```
class pypdf.constants.AnnotationFlag(*values)
```

```
    Bases: IntFlag
```

```
    See §12.5.3 “Annotation Flags”.
```

```
    INVISIBLE = 1
```

```
    HIDDEN = 2
```

```
    PRINT = 4
```

```
    NO_ZOOM = 8
```

```
    NO_ROTATE = 16
```

```
    NO_VIEW = 32
```

```
    READ_ONLY = 64
```

```
    LOCKED = 128
```

```
    TOGGLE_NO_VIEW = 256
```

```
    LOCKED_CONTENTS = 512
```

```
class pypdf.constants.ImageType(*values)
```

```
    Bases: IntFlag
```

```
    NONE = 0
```

```
    XOBJECT_IMAGES = 1
```

```
    INLINE_IMAGES = 2
```

```
    DRAWING_IMAGES = 4
```

```
    ALL = 7
```

```
    IMAGES = 7
```

```
class pypdf.constants.PageLabelStyle
```

```
    Bases: object
```

```
    Table 8.10 in the 1.7 reference. Table 161 in the 2.0 reference.
```

```
    DECIMAL = '/D'
```

```
UPPERCASE_ROMAN = '/R'
LOWERCASE_ROMAN = '/r'
UPPERCASE_LETTER = '/A'
LOWERCASE_LETTER = '/a'
```

```
class pypdf.constants.UserAccessPermissions(*values)
```

```
    Bases: IntFlag
```

```
    Table 3.20 User access permissions. Table 22 of the 2.0 manual.
```

```
    R1 = 1
    R2 = 2

    PRINT = 4
    MODIFY = 8
    EXTRACT = 16
    ADD_OR_MODIFY = 32

    R7 = 64
    R8 = 128

    FILL_FORM_FIELDS = 256
    EXTRACT_TEXT_AND_GRAPHICS = 512
    ASSEMBLE_DOC = 1024
    PRINT_TO_REPRESENTATION = 2048

    R13 = 4096
    R14 = 8192
    R15 = 16384
    R16 = 32768
    R17 = 65536
    R18 = 131072
    R19 = 262144
    R20 = 524288
    R21 = 1048576
    R22 = 2097152
    R23 = 4194304
    R24 = 8388608
```

**R25 = 16777216**

**R26 = 33554432**

**R27 = 67108864**

**R28 = 134217728**

**R29 = 268435456**

**R30 = 536870912**

**R31 = 1073741824**

**R32 = 2147483648**

**to\_dict()** → dict[str, bool]

Convert the given flag value to a corresponding verbose name mapping.

**classmethod from\_dict(value: dict[str, bool])** → UserAccessPermissions

Convert the verbose name mapping to the corresponding flag value.

**classmethod all()** → UserAccessPermissions

**class pypdf.constants.FieldDictionaryAttributes**

Bases: object

Entries common to all field dictionaries (Table 8.69 PDF 1.7 reference) (*very partially documented here*).

FFBits provides the constants used for /Ff from Table 8.70/8.75/8.77/8.79

**Ff = '/Ff'**

**class FfBits(\*values)**

Bases: IntFlag

Ease building /Ff flags Some entries may be specific to:

- Text (Tx) (Table 8.75 PDF 1.7 reference)
- Buttons (Btn) (Table 8.77 PDF 1.7 reference)
- Choice (Ch) (Table 8.79 PDF 1.7 reference)

**ReadOnly = 1**

common to Tx/Btn/Ch in Table 8.70

**Required = 2**

common to Tx/Btn/Ch in Table 8.70

**NoExport = 4**

common to Tx/Btn/Ch in Table 8.70

**Multiline = 4096**

Tx

**Password = 8192**

Tx

**NoToggleToOff = 16384**

Btn

**Radio = 32768**  
    Btn

**Pushbutton = 65536**  
    Btn

**Combo = 131072**  
    Ch

**Edit = 262144**  
    Ch

**Sort = 524288**  
    Ch

**FileSelect = 1048576**  
    Tx

**MultiSelect = 2097152**  
    Tx

**DoNotSpellCheck = 4194304**  
    Tx/Ch

**DoNotScroll = 8388608**  
    Tx

**Comb = 16777216**  
    Tx

**RadiosInUnison = 33554432**  
    Btn

**RichText = 33554432**  
    Tx

**CommitOnSelChange = 67108864**  
    Ch



## ERRORS

All errors/exceptions pypdf raises and all of the warnings it uses.

Please note that broken PDF files might cause other Exceptions.

**exception** `pypdf.errors.DeprecationError`

Bases: `Exception`

Raised when a deprecated feature is used.

**exception** `pypdf.errors.DependencyError`

Bases: `Exception`

Raised when a required dependency (a library or module that pypdf depends on) is not available or cannot be imported.

**exception** `pypdf.errors.PyPdfError`

Bases: `Exception`

Base class for all exceptions raised by pypdf.

**exception** `pypdf.errors.PdfReadError`

Bases: `PyPdfError`

Raised when there is an issue reading a PDF file.

**exception** `pypdf.errors.PageSizeNotDefinedError`

Bases: `PyPdfError`

Raised when the page size of a PDF document is not defined.

**exception** `pypdf.errors.PdfReadWarning`

Bases: `UserWarning`

Issued when there is a potential issue reading a PDF file, but it can still be read.

**exception** `pypdf.errors.PdfStreamError`

Bases: `PdfReadError`

Raised when there is an issue reading the stream of data in a PDF file.

**exception** `pypdf.errors.ParseError`

Bases: `PyPdfError`

Raised when there is an issue parsing (analyzing and understanding the structure and meaning of) a PDF file.

**exception** `pypdf.errors.FileNotDecryptedError`

Bases: *PdfReadError*

Raised when a PDF file that has been encrypted (meaning it requires a password to be accessed) has not been successfully decrypted.

**exception** `pypdf.errors.WrongPasswordError`

Bases: *FileNotDecryptedError*

Raised when the wrong password is used to try to decrypt an encrypted PDF file.

**exception** `pypdf.errors.EmptyFileError`

Bases: *PdfReadError*

Raised when a PDF file is empty or has no content.

**exception** `pypdf.errors.EmptyImageDataError`

Bases: *PyPdfError*

Raised when trying to process an image that has no data.

**exception** `pypdf.errors.LimitReachedError`

Bases: *PyPdfError*

Raised when a limit is reached.

**exception** `pypdf.errors.XmpDocumentError`

Bases: *PyPdfError*, *RuntimeError*

Raised when the XMP XML document context is invalid or missing.

## GENERIC PDF OBJECTS

Implementation of generic PDF objects (dictionary, number, string, ...).

```
class pypdf.generic.ArrayObject(iterable=(), /)
    Bases: list[Any], PdfObject
    replicate(pdf_dest: PdfWriterProtocol) → ArrayObject
    clone(pdf_dest: PdfWriterProtocol, force_duplicate: bool = False, ignore_fields: Sequence[str | int] | None =
        ()) → ArrayObject
        Clone object into pdf_dest.
    hash_bin() → int
        Used to detect modified object.

        Returns
            Hash considering type and value.
    items() → Iterable[Any]
        Emulate DictionaryObject.items for a list (index, object).
    write_to_stream(stream: IO[Any], encryption_key: None | str | bytes = None) → None
    static read_from_stream(stream: IO[Any], pdf: PdfReaderProtocol | None, forced_encoding: None | str |
        list[str] | dict[int, str] = None) → ArrayObject

class pypdf.generic.BooleanObject(value: Any)
    Bases: PdfObject
    clone(pdf_dest: PdfWriterProtocol, force_duplicate: bool = False, ignore_fields: Sequence[str | int] | None =
        ()) → BooleanObject
        Clone object into pdf_dest.
    hash_bin() → int
        Used to detect modified object.

        Returns
            Hash considering type and value.
    write_to_stream(stream: IO[Any], encryption_key: None | str | bytes = None) → None
    static read_from_stream(stream: IO[Any]) → BooleanObject
```

**class** pypdf.generic.ByteStringObject(\*args, \*\*kwargs)

Bases: [bytes](#), [PdfObject](#)

Represents a string object where the text encoding could not be determined.

This occurs quite often, as the PDF spec doesn't provide an alternate way to represent strings – for example, the encryption data stored in files (like /O) is clearly not text, but is still stored in a “String” object.

**clone**(pdf\_dest: *Any*, force\_duplicate: *bool* = *False*, ignore\_fields: *Sequence*[*str* | *int*] | *None* = ()) → [ByteStringObject](#)

Clone object into pdf\_dest.

**hash\_bin**() → *int*

Used to detect modified object.

#### Returns

Hash considering type and value.

**property original\_bytes:** [bytes](#)

For compatibility with TextStringObject.original\_bytes.

**write\_to\_stream**(stream: *IO*[*Any*], encryption\_key: *None* | *str* | *bytes* = *None*) → *None*

**class** pypdf.generic.ContentStream(stream: *Any*, pdf: *Any*, forced\_encoding: *None* | *str* | *list*[*str*] | *dict*[*int*, *str*] = *None*)

Bases: [DecodedStreamObject](#)

In order to be fast, this data structure can contain either:

- raw data in `._data`
- parsed stream operations in `._operations`.

At any time, ContentStream object can either have both of those fields defined, or one field defined and the other set to *None*.

These fields are “rebuilt” lazily, when accessed:

- when `.get_data()` is called, if `._data` is *None*, it is rebuilt from `._operations`.
- when `.operations` is called, if `._operations` is *None*, it is rebuilt from `._data`.

Conversely, these fields can be invalidated:

- when `.set_data()` is called, `._operations` is set to *None*.
- when `.operations` is set, `._data` is set to *None*.

**replicate**(pdf\_dest: [PdfWriterProtocol](#)) → [ContentStream](#)

**clone**(pdf\_dest: *Any*, force\_duplicate: *bool* = *False*, ignore\_fields: *Sequence*[*str* | *int*] | *None* = ()) → [ContentStream](#)

Clone object into pdf\_dest.

#### Parameters

- **pdf\_dest**
- **force\_duplicate**
- **ignore\_fields**

#### Returns

The cloned ContentStream

**get\_data**() → [bytes](#)

**set\_data**(data: *bytes*) → *None*

property operations: `list[tuple[Any, bytes]]`

`isolate_graphics_state()` → `None`

`write_to_stream(stream: IO[Any], encryption_key: None | str | bytes = None)` → `None`

**class** `pypdf.generic.DecodedStreamObject`

Bases: `StreamObject`

**class** `pypdf.generic.DictionaryObject`

Bases: `dict[Any, Any]`, `PdfObject`

**replicate**(`pdf_dest: PdfWriterProtocol`) → `DictionaryObject`

**clone**(`pdf_dest: PdfWriterProtocol`, `force_duplicate: bool = False`, `ignore_fields: Sequence[str | int] | None = ()`) → `DictionaryObject`

Clone object into pdf\_dest.

**hash\_bin()** → `int`

Used to detect modified object.

#### Returns

Hash considering type and value.

**raw\_get**(`key: Any`) → `Any`

**get\_inherited**(`key: str`, `default: Any = None`) → `Any`

Returns the value of a key or from the parent if not found. If not found returns default.

#### Parameters

- **key** – string identifying the field to return
- **default** – default value to return

#### Returns

Current key or inherited one, otherwise default value.

**setdefault**(`key: Any`, `value: Any | None = None`) → `Any`

property **xmp\_metadata**: `XmpInformationProtocol | None`

Retrieve XMP (Extensible Metadata Platform) data relevant to this object, if available.

See Table 347 — Additional entries in a metadata stream dictionary.

#### Returns

Returns a `XmpInformation` instance that can be used to access XMP metadata from the document. Can also return `None` if no metadata was found on the document root.

**write\_to\_stream**(`stream: IO[Any]`, `encryption_key: None | str | bytes = None`) → `None`

**static read\_from\_stream**(`stream: IO[Any]`, `pdf: PdfReaderProtocol | None`, `forced_encoding: None | str | list[str] | dict[int, str] = None`) → `DictionaryObject`

**class** `pypdf.generic.DirectReferenceLink`(`reference: ArrayObject`)

Bases: `object`

Direct reference link being preserved until we can resolve it correctly.

**find\_referenced\_page()** → `IndirectObject`

**patch\_reference**(*target\_pdf*: PdfWriter, *new\_page*: IndirectObject) → None

*target\_pdf*: PdfWriter which the new link went into

**class** pypdf.generic.**EmbeddedFile**(*name*: str, *pdf\_object*: DictionaryObject, *parent*: ArrayObject | None = None)

Bases: *object*

Container holding the information on an embedded file.

Attributes are evaluated lazily if possible.

Further information on embedded files can be found in section 7.11 of the PDF 2.0 specification.

**property name:** str

The (primary) name of the embedded file as provided in the name tree.

**property alternative\_name:** str | None

Retrieve the alternative name (file specification).

**property description:** str | None

Retrieve the description.

**property associated\_file\_relationship:** str

Retrieve the relationship of the referring document to this embedded file.

**property subtype:** str | None

Retrieve the subtype. This is a MIME media type, prefixed by a slash.

**property content:** bytes

Retrieve the actual file content.

**property size:** int | None

Retrieve the size of the uncompressed file in bytes.

**property creation\_date:** datetime | None

Retrieve the file creation datetime.

**property modification\_date:** datetime | None

Retrieve the datetime of the last file modification.

**property checksum:** bytes | None

Retrieve the MD5 checksum of the (uncompressed) file.

**delete()** → None

Delete the file from the document.

**class** pypdf.generic.**EncodedStreamObject**

Bases: *StreamObject*

**get\_data()** → bytes

**set\_data**(*data*: bytes) → None

**class** pypdf.generic.**FloatObject**(*value*: Any = '0.0', *context*: Any | None = None)

Bases: *float*, *PdfObject*

**clone**(*pdf\_dest*: Any, *force\_duplicate*: bool = False, *ignore\_fields*: Sequence[str | int] | None = ()) → *FloatObject*

Clone object into pdf\_dest.

**hash\_bin()** → int

Used to detect modified object.

**Returns**

Hash considering type and value.

**myrepr()** → str

**as\_numeric()** → float

**write\_to\_stream**(stream: IO[Any], encryption\_key: None | str | bytes = None) → None

**class** pypdf.generic.IndirectObject(idnum: int, generation: int, pdf: Any)

Bases: PdfObject

**hash\_bin()** → int

Used to detect modified object.

**Returns**

Hash considering type and value.

**replicate**(pdf\_dest: PdfWriterProtocol) → PdfObject

**clone**(pdf\_dest: PdfWriterProtocol, force\_duplicate: bool = False, ignore\_fields: Sequence[str | int] | None = ()) → IndirectObject

Clone object into pdf\_dest.

**property indirect\_reference:** IndirectObject

**get\_object()** → PdfObject | None

**write\_to\_stream**(stream: IO[Any], encryption\_key: None | str | bytes = None) → None

**static read\_from\_stream**(stream: IO[Any], pdf: Any) → IndirectObject

**class** pypdf.generic.NameObject

Bases: str, PdfObject

**delimiter\_pattern** = re.compile(b'\\s+|[\\(\\)\\<\\>\\[\\]\\{\\}/%]'')

**prefix** = b'/'

**renumber\_table:** ClassVar[dict[str, bytes]] = {'\x00': b'#00', '\x01': b'#01', '\x02': b'#02', '\x03': b'#03', '\x04': b'#04', '\x05': b'#05', '\x06': b'#06', '\x07': b'#07', '\x08': b'#08', '\t': b'#09', '\n': b'#0A', '\x0b': b'#0B', '\x0c': b'#0C', '\r': b'#0D', '\x0e': b'#0E', '\x0f': b'#0F', '\x10': b'#10', '\x11': b'#11', '\x12': b'#12', '\x13': b'#13', '\x14': b'#14', '\x15': b'#15', '\x16': b'#16', '\x17': b'#17', '\x18': b'#18', '\x19': b'#19', '\x1a': b'#1A', '\x1b': b'#1B', '\x1c': b'#1C', '\x1d': b'#1D', '\x1e': b'#1E', '\x1f': b'#1F', ' ': b'#20', '#': b'#23', '%': b'#25', '(': b'#28', ')': b'#29', '/': b'#2F', '<': b'#3C', '>': b'#3E', '[': b'#5B', ']': b'#5D', '{': b'#7B', '}': b'#7D'}

**clone**(pdf\_dest: Any, force\_duplicate: bool = False, ignore\_fields: Sequence[str | int] | None = ()) → NameObject

Clone object into pdf\_dest.

**hash\_bin()** → *int*

Used to detect modified object.

**Returns**

Hash considering type and value.

**write\_to\_stream**(*stream: IO[Any]*, *encryption\_key: None | str | bytes = None*) → *None*

**renumber()** → *bytes*

**surfix**

Decorator that converts a method with a single cls argument into a property that can be accessed directly from the class.

**static unnumber**(*sin: bytes*) → *bytes*

**CHARSETS** = ('utf-8', 'gbk', 'latin1')

**static read\_from\_stream**(*stream: IO[Any]*, *pdf: Any*) → *NameObject*

**class** pypdf.generic.**NamedReferenceLink**(*reference: TextStringObject*, *source\_pdf: PdfReader*)

Bases: *object*

Named reference link being preserved until we can resolve it correctly.

**find\_referenced\_page()** → *IndirectObject | None*

**patch\_reference**(*target\_pdf: PdfWriter*, *new\_page: IndirectObject*) → *None*

target\_pdf: PdfWriter which the new link went into

**class** pypdf.generic.**NullObject**(\*args, \*\*kwargs)

Bases: *PdfObject*

**clone**(*pdf\_dest: PdfWriterProtocol*, *force\_duplicate: bool = False*, *ignore\_fields: Sequence[str | int] | None = ()*) → *NullObject*

Clone object into pdf\_dest.

**hash\_bin()** → *int*

Used to detect modified object.

**Returns**

Hash considering type and value.

**write\_to\_stream**(*stream: IO[Any]*, *encryption\_key: None | str | bytes = None*) → *None*

**static read\_from\_stream**(*stream: IO[Any]*) → *NullObject*

**class** pypdf.generic.**NumberObject**(*value: Any*)

Bases: *int, PdfObject*

**NumberPattern** = *re.compile(b'^+-.0-9')*

**clone**(*pdf\_dest: Any*, *force\_duplicate: bool = False*, *ignore\_fields: Sequence[str | int] | None = ()*) → *NumberObject*

Clone object into pdf\_dest.

**hash\_bin()** → *int*

Used to detect modified object.

**Returns**

Hash considering type and value.



**as\_numeric()** → int

**write\_to\_stream**(stream: *IO[Any]*, encryption\_key: *None* | *str* | *bytes* = *None*) → *None*

**static read\_from\_stream**(stream: *IO[Any]*) → *NumberObject* | *FloatObject*

**class** pypdf.generic.**OutlineFontFlag**(\*values)

Bases: *IntFlag*

A class used as an enumerable flag for formatting an outline font.

**italic** = 1

**bold** = 2

**class** pypdf.generic.**OutlineItem**(title: *str* | *bytes*, page: *NumberObject* | *IndirectObject* | *NullObject* | *DictionaryObject*, fit: *Fit*)

Bases: *Destination*

**write\_to\_stream**(stream: *IO[Any]*, encryption\_key: *None* | *str* | *bytes* = *None*) → *None*

**class** pypdf.generic.**PdfObject**(\*args, \*\*kwargs)

Bases: *PdfObjectProtocol*

**hash\_func**(\*, usedforsecurity=True, string=None)

Returns a sha1 hash object; optionally initialized with a string

**indirect\_reference**: **ForwardRef('IndirectObject')** | **None**

**hash\_bin**() → int

Used to detect modified object.

#### Returns

Hash considering type and value.

**hash\_value\_data**() → *bytes*

**hash\_value**() → *bytes*

**replicate**(pdf\_dest: *PdfWriterProtocol*) → *PdfObject*

Clone object into pdf\_dest (PdfWriterProtocol which is an interface for PdfWriter) without ensuring links. This is used in clone\_document\_from\_root with incremental = True.

#### Parameters

**pdf\_dest** – Target to clone to.

#### Returns

The cloned PdfObject

**clone**(pdf\_dest: *PdfWriterProtocol*, force\_duplicate: *bool* = *False*, ignore\_fields: *Sequence[str | int]* | *None* = *()*) → *PdfObject*

Clone object into pdf\_dest (PdfWriterProtocol which is an interface for PdfWriter).

By default, this method will call `_reference_clone` (see `_reference`).

#### Parameters

- **pdf\_dest** – Target to clone to.
- **force\_duplicate** – By default, if the object has already been cloned and referenced, the copy will be returned; when *True*, a new copy will be created. (Default value = *False*)

- **ignore\_fields** – List/tuple of field names (for dictionaries) that will be ignored during cloning (applies to children duplication as well). If fields are to be considered for a limited number of levels, you have to add it as integer, for example [1, "/B", "/TOT0"] means that "/B" will be ignored at the first level only but "/TOT0" on all levels.

**Returns**

The cloned PdfObject

**get\_object()** → *PdfObject* | *None*

Resolve indirect references.

**write\_to\_stream**(*stream: IO[Any]*, *encryption\_key: None* | *str* | *bytes* = *None*) → *None*

**class** pypdf.generic.**StreamObject**

Bases: *DictionaryObject*

**replicate**(*pdf\_dest: PdfWriterProtocol*) → *StreamObject*

**hash\_bin()** → *int*

Used to detect modified object.

**Returns**

Hash considering type and value.

**get\_data()** → *bytes*

**set\_data**(*data: bytes*) → *None*

**hash\_value\_data()** → *bytes*

**write\_to\_stream**(*stream: IO[Any]*, *encryption\_key: None* | *str* | *bytes* = *None*) → *None*

**static initialize\_from\_dictionary**(*data: dict[str, Any]*) → *EncodedStreamObject* | *DecodedStreamObject*

**flate\_encode**(*level: int* = -1) → *EncodedStreamObject*

**decode\_as\_image**(*pillow\_parameters: dict[str, Any]* | *None* = *None*) → *Any*

Try to decode the stream object as an image

**Parameters**

**pillow\_parameters** – parameters provided to Pillow Image.save() method, cf. <<https://pillow.readthedocs.io/en/stable/reference/Image.html#PIL.Image.Image.save>>

**Returns**

a PIL image if proper decoding has been found

**Raises**

**Exception** – Errors during decoding will be reported. It is recommended to catch exceptions to prevent stops in your program.

**class** pypdf.generic.**TextStringObject**(*value: Any*)

Bases: *str*, *PdfObject*

A string object that has been decoded into a real unicode string.

If read from a PDF document, this string appeared to match the PDFDocEncoding, or contained a UTF-16BE BOM mark to cause UTF-16 decoding to occur.

**autodetect\_pdfdocencoding:** `bool`

**autodetect\_utf16:** `bool`

**utf16\_bom:** `bytes`

**clone**(pdf\_dest: *Any*, force\_duplicate: *bool* = *False*, ignore\_fields: *Sequence*[*str* | *int*] | *None* = ()) → *TextStringObject*

Clone object into pdf\_dest.

**hash\_bin**() → *int*

Used to detect modified object.

#### Returns

Hash considering type and value.

**property original\_bytes:** `bytes`

It is occasionally possible that a text string object gets created where a byte string object was expected due to the autodetection mechanism – if that occurs, this “original\_bytes” property can be used to back-calculate what the original encoded bytes were.

**get\_original\_bytes**() → *bytes*

**get\_encoded\_bytes**() → *bytes*

**write\_to\_stream**(stream: *IO*[*Any*], encryption\_key: *None* | *str* | *bytes* = *None*) → *None*

**class** pypdf.generic.**TreeObject**(dct: *DictionaryObject* | *None* = *None*)

Bases: *DictionaryObject*

**has\_children**() → *bool*

**children**() → *Iterable*[*Any*]

**add\_child**(child: *Any*, pdf: *PdfWriterProtocol*) → *None*

**inc\_parent\_counter\_default**(parent: *None* | *IndirectObject* | *TreeObject*, n: *int*) → *None*

**inc\_parent\_counter\_outline**(parent: *None* | *IndirectObject* | *TreeObject*, n: *int*) → *None*

**insert\_child**(child: *Any*, before: *Any*, pdf: *PdfWriterProtocol*, inc\_parent\_counter: *Callable*[[*...*], *Any*] | *None* = *None*) → *IndirectObject*

**remove\_child**(child: *Any*) → *None*

**remove\_from\_tree**() → *None*

Remove the object from the tree it is in.

**empty\_tree**() → *None*

**class** pypdf.generic.**ViewerPreferences**(value: *Any* = *None*)

Bases: *DictionaryObject*

**property PRINT\_SCALING:** *NameObject*

**pypdf.generic.create\_string\_object**(string: *str* | *bytes*, forced\_encoding: *None* | *str* | *list*[*str*] | *dict*[*int*, *str*] = *None*) → *TextStringObject* | *ByteStringObject*

Create a *ByteStringObject* or a *TextStringObject* from a string to represent the string.

#### Parameters

- **string** – The data being used
- **forced\_encoding** – Typically None, or an encoding string

**Returns**

A ByteStringObject

**Raises**

**TypeError** – If string is not of type str or bytes.

`pypdf.generic.decode_pdfdocencoding(byte_array: bytes) → str`

`pypdf.generic.encode_pdfdocencoding(unicode_string: str) → bytes`

`pypdf.generic.extract_links(new_page: PageObject, old_page: PageObject) → list[tuple[NamedReferenceLink | DirectReferenceLink, NamedReferenceLink | DirectReferenceLink]]`

Extracts links from two pages on the assumption that the two pages are the same. Produces one list of (new link, old link) tuples.

Non-link annotations are ignored before pairing to avoid dropping valid links when one page includes additional non-link annotation entries.

`pypdf.generic.hex_to_rgb(value: str) → tuple[float, float, float]`

`pypdf.generic.is_null_or_none(x: Any) → TypeGuard[None | NullObject | IndirectObject]`

**Returns**

True if x is None or NullObject.

`pypdf.generic.read_hex_string_from_stream(stream: IO[Any], forced_encoding: None | str | list[str] | dict[int, str] = None) → TextStringObject | ByteStringObject`

`pypdf.generic.read_object(stream: IO[Any], pdf: PdfReaderProtocol | None, forced_encoding: None | str | list[str] | dict[int, str] = None) → PdfObject | int | str | ContentStream`

`pypdf.generic.read_string_from_stream(stream: IO[Any], forced_encoding: None | str | list[str] | dict[int, str] = None) → TextStringObject | ByteStringObject`

`class pypdf._protocols.PdfObjectProtocol(*args, **kwargs)`

Bases: `Protocol`

**indirect\_reference:** `Any`

**clone**(pdf\_dest: Any, force\_duplicate: bool = False, ignore\_fields: tuple[str, ...] | list[str] | None = ()) → Any

**get\_object**() → PdfObjectProtocol | None

**hash\_value**() → bytes

**write\_to\_stream**(stream: IO[Any], encryption\_key: None | str | bytes = None) → None

`class pypdf._protocols.XmpInformationProtocol(*args, **kwargs)`

Bases: `PdfObjectProtocol`

`class pypdf._protocols.PdfCommonDocProtocol(*args, **kwargs)`

Bases: `Protocol`

**property pdf\_header:** `str`

```
property pages: list[Any]

property root_object: PdfObjectProtocol

get_object(indirect_reference: Any) → PdfObjectProtocol | None

property strict: bool

class pypdf._protocols.PdfReaderProtocol(*args, **kwargs)
    Bases: PdfCommonDocProtocol, Protocol
    abstract property xref: dict[int, dict[int, Any]]

    abstract property trailer: dict[str, Any]

class pypdf._protocols.PdfWriterProtocol(*args, **kwargs)
    Bases: PdfCommonDocProtocol, Protocol
    incremental: bool

    abstract write(stream: Path | str | IO[Any]) → tuple[bool, IO[Any]]
```



## THE PDFDOCCOMMON CLASS

**PdfDocCommon** is an abstract class which is inherited by *PdfReader* and *PdfWriter*.

Where identified in the API, you can use any of the derived class.

**class** pypdf.\_doc\_common.**PdfDocCommon**

Bases: *object*

Common functions from PdfWriter and PdfReader objects.

This root class is strongly abstracted.

**strict:** *bool* = *False*

**flattened\_pages:** *list[PageObject]* | *None* = *None*

**abstract property root\_object:** *DictionaryObject*

**abstract property pdf\_header:** *str*

**abstract get\_object**(*indirect\_reference: int* | *IndirectObject*) → *PdfObject* | *None*

**property metadata:** *DocumentInformation* | *None*

Retrieve the PDF file's document information dictionary, if it exists.

Note that some PDF files use metadata streams instead of document information dictionaries, and these metadata streams will not be accessed by this function.

**property xmp\_metadata:** *XmpInformation* | *None*

**property viewer\_preferences:** *ViewerPreferences* | *None*

Returns the existing ViewerPreferences as an overloaded dictionary.

**get\_num\_pages**() → *int*

Calculate the number of pages in this PDF file.

**Returns**

The number of pages of the parsed PDF file.

**Raises**

*PdfReadError* – If restrictions prevent this action.

**get\_page**(*page\_number: int*) → *PageObject*

Retrieve a page by number from this PDF file. Most of the time `.pages[page_number]` is preferred.

**Parameters**

**page\_number** – The page number to retrieve (pages begin at zero)

**Returns**

A *PageObject* instance.

**property named\_destinations:** `dict[str, Destination]`

A read-only dictionary which maps names to destinations.

**get\_named\_dest\_root()**  $\rightarrow$  *ArrayObject*

**get\_fields**(*tree*: *TreeObject* | *None* = *None*, *retval*: `dict[Any, Any]` | *None* = *None*, *fileobj*: *Any* | *None* = *None*, *stack*: `list[PdfObject]` | *None* = *None*)  $\rightarrow$  `dict[str, Any]` | *None*

Extract field data if this PDF contains interactive form fields.

The *tree*, *retval*, *stack* parameters are for recursive use.

**Parameters**

- **tree** – Current object to parse.
- **retval** – In-progress list of fields.
- **fileobj** – A file object (usually a text file) to write a report to on all interactive form fields found.
- **stack** – List of already parsed objects.

**Returns**

A dictionary where each key is a field name, and each value is a *Field* object. By default, the mapping name is used for keys. *None* if form data could not be located.

**get\_form\_text\_fields**(*full\_qualified\_name*: *bool* = *False*)  $\rightarrow$  `dict[str, Any]`

Retrieve form fields from the document with textual data.

**Parameters**

**full\_qualified\_name** – to get full name

**Returns**

A dictionary. The key is the name of the form field, the value is the content of the field.

If the document contains multiple form fields with the same name, the second and following will get the suffix .2, .3, ...

**get\_pages\_showing\_field**(*field*: *Field* | *PdfObject* | *IndirectObject*)  $\rightarrow$  `list[PageObject]`

Provides list of pages where the field is called.

**Parameters**

**field** – Field Object, PdfObject or IndirectObject referencing a Field

**Returns**

*List of pages* –

- **Empty list:**  
The field has no widgets attached (either hidden field or ancestor field).
- **Single page list:**  
Page where the widget is present (most common).
- **Multi-page list:**  
Field with multiple kids widgets (example: radio buttons, field repeated on multiple pages).



**property open\_destination:** `None | Destination | TextStringObject | ByteStringObject`

Property to access the opening destination (/OpenAction entry in the PDF catalog). It returns `None` if the entry does not exist or is not set.

**Raises**

**Exception** – If a destination is invalid.

**property outline:** `list[Destination | list[Destination | list[Destination]]]`

Read-only property for the outline present in the document (i.e., a collection of ‘outline items’ which are also known as ‘bookmarks’).

**property threads:** `ArrayObject | None`

Read-only property for the list of threads.

See §12.4.3 from the PDF 1.7 or 2.0 specification.

It is an array of dictionaries with “/F” (the first bead in the thread) and “/I” (a thread information dictionary containing information about the thread, such as its title, author, and creation date) properties or `None` if there are no articles.

Since PDF 2.0 it can also contain an indirect reference to a metadata stream containing information about the thread, such as its title, author, and creation date.

**get\_page\_number**(page: `PageObject`) → `int | None`

Retrieve page number of a given `PageObject`.

**Parameters**

**page** – The page to get page number. Should be an instance of `PageObject`

**Returns**

The page number or `None` if page is not found

**get\_destination\_page\_number**(destination: `Destination`) → `int | None`

Retrieve page number of a given `Destination` object.

**Parameters**

**destination** – The destination to get page number.

**Returns**

The page number or `None` if page is not found

**property pages:** `list[PageObject]`

Property that emulates a list of `PageObject`. This property allows to get a page or a range of pages.

**Note**

For PdfWriter only: Provides the capability to remove a page/range of page from the list (using the del operator). Remember: Only the page entry is removed, as the objects beneath can be used elsewhere. A solution to completely remove them - if they are not used anywhere - is to write to a buffer/temporary file and then load it into a new PdfWriter.

**property page\_labels:** `list[str]`

A list of labels for the pages in this document.

This property is read-only. The labels are in the order that the pages appear in the document.

**property page\_layout:** `str` | `None`

Get the page layout currently being used.

Table 1: Valid layout values

|                               |                                                            |
|-------------------------------|------------------------------------------------------------|
| <code>/NoLayout</code>        | Layout explicitly not specified                            |
| <code>/SinglePage</code>      | Show one page at a time                                    |
| <code>/OneColumn</code>       | Show one column at a time                                  |
| <code>/TwoColumnLeft</code>   | Show pages in two columns, odd-numbered pages on the left  |
| <code>/TwoColumn-Right</code> | Show pages in two columns, odd-numbered pages on the right |
| <code>/TwoPageLeft</code>     | Show two pages at a time, odd-numbered pages on the left   |
| <code>/TwoPageRight</code>    | Show two pages at a time, odd-numbered pages on the right  |

**property page\_mode:** `Literal['/UseNone', '/UseOutlines', '/UseThumbs', '/FullScreen', '/UseOC', '/UseAttachments']` | `None`

Get the page mode currently being used.

Table 2: Valid mode values

|                              |                                          |
|------------------------------|------------------------------------------|
| <code>/UseNone</code>        | Do not show outline or thumbnails panels |
| <code>/UseOutlines</code>    | Show outline (aka bookmarks) panel       |
| <code>/UseThumbs</code>      | Show page thumbnails panel               |
| <code>/FullScreen</code>     | Fullscreen view                          |
| <code>/UseOC</code>          | Show Optional Content Group (OCG) panel  |
| <code>/UseAttachments</code> | Show attachments panel                   |

**remove\_page**(*page*: `int` | `PageObject` | `IndirectObject`, *clean*: `bool = False`) → `None`

Remove page from pages list.

#### Parameters

- **page** –
  - `int`: Page number to be removed.
  - `PageObject`: page to be removed. If the page appears many times only the first one will be removed.
  - `IndirectObject`: Reference to page to be removed.
- **clean** – replace `PageObject` with `NullObject` to prevent annotations or destinations to reference a detached page.

**decode\_permissions**(*permissions\_code*: `int`) → `dict[str, bool]`

Take the permissions as an integer, return the allowed access.

**property user\_access\_permissions:** `UserAccessPermissions` | `None`

Get the user access permissions for encrypted documents. Returns `None` if not encrypted.

#### Warning

For AES-256 encrypted documents (R=5/R=6), the returned permissions are derived from the `/P` field, which is only trustworthy if the `/Perms` integrity check passed. Check [are\\_permissions\\_valid](#) to verify.

**property are\_permissions\_valid:** `bool` | `None`

Whether the /Perms integrity check passed for this document.

For AES-256 encrypted documents (R=5/R=6), the /Perms field is an encrypted copy of the permissions that can be verified independently. Returns `False` if this check fails (the /P permissions may have been tampered with).

Returns `None` if the document is not encrypted or has not yet been decrypted via `decrypt()`. Returns `True` for non-AES-256 encryption (no /Perms to check).

**abstract property is\_encrypted:** `bool`

Read-only boolean property showing whether this PDF file is encrypted.

Note that this property, if true, will remain true even after the `decrypt()` method is called.

**property xfa:** `dict[str, Any]` | `None`

**property attachments:** `Mapping[str, list[bytes]]`

Mapping of attachment filenames to their content.

**property attachment\_list:** `Generator[EmbeddedFile, None, None]`

Iterable of attachment objects.



## DEVELOPER INTRO

pypdf is a library and hence its users are developers. This document is not for the users, but for people who want to work on pypdf itself.

### Note

Our CI (continuous integration) validates that relevant standards are met with your contribution. Especially for regular contributors or larger changes, it is highly recommended that you set up your own development environment to already cover the most important aspects locally. This greatly helps us to reduce the noise compared to when you open an untested PR early and use our CI to do your debugging and improvements from there. The maintainers usually receive a notification on every push to a branch where a corresponding PR is open, possibly hiding important notifications.

## 41.1 Installing Requirements

```
pip install -r requirements/dev.txt
```

## 41.2 Running Tests

See *testing pypdf with pytest*.

## 41.3 The sample-files git submodule

The reason for having the submodule `sample-files` is that we want to keep the size of the pypdf repository small while we also want to have an extensive test suite. Those two goals contradict each other.

The `resources` folder should contain a select set of core examples that cover most cases we typically want to test for. The `sample-files` might cover a lot more edge cases, the behavior we get when file sizes get bigger, different PDF producers.

To get the `sample-files` folder, you need to execute:

```
git submodule update --init
```

## 41.4 Tools: git and pre-commit

Git is a command line application for version control. If you don't know it, you can [play ohmygit](#) to learn it.

GitHub is the service where the pypdf project is hosted. While git is free and open source, GitHub is a paid service by Microsoft, but free in a lot of cases.

`pre-commit` is a command line application that uses git hooks to automatically execute code. This allows you to avoid style issues and other code quality issues. After you entered `pre-commit` install once in your local copy of pypdf, it will automatically be executed when you `git commit`.

## 41.5 Commit Messages

Having a clean commit message helps people to quickly understand what the commit is about, without actually looking at the changes. The first line of the commit message is used to [auto-generate the CHANGELOG](#). For this reason, the format should be:

PREFIX: DESCRIPTION

BODY

The PREFIX can be:

- SEC: Security improvements. Typically, an infinite loop that was possible.
- BUG: A bug was fixed. Likely there are one or multiple issues. Then write in the BODY: Closes #123 where 123 is the issue number on GitHub. It would be absolutely amazing if you could write a regression test in those cases. That is a test that would fail without the fix. A bug is always an issue for pypdf users - test code or CI that was fixed is not considered a bug here.
- ENH: A new feature! Describe in the body what it can be used for.
- DEP: Deprecation. Either marking something as “this is going to be removed” or actually removing it.
- PI: A performance improvement. This could also be a reduction in the file size of PDF files generated by pypdf.
- ROB: A robustness change. Dealing better with broken PDF files.
- DOC: A documentation change.
- TST: Adding or adjusting tests.
- DEV: Developer experience improvements, e.g., pre-commit or setting up CI.
- MAINT: Quite a lot of different stuff. Performance improvements are, for sure, the most interesting changes in here. Refactorings as well.
- STY: A style change. Something that makes pypdf code more consistent. Typically, a small change. It could also be better error messages for end users.

The prefix is used to generate the CHANGELOG. Every PR must have exactly one - if you feel like several match, take the top one from this list that matches for your PR.

## 41.6 Pull Request Size

Smaller Pull Requests (PRs) are preferred as it's typically easier to merge them. For example, if you have some typos, a few code-style changes, a new feature, and a bug-fix, that could be three or four PRs.

A PR must be complete. That means if you introduce a new feature, it must be finished within the PR and have a test for that feature.

## 41.7 Benchmarks

We need to keep an eye on performance, and thus we have a few benchmarks.

See [py-pdf.github.io/pypdf/dev/bench](https://py-pdf.github.io/pypdf/dev/bench)





## THE PDF FORMAT

It is recommended to look in the PDF specification for details and clarifications.

- [PDF Specification Archive](#)
- [Portable Document Format Reference Manual](#), 1993. ISBN 0-201-62628-4
- [ISO 32000-1:2008 \(PDF 1.7\)](#)
- [ISO 32000-2:2020 \(PDF 2.0\)](#)

### Note

We currently generate files with a header for PDF 1.3 by default. At the same time, we strive to support the PDF 1.7 specification.

Features specific to PDF 2.0 might be available, but we always ensure that older versions do not break due to the rather limited general PDF 2.0 support in the wild and to not break for old PDF files. For this reason, some historical aspects (like insecure encryption algorithms) are required to be supported, although PDF 2.0 deprecates most of them and allows more secure variants.

Below is only intended to give a very rough overview of the format.

## 42.1 Overall Structure

A PDF consists of:

1. Header: Contains the version of the PDF, e.g. %PDF-1.7
2. Body: Contains a sequence of indirect objects
3. Cross-reference table (xref): Contains a list of the indirect objects in the body
4. Trailer

## 42.2 The xref table

A cross-reference table (xref) is a table of the indirect objects in the body. It allows quick access to those objects by pointing to their location in the file.

It looks like this:

```
xref 42 5
00000001000 65535 f
00000001234 00000 n
00000001987 00000 n
00000011987 00000 n
00000031987 00000 n
```

Let's go through it step-by-step:

- `xref` is just a keyword that specifies the start of the xref table.
- 42 is the numerical ID of the first object in this xref section; 5 is the number of entries in the xref table.
- Now every object has 3 entries `nnnnnnnnnn ggggg n`: a 10-digit byte offset, a 5-digit generation number, and a literal keyword which is either `n` or `f`.
  - `nnnnnnnnnn` is the byte offset of the object. It tells the reader where the object is in the file.
  - `ggggg` is the generation number. It tells the reader how old the object is.
  - `n` means that the object is a normal in-use object, `f` means that the object is a free object.
    - \* The first free object always has a generation number of 65535. It forms the head of a linked-list of all free objects.
    - \* The generation number of a normal object is always 0. The generation number allows the PDF format to contain multiple versions of the same object. This is a version history mechanism.

## 42.3 The body

The body is a sequence of indirect objects:

```
counter generation_number << the_object >> endobj
```

- `counter` (integer) is a unique identifier for the object.
- `generation_number` (integer) is the generation number of the object.
- `the_object` is the object itself. It can be empty. Starts with `/Keyword` to specify which kind of object it is.
- `endobj` marks the end of the object.

A concrete example can be found in `test_reader.py::test_get_images_raw`:

```
1 0 obj << /Count 1 /Kids [4 0 R] /Type /Pages >> endobj
2 0 obj << >> endobj
3 0 obj << >> endobj
4 0 obj << /Contents 3 0 R /CropBox [0.0 0.0 2550.0 3508.0]
  /MediaBox [0.0 0.0 2550.0 3508.0] /Parent 1 0 R
  /Resources << /Font << >> >>
  /Rotate 0 /Type /Page >> endobj
5 0 obj << /Pages 1 0 R /Type /Catalog >> endobj
```

## 42.4 The trailer

The trailer looks like this:

```
trailer << /Root 5 0 R
          /Size 6
          >>
startxref 1234
%%EOF
```

Let's go through it:

- **trailer** << indicates that the *trailer dictionary* starts. It ends with >>.
- **startxref** is a keyword followed by the byte-location of the **xref** keyword. As the trailer is always at the bottom of the file, this allows readers to quickly find the xref table.
- **%%EOF** is the end-of-file marker.

The trailer dictionary is a key-value list. The keys are specified in Table 15 of the PDF Reference 1.7, e.g. **/Root** and **/Size** (both are required).

- **/Root** (dictionary) contains the document catalog.
  - The 5 is the object number of the catalog dictionary.
  - 0 is the generation number of the catalog dictionary.
  - R is the keyword that indicates that the object is a reference to the catalog dictionary.
- **/Size** (integer) contains the total number of entries in the files xref table.

## 42.5 Reading PDF files

Most PDF files are compressed. If you want to read them, first uncompress them:

```
pdftk crazyones.pdf output crazyones-uncomp.pdf uncompress
```

Then rename **crazyones-uncomp.pdf** to **crazyones-uncomp.txt** and open it in your favorite IDE / text editor.



## HOW PYPDF PARSES PDF FILES

pypdf uses *PdfReader* to parse PDF files. The method *PdfReader.read* shows the basic structure of parsing:

1. **Finding and reading the cross-reference tables / trailer:** The cross-reference table (xref table) is a table of byte offsets that indicate the locations of objects within the file. The trailer provides additional information such as the root object (Catalog) and the Info object containing metadata.
2. **Parsing the objects:** After locating the xref table and the trailer, pypdf proceeds to parse the objects in the PDF. Objects in a PDF can be of various types such as dictionaries, arrays, streams, and simple data types (e.g., integers, strings). pypdf parses these objects and stores them in *PdfReader.resolved\_objects*, populated by *cache\_indirect\_object*.
3. **Decoding content streams:** The content of a PDF is typically stored in content streams, which are sequences of PDF operators and operands. pypdf decodes these content streams by applying filters (e.g., *FlateDecode*, *LZWDecode*) specified in the stream's dictionary. This is only done when the object is requested by *PdfReader.get\_object* which uses the *PdfReader.\_get\_object\_from\_stream* method.

### 43.1 References

PDF 1.7 specification:

- 7.5 File Structure
- 7.5.4 Cross-Reference Table
- 7.8 Content Streams and Resources



## HOW PYPDF WRITES PDF FILES

pypdf uses *PdfWriter* to write PDF files. pypdf has *PdfObject* and several subclasses with the *write\_to\_stream* method. The *PdfWriter.write* method uses the *write\_to\_stream* methods of the referenced objects.

The *PdfWriter.write\_stream* method has the following core steps:

1. **\_sweep\_indirect\_references**: This step ensures that any circular references to objects are correctly handled. It adds the object reference numbers of any circularly referenced objects to an external reference map, so that self-page-referencing trees can reference the correct new object location, rather than copying in a new copy of the page object.
2. **Write the File Header and Body** with *\_write\_pdf\_structure*: In this step, the PDF header and objects are written to the output stream. This includes the PDF version (e.g., %PDF-1.7) and the objects that make up the content of the PDF, such as pages, annotations, and form fields. The locations (byte offsets) of these objects are stored for later use in generating the xref table.
3. **Write the Cross-Reference Table** with *\_write\_xref\_table*: Using the stored object locations, this step generates and writes the cross-reference table (xref table) to the output stream. The cross-reference table contains the byte offsets for each object in the PDF file, allowing for quick random access to objects when reading the PDF.
4. **Write the File Trailer** with *\_write\_trailer*: The trailer is written to the output stream in this step. The trailer contains essential information, such as the number of objects in the PDF, the location of the root object (Catalog), and the Info object containing metadata. The trailer also specifies the location of the xref table.

### 44.1 How others do it

Looking at alternative software designs and implementations can help to improve our choices.

#### 44.1.1 fpdf2

fpdf2 has a *PDFObject* class with a *serialize* method which roughly maps to *pypdf.PdfObject.write\_to\_stream*. Some other similarities include:

- *fpdf.output.OutputProducer.buffer\_size* vs. *pypdf.PdfWriter.write\_stream*
- *fpdf.syntax.Name* vs. *pypdf.generic.NameObject*
- *fpdf.syntax.build\_obj\_dict* vs. *pypdf.generic.DictionaryObject*
- *fpdf.structure\_tree.NumberTree* vs. *pypdf.generic.TreeObject*

### **44.1.2 pdfwr**

`pdfwr`, in contrast, seems to work more with the standard Python objects (bool, float, string) and not wrap them in custom objects, if possible. It still has:

- PdfArray
- PdfDict
- PdfName
- PdfString
- PdfIndirect

The core classes of `pdfwr` are PdfReader and PdfWriter



## CMAPS

Looking at the cmap of “crazyones”:

```
pdftk crazyones.pdf output crazyones-uncomp.pdf uncompress
```

You can see this:

```
begincmap
/CMAPName /T1Encoding-UTF16 def
/CMAPType 2 def
/CIDSystemInfo <<
  /Registry (Adobe)
  /Ordering (UCS)
  /Supplement 0
>> def
1 begincodespacerange
<00> <FF>
endcodespacerange
1 beginbfchar
<1B> <FB00>
endbfchar
endcmap
CMAPName currentdict /CMAP defineresource pop
```

### 45.1 codespacerange

A codespacerange maps a complete sequence of bytes to a range of Unicode glyphs. It defines a starting point:

```
1 beginbfchar
<1B> <FB00>
```

That means that 1B (Hex for 27) maps to the Unicode character **FB00** - the ligature ff (two lowercase f’s).

The two numbers in **begincodespacerange** mean that it starts with an offset of 0 (hence from 1B **FB00**) up to an offset of FF (dec: 255), hence 1B+FF = 282 **FBFF**.

Within the text stream, there is

```
(The)-342(mis\034ts.)
```

`\034` is octal for the decimal value 28.



## THE DEPRECATION PROCESS

pypdf strives to be an excellent library for its current users and for new ones. We are careful with introducing potentially breaking changes, but we will do them if they provide value for the community in the long run.

We hope and think that deprecations will not happen frequently. If they do, users can rely on the following procedure.

### 46.1 Semantic Versioning

pypdf uses [semantic versioning](#). If you want to avoid breaking changes, please use dependency pinning (also known as version pinning). In Python, this is done by specifying the exact version you want to use in a `requirements.txt` file. A tool that can support you is `pip-compile` from [pip-tools](#).

If you are using [Poetry](#) it is done with the `poetry.lock` file.

### 46.2 How pypdf deprecates features

Assume the current version of pypdf is `x.y.z`. After a discussion (e.g., via GitHub issues), we decided to remove a class / function / method. This is how we do it:

1. `x.y.(z+1)`: Add a `DeprecationWarning`. If there is a replacement, the replacement is also introduced and the warning informs about the change and when it will happen. The docs let users know about the deprecation and when it will happen and the new function. The CHANGELOG informs about it.
2. `(x+1).0.0`: Remove / change the code in the breaking way by replacing `DeprecationWarnings` by `DeprecationErrors`. We do this to help people who didn't look at the warnings before. The CHANGELOG informs about it.
3. `(x+2).0.0`: The `DeprecationErrors` are removed.

This means the users have three warnings in the CHANGELOG, a `DeprecationWarning` until the next major release and a `DeprecationError` until the major release after that.

Please note that adding warnings can be a breaking change for some users; most likely just in the CI. This means it needs to be properly documented.



## DOCUMENTATION

This documentation is build with [Sphinx](#) and hosted by [Read the Docs](#)

### 47.1 Testing code snippets

Almost all python code snippets in documentation tested using Sphinx's extension `sphinx.ext.doctest`. This allows to make sure that we have no typos, missed imports and other problems in:

- code snippets marked with `testcode` directive in `*.md` files
- code snippets from python's docstrings imported via `autoclass` directive in `*.rst` files

CI pipeline is configured run Sphinx's `doctest` build automatically for each PR. It is also possible to run it locally:

1. First you need to install docs requirements

```
pip install -r requirements/docs.txt
```

2. Change current directory

```
cd docs
```

3. Run `doctest` build. It uses indirectly `sphinx-build` command line tool installed with docs requirements. See [Sphinx's docs](#) for details.

```
make doctest
```

4. If everything is okay you should see in output `Doctest summary` without failures

### 47.2 API Reference

#### 47.2.1 Method / Function Docstrings

We use Google-Style Docstrings:

```
def example(param1: int, param2: str) -> bool:
    """
    Example function with PEP 484 type annotations.

    Args:
        param1: The first parameter.
        param2: The second parameter.
```

(continues on next page)

(continued from previous page)

**Returns:**

*The return value. True for success, False otherwise.*

**Raises:**

*AttributeError: The ``Raises`` section is a list of all exceptions that are relevant to the interface.*

*ValueError: If `param2` is equal to `param1`.*

**Examples:**

*Examples should be written in doctest format, and should illustrate how to use the function.*

```
>>> print([i for i in example_generator(4)])
[0, 1, 2, 3]
"""
```

- The order of sections is (1) Args (2) Returns (3) Raises (4) Examples
- If there is no return value, remove the 'Returns' block
- Properties should not have any sections

## 47.3 Issues and PRs

An issue can be used to discuss what we want to achieve.

A PR can be used to discuss how we achieve it.

## 47.4 Commit Messages

We want to have descriptive commits in the main branch. For this reason, every pull request (PR) is squashed. That means no matter how many commits a PR has, in the end only one combined commit will be in main.

The title of the PR will be used as the first line of that combined commit message.

The first comment within the commit will be used as the message body.

See [developer intro](#) for more details.

## TESTING

pypdf uses `pytest` for testing.

To run the tests, you need to install the CI (Continuous Integration) requirements by running `pip install -r requirements/ci.txt` or `pip install -r requirements/ci-3.11.txt` if running Python 3.11.

### 48.1 Deselecting groups of tests

pypdf makes use of the following pytest markers:

- `slow`: Tests that require more than 5 seconds.
- `samples`: Tests that require the `sample-files` git submodule to be initialized. As of October 2022, this is about 25 MB.
- `enable_socket`: Tests that download PDF documents. They are stored locally and thus only need to be downloaded once. As of October 2022, this is about 200 MB.
  - To successfully run the tests, please download most of the documents beforehand: `python -c "from tests import download_test_pdfs; download_test_pdfs()"`

You can disable them by `pytest -m "not enable_socket"` or `pytest -m "not samples"`. You can even disable all of them: `pytest -m "not enable_socket" -m "not samples" -m "not slow"`.

Please note that this reduces test coverage. The CI will always test all files.

### 48.2 Docstrings in Unit tests

The first line of a docstring in a unit test should be written in a way that you could prefix it with “This tests ensures that ...”, e.g.

- Invalid XML in `xmp_metadata` is gracefully handled.
- The identity is returning its input.
- `xmp_modify_date` is extracted correctly.

This way, plugins like `pytest-testdox` can generate really nice output when the tests are running. This looks similar to the output of `mocha.js`.

If the test is a regression test, write

This test is a regression test for issue #1234

If the regression test is just one parameter of other tests, then add it as a comment for that parameter.

## 48.3 Evaluate a PR in-progress version

You may want to test a version from a PR which has not been released yet. The easiest way is to use pip and install a version from git:

a) Go the PR and identify the repository and branch.

### BUG: invalid cm/tm in visitor

 Open

pubpub-zz wants to merge 4 commits into [py-pdf:1](#)

 Conversation 5

 Commits 4

 Checks 1



pubpub-zz commented 2 days ago • edited ▾

Example from below : repository: **pubpub-zz** / branch: **iss2200** :

b) you can then install the version using pip from git:

Example:

```
pip install git+https://github.com/pubpub-zz/pypdf.git@iss2200
```



## RELEASING

A pypdf release contains the following artifacts:

- A new [release on PyPI](#)
- A [release commit](#)
  - Containing a changelog update
  - A new [git tag](#)
    - \* A [GitHub release](#)

### 49.1 Who does it?

pypdf should typically only be released by one of the core maintainers / the core maintainer. At the moment, this usually is stefan6419846.

Any owner of the py-pdf organization also has the technical permissions to release.

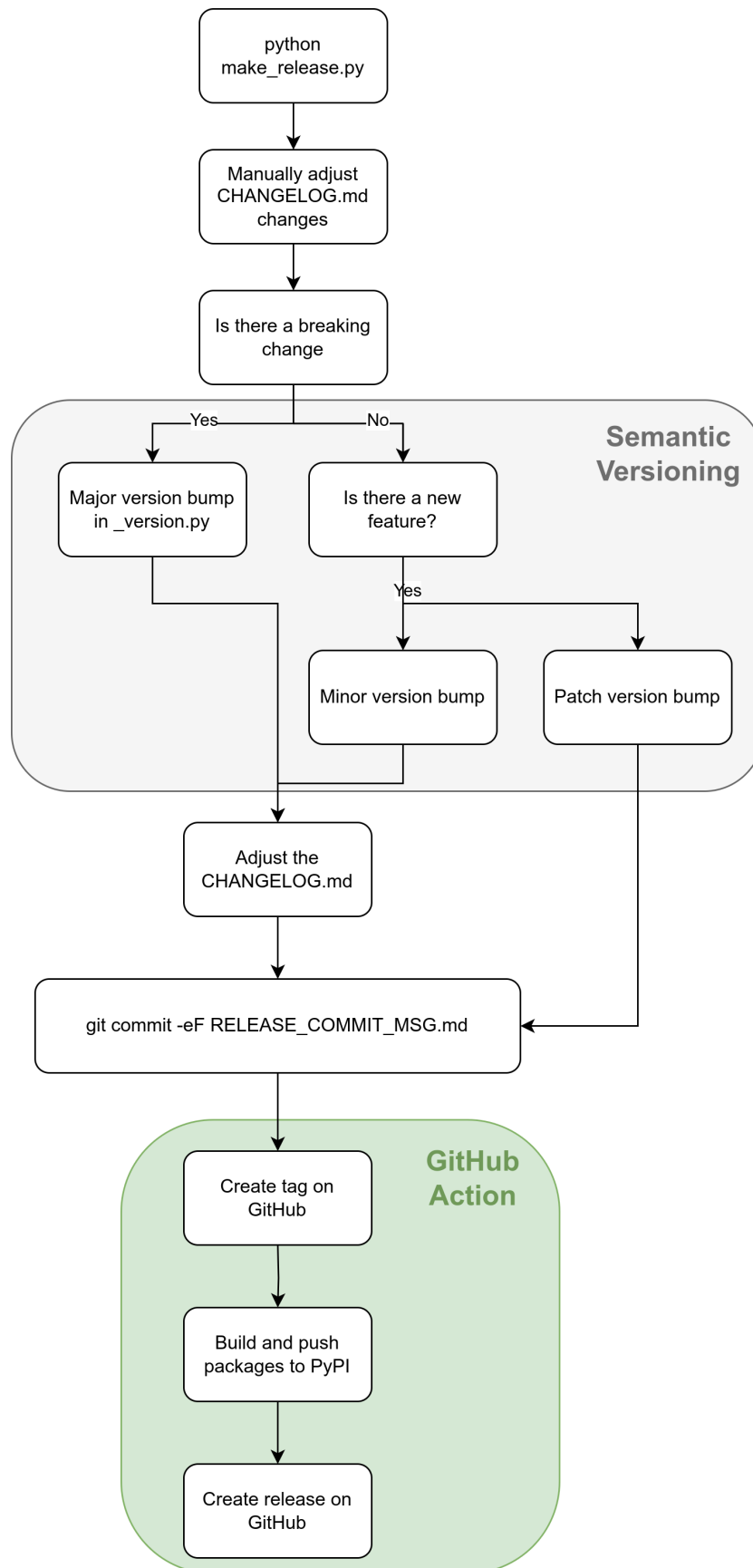
### 49.2 How is it done?

#### 49.2.1 With direct push permissions

This is the typical way for the core maintainer/benevolent dictator.

The release contains the following steps:

1. Update the CHANGELOG.md and the \_version.py via `python make_release.py`. This also prepares the release commit message.
2. Create a release commit: `git commit -eF RELEASE_COMMIT_MSG.md`.
3. Push commit: `git push`.
4. Create the tag: `git tag -s 6.7.1 -eF RELEASE_COMMIT_MSG.md`.
5. Push the tag: `git push origin 6.7.1`.
6. CI now builds a source and a wheels package which it pushes to PyPI. It also creates the corresponding GitHub release.



### 49.2.2 Using a Pull Request

This is the typical way for collaborators which do not have direct push permissions for the `main` branch.

The release contains the following steps:

1. Update the `CHANGELOG.md` and the `_version.py` via `python make_release.py`. This also prepares the release commit message.
2. Push the changes to a dedicated branch.
3. Open a pull request starting with `REL:` , followed by the new version number.
4. Wait for the approval of another eligible maintainer.
5. Merge the pull request with the name being the PR title and the body being the content of `RELEASE_COMMIT_MSG.md`.
6. Create the tag: `git tag -s 6.7.1 -eF RELEASE_COMMIT_MSG.md`.
7. Push the tag: `git push origin 6.7.1`.
8. CI now builds a source and a wheels package which it pushes to PyPI. It also creates the corresponding GitHub release.

### 49.2.3 The Release Tag

- Use the release version as the tag name. No need for a leading “v”.
- Use the changelog entry as the body.

## 49.3 When are releases done?

There is no need to wait for anything. If the CI is green (all tests succeeded), we can release.

At the moment, there is no fixed release cycle - except that we usually release on Sunday.



## CHANGELOG

### 50.1 Version 6.10.1, 2026-04-14

#### 50.1.1 Security (SEC)

- Limit the allowed size of xref and object streams (#3733)

#### 50.1.2 Robustness (ROB)

- Consider strict mode setting for decryption errors (#3731)

#### 50.1.3 Documentation (DOC)

- Use new parameter names for compress\_identical\_objects

[Full Changelog](#)

### 50.2 Version 6.10.0, 2026-04-10

#### 50.2.1 Security (SEC)

- Disallow custom XML entity declarations for XMP metadata (#3724)

#### 50.2.2 New Features (ENH)

- Skip MD5 key derivation for AES-256 encrypted PDFs (#3694)

#### 50.2.3 Bug Fixes (BUG)

- Use remove\_orphans in compress\_identical\_objects (#3310)
- Fix PdfReadError when xref table contains comments before trailer (#3710)
- Correctly verify AES padding during decryption (#3699)
- Fix stale object cache from non-authoritative object streams (#3698)
- Fix extract\_links pairing when annotations include non-links (#3687)

#### 50.2.4 Documentation (DOC)

- Add AI policy (#3717)

[Full Changelog](#)

## 50.3 Version 6.9.2, 2026-03-23

### 50.3.1 Security (SEC)

- Avoid infinite loop in `read_from_stream` for broken files (#3693)

### 50.3.2 Robustness (ROB)

- Resolve `UnboundLocalError` for `xobjs` in `_get_image` (#3684)

[Full Changelog](#)

## 50.4 Version 6.9.1, 2026-03-17

### 50.4.1 Security (SEC)

- Improve performance and limit length of array-based content streams (#3686)

[Full Changelog](#)

## 50.5 Version 6.9.0, 2026-03-15

### 50.5.1 New Features (ENH)

- Expose `/Perms` verification result on Encryption object (#3672)

### 50.5.2 Performance Improvements (PI)

- Fix  $O(n^2)$  performance in `NameObject` read/write (#3679)
- Batch-parse all objects in `ObjStm` on first access (#3677)

### 50.5.3 Bug Fixes (BUG)

- Avoid sharing array-based content streams between pages (#3681)
- Avoid accessing invalid page when inserting blank page under some conditions (#3529)

[Full Changelog](#)

## 50.6 Version 6.8.0, 2026-03-09

### 50.6.1 Security (SEC)

- Limit allowed `/Length` value of stream (#3675)

### 50.6.2 New Features (ENH)

- Add `/IRT` (in-reply-to) support for markup annotations (#3631)

### 50.6.3 Documentation (DOC)

- Avoid using `PageObject.replace_contents` on `PdfReader` (#3669)
- Document how to disable `jbig2dec` calls

[Full Changelog](#)

## 50.7 Version 6.7.5, 2026-03-02

### 50.7.1 Security (SEC)

- Improve the performance of the ASCIIHexDecode filter (#3666)

[Full Changelog](#)

## 50.8 Version 6.7.4, 2026-02-27

### 50.8.1 Security (SEC)

- Allow limiting output length for RunLengthDecode filter (#3664)

### 50.8.2 Robustness (ROB)

- Deal with invalid annotations in `extract_links` (#3659)

[Full Changelog](#)

## 50.9 Version 6.7.3, 2026-02-24

### 50.9.1 Security (SEC)

- Use zlib decompression limit when retrieving XFA data (#3658)

[Full Changelog](#)

## 50.10 Version 6.7.2, 2026-02-22

### 50.10.1 Security (SEC)

- Prevent infinite loop from circular `xref /Prev` references (#3655)

### 50.10.2 Bug Fixes (BUG)

- Fix wrong LUT size error (#3651)
- Fix handling of page boxes defined on `/Pages` (#3650)

[Full Changelog](#)

## 50.11 Version 6.7.1, 2026-02-17

### 50.11.1 Security (SEC)

- Detect cyclic references when accessing `TreeObject.children` (#3645)
- Limit size of `/ToUnicode` entries (#3646)
- Limit `FlateDecode` recovery attempts (#3644)

### **50.11.2 Bug Fixes (BUG)**

- Avoid own object replacement logic in `PageObject.replace_contents` (#3638)
- Fix `UnboundLocalError` when `update_page_form_field_values` with `/Sig` (#3634)

### **50.11.3 Robustness (ROB)**

- Avoid division by zero when decoding `FlateDecode` PNG prediction (#3641)

[Full Changelog](#)

## **50.12 Version 6.7.0, 2026-02-08**

### **50.12.1 Deprecations (DEP)**

- Deprecate support for abbreviations in `decode_stream_data` (#3617)

### **50.12.2 New Features (ENH)**

- Add ability to add font resources for 14 Adobe Core fonts in text widget annotations (#3624)

### **50.12.3 Bug Fixes (BUG)**

- Avoid invalid load for `ICCBased` `FlateDecode` images in mode 1 (#3619)

### **50.12.4 Robustness (ROB)**

- Fix AESV2 decryption when `/Length` missing in encrypt dict (#3629)
- Fix merging when annotations point to `NullObject` (#3613)
- Check for `self._info` being `None` in `compress_identical_objects` (#3612)

[Full Changelog](#)

## **50.13 Version 6.6.2, 2026-01-26**

### **50.13.1 Security (SEC)**

- Detect cyclic references when retrieving outlines (#3610)

[Full Changelog](#)

## **50.14 Version 6.6.1, 2026-01-25**

### **50.14.1 Robustness (ROB)**

- `/AcroForm` might be `NullObject` (#3601)
- Handle missing font bounding boxes gracefully (#3600)

[Full Changelog](#)



## 50.15 Version 6.6.0, 2026-01-09

### 50.15.1 Security (SEC)

- Improve handling of partially broken PDF files (#3594)

### 50.15.2 Deprecations (DEP)

- Block common page content modifications when assigned to reader (#3582)

### 50.15.3 New Features (ENH)

- Embellishments to generated text appearance streams (#3571)

### 50.15.4 Bug Fixes (BUG)

- Do not consider multi-byte BOM-like sequences as BOMs (#3589)

### 50.15.5 Robustness (ROB)

- Avoid empty FlateDecode outputs without warning (#3579)

### 50.15.6 Documentation (DOC)

- Add outlines documentation and link it in User Guide (#3511)

### 50.15.7 Developer Experience (DEV)

- Add PyPy 3.11 to test matrix and benchmarks (#3574)

### 50.15.8 Maintenance (MAINT)

- Fix compatibility with Pillow >= 12.1.0 (#3590)

[Full Changelog](#)

## 50.16 Version 6.5.0, 2025-12-21

### 50.16.1 New Features (ENH)

- Limit jbig2dec memory usage (#3576)
- FontDescriptor: Initiate from embedded font resource (#3551)

### 50.16.2 Robustness (ROB)

- Allow fallback to PBM files for jbig2dec without PNG support (#3567)
- Use warning instead of error for early EOD for RunLengthDecode (#3548)

### 50.16.3 Developer Experience (DEV)

- Test with macOS as well (#3401)

[Full Changelog](#)

## 50.17 Version 6.4.2, 2025-12-14

### 50.17.1 Bug Fixes (BUG)

- Fix KeyError when flattening form field without /Font in resources (#3554)

### 50.17.2 Robustness (ROB)

- Allow deleting non-existent annotations (#3559)

### 50.17.3 Documentation (DOC)

- Fix level of attachment heading (#3560)

[Full Changelog](#)

## 50.18 Version 6.4.1, 2025-12-07

### 50.18.1 Performance Improvements (PI)

- Optimize loop for layout mode text extraction (#3543)

### 50.18.2 Bug Fixes (BUG)

- Do not fail on choice field without /Opt key (#3540)

### 50.18.3 Documentation (DOC)

- Document possible issues with merge\_page and clipping (#3546)
- Add some notes about library security (#3545)

### 50.18.4 Maintenance (MAINT)

- Use CORE\_FONT\_METRICS for widths where possible (#3526)

[Full Changelog](#)

## 50.19 Version 6.4.0, 2025-11-23

### 50.19.1 Security (SEC)

- Reduce default limit for LZW decoding

### 50.19.2 New Features (ENH)

- Parse and format comb fields in text widget annotations (#3519)

### 50.19.3 Robustness (ROB)

- Silently ignore Adobe Ascii85 whitespace for suffix detection (#3528)

[Full Changelog](#)

## 50.20 Version 6.3.0, 2025-11-16

### 50.20.1 New Features (ENH)

- Wrap and align text in flattened PDF forms (#3465)

### 50.20.2 Bug Fixes (BUG)

- Fix missing “PreventGC” when cloning (#3520)
- Preserve JPEG image quality by default (#3516)

[Full Changelog](#)

## 50.21 Version 6.2.0, 2025-11-09

### 50.21.1 New Features (ENH)

- Add ‘strict’ parameter to PDFWriter (#3503)

### 50.21.2 Bug Fixes (BUG)

- PdfWriter.append fails when there are articles being None (#3509)

### 50.21.3 Documentation (DOC)

- Execute docs examples in CI (#3507)

[Full Changelog](#)

## 50.22 Version 6.1.3, 2025-10-22

### 50.22.1 Security (SEC)

- Allow limiting size of LZWDecode streams (#3502)
- Avoid infinite loop when reading broken DCT-based inline images (#3501)

### 50.22.2 Bug Fixes (BUG)

- PageObject.scale() scales media box incorrectly (#3489)

### 50.22.3 Robustness (ROB)

- Fail with explicit exception when image mode is an empty array (#3500)

[Full Changelog](#)

## 50.23 Version 6.1.2, 2025-10-19

### 50.23.1 Bug Fixes (BUG)

- Fix handling of zero-length StreamObject (#3485)

### 50.23.2 Robustness (ROB)

- Deal with wrong size for incremental PDF files (#3495)
- Improve handling for malformed cross-reference tables (#3483)

### 50.23.3 Developer Experience (DEV)

- Use released Python 3.14
- Use Mapping instead of dict in type hint of `update_page_form_field_values` (#3490)

[Full Changelog](#)

## 50.24 Version 6.1.1, 2025-09-28

### 50.24.1 Bug Fixes (BUG)

- Insert new embedded files in a sorted manner (#3477)
- Fix name tree handling for embedded files with Kids-based inputs (#3475)
- Make embedding files not break PDF/A-3 compliance (#3472)

### 50.24.2 Documentation (DOC)

- Document AFRelationship handling for PDF/A and provide constants (#3478)

[Full Changelog](#)

## 50.25 Version 6.1.0, 2025-09-21

### 50.25.1 New Features (ENH)

- Enhance XMP metadata handling with creation and setter methods (#3410)
- Add all font metrics for base 14 Type 1 PDF fonts (#3363)
- Allow deleting embedded files (#3461)
- Add support for Python in FIPS mode for document identifier (#3438)

### 50.25.2 Bug Fixes (BUG)

- Fix handling of UTF-16 encoded destination titles (#3463)
- Guard empty input to prevent IndexError (#3448)

### 50.25.3 Developer Experience (DEV)

- Fix type hint for XMP metadata setter to add bytes type (#3464)

[Full Changelog](#)

## 50.26 Version 6.0.0, 2025-08-11

### 50.26.1 Security (SEC)

- Limit decompressed size for FlateDecode filter (#3430)

### 50.26.2 Deprecations (DEP)

- Drop Python 3.8 support (#3412)

### 50.26.3 New Features (ENH)

- Move BlackIs1 functionality to tiff\_header (#3421)

### 50.26.4 Robustness (ROB)

- Skip Go-To actions without a destination (#3420)

### 50.26.5 Developer Experience (DEV)

- Update code style related libraries (#3414)
- Update mypy to 1.17.0 (#3413)
- Stop testing on Python 3.8 and start testing on Python 3.14 (#3411)

### 50.26.6 Maintenance (MAINT)

- Cleanup deprecations (#3424)

[Full Changelog](#)

## 50.27 Version 5.9.0, 2025-07-27

### 50.27.1 New Features (ENH)

- Automatically preserve links in added pages (#3298)
- Allow writing/updating all properties of an embedded file (#3374)

### 50.27.2 Bug Fixes (BUG)

- Fix XMP handling dropping indirect references (#3392)

### 50.27.3 Robustness (ROB)

- Deal with DecodeParms being empty list (#3388)

### 50.27.4 Documentation (DOC)

- Document how to read and modify XMP metadata (#3383)

[Full Changelog](#)

## 50.28 Version 5.8.0, 2025-07-13

### 50.28.1 New Features (ENH)

- Implement flattening for writer (#3312)

### 50.28.2 Bug Fixes (BUG)

- Unterminated object when using PdfWriter with incremental=True (#3345)

### 50.28.3 Robustness (ROB)

- Resolve some image extraction edge cases (#3371)
- Ignore faulty trailing newline during RLE decoding (#3355)
- Gracefully handle odd-length strings in parse\_bfchar (#3348)

### 50.28.4 Developer Experience (DEV)

- Modernize license specifiers (#3338)

### 50.28.5 Maintenance (MAINT)

- Reduce max-complexity of tool.ruff.lint.mccabe (#3365)
- Refactor text extraction code

[Full Changelog](#)

## 50.29 Version 5.7.0, 2025-06-29

### 50.29.1 Performance Improvements (PI)

- Performance optimization for LZW decoding (#3329)

### 50.29.2 Robustness (ROB)

- Flate decoding for streams with faulty tail bytes (#3332)
- dc\_creator could be a Bag as well (#3333)
- Handle tree being NullObject when retrieving named destinations (#3331)

### 50.29.3 Maintenance (MAINT)

- Move inline-image mappings to constants (#3328)

[Full Changelog](#)

## 50.30 Version 5.6.1, 2025-06-22

### 50.30.1 New Features (ENH)

- Add PDF/A XMP metadata support (#3314)

### 50.30.2 Robustness (ROB)

- Deal with annotations not being lists on merge (#3321)
- Handle NullObject for cmap encoding Differences entry (#3317)

### 50.30.3 Developer Experience (DEV)

- Update ruff to 0.12.0 (#3316)

[Full Changelog](#)

## 50.31 Version 5.6.0, 2025-06-01

### 50.31.1 New Features (ENH)

- Add basic support for JBIG2 by using jbig2dec (#3163)

### 50.31.2 Bug Fixes (BUG)

- Fix crashes by removing unnecessary line (#3293)
- Add delimiters to NameObject.renumber\_table (#3286)

### 50.31.3 Robustness (ROB)

- Handle DecodeParms being a NullObject (#3285)

### 50.31.4 Code Style (STY)

- Update to mypy 1.16.0 (#3300)

[Full Changelog](#)

## 50.32 Version 5.5.0, 2025-05-11

### 50.32.1 New Features (ENH)

- Add support for IndirectObject.**iter** (#3228)
- Allow filtering by font when removing text (#3216)

### 50.32.2 Bug Fixes (BUG)

- Add missing named destinations being ByteStringObjects (#3282)
- Get font information more reliably when removing text (#3252)
- T\* 2D Translation consistent with PDF 1.7 Spec (#3250)
- Add font stack to q/Q operations in layout mode (#3225)
- Avoid completely hiding image loading issues like exceeding image size limits (#3221)
- Using compress\_identical\_objects on transformed content duplicates differing content (#3197)
- Consider BlackIs1 parameter for CCITTFaxDecode filter (#3196)

### 50.32.3 Robustness (ROB)

- Deal with insufficient cm matrix during text extraction (#3283)
- Allow merging when annotations miss D entry (#3281)
- Fix merging documents if there are no Dests (#3280)
- Fix crash on malformed action in outline (#3278)

- Fix compression issues for removed images which might be None (#3246)
- Attempt to deal with non-rectangular FlateDecode streams (#3245)
- Handle some None values for broken PDF files (#3230)

#### **50.32.4 Developer Experience (DEV)**

- Multiple style improvements
- Update ruff to 0.11.0

#### **50.32.5 Maintenance (MAINT)**

- Conform ASCIIHexDecode implementation to specification (#3274)
- Modify comments of filters that do not use decode\_parms (#3260)

#### **50.32.6 Code Style (STY)**

- Simplify warnings & debugging in layout mode text extraction (#3271)
- Standardize mypy assert statements (#3276)

[Full Changelog](#)

### **50.33 Version 5.4.0, 2025-03-16**

#### **50.33.1 New Features (ENH)**

- Add support for IndirectObject.\_\_contains\_\_ (#3155)

#### **50.33.2 Bug Fixes (BUG)**

- Fix detection of inline images followed by names or numbers (#3173)

#### **50.33.3 Robustness (ROB)**

- Consider root objects without catalog type as fallback (#3175)
- Raise proper error on infinite loop when reading objects (#3169)

#### **50.33.4 Documentation (DOC)**

- Mention memory consumption of text extraction (#3168)

#### **50.33.5 Developer Experience (DEV)**

- Upgrade to ruff 0.10.0 (#3191)

[Full Changelog](#)

### **50.34 Version 5.3.1, 2025-03-02**

#### **50.34.1 Bug Fixes (BUG)**

- Use the correct name StandardEncoding for the predefined cmap (#3156)
- Handle inline images containing EI sequences (#3152)



- Fix check box value which should be name object (#3124)
- Fix stream position on inline image fallback extraction (#3120)
- Fix object count for incremental writer (#3117)

### 50.34.2 Robustness (ROB)

- Avoid index errors on empty lines in xref table (#3162)
- Improve handling of LZW decoder table overflow (#3159)
- Ignore non-numbers for width when building font width map (#3158)
- Avoid negative seek values when reading partially broken files (#3157)

### 50.34.3 Documentation (DOC)

- Fixed PageObject.images example usage for replacing image (#3149)

[Full Changelog](#)

## 50.35 Version 5.3.0, 2025-02-09

### 50.35.1 New Features (ENH)

- Handle attachments in /Kids and provide object-oriented API (#3108)

### 50.35.2 Bug Fixes (BUG)

- Handle annotations being None on merging (#3111)

### 50.35.3 Robustness (ROB)

- Prevent excessive layout mode text output from Type3 fonts (#3082)

### 50.35.4 Documentation (DOC)

- stefan6419846 becomes BDFL of pypdf (#3078)
- Tidy the visitor function description (#3086)

### 50.35.5 Developer Experience (DEV)

- Remove ignoring multiple Ruff rules
- Remove unused mutmut configuration (#3092)

### 50.35.6 Testing (TST)

- Fix warning assertions to use `pytest.warns()` (#3083)

[Full Changelog](#)

## 50.36 Version 5.2.0, 2025-01-26

### 50.36.1 Deprecations (DEP)

- Deprecate with replacement CCITParameters (#3019)
- Correct deprecation of interior\_color (#2947)

### 50.36.2 New Features (ENH)

- Support alternative (U)F names for embedded file retrieval (#3072)
- Adding support for reading .metadata.keywords (#2939)

### 50.36.3 Bug Fixes (BUG)

- Handle further Tf operators in text extraction layout mode (#3073)
- Ensure add\_metadata can deal with \_info = None (#3040)
- Handle IndirectObject in CCITTFaxDecode filter (#2965)
- Handle chained colorspace for inline images when no filter is set (#3008)
- Avoid extracting inline images twice and dropping other operators (#3002)
- Fixed reference of value with str.\_\_new\_\_ in TextStringObject (#2952)
- Handle indirect objects in font width calculations (#2967)
- Title sometimes is bytes and not str (#2930)
- Fix undefined variable for text extraction (regression) (#2934)
- Don't close stream passed to PdfWriter.write() (#2909)

### 50.36.4 Robustness (ROB)

- Handle zero height fonts when extracting text (#3075)
- Deal with content streams not containing streams (#3005)
- Gracefully handle some text operators when the operands are missing (#3006)
- Fall back to non-Adobe Ascii85 format for missing end markers (#3007)
- Ignore odd-length strings when processing cmap lines (#3009)
- Skip annotation destination being NullObject in PdfWriter (#2964)
- Skip destination page being None in PdfWriter (#2963)
- Fix infinite loop case when reading null objects within an Array
- Fixing infinite loop in ArrayObject read\_from\_stream (#2928)

### 50.36.5 Documentation (DOC)

- Add note about default line colors (#3014)

### 50.36.6 Developer Experience (DEV)

- Remove ignoring Ruff rule PGH004 (#3071)
- Tidy ignore array in tool.ruff.lint (#3069)
- Move Windows CI to Python 3.13 (#3003)
- Move to Ubuntu 22.04 (#3004)

### 50.36.7 Maintenance (MAINT)

- Fix formatting of warning message and include exception message (#3076)
- Narrow return type for `ContentStream.operations` (#2941)

### 50.36.8 Testing (TST)

- Fix image similarity for upcoming Ubuntu 24.04 (#3039)
- Replace broken Apache Tika Corpora urls (#3041)

### 50.36.9 Code Style (STY)

- Add form feed to WHITESPACES (#3054)
- Lots of small internal changes

[Full Changelog](#)

## 50.37 Version 5.1.0, 2024-10-27

### 50.37.1 New Features (ENH)

- Add `layout_mode_font_height_weight` argument to `PageObject.extract_text()` (#2920)

### 50.37.2 Bug Fixes (BUG)

- Fix font specifier for `FreeText` annotation (#2893)
- Line breaks are not generated due to incorrect calculation of text leading (#2890)
- Improve handling of spaces in text extraction (#2882)

### 50.37.3 Robustness (ROB)

- Soft failure for flate encode image mode 1 with wrong LUT size (#2900)

### 50.37.4 Documentation (DOC)

- Use latest package versions (#2907)
- Correct example of reading `FileAttachment` annotation (#2906)

### 50.37.5 Developer Experience (DEV)

- Update pinned requirements (#2918)
- Make `make_release.py` compatible with Windows environment (#2894)

### 50.37.6 Maintenance (MAINT)

- Remove references to outdated Python versions (#2919)
- Generalize the method of obtaining space\_code (#2891)
- Unnecessary character mapping process (#2888)
- New LZW decoding implementation (#2887)

### 50.37.7 Testing (TST)

- Add LzwCodec for encoding (#2883)

### 50.37.8 Code Style (STY)

- Capitalize error messages (#2903)
- Modify error messages in PdfWriter (#2902)

[Full Changelog](#)

## 50.38 Version 5.0.1, 2024-09-29

### 50.38.1 New Features (ENH)

- Add full parameter to PdfWriter constructor (#2865)

### 50.38.2 Bug Fixes (BUG)

- Update pyproject.toml with minimum Python version of 3.8 (#2859)
- Cope with unbalanced delimiters in dictionary object (#2878)
- Cope with encoding with too many differences (#2873)
- Missing spaces in extract\_text() method (#1328) (#2868)
- Tolerate truncated files and no warning when jumping startxref (#2855)

### 50.38.3 Robustness (ROB)

- Repair PDF with invalid Root object (#2880)
- Continue parsing dictionary object when error is detected (#2872)
- Merge documents with invalid pages in named destinations (#2857)
- Tolerate comments in arrays (#2856)

### 50.38.4 Developer Experience (DEV)

- Use latest Python version for benchmarking (#2879)

### 50.38.5 Maintenance (MAINT)

- Add tests to source distributions (#2874)
- Refactor \_update\_field\_annotation (#2862)

[Full Changelog](#)

## 50.39 Version 5.0.0, 2024-09-15

This version drops support for Python 3.7 (not maintained since July 2023), PdfMerger (use PdfWriter instead) and AnnotationBuilder (use annotations instead).

### 50.39.1 Deprecations (DEP)

- Remove the deprecated PdfMerger and AnnotationBuilder classes and other deprecations cleanup (#2813)
- Drop Python 3.7 support (#2793)

### 50.39.2 New Features (ENH)

- Add capability to remove /Info from PDF (#2820)
- Add incremental capability to PdfWriter (#2811)
- Add UniGB-UTF16 encodings (#2819)
- Accept utf strings for metadata (#2802)
- Report PdfReadError instead of RecursionError (#2800)
- Compress PDF files merging identical objects (#2795)

### 50.39.3 Bug Fixes (BUG)

- Fix sheared image (#2801)

### 50.39.4 Robustness (ROB)

- Robustify .set\_data() (#2821)
- Raise PdfReadError when missing /Root in trailer (#2808)
- Fix extract\_text() issues on damaged PDFs (#2760)
- Handle images with empty data when processing an image from bytes (#2786)

### 50.39.5 Developer Experience (DEV)

- Fix coverage uploads (#2832)
- Test against Python 3.13 (#2776)

[Full Changelog](#)

## 50.40 Version 4.3.1, 2024-07-21

### 50.40.1 Bug Fixes (BUG)

- Cope with Matrix entry in field annotations (#2736)

### 50.40.2 Robustness (ROB)

- Cope with fields with upside down box/rectangle (#2729)

### 50.40.3 Maintenance (MAINT)

- Add `deprecate_with_replacement` to `StreamObject.initializeFromD...` (#2728)
- Deal with cryptography>=43 moving ARC4 (#2765)

[Full Changelog](#)

## 50.41 Version 4.3.0, 2024-06-23

### 50.41.1 New Features (ENH)

- Accept ETen-B5 and UniCNS-UTF16 encodings (#2721)
- Add `decode_as_image()` to `ContentStreams` (#2615)
- context manager for `PdfReader` (#2666)
- Add capability to set font and size in fields (#2636)
- Allow to pass input file without named argument (#2576)

### 50.41.2 Bug Fixes (BUG)

- Fix deprecation for `Ressources` when using old constants (#2705)
- Fix images issue 4 bits encoding and LUT starting with UTF16\_BOM (#2675)
- Reading large compressed images takes huge time to process (#2644)
- Highlighted Text Cannot Be Printed (#2604)
- Fix `UnboundLocalError` on malformed pdf (#2619)

### 50.41.3 Robustness (ROB)

- Cope with missing Standard 14 fonts in fields (#2677)
- Improve inline image extraction (#2622)
- Cope with loops in Fields tree (#2656)
- Discard /I in choice fields for compatibility with Acrobat (#2614)
- Cope with some issues in pillow (#2595)
- Cope with some image extraction issues (#2591)

### 50.41.4 Documentation (DOC)

- Various improvements on docstrings and examples

### 50.41.5 Maintenance (MAINT)

- Deprecate `interiour_color` with replacement `interior_color` (#2706)
- Add `deprecate_with_replacement` to `PdfWriter.find_bookmark` (#2674)

### 50.41.6 Code Style (STY)

- Change Link to be a non-markup annotation (#2714)

[Full Changelog](#)

## 50.42 Version 4.2.0, 2024-04-07

### 50.42.1 New Features (ENH)

- Allow multiple charsets for NameObject.read\_from\_stream (#2585)
- Add support for /Kids in page labels (#2562)
- Allow to update fields on many pages (#2571)
- Tolerate PDF with invalid xref pointed objects (#2335)
- Add Enforce from PDF2.0 in viewer\_preferences (#2511)
- Add += and -= operators to ArrayObject (#2510)

### 50.42.2 Bug Fixes (BUG)

- Fix merge\_page sometimes generating unknown operator 'QQ' (#2588)
- Fix fields update where annotations are kids of field (#2570)
- Process CMYK images without a filter correctly (#2557)
- Extract text in layout mode without finding resources (#2555)
- Prevent recursive loop in some PDF files (#2505)

### 50.42.3 Robustness (ROB)

- Tolerate “truncated” xref (#2580)
- Replace error by warning for EOD in RunLengthDecode/ASCIHexDecode (#2334)
- Rebuild xref table if one entry is invalid (#2528)
- Robustify stream extraction (#2526)

### 50.42.4 Documentation (DOC)

- Update release process for latest changes (#2564)
- Encryption/decryption: Clone document instead of copying all pages (#2546)
- Minor improvements (#2542)
- Update annotation list (#2534)
- Update references and formatting (#2529)
- Correct threads reference, plus minor changes (#2521)
- Minor readability increases (#2515)
- Simplify PaperSize examples (#2504)
- Minor improvements (#2501)

### 50.42.5 Developer Experience (DEV)

- Remove unused dependencies (#2572)
- Remove page labels PR link from message (#2561)
- Fix changelog generator regarding whitespace and handling of “Other” group (#2492)
- Add REL to known PR prefixes (#2554)
- Release using the REL commit instead of git tag (#2500)
- Unify code between PdfReader and PdfWriter (#2497)
- Bump softprops/action-gh-release from 1 to 2 (#2514)

### 50.42.6 Maintenance (MAINT)

- Ressources → Resources (and internal name childs) (#2550)
- Fix typos found by codespell (#2549)
- Update Read the Docs configuration (#2538)
- Add root\_object, \_info and \_ID to PdfReader (#2495)

### 50.42.7 Testing (TST)

- Allow loading truncated images if required (#2586)
- Fix download issues from #2562 (#2578)
- Improve test\_get\_contents\_from\_nullobject to show real use-case (#2524)
- Add missing test annotations (#2507)

[Full Changelog](#)

## 50.43 Version 4.1.0, 2024-03-03

Generating name objects (`NameObject`) without a leading slash is considered deprecated now. Previously, just a plain warning would be logged, leading to possibly invalid PDF files. According to our deprecation policy, this will log a *DeprecationWarning* for now.

### 50.43.1 New Features (ENH)

- Add `get_pages_from_field` (#2494)
- Add `reattach_fields` function (#2480)
- Automatic access to pointed object for `IndirectObject` (#2464)

### 50.43.2 Bug Fixes (BUG)

- Missing error on name without leading / (#2387)
- `encode_pdfdocencoding()` always returns bytes (#2440)
- BI in text content identified as image tag (#2459)



### 50.43.3 Robustness (ROB)

- Missing basefont entry in type 3 font (#2469)

### 50.43.4 Documentation (DOC)

- Improve lossless compression example (#2488)
- Amend robustness documentation (#2479)

### 50.43.5 Developer Experience (DEV)

- Fix changelog for UTF-8 characters (#2462)

### 50.43.6 Maintenance (MAINT)

- Add `_get_page_number_from_indirect` in writer (#2493)
- Remove user assignment for feature requests (#2483)
- Remove reference to old 2.0.0 branch (#2482)

### 50.43.7 Testing (TST)

- Fix benchmark failures (#2481)
- Broken test due to expired test file URL (#2468)
- Resolve file naming conflict in `test_iss1767` (#2445)

[Full Changelog](#)

## 50.44 Version 4.0.2, 2024-02-18

### 50.44.1 Bug Fixes (BUG)

- Use `NumberObject` for `/Border` elements of annotations (#2451)

[Full Changelog](#)

## 50.45 Version 4.0.1, 2024-01-28

### 50.45.1 Bug Fixes (BUG)

- layout mode text extraction `ZeroDivisionError` (#2417)

### 50.45.2 Testing (TST)

- Skip tests using `fpdf2` if it's not installed (#2419)

[Full Changelog](#)

## 50.46 Version 4.0.0, 2024-01-19

### 50.46.1 Deprecations (DEP)

- Drop Python 3.6 support (#2369)

- Remove deprecated code (#2367)
- Remove deprecated XMP properties (#2386)

### **50.46.2 New Features (ENH)**

- Add “layout” mode for text extraction (#2388)
- Add Jupyter Notebook integration for PdfReader (#2375)
- Improve/rewrite PDF permission retrieval (#2400)

### **50.46.3 Bug Fixes (BUG)**

- PdfWriter.add\_uri was setting the wrong type (#2406)
- Add support for GBK2K cmaps (#2385)

### **50.46.4 Maintenance (MAINT)**

- Return None instead of -1 when page is not attached (#2376)
- Complete FileSpecificationDictionaryEntries constants (#2416)
- Replace warning with logging.error (#2377)

[Full Changelog](#)

## **50.47 Version 3.17.4, 2023-12-24**

### **50.47.1 Bug Fixes (BUG)**

- Handle IndirectObject as image filter (#2355)

[Full Changelog](#)

## **50.48 Version 3.17.3, 2023-12-17**

### **50.48.1 Robustness (ROB)**

- Out-of-bounds issue in handle\_tj (text extraction) (#2342)

### **50.48.2 Developer Experience (DEV)**

- Make make\_release.py easier to configure (#2348)

### **50.48.3 Maintenance (MAINT)**

- Bump actions/download-artifact from 3 to 4 (#2344)

[Full Changelog](#)

## **50.49 Version 3.17.2, 2023-12-10**

### **50.49.1 Bug Fixes (BUG)**

- Cope with deflated images with CMYK Black Only (#2322)

- Handle indirect objects as parameters for CCITTFaxDecode (#2307)
- check words length in \_cmap type1\_alternative function (#2310)

### 50.49.2 Robustness (ROB)

- Relax flate decoding for too many lookup values (#2331)
- Let \_build\_destination skip in case of missing /D key (#2018)

[Full Changelog](#)

## 50.50 Version 3.17.1, 2023-11-14

### 50.50.1 Bug Fixes (BUG)

- Mediabox expansion size when applying non-right angle rotation (#2282)

### 50.50.2 Robustness (ROB)

- MissingWidth is IndirectObject (#2288)
- Initialize states array with an empty value (#2280)

[Full Changelog](#)

## 50.51 Version 3.17.0, 2023-10-29

### 50.51.1 Security (SEC)

- Infinite recursion when using PdfWriter(clone\_from=reader) (#2264)

### 50.51.2 New Features (ENH)

- Add parameter to select images to be removed (#2214)

### 50.51.3 Bug Fixes (BUG)

- Correctly handle image mode 1 with FlateDecode (#2249)
- Error when filling a value with parentheses #2268 (#2269)
- Handle empty root outline (#2239)

[Full Changelog](#)

## 50.52 Version 3.16.4, 2023-10-10

### 50.52.1 Bug Fixes (BUG)

- Avoid exceeding recursion depth when retrieving image mode (#2251)

[Full Changelog](#)

## **50.53 Version 3.16.3, 2023-10-08**

### **50.53.1 Bug Fixes (BUG)**

- Invalid cm/tm in visitor functions (#2206)
- Encrypt / decrypt Stream object dictionaries (#2228)
- Support nested color spaces for the /DeviceN color space (#2241)
- Images property fails if NullObject in list (#2215)

### **50.53.2 Developer Experience (DEV)**

- Unify mypy options and warn redundant workarounds (#2223)

[Full Changelog](#)

## **50.54 Version 3.16.2, 2023-09-24**

### **50.54.1 Bug Fixes (BUG)**

- PDF size increases because of too high float writing precision (#2213)
- Fix test\_watermarking\_reportlab\_rendering() (#2203)

[Full Changelog](#)

## **50.55 Version 3.16.1, 2023-09-17**

The ‘rename PdfWriter.create\_viewer\_preference to PdfWriter.create\_viewer\_preferences (#2190)’ could be a breaking change for you, if you use it. As it was only introduced last week I’m confident enough that nobody will be affected though. Hence only the patch update.

### **50.55.1 Bug Fixes (BUG)**

- Missing new line in extract\_text with cm operations (#2142)
- \_get\_fonts not processing properly CIDFonts and annotations (#2194)

### **50.55.2 Maintenance (MAINT)**

- Rename PdfWriter.create\_viewer\_preference to PdfWriter.create\_viewer\_preferences (#2190)

[Full Changelog](#)

## **50.56 Version 3.16.0, 2023-09-10**

### **50.56.1 Security (SEC)**

- Infinite recursion caused by IndirectObject clone (#2156)

### 50.56.2 New Features (ENH)

- Ease access to ViewerPreferences (#2144)

### 50.56.3 Bug Fixes (BUG)

- Catch the case where w[0] is an IndirectObject instead of an int (#2154)
- Cope with indirect objects in filters and remove deprecated code (#2177)
- Accept tabs in cmap (#2174) / cope with extra space (#2151)
- Merge pages without resources (#2150)
- getcontents() shall return None if contents is NullObject (#2161)
- Fix conversion from 1 to LA (#2175)

### 50.56.4 Robustness (ROB)

- Accept XYZ with no arguments (#2178)

[Full Changelog](#)

## 50.57 Version 3.15.5, 2023-09-03

### 50.57.1 Bug Fixes (BUG)

- Cope with missing /I in articles (#2134)
- Fix image look-up table in EncodedStreamObject (#2128)
- remove\_images not operating in sub level forms (#2133)

### 50.57.2 Robustness (ROB)

- Cope with damaged PDF (#2129)

[Full Changelog](#)

## 50.58 Version 3.15.4, 2023-08-27

### 50.58.1 Performance Improvements (PI)

- Making pypdf as fast as pdfrw (#2086)

### 50.58.2 Maintenance (MAINT)

- Relax typing\_extensions version (#2104)

[Full Changelog](#)

## 50.59 Version 3.15.3, 2023-08-26

### 50.59.1 Bug Fixes (BUG)

- Check version of crypt provider (#2115)
- TypeError: can't concat str to bytes (#2114)

- Require flit\_core >= 3.9 (#2091)

[Full Changelog](#)

## **50.60 Version 3.15.2, 2023-08-20**

### **50.60.1 Security (SEC)**

- Avoid endless recursion of reading damaged PDF file (#2093)

### **50.60.2 Performance Improvements (PI)**

- Reuse content stream (#2101)

### **50.60.3 Maintenance (MAINT)**

- Make ParseError inherit from PyPdfError (#2097)

[Full Changelog](#)

## **50.61 Version 3.15.1, 2023-08-13**

### **50.61.1 Performance Improvements (PI)**

- optimize \_decode\_png\_prediction (#2068)

### **50.61.2 Bug Fixes (BUG)**

- Fix incorrect tm\_matrix in call to visitor\_text (#2060)
- Writing German characters into form fields (#2047)
- Prevent stall when accessing image in corrupted pdf (#2081)
- append() fails when articles do not have /T (#2080)

### **50.61.3 Robustness (ROB)**

- Cope with xref not followed by separator (#2083)

[Full Changelog](#)

## **50.62 Version 3.15.0, 2023-08-06**

### **50.62.1 New Features (ENH)**

- Add level parameter to compress\_content\_streams (#2044)
- Process /uniHHHH for text\_extract (#2043)

### **50.62.2 Bug Fixes (BUG)**

- Fix AnnotationBuilder.link (#2066)
- JPX image without ColorSpace (#2062)
- Added check for field /Info when cloning reader document (#2055)
- Fix indexed/CMYK images (#2039)

### 50.62.3 Maintenance (MAINT)

- Cryptography as primary dependency (#2053)

[Full Changelog](#)

## 50.63 Version 3.14.0, 2023-07-29

### 50.63.1 New Features (ENH)

- Accelerate image list keys generation (#2014)
- Use `cryptography` for encryption/decryption as a fallback for `PyCryptodome` (#2000)
- Extract LaTeX characters (#2016)
- `ASCIIStr.decode` now returns bytes instead of str (#1994)

### 50.63.2 Bug Fixes (BUG)

- Add `RunLengthDecode` filter (#2012)
- Process `/Separation ColorSpace` (#2007)
- Handle single element `ColorSpace` list (#2026)
- Process lookup decoded as `TextStringObjects` (#2008)

### 50.63.3 Robustness (ROB)

- Cope with garbage collector during cloning (#1841)

### 50.63.4 Maintenance (MAINT)

- Cleanup of annotations (#1745)

[Full Changelog](#)

## 50.64 Version 3.13.0, 2023-07-23

### 50.64.1 New Features (ENH)

- Add `is_open` in outlines in `PdfReader` and `PdfWriter` (#1960)

### 50.64.2 Bug Fixes (BUG)

- Search `/DA` in hierarchy fields (#2002)
- Cope with different ISO date length (#1999)
- Decode Black only/CMYK deviceN images (#1984)
- Process CMYK in deflate images (#1977)

### 50.64.3 Developer Experience (DEV)

- Add `mypy` to pre-commit (#2001)
- Release automation (#1991, #1985)

[Full Changelog](#)

## 50.65 Version 3.12.2, 2023-07-16

### 50.65.1 Bug Fixes (BUG)

- Accept calRGB and calGray color\_spaces (#1968)
- Process 2bits and 4bits images (#1967)
- Check for AcroForm and ensure it is not None (#1965)

### 50.65.2 Developer Experience (DEV)

- Automate the release process (#1970)

[Full Changelog](#)

## 50.66 Version 3.12.1, 2023-07-09

### 50.66.1 Bug Fixes (BUG)

- Prevent updating page contents after merging page (stamping/watermarking) (#1952)
- 
- Inverse color in CMYK images (#1947)
- Dates conversion not working with Z00'00' (#1946)
- Support UTF-16-LE Strings (#1884)

[Full Changelog](#)

## 50.67 Version 3.12.0, 2023-07-02

### 50.67.1 New Features (ENH)

- Add AES support for encrypting PDF files (#1918, #1935, #1936, #1938)
- Add page deletion feature to PdfWriter (#1843)

### 50.67.2 Bug Fixes (BUG)

- PdfReader.get\_fields() attempts to delete non-existing index “/Off” (#1933)
- Remove unused objects when cloning\_from (#1926)
- Add the TK.SIZE into the trailer (#1911)
- add\_named\_destination() maintains named destination list sort order (#1930)

[Full Changelog](#)

## 50.68 Version 3.11.1, 2023-06-25

### 50.68.1 Bug Fixes (BUG)

- Cascaded filters in image objects (#1913)
- Append pdf with named destination using numbers for pages (#1858)



- Ignore “/B” fields only on pages in PdfWriter.append() (#1875)

[Full Changelog](#)

## 50.69 Version 3.11.0, 2023-06-23

### 50.69.1 New Features (ENH)

- Add page\_number property (#1856)

### 50.69.2 Bug Fixes (BUG)

- File expansion when updating with Page Contents (#1906)
- Missing Alternate in indexed/ICCbased colorspaces (#1896)

[Full Changelog](#)

## 50.70 Version 3.10.0, 2023-06-18

### 50.70.1 New Features (ENH)

- Extraction of inline images (#1850)
- Add capability to replace image (#1849)
- Extend images interface by returning an ImageFile(File) class (#1848)
- Add set\_data to EncodedStreamObject (#1854)

### 50.70.2 Bug Fixes (BUG)

- Fix RGB FlateEncode Images(PNG) and transparency (#1834)
- Generate static appearance for fields (#1864)

[Full Changelog](#)

## 50.71 Version 3.9.1, 2023-06-04

### 50.71.1 Deprecations (DEP)

- Deprecate PdfMerger (#1866)

### 50.71.2 Bug Fixes (BUG)

- Ignore UTF-8 decode errors (#1865)

### 50.71.3 Robustness (ROB)

- Handle missing /Type entry in Page tree (#1859)

[Full Changelog](#)

## 50.72 Version 3.9.0, 2023-05-21

### 50.72.1 New Features (ENH)

- Simplify metadata input (Document Information Dictionary) (#1851)
- Extend cmap compatibility to GBK\_EUC\_H/V (#1812)

### 50.72.2 Bug Fixes (BUG)

- Prevent infinite loop when no character follows after a comment (#1828)
- get\_contents does not return ContentStream (#1847)
- Accept XYZ destination with zoom missing (default to zoom=0.0) (#1844)
- Cope with 1 Bit images (#1815)

### 50.72.3 Robustness (ROB)

- Handle missing /Type entry in Page tree (#1845)

### 50.72.4 Documentation (DOC)

- Expand file size explanations (#1835)
- Add comparison with pdfplumber (#1837)
- Clarify that PyPDF2 is dead (#1827)
- Add Hunter King as Contributor for #1806

### 50.72.5 Maintenance (MAINT)

- Refactor internal Encryption class (#1821)
- Add R parameter to generate\_values (#1820)
- Make encryption\_key parameter of write\_to\_stream optional (#1819)
- Prepare for adding AES encryption support (#1818)

[Full Changelog](#)

## 50.73 Version 3.8.1, 2023-04-23

### 50.73.1 Bug Fixes (BUG)

- Convert color space before saving (#1802)

### 50.73.2 Documentation (DOC)

- PDF/A (#1807)
- Use append instead of add\_page
- Document core mechanics of pypdf (#1783)

[Full Changelog](#)

## 50.74 Version 3.8.0, 2023-04-16

### 50.74.1 New Features (ENH)

- Add transform method to Transformation class (#1765)
- Cope with UC2 fonts in text\_extraction (#1785)

### 50.74.2 Robustness (ROB)

- Invalid startxref pointing 1 char before (#1784)

### 50.74.3 Maintenance (MAINT)

- Mark code handling old parameters as deprecated (#1798)

[Full Changelog](#)

## 50.75 Version 3.7.1, 2023-04-09

### 50.75.1 Security (SEC)

- Warn about PDF encryption security (#1755)

### 50.75.2 Robustness (ROB)

- Prevent loop in Cloning (#1770)
- Capture UnicodeDecodeError at PdfReader.pdf\_header (#1768)

### 50.75.3 Documentation (DOC)

- Add .readthedocs.yaml and bump docs dependencies using `tox -e deps` (#1750, #1752)

### 50.75.4 Developer Experience (DEV)

- Make `make_changelog.py` idempotent

### 50.75.5 Maintenance (MAINT)

- Move generation of file identifiers to a method (#1760)

### 50.75.6 Testing (TST)

- Add xmp test (#1775)

[Full Changelog](#)

## 50.76 Version 3.7.0, 2023-03-26

### 50.76.1 Security (SEC)

- Use Python's secrets module instead of random module (#1748)

### 50.76.2 New Features (ENH)

- Add AnnotationBuilder.highlight text markup annotation (#1740)
- Add AnnotationBuilder.popup (#1665)
- Add AnnotationBuilder.polyline annotation support (#1726)
- Add clone\_from parameter in PdfWriter constructor (#1703)

### 50.76.3 Bug Fixes (BUG)

- ‘DictionaryObject’ object has no attribute ‘indirect\_reference’ (#1729)

### 50.76.4 Robustness (ROB)

- Handle params NullObject in decode\_stream\_data (#1738)

### 50.76.5 Documentation (DOC)

- Project scope (#1743)

### 50.76.6 Maintenance (MAINT)

- Add AnnotationFlag (#1746)
- Add LazyDict.str (#1727)

[Full Changelog](#)

## 50.77 Version 3.6.0, 2023-03-18

### 50.77.1 New Features (ENH)

- Extend PdfWriter.append() to PageObjects (#1704)
- Support qualified names in update\_page\_form\_field\_values (#1695)

### 50.77.2 Robustness (ROB)

- Tolerate streams without length field (#1717)
- Accept DictionaryObject in /D of NamedDestination (#1720)
- Widths def in cmap calls IndirectObject (#1719)

[Full Changelog](#)

## 50.78 Version 3.5.2, 2023-03-12

We discovered that compress\_content\_stream has to be applied to a page of the PdfWriter. It may not be applied to a page of the PdfReader!

### 50.78.1 Bug Fixes (BUG)

- compress\_content\_stream not readable in Adobe Acrobat (#1698)
- Pass logging parameters correctly in set\_need\_appearances\_writer (#1697)
- Write /Root/AcroForm in set\_need\_appearances\_writer (#1639)

### 50.78.2 Robustness (ROB)

- Allow more whitespaces within linearized file (#1701)

[Full Changelog](#)

## 50.79 Version 3.5.1, 2023-03-05

### 50.79.1 Robustness (ROB)

- Some attributes not copied in DictionaryObject.clone (#1635)
- Allow merging multiple time pages with annots (#1624)

### 50.79.2 Testing (TST)

- Replace pytest.mark.external by enable\_socket (#1657)

[Full Changelog](#)

## 50.80 Version 3.5.0, 2023-02-26

### 50.80.1 New Features (ENH)

- Add reader.attachments public interface (#1611, #1661)
- Add PdfWriter.remove\_objects\_from\_page(page: PageObject, to\_delete: ObjectDeletionFlag) (#1648)
- Allow free-text annotation to have transparent border/background (#1664)

### 50.80.2 Bug Fixes (BUG)

- Allow decryption with empty password for AlgV5 (#1663)
- Let PdfWriter.pages return PageObject after calling clone\_document\_from\_reader() (#1613)
- Invalid font pointed during merge\_resources (#1641)

### 50.80.3 Robustness (ROB)

- Cope with invalid objects in IndirectObject.clone (#1637)
- Improve tolerance to invalid Names/Dests (#1658)
- Decode encoded values in get\_fields (#1636)
- Let PdfWriter.merge cope with missing “/Fields” (#1628)

[Full Changelog](#)

## 50.81 Version 3.4.1, 2023-02-12

### 50.81.1 Bug Fixes (BUG)

- Switch from trimbox to cropbox when merging pages (#1622)
- Text extraction not working with one glyph to char sequence (#1620)

### **50.81.2 Robustness (ROB)**

- Fix 2 cases of “object has no attribute ‘indirect\_reference’” (#1616)

### **50.81.3 Testing (TST)**

- Add multiple retry on get\_url for external PDF downloads (#1626)

[Full Changelog](#)

## **50.82 Version 3.4.0, 2023-02-05**

NOTICE: pypdf changed the way it represents numbers parsed from PDF files. pypdf<3.4.0 represented numbers as Decimal, pypdf>=3.4.0 represents them as floats. Several other PDF libraries to this, as well as many PDF viewers. We hope to fix issues with too high precision like this and get a speed boost. In case your PDF documents rely on more than 18 decimals of precision you should check if it still works as expected. To clarify: This does not affect the text shown in PDF documents. It affects numbers, e.g. when graphics are drawn on the PDF or very exact positions are used. Typically, 5 decimals should be enough.

### **50.82.1 New Features (ENH)**

- Enable merging forms with overlapping names (#1553)
- Add ‘over’ parameter to merge\_transformend\_page & co (#1567)

### **50.82.2 Bug Fixes (BUG)**

- Fix getter of the PageObject.rotation property with an indirect object (#1602)
- Restore merge\_transformed\_page & co (#1567)
- Replace decimal by float (#1563)

### **50.82.3 Robustness (ROB)**

- PdfWriter.remove\_images: /Contents might not be in page\_ref (#1598)

### **50.82.4 Developer Experience (DEV)**

- Introduce ruff (#1586, #1609)

### **50.82.5 Maintenance (MAINT)**

- Remove decimal (#1608)

[Full Changelog](#)

## **50.83 Version 3.3.0, 2023-01-22**

### **50.83.1 New Features (ENH)**

- Add page label support to PdfWriter (#1558)
- Accept inline images with space before EI (#1552)
- Add circle annotation support (#1556)
- Add polygon annotation support (#1557)

- Make merging pages produce a deterministic PDF (#1542, #1543)

### 50.83.2 Bug Fixes (BUG)

- Fix error in cmap extraction (#1544)
- Remove erroneous assertion check (#1564)
- Fix dictionary access of optional page label keys (#1562)

### 50.83.3 Robustness (ROB)

- Set ignore\_eof=True for read\_until\_regex (#1521)

### 50.83.4 Documentation (DOC)

- Paper size (#1550)

### 50.83.5 Developer Experience (DEV)

- Fix broken combination of dependencies of docs.txt
- Annotate tests appropriately (#1551)

[Full Changelog](#)

## 50.84 Version 3.2.1, 2023-01-08

### 50.84.1 Bug Fixes (BUG)

- Accept hierarchical fields (#1529)

### 50.84.2 Documentation (DOC)

- Use google style docstrings (#1534)
- Fix linked markdown documents (#1537)

### 50.84.3 Developer Experience (DEV)

- Update docs config (#1535)

## 50.85 Version 3.2.0, 2022-12-31

### 50.85.1 Performance Improvement (PI)

- Help the specializing adaptive interpreter (#1522)

### 50.85.2 New Features (ENH)

- Add support for page labels (#1519)

### **50.85.3 Bug Fixes (BUG)**

- upgrade clone\_document\_root (#1520)

[Full Changelog](#)

## **50.86 Version 3.1.0, 2022-12-23**

Move PyPDF2 to pypdf (#1513). This now it's all lowercase, no number in the name. For installation and for import. PyPDF2 will no longer receive updates. The community should move back to its roots.

If you were still using pyPdf or PyPDF2 < 2.0.0, I recommend reading the migration guide: <https://pypdf.readthedocs.io/en/latest/user/migration-1-to-2.html>

pypdf==3.1.0 is only different from PyPDF2==3.0.0 in the package name. Replacing “PyPDF2” by “pypdf” should be enough if you migrate from PyPDF2==3.0.0 to pypdf==3.1.0.

[Full Changelog](#)

## **50.87 Version 3.0.0, 2022-12-22**

### **50.87.1 BREAKING CHANGES**

- Deprecate features with PyPDF2==3.0.0 (#1489)
- Refactor Fit / Zoom parameters (#1437)

### **50.87.2 New Features (ENH)**

- Add Cloning (#1371)
- Allow int for indirect\_reference in PdfWriter.get\_object (#1490)

### **50.87.3 Documentation (DOC)**

- How to read PDFs from S3 (#1509)
- Make MyST parse all links as simple hyperlinks (#1506)
- Changed ‘latest’ for ‘stable’ generated docs (#1495)
- Adjust deprecation procedure (#1487)

### **50.87.4 Maintenance (MAINT)**

- Use typing.IO for file streams (#1498)

[Full Changelog](#)

## **50.88 Version 2.12.1, 2022-12-10**

### **50.88.1 Documentation (DOC)**

- Deduplicate extract\_text docstring (#1485)
- How to cite PyPDF2 (#1476)



## 50.88.2 Maintenance (MAINT)

Consistency changes:

- indirect\_ref/ido indirect\_reference, dest page\_destination (#1467)
- owner\_pwd/user\_pwd owner\_password/user\_password (#1483)
- position page\_number in Merger.merge (#1482)
- indirect\_ref indirect\_reference (#1484)

[Full Changelog](#)

## 50.89 Version 2.12.0, 2022-12-10

### 50.89.1 New Features (ENH)

- Add support to extract gray scale images (#1460)
- Add 'threads' property to PdfWriter (#1458)
- Add 'open\_destination' property to PdfWriter (#1431)
- Make PdfReader.get\_object accept integer arguments (#1459)

### 50.89.2 Bug Fixes (BUG)

- Scale PDF annotations (#1479)

### 50.89.3 Robustness (ROB)

- Padding issue with AES encryption (#1469)
- Accept empty object as null objects (#1477)

### 50.89.4 Documentation (DOC)

- Add module documentation the PaperSize class (#1447)

### 50.89.5 Maintenance (MAINT)

- Use 'page\_number' instead of 'pagenum' (#1365)
- Add List of pages to PageRangeSpec (#1456)

### 50.89.6 Testing (TST)

- Cleanup temporary files (#1454)
- Mark test\_tounicode\_is\_identity as external (#1449)
- Use Ubuntu 20.04 for running CI test suite (#1452)

[Full Changelog](#)

## 50.90 Version 2.11.2, 2022-11-20

### 50.90.1 New Features (ENH)

- Add `remove_from_tree` (#1432)
- Add `AnnotationBuilder.rectangle` (#1388)

### 50.90.2 Bug Fixes (BUG)

- JavaScript executed twice (#1439)
- `ToUnicode` stores `/Identity-H` instead of stream (#1433)
- Declare Pillow as optional dependency (#1392)

### 50.90.3 Developer Experience (DEV)

- Link 'Full Changelog' automatically
- Modify `read_string_from_stream` to a benchmark (#1415)
- Improve error reporting of `read_object` (#1412)
- Test Python 3.11 (#1404)
- Extend Flake8 ignore list (#1410)
- Use correct pytest markers (#1407)
- Move project configuration to `pyproject.toml` (#1382)

[Full Changelog](#)

## 50.91 Version 2.11.1, 2022-10-09

### 50.91.1 Bug Fixes (BUG)

- `td matrix` (#1373)
- Cope with `cmap` from #1322 (#1372)

### 50.91.2 Robustness (ROB)

- Cope with `str` returned from `get_data` in `cmap` (#1380)

[Full Changelog](#)

## 50.92 Version 2.11.0, 2022-09-25

### 50.92.1 New Features (ENH)

- Addition of optional visitor-functions in `extract_text()` (#1252)
- Add `metadata.creation_date` and `modification_date` (#1364)
- Add `PageObject.images` attribute (#1330)

### 50.92.2 Bug Fixes (BUG)

- Lookup index in `_xobj_to_image` can be `ByteStringObject` (#1366)
- 'IndexError: index out of range' when using `extract_text` (#1361)
- Errors in `transfer_rotation_to_content()` (#1356)

### 50.92.3 Robustness (ROB)

- Ensure `update_page_form_field_values` does not fail if no fields (#1346)

[Full Changelog](#)

## 50.93 Version 2.10.9, 2022-09-18

### 50.93.1 New Features (ENH)

- Add rotation property and `transfer_rotate_to_content` (#1348)

### 50.93.2 Performance Improvements (PI)

- Avoid string concatenation with large embedded base64-encoded images (#1350)

### 50.93.3 Bug Fixes (BUG)

- Format floats using their intrinsic decimal precision (#1267)

### 50.93.4 Robustness (ROB)

- Fix `merge_page` for pages without resources (#1349)

[Full Changelog](#)

## 50.94 Version 2.10.8, 2022-09-14

### 50.94.1 New Features (ENH)

- Add `PageObject.user_unit` property (#1336)

### 50.94.2 Robustness (ROB)

- Improve `NameObject` reading/writing (#1345)

[Full Changelog](#)

## 50.95 Version 2.10.7, 2022-09-11

### 50.95.1 Bug Fixes (BUG)

- Fix Error in transformations (#1341)
- Decode #23 in `NameObject` (#1342)

### **50.95.2 Testing (TST)**

- Use `pytest.warns()` for warnings, and `.raises()` for exceptions (#1325)

[Full Changelog](#)

## **50.96 Version 2.10.6, 2022-09-09**

### **50.96.1 Robustness (ROB)**

- Fix infinite loop due to Invalid object (#1331)
- Fix image extraction issue with superfluous whitespaces (#1327)

[Full Changelog](#)

## **50.97 Version 2.10.5, 2022-09-04**

### **50.97.1 New Features (ENH)**

- Process XRefStm (#1297)
- Auto-detect RTL for text extraction (#1309)

### **50.97.2 Bug Fixes (BUG)**

- Avoid scaling cropbox twice (#1314)

### **50.97.3 Robustness (ROB)**

- Fix offset correction in revised PDF (#1318)
- Crop data of /U and /O in encryption dictionary to 48 bytes (#1317)
- MultiLine bfrange in cmap (#1299)
- Cope with 2 digit codes in bfchar (#1310)
- Accept ‘/annn’ charset as ASCII code (#1316)
- Log errors during Float / NumberObject initialization (#1315)
- Cope with corrupted entries in xref table (#1300)

### **50.97.4 Documentation (DOC)**

- Migration guide (PyPDF2 1.x 2.x) (#1324)
- Creating a coverage report (#1319)
- Fix AnnotationBuilder.free\_text example (#1311)
- Fix usage of page.scale by replacing it with page.scale\_by (#1313)

### **50.97.5 Maintenance (MAINT)**

- PdfReaderProtocol (#1303)
- Throw PdfReadError if Trailer can't be read (#1298)
- Remove catching OverflowException (#1302)

[Full Changelog](#)

## 50.98 Version 2.10.4, 2022-08-28

### 50.98.1 Robustness (ROB)

- Fix errors/warnings on no /Resources within extract\_text (#1276)
- Add required line separators in ContentStream ArrayObjects (#1281)

### 50.98.2 Maintenance (MAINT)

- Use NameObject idempotency (#1290)

### 50.98.3 Testing (TST)

- Rectangle deletion (#1289)
- Add workflow tests (#1287)
- Remove files after tests ran (#1286)

### 50.98.4 Packaging (PKG)

- Add minimum version for typing\_extensions requirement (#1277)

[Full Changelog](#)

## 50.99 Version 2.10.3, 2022-08-21

### 50.99.1 Robustness (ROB)

- Decrypt returns empty bytestring (#1258)

### 50.99.2 Developer Experience (DEV)

- Modify CI to better verify built package contents (#1244)

### 50.99.3 Maintenance (MAINT)

- Remove 'mine' as PdfMerger always creates the stream (#1261)
- Let PdfMerger.\_create\_stream raise NotImplemented (#1251)
- password param of \_security.\_alg32(...) is only a string, not bytes (#1259)
- Remove unreachable code in read\_block\_backwards (#1250) and sign function in \_extract\_text (#1262)

### 50.99.4 Testing (TST)

- Delete annotations (#1263)
- Close PdfMerger in tests (#1260)
- PdfReader.xmp\_metadata workflow (#1257)
- Various PdfWriter (Layout, Bookmark deprecation) (#1249)

[Full Changelog](#)

## 50.100 Version 2.10.2, 2022-08-15

BUG: Add PyPDF2.generic to PyPI distribution

## 50.101 Version 2.10.1, 2022-08-15

### 50.101.1 Bug Fixes (BUG)

- TreeObject.remove\_child had a non-PdfObject assignment for Count (#1233, #1234)
- Fix stream truncated prematurely (#1223)

### 50.101.2 Documentation (DOC)

- Fix docstring formatting (#1228)

### 50.101.3 Maintenance (MAINT)

- Split generic.py (#1229)

### 50.101.4 Testing (TST)

- Decrypt AlgV4 with owner password (#1239)
- AlgV5.generate\_values (#1238)
- TreeObject.remove\_child / empty\_tree (#1235, #1236)
- create\_string\_object (#1232)
- Free-Text annotations (#1231)
- generic.\_base (#1230)
- Strict get fonts (#1226)
- Increase PdfReader coverage (#1219, #1225)
- Increase PdfWriter coverage (#1237)
- 100% coverage for utils.py (#1217)
- PdfWriter exception non-binary stream (#1218)
- Don't check coverage for deprecated code (#1216)

[Full Changelog](#)

## 50.102 Version 2.10.0, 2022-08-07

### 50.102.1 New Features (ENH)

- “with” support for PdfMerger and PdfWriter (#1193)
- Add AnnotationBuilder.text(...) to build text annotations (#1202)

### 50.102.2 Bug Fixes (BUG)

- Allow IndirectObjects as stream filters (#1211)

### 50.102.3 Documentation (DOC)

- Font scrambling
- Page vs Content scaling (#1208)
- Example for orientation parameter of `extract_text` (#1206)
- Fix `AnnotationBuilder` parameter formatting (#1204)

### 50.102.4 Developer Experience (DEV)

- Add `flake8-print` (#1203)

### 50.102.5 Maintenance (MAINT)

- Introduce `WrongPasswordError` / `FileNotDecryptedError` / `EmptyFileError` (#1201)

[Full Changelog](#)

## 50.103 Version 2.9.0, 2022-07-31

### 50.103.1 New Features (ENH)

- Add ability to add hex encoded colors to outline items (#1186)
- Add support for `pathlib.Path` in `PdfMerger.merge` (#1190)
- Add link annotation (#1189)
- Add capability to filter text extraction by orientation (#1175)

### 50.103.2 Bug Fixes (BUG)

- Named Dest in PDF1.1 (#1174)
- Incomplete Graphic State save/restore (#1172)

### 50.103.3 Documentation (DOC)

- Update changelog url in package metadata (#1180)
- Mention camelot for table extraction (#1179)
- Mention pyHanko for signing PDF documents (#1178)
- We now have CMAP support since a while (#1177)

### 50.103.4 Maintenance (MAINT)

- Consistent usage of warnings / log messages (#1164)
- Consistent terminology for outline items (#1156)

[Full Changelog](#)

## 50.104 Version 2.8.1, 2022-07-25

### 50.104.1 Bug Fixes (BUG)

- u\_hash in AlgV4.compute\_key (#1170)

### 50.104.2 Robustness (ROB)

- Fix loading of file from #134 (#1167)
- Cope with empty DecodeParams (#1165)

### 50.104.3 Documentation (DOC)

- Typo in merger deprecation warning message (#1166)

### 50.104.4 Maintenance (MAINT)

- Package updates; solve mypy strict remarks (#1163)

### 50.104.5 Testing (TST)

- Add test from #325 (#1169)

[Full Changelog](#)

## 50.105 Version 2.8.0, 2022-07-24

### 50.105.1 New Features (ENH)

- Add writer.add\_annotation, page.annotations, and generic.AnnotationBuilder (#1120)

### 50.105.2 Bug Fixes (BUG)

- Set /AS for /Btn form fields in writer (#1161)
- Ignore if /Perms verify failed (#1157)

### 50.105.3 Robustness (ROB)

- Cope with utf16 character for space calculation (#1155)
- Cope with null params for FitH / FitV destination (#1152)
- Handle outlines without valid destination (#1076)

### 50.105.4 Developer Experience (DEV)

- Introduce \_utils.logger\_warning (#1148)

### 50.105.5 Maintenance (MAINT)

- Break up parse\_to\_unicode (#1162)
- Add diagnostic output to exception in read\_from\_stream (#1159)
- Reduce PdfReader.read complexity (#1151)



### 50.105.6 Testing (TST)

- Add workflow tests found by arc testing (#1154)
- Decrypt file which is not encrypted (#1149)
- Test CryptRC4 encryption class; test image extraction filters (#1147)

[Full Changelog](#)

## 50.106 Version 2.7.0, 2022-07-21

### 50.106.1 New Features (ENH)

- Add `outline_count` property (#1129)

### 50.106.2 Bug Fixes (BUG)

- Make `reader.get_fields` also return dropdowns with options (#1114)
- Add deprecated `EncodedStreamObject` functions back until `PyPDF2==3.0.0` (#1139)

### 50.106.3 Robustness (ROB)

- Cope with missing `/W` entry (#1136)
- Cope with invalid parent `xref` (#1133)

### 50.106.4 Documentation (DOC)

- Contributors file (#1132)
- Fix type in signature of `PdfWriter.add_uri` (#1131)

### 50.106.5 Developer Experience (DEV)

- Add `.git-blame-ignore-revs` (#1141)

### 50.106.6 Code Style (STY)

- Fixing typos (#1137)
- Reuse code via `get_outlines_property` in tests (#1130)

[Full Changelog](#)

## 50.107 Version 2.6.0, 2022-07-17

### 50.107.1 New Features (ENH)

- Add `color` and `font_format` to `PdfReader.outlines[i]` (#1104)
- Extract Text Enhancement (whitespaces) (#1084)

### 50.107.2 Bug Fixes (BUG)

- Use `build_destination` for named destination outlines (#1128)
- Avoid a crash when a ToUnicode CMap has an empty `dstString` in `beginbfchar` (#1118)
- Prevent deduplication of `PageObject` (#1105)
- None-check in `DictionaryObject.read_from_stream` (#1113)
- Avoid `IndexError` in `_cmap.parse_to_unicode` (#1110)

### 50.107.3 Documentation (DOC)

- Explanation for git submodule
- Watermark and stamp (#1095)

### 50.107.4 Maintenance (MAINT)

- Text extraction improvements (#1126)
- `Destination.color` returns `ArrayObject` instead of tuple as fallback (#1119)
- Use `add_bookmark_destination` in `add_bookmark` (#1100)
- Use `add_bookmark_destination` in `add_bookmark_dict` (#1099)

### 50.107.5 Testing (TST)

- Add test for arab text (#1127)
- Add xfail for decryption fail (#1125)
- Add xfail test for `IndexError` when extracting text (#1124)
- Add MCVE showing outline title issue (#1123)

### 50.107.6 Code Style (STY)

- Use `IntFlag` for `permissions_flag` / `update_page_form_field_values` (#1094)
- Simplify code (#1101)

[Full Changelog](#)

## 50.108 Version 2.5.0, 2022-07-10

### 50.108.1 New Features (ENH)

- Add support for indexed color spaces / `BitsPerComponent` for decoding PNGs (#1067)
- Add `PageObject._get_fonts` (#1083)

### 50.108.2 Performance Improvements (PI)

- Use iterative DFS in `PdfWriter._sweep_indirect_references` (#1072)

### 50.108.3 Bug Fixes (BUG)

- Let Page.scale also scale the crop-/trim-/bleed-/artbox (#1066)
- Column default for CCITTFaxDecode (#1079)

### 50.108.4 Robustness (ROB)

- Guard against None-value in \_get\_outlines (#1060)

### 50.108.5 Documentation (DOC)

- Stamps and watermarks (#1082)
- OCR vs PDF text extraction (#1081)
- Python Version support
- Formatting of CHANGELOG

### 50.108.6 Developer Experience (DEV)

- Cache downloaded files (#1070)
- Speed-up for CI (#1069)

### 50.108.7 Maintenance (MAINT)

- Set page.rotate(angle: int) (#1092)
- Issue #416 was fixed by #1015 (#1078)

### 50.108.8 Testing (TST)

- Image extraction (#1080)
- Image extraction (#1077)

### 50.108.9 Code Style (STY)

- Apply black
- Typo in Changelog

[Full Changelog](#)

## 50.109 Version 2.4.2, 2022-07-05

### 50.109.1 New Features (ENH)

- Add PdfReader.xfa attribute (#1026)

### 50.109.2 Bug Fixes (BUG)

- Wrong page inserted when PdfMerger.merge is done (#1063)
- Resolve IndirectObject when it refers to a free entry (#1054)

### 50.109.3 Developer Experience (DEV)

- Added {posargs} to tox.ini (#1055)

### 50.109.4 Maintenance (MAINT)

- Remove PyPDF2.\_utils.bytes\_type (#1053)

### 50.109.5 Testing (TST)

- Scale page (indirect rect object) (#1057)
- Simplify pathlib PdfReader test (#1056)
- IndexError of VirtualList (#1052)
- Invalid XML in xmp information (#1051)
- No pycryptodome (#1050)
- Increase test coverage (#1045)

### 50.109.6 Code Style (STY)

- DOC of compress\_content\_streams (#1061)
- Minimize diff for #879 (#1049)

[Full Changelog](#)

## 50.110 Version 2.4.1, 2022-06-30

### 50.110.1 New Features (ENH)

- Add writer.pdf\_header property (getter and setter) (#1038)

### 50.110.2 Performance Improvements (PI)

- Remove b\_call in FloatObject.write\_to\_stream (#1044)
- Check duplicate objects in writer.\_sweep\_indirect\_references (#207)

### 50.110.3 Documentation (DOC)

- How to suppress exceptions/warnings/log messages (#1037)
- Remove hyphen from lossless (#1041)
- Compression of content streams (#1040)
- Fix inconsistent variable names in add-watermark.md (#1039)
- File size reduction
- Add CHANGELOG to the rendered docs (#1023)

### 50.110.4 Maintenance (MAINT)

- Handle XML error when reading XmpInformation (#1030)
- Deduplicate Code / add mutmut config (#1022)

### 50.110.5 Code Style (STY)

- Use unnecessary one-line function / class attribute (#1043)
- Docstring formatting (#1033)

[Full Changelog](#)

## 50.111 Version 2.4.0, 2022-06-26

### 50.111.1 New Features (ENH):

- Support R6 decrypting (#1015)
- Add PdfReader.pdf\_header (#1013)

### 50.111.2 Performance Improvements (PI):

- Remove ord\_ calls (#1014)

### 50.111.3 Bug Fixes (BUG):

- Fix missing page for bookmark (#1016)

### 50.111.4 Robustness (ROB):

- Deal with invalid Destinations (#1028)

### 50.111.5 Documentation (DOC):

- get\_form\_text\_fields does not extract dropdown data (#1029)
- Adjust PdfWriter.add\_uri docstring
- Mention crypto extra\_requires for installation (#1017)

### 50.111.6 Developer Experience (DEV):

- Use /n line endings everywhere (#1027)
- Adjust string formatting to be able to use mutmut (#1020)
- Update Bug report template

[Full Changelog](#)

## 50.112 Version 2.3.1, 2022-06-19

BUG: Forgot to add the internal \_codecs subpackage.

[Full Changelog](#)

## 50.113 Version 2.3.0, 2022-06-19

The highlight of this release is improved support for file encryption (AES-128 and AES-256, R5 only). See #749 for the amazing work of @exiledkingcc Thank you

### 50.113.1 Deprecations (DEP)

- Rename names to be PEP8-compliant (#967)
- PdfWriter.get\_page: the pageNumber parameter is renamed to page\_number
- PyPDF2.filters:
  - For all classes, a parameter rename: decodeParms decode\_parms
  - decodeStreamData decode\_stream\_data
- PyPDF2.xmp:
  - XmpInformation.rdfRoot XmpInformation.rdf\_root
  - XmpInformation.xmp\_createDate XmpInformation.xmp\_create\_date
  - XmpInformation.xmp\_creatorTool XmpInformation.xmp\_creator\_tool
  - XmpInformation.xmp\_metadataDate XmpInformation.xmp\_metadata\_date
  - XmpInformation.xmp\_modifyDate XmpInformation.xmp\_modify\_date
  - XmpInformation.xmpMetadata XmpInformation.xmp\_metadata
  - XmpInformation.xmpmm\_documentId XmpInformation.xmpmm\_document\_id
  - XmpInformation.xmpmm\_instanceId XmpInformation.xmpmm\_instance\_id
- PyPDF2.generic:
  - readHexStringFromStream read\_hex\_string\_from\_stream
  - initializeFromDictionary initialize\_from\_dictionary
  - createStringObject create\_string\_object
  - TreeObject.hasChildren TreeObject.has\_children
  - TreeObject.emptyTree TreeObject.empty\_tree

### 50.113.2 New Features (ENH)

- Add decrypt support for V5 and AES-128, AES-256 (R5 only) (#749)

### 50.113.3 Robustness (ROB)

- Fix corrupted (wrongly) linear PDF (#1008)

### 50.113.4 Maintenance (MAINT)

- Move PDF\_Samples folder into resources
- Fix typos (#1007)

### 50.113.5 Testing (TST)

- Improve encryption/decryption test (#1009)
- Add merger test cases with real PDFs (#1006)
- Add mutmut config

### 50.113.6 Code Style (STY)

- Put pure data mappings in separate files (#1005)
- Make encryption module private, apply pre-commit (#1010)

[Full Changelog](#)

## 50.114 Version 2.2.1, 2022-06-17

### 50.114.1 Performance Improvements (PI)

- Remove b\_ calls (#992, #986)
- Apply improvements to \_utils suggested by perflint (#993)

### 50.114.2 Robustness (ROB)

- utf-16-be codec can't decode (...) (#995)

### 50.114.3 Documentation (DOC)

- Remove reference to Scripts (#987)

### 50.114.4 Developer Experience (DEV)

- Fix type annotations for add\_bookmarks (#1000)

### 50.114.5 Testing (TST)

- Add test for PdfMerger (#1001)
- Add tests for XMP information (#996)
- reader.get\_fields / zlib issue / LZW decode issue (#1004)
- reader.get\_fields with report generation (#1002)
- Improve test coverage by extracting texts (#998)

### 50.114.6 Code Style (STY)

- Apply fixes suggested by pylint (#999)

[Full Changelog](#)

## 50.115 Version 2.2.0, 2022-06-13

The 2.2.0 release improves text extraction again via (#969):

- Improvements around /Encoding / /ToUnicode
- Extraction of CMaps improved
- Fallback for font def missing
- Support for /Identity-H and /Identity-V: utf-16-be
- Support for /GB-EUC-H / /GB-EUC-V / GBp/c-EUC-H / /GBpc-EUC-V (beta release for evaluation)
- Arabic (for evaluation)

- Whitespace extraction improvements

Those changes should mainly improve the text extraction for non-ASCII alphabets, e.g. Russian / Chinese / Japanese / Korean / Arabic.

[Full Changelog](#)

## **50.116 Version 2.1.1, 2022-06-12**

### **50.116.1 New Features (ENH)**

- Add support for pathlib as input for PdfReader (#979)

### **50.116.2 Performance Improvements (PI)**

- Optimize read\_next\_end\_line (#646)

### **50.116.3 Bug Fixes (BUG)**

- Adobe Acrobat ‘Would you like to save this file?’ (#970)

### **50.116.4 Documentation (DOC)**

- Notes on annotations (#982)
- Who uses PyPDF2
- intendet \xe2\x9e\x94 in robustness page (#958)

### **50.116.5 Maintenance (MAINT)**

- pre-commit / requirements.txt updates (#977)
- Mark read\_next\_end\_line as deprecated (#965)
- Export PageObject in PyPDF2 root (#960)

### **50.116.6 Testing (TST)**

- Add MCVE of issue #416 (#980)
- FlateDecode.decode decodeParms (#964)
- Xmp module (#962)
- utils.paeth\_predictor (#959)

### **50.116.7 Code Style (STY)**

- Use more tuples and list/dict comprehensions (#976)

[Full Changelog](#)

## **50.117 Version 2.1.0, 2022-06-06**

The highlight of the 2.1.0 release is the most massive improvement to the text extraction capabilities of PyPDF2 since 2016. A very big thank you goes to [pubpub-zz](#) who took a lot of time and knowledge about the PDF format to finally get those improvements into PyPDF2. Thank you



In case the new function causes any issues, you can use `_extract_text_old` for the old functionality. Please also open a bug ticket in that case.

There were several people who have attempted to bring similar improvements to PyPDF2. All of those were valuable. The main reason why they didn't get merged is the big amount of open PRs / issues. `pubpub-zz` was the most comprehensive PR which also incorporated the latest changes of PyPDF2 2.0.0.

Thank you to [VictorCarlquist](#) for #858 and [asabramo](#) for #464

### 50.117.1 New Features (ENH)

- Massive text extraction improvement (#924). Closed many open issues:
  - Exceptions / missing spaces in `extract_text()` method (#17)
    - \* Whitespace issues in `extract_text()` (#42)
    - \* pypdf2 reads the hifenated words in a new line (#246)
  - PyPDF2 failing to read unicode character (#37)
    - \* Unable to read bullets (#230)
  - `ExtractText` yields nothing for apparently good PDF (#168)
  - Encoding issue in `extract_text()` (#235)
  - `extractText()` doesn't work on Chinese PDF (#252)
  - encoding error (#260)
  - Trouble with apostrophes in names in text "O'Doul" (#384)
  - `extract_text` works for some PDF files, but not the others (#437)
  - Euro sign not being recognized by `extractText` (#443)
  - Failed extracting text from French texts (#524)
  - `extract_text` doesn't extract ligatures correctly (#598)
  - reading spanish text - mark convert issue (#635)
  - Read PDF changed from text to random symbols (#654)
  - `.extractText()` reads / as 1. (#789)
- Update glyphlist (#947) - inspired by #464
- Allow adding `PageRange` objects (#948)

### 50.117.2 Bug Fixes (BUG)

- Delete `.python-version` file (#944)
- Compare `StreamObject.decoded_self` with `None` (#931)

### 50.117.3 Robustness (ROB)

- Fix some conversion errors on non conform PDF (#932)

#### 50.117.4 Documentation (DOC)

- Elaborate on PDF text extraction difficulties (#939)
- Add logo (#942)
- rotate vs Transformation().rotate (#937)
- Example how to use PyPDF2 with AWS S3 (#938)
- How to deprecate (#930)
- Fix typos on robustness page (#935)
- Remove scripts (pdfcat) from docs (#934)

#### 50.117.5 Developer Experience (DEV)

- Ignore .python-version file
- Mark deprecated code with no-cover (#943)
- Automatically create Github releases from tags (#870)

#### 50.117.6 Testing (TST)

- Text extraction for non-latin alphabets (#954)
- Ignore PdfReadWarning in benchmark (#949)
- writer.remove\_text (#946)
- Add test for Tree and \_security (#945)

#### 50.117.7 Code Style (STY)

- black, isort, Flake8, splitting buildCharMap (#950)

[Full Changelog](#)

### 50.118 Version 2.0.0, 2022-06-01

The 2.0.0 release of PyPDF2 includes three core changes:

1. Dropping support for Python 3.5 and older.
2. Introducing type annotations.
3. Interface changes, mostly to have PEP8-compliant names

We introduced a [deprecation process](#) that hopefully helps users to avoid unexpected breaking changes.

#### 50.118.1 Breaking Changes (DEP)

- PyPDF2 2.0 requires Python 3.6+. Python 2.7 and 3.5 support were dropped.
- PdfFileReader: The “warndest” parameter was removed
- PdfFileReader and PdfFileMerger no longer have the `overwriteWarnings` parameter. The new behavior is `overwriteWarnings=False`.
- merger: OutlinesObject was removed without replacement.
- merger.py \_merger.py: You must import PdfFileMerger from PyPDF2 directly.

- `utils`:
  - `ConvertFunctionsToVirtualList` was removed
  - `formatWarning` was removed
  - `isInt(obj)`: Use `instance(obj, int)` instead
  - `u_(s)`: Use `s` directly
  - `chr_(c)`: Use `chr(c)` instead
  - `barray(b)`: Use `bytearray(b)` instead
  - `isBytes(b)`: Use `instance(b, type(bytes()))` instead
  - `xrange_fn`: Use `range` instead
  - `string_type`: Use `str` instead
  - `isString(s)`: Use `instance(s, str)` instead
  - `_basestring`: Use `str` instead
  - All Exceptions are now in `PyPDF2.errors`:
    - \* `PageSizeNotDefinedError`
    - \* `PdfReadError`
    - \* `PdfReadWarning`
    - \* `PyPdfError`
- `PyPDF2.pdf` (the `pdf` module) no longer exists. The contents were moved with the library. You should most likely import directly from `PyPDF2` instead. The `RectangleObject` is in `PyPDF2.generic`.
- The `Resources`, `Scripts`, and `Tests` will no longer be part of the distribution files on PyPI. This should have little to no impact on most people. The `Tests` are renamed to `tests`, the `Resources` are renamed to `resources`. Both are still in the git repository. The `Scripts` are now in `cpdf`. `Sample_Code` was moved to the docs.

For a full list of deprecated functions, please see the changelog of version 1.28.0.

## 50.118.2 New Features (ENH)

- Improve space setting for text extraction (#922)
- Allow setting the decryption password in `PdfReader.__init__` (#920)
- Add `Page.add_transformation` (#883)

## 50.118.3 Bug Fixes (BUG)

- Fix error adding transformation to page without `/Contents` (#908)

## 50.118.4 Robustness (ROB)

- Cope with invalid length in streams (#861)

## 50.118.5 Documentation (DOC)

- Fix style of 1.25 and 1.27 patch notes (#927)
- Transformation (#907)

### 50.118.6 Developer Experience (DEV)

- Create flake8 config file (#916)
- Use relative imports (#875)

### 50.118.7 Maintenance (MAINT)

- Use Python 3.6 language features (#849)
- Add wrapper function for PendingDeprecationWarnings (#928)
- Use new PEP8 compliant names (#884)
- Explicitly represent transformation matrix (#878)
- Inline PAGE\_RANGE\_HELP string (#874)
- Remove unnecessary generics imports (#873)
- Remove star imports (#865)
- merger.py \_merger.py (#864)
- Type annotations for all functions/methods (#854)
- Add initial type support with mypy (#853)

### 50.118.8 Testing (TST)

- Regression test for xmp\_metadata converter (#923)
- Checkout submodule sample-files for benchmark
- Add text extracting performance benchmark
- Use new PyPDF2 API in benchmark (#902)
- Make test suite fail for uncaught warnings (#892)
- Remove -OO testrun from CI (#901)
- Improve tests for convert\_to\_int (#899)

[Full Changelog](#)

## 50.119 PyPDF2 1.X

See *CHANGELOG PyPDF2 1.X*

## CHANGELOG OF PYPDF2 1.X

### 51.1 Version 1.28.4, 2022-05-29

Bug Fixes (BUG):

- XmpInformation.\_converter\_date was unusable (#921)

[Full Changelog](#)

### 51.2 Version 1.28.3, 2022-05-28

#### 51.2.1 Deprecations (DEP)

- PEP8 renaming (#905)

#### 51.2.2 Bug Fixes (BUG)

- XmpInformation missing method \_getText (#917)
- Fix PendingDeprecationWarning on \_merge\_page (#904)

[Full Changelog](#)

### 51.3 Version 1.28.2, 2022-05-23

#### 51.3.1 Bug Fixes (BUG)

- PendingDeprecationWarning for getContents (#893)
- PendingDeprecationWarning on using PdfMerger (#891)

[Full Changelog](#)

### 51.4 Version 1.28.1, 2022-05-22

#### 51.4.1 Bug Fixes (BUG)

- Incorrectly show deprecation warnings on internal usage (#887)

## 51.4.2 Maintenance (MAINT)

- Add `stacklevel=2` to deprecation warnings (#889)
- Remove duplicate warnings imports (#888)

[Full Changelog](#)

## 51.5 Version 1.28.0, 2022-05-22

This release adds a lot of deprecation warnings in preparation of the PyPDF2 2.0.0 release. The changes are mostly using `snake_case` function-, method-, and variable-names as well as using properties instead of getter-methods.

Maintenance (MAINT):

- Remove IronPython Fallback for zlib (#868)

[Full Changelog](#)

### 51.5.1 Deprecations (DEP)

- Make the `PyPDF2.utils` module private
- Rename of core classes:
  - `PdfFileReader` `PdfReader`
  - `PdfFileWriter` `PdfWriter`
  - `PdfFileMerger` `PdfMerger`
- Use PEP8 conventions for function names and parameters
- If a property and a getter-method are both present, use the property

### Details

In many places:

- `getObject` `get_object`
- `writeToStream` `write_to_stream`
- `readFromStream` `read_from_stream`

`PyPDF2.generic`

- `readObject` `read_object`
- `convertToInt` `convert_to_int`
- `DocumentInformation.getText` `DocumentInformation._get_text` : This method should typically not be used; please let me know if you need it.

`PdfReader` class:

- `reader.getPage(pageNumber)` `reader.pages[page_number]`
- `reader.getNumPages()` / `reader.numPages` `len(reader.pages)`
- `getDocumentInfo` `metadata`
- `flattenedPages` attribute `flattened_pages`
- `resolvedObjects` attribute `resolved_objects`
- `xrefIndex` attribute `xref_index`

- `getNamedDestinations` / `namedDestinations` attribute `named_destinations`
- `getPageLayout` / `pageLayout` `page_layout` attribute
- `getPageMode` / `pageMode` `page_mode` attribute
- `getIsEncrypted` / `isEncrypted` `is_encrypted` attribute
- `getOutlines` `get_outlines`
- `readObjectHeader` `read_object_header`
- `cacheGetIndirectObject` `cache_get_indirect_object`
- `cacheIndirectObject` `cache_indirect_object`
- `getDestinationPageNumber` `get_destination_page_number`
- `readNextEndLine` `read_next_end_line`
- `_zeroXref` `_zero_xref`
- `_authenticateUserPassword` `_authenticate_user_password`
- `_pageId2Num` attribute `_page_id2num`
- `_buildDestination` `_build_destination`
- `_buildOutline` `_build_outline`
- `_getPageNumberByIndirect(indirectRef)` `_get_page_number_by_indirect(indirect_ref)`
- `_getObjectFromStream` `_get_object_from_stream`
- `_decryptObject` `_decrypt_object`
- `_flatten(..., indirectRef)` `_flatten(..., indirect_ref)`
- `_buildField` `_build_field`
- `_checkKids` `_check_kids`
- `_writeField` `_write_field`
- `_write_field(..., fieldAttributes)` `_write_field(..., field_attributes)`
- `_read_xref_subsections(..., getEntry, ...)` `_read_xref_subsections(..., get_entry, ...)`

PdfWriter class:

- `writer.getPage(pageNumber)` `writer.pages[page_number]`
- `writer.getNumPages()` `len(writer.pages)`
- `addMetadata` `add_metadata`
- `addPage` `add_page`
- `addBlankPage` `add_blank_page`
- `addAttachment(fname, fdata)` `add_attachment(filename, data)`
- `insertPage` `insert_page`
- `insertBlankPage` `insert_blank_page`
- `appendPagesFromReader` `append_pages_from_reader`
- `updatePageFormFieldValues` `update_page_form_field_values`
- `cloneReaderDocumentRoot` `clone_reader_document_root`

- cloneDocumentFromReader clone\_document\_from\_reader
- getReference get\_reference
- getOutlineRoot get\_outline\_root
- getNamedDestRoot get\_named\_dest\_root
- addBookmarkDestination add\_bookmark\_destination
- addBookmarkDict add\_bookmark\_dict
- addBookmark add\_bookmark
- addNamedDestinationObject add\_named\_destination\_object
- addNamedDestination add\_named\_destination
- removeLinks remove\_links
- removeImages(ignoreByteStringObject) remove\_images(ignore\_byte\_string\_object)
- removeText(ignoreByteStringObject) remove\_text(ignore\_byte\_string\_object)
- addURI add\_uri
- addLink add\_link
- getPage(pageNumber) get\_page(page\_number)
- getPageLayout / setPageLayout / pageLayout page\_layout attribute
- getPageMode / setPageMode / pageMode page\_mode attribute
- \_addObject \_add\_object
- \_addPage \_add\_page
- \_sweepIndirectReferences \_sweep\_indirect\_references

PdfMerger class

- \_\_init\_\_ parameter: strict=True strict=False (the PdfFileMerger still has the old default)
- addMetadata add\_metadata
- addNamedDestination add\_named\_destination
- setPageLayout set\_page\_layout
- setPageMode set\_page\_mode

Page class:

- artBox / bleedBox/ cropBox/ mediaBox / trimBox artbox / bleedbox/ cropbox/ mediabox / trimbox
  - getWidth, getHeight width / height
  - getLowerLeft\_x / getUpperLeft\_x left
  - getUpperRight\_x / getLowerRight\_x right
  - getLowerLeft\_y / getLowerRight\_y bottom
  - getUpperRight\_y / getUpperLeft\_y top
  - getLowerLeft / setLowerLeft lower\_left property
  - upperRight upper\_right
- mergePage merge\_page



- rotateClockwise / rotateCounterClockwise rotate\_clockwise
- \_mergeResources \_merge\_resources
- \_contentStreamRename \_content\_stream\_rename
- \_pushPopGS \_push\_pop\_gs
- \_addTransformationMatrix \_add\_transformation\_matrix
- \_mergePage \_merge\_page

XmpInformation class:

- getElement(..., aboutUri, ...) get\_element(..., about\_uri, ...)
- getNodesInNamespace(..., aboutUri, ...) get\_nodes\_in\_namespace(..., aboutUri, ...)
- \_getText \_get\_text

utils.py:

- matrixMultiply matrix\_multiply
- RC4\_encrypt is moved to the security module

## 51.6 Version 1.27.12, 2022-05-02

### 51.6.1 Bug Fixes (BUG)

- \_rebuild\_xref\_table expects trailer to be a dict (#857)

### 51.6.2 Documentation (DOC)

- Security Policy

[Full Changelog](#)

## 51.7 Version 1.27.11, 2022-05-02

### 51.7.1 Bug Fixes (BUG)

- Incorrectly issued xref warning/exception (#855)

[Full Changelog](#)

## 51.8 Version 1.27.10, 2022-05-01

### 51.8.1 Robustness (ROB)

- Handle missing destinations in reader (#840)
- warn-only in readStringFromStream (#837)
- Fix corruption in startxref or xref table (#788 and #830)

### **51.8.2 Documentation (DOC)**

- Project Governance (#799)
- History of PyPDF2
- PDF feature/version support (#816)
- More details on text parsing issues (#815)

### **51.8.3 Developer Experience (DEV)**

- Add benchmark command to Makefile
- Ignore IronPython parts for code coverage (#826)

### **51.8.4 Maintenance (MAINT)**

- Split pdf module (#836)
- Separated CCITTFax param parsing/decoding (#841)
- Update requirements files

### **51.8.5 Testing (TST)**

- Use external repository for larger/more PDFs for testing (#820)
- Swap incorrect test names (#838)
- Add test for PdfFileReader and page properties (#835)
- Add tests for PyPDF2.generic (#831)
- Add tests for utils, form fields, PageRange (#827)
- Add test for ASCII85Decode (#825)
- Add test for FlateDecode (#823)
- Add test for filters.ASCIIHexDecode (#822)

### **51.8.6 Code Style (STY)**

- Apply pre-commit (black, isort) + use snake\_case variables (#832)
- Remove debug code (#828)
- Documentation, Variable names (#839)

[Full Changelog](#)

## **51.9 Version 1.27.9, 2022-04-24**

A change I would like to highlight is the performance improvement for large PDF files (#808)

### **51.9.1 New Features (ENH)**

- Add papersizes (#800)
- Allow setting permission flags when encrypting (#803)
- Allow setting form field flags (#802)

### 51.9.2 Bug Fixes (BUG)

- TypeError in xmp.\_converter\_date (#813)
- Improve spacing for text extraction (#806)
- Fix PDFDocEncoding Character Set (#809)

### 51.9.3 Robustness (ROB)

- Use null ID when encrypted but no ID given (#812)
- Handle recursion error (#804)

### 51.9.4 Documentation (DOC)

- CMaps (#811)
- The PDF Format + commit prefixes (#810)
- Add compression example (#792)

### 51.9.5 Developer Experience (DEV)

- Add Benchmark for Performance Testing (#781)

### 51.9.6 Maintenance (MAINT)

- Validate PDF magic byte in strict mode (#814)
- Make PdfFileMerger.addBookmark() behave like PdfFileWriters' (#339)
- Quadratic runtime while parsing reduced to linear (#808)

### 51.9.7 Testing (TST)

- Newlines in text extraction (#807)

[Full Changelog](#)

## 51.10 Version 1.27.8, 2022-04-21

### 51.10.1 Bug Fixes (BUG)

- Use 1MB as offset for readNextEndLine (#321)
- 'PdfFileWriter' object has no attribute 'stream' (#787)

### 51.10.2 Robustness (ROB)

- Invalid float object; use 0 as fallback (#782)

### 51.10.3 Documentation (DOC)

- Robustness (#785)

[Full Changelog](#)

## 51.11 Version 1.27.7, 2022-04-19

### 51.11.1 Bug Fixes (BUG)

- Import exceptions from PyPDF2.errors in PyPDF2.utils (#780)

### 51.11.2 Code Style (STY)

- Naming in 'make\_changelog.py'

## 51.12 Version 1.27.6, 2022-04-18

### 51.12.1 Deprecations (DEP)

- Remove support for Python 2.6 and older (#776)

### 51.12.2 New Features (ENH)

- Extract document permissions (#320)

### 51.12.3 Bug Fixes (BUG)

- Clip by trimBox when merging pages, which would otherwise be ignored (#240)
- Add overwriteWarnings parameter PdfFileMerger (#243)
- IndexError for getPage() of decrypted file (#359)
- Handle cases where decodeParms is an ArrayObject (#405)
- Updated PDF fields don't show up when page is written (#412)
- Set Linked Form Value (#414)
- Fix zlib -5 error for corrupt files (#603)
- Fix reading more than last 1K for EOF (#642)
- Accidental import

### 51.12.4 Robustness (ROB)

- Allow extra whitespace before "obj" in readObjectHeader (#567)

### 51.12.5 Documentation (DOC)

- Link to pdftoc in Sample\_Code (#628)
- Working with annotations (#764)
- Structure history

### 51.12.6 Developer Experience (DEV)

- Add issue templates (#765)
- Add tool to generate changelog

### 51.12.7 Maintenance (MAINT)

- Use grouped constants instead of string literals (#745)
- Add error module (#768)
- Use decorators for @staticmethod (#775)
- Split long functions (#777)

### 51.12.8 Testing (TST)

- Run tests in CI once with -OO Flags (#770)
- Filling out forms (#771)
- Add tests for Writer (#772)
- Error cases (#773)
- Check Error messages (#769)
- Regression test for issue #88
- Regression test for issue #327

### 51.12.9 Code Style (STY)

- Make variable naming more consistent in tests

[Full changelog](#)

## 51.13 Version 1.27.5, 2022-04-15

### 51.13.1 Security (SEC)

- ContentStream\_readInlineImage had potential infinite loop (#740)

### 51.13.2 Bug fixes (BUG)

- Fix merging encrypted files (#757)
- CCITTFaxDecode decodeParms can be an ArrayObject (#756)

### 51.13.3 Robustness improvements (ROBUST)

- title sometimes None (#744)

### 51.13.4 Documentation (DOC)

- Adjust short description of the package

### 51.13.5 Tests and Test setup (TST)

- Rewrite JS tests from unittest to pytest (#746)
- Increase Test coverage, mainly with filters (#756)
- Add test for inline images (#758)

### **51.13.6 Developer Experience Improvements (DEV)**

- Remove unused Travis-CI configuration (#747)
- Show code coverage (#754, #755)
- Add mutmut (#760)

### **51.13.7 Miscellaneous**

- STY: Closing file handles, explicit exports, ... (#743)

[Full Changelog](#)

## **51.14 Version 1.27.4, 2022-04-12**

### **51.14.1 Bug fixes (BUG)**

- Guard formatting of `__init__.__doc__` string (#738)

### **51.14.2 Packaging (PKG)**

- Add more precise license field to setup (#733)

### **51.14.3 Testing (TST)**

- Add test for issue #297

### **51.14.4 Miscellaneous**

- DOC: Miscallenious Miscellaneous (Typo)
- TST: Fix CI triggering (master main) (#739)
- STY: Fix various style issues (#742)

[Full Changelog](#)

## **51.15 Version 1.27.3, 2022-04-10**

- PKG: Make Tests not a subpackage (#728)
- BUG: Fix ASCII85Decode.decode assertion (#729)
- BUG: Error in Chinese character encoding (#463)
- BUG: Code duplication in Scripts/2-up.py
- ROBUST: Guard `'obj.writeToStream'` with `'if obj is not None'`
- ROBUST: Ignore `/Prev` entry with value 0 in the trailer
- MAINT: Remove `Sample_Code` (#726)
- TST: Close file handle in `test_writer` (#722)
- TST: Fix `test_get_images` (#730)
- DEV: Make tox use pytest and add more Python versions (#721)
- DOC: Many (#720, #723-725, #469)

[Full Changelog](#)

## 51.16 Version 1.27.2, 2022-04-09

- Add Scripts (including pdfcat), Resources, Tests, and Sample\_Code back to PyPDF2. It was removed by accident in 1.27.0, but might get removed with 2.0.0 See [discussions/718](#).

[Full Changelog](#)

## 51.17 Version 1.27.1, 2022-04-08

- Fixed project links on PyPI page after migration from mstamy2 to MartinThoma to the py-pdf organization on GitHub
- Documentation is now at [pypdf2.readthedocs.io](https://pypdf2.readthedocs.io)

[Full Changelog](#)

## 51.18 Version 1.27.0, 2022-04-07

Features:

- Add alpha channel support for png files in Script (#614)

### 51.18.1 Bug fixes (BUG)

- Fix formatWarning for filename without slash (#612)
- Add whitespace between words for extractText() (#569, #334)
- “invalid escape sequence” SyntaxError (#522)
- Avoid error when printing warning in pythonw (#486)
- Stream operations can be List or Dict (#665)

### 51.18.2 Documentation (DOC)

- Added Scripts/pdf-image-extractor.py
- Documentation improvements (#550, #538, #324, #426, #394)

### 51.18.3 Tests and Test setup (TST)

- Add GitHub Action which automatically runs unit tests via pytest and static code analysis with Flake8 (#660)
- Add several unit tests (#661, #663)
- Add .coveragerc to create coverage reports

### 51.18.4 Developer Experience Improvements (DEV)

- Pre commit: Developers can now `pre-commit install` to avoid tiny issues like trailing whitespaces

### 51.18.5 Miscellaneous

- Add the LICENSE file to the distributed packages (#288)
- Use setuptools instead of distutils (#599)
- Improvements for the PyPI page (#644)

- Python 3 changes (#504, #366)

[Full Changelog](#)

## **51.19 Version 1.26.0, 2016-05-18**

- NOTE: Active maintenance on PyPDF2 is resuming after a hiatus
- Fixed a bug where image resources were incorrectly overwritten when merging pages
- Added dictionary for JavaScript actions to the root (louib)
- Added unit tests for the JS functionality (louib)
- Add more Python 3 compatibility when reading inline images (im2703 and (VyacheslavHashov)
- Return NullObject instead of raising error when failing to resolve object (ctate)
- Don't output warning for non-zeroed xref table when strict=False (BenRussert)
- Remove extraneous zeroes from output formatting (speedplane)
- Fix bug where reading an inline image would cut off prematurely in certain cases (speedplane)

## **51.20 Version 1.25.1, 2015-07-20**

- Fix bug when parsing inline images. Occurred when merging certain pages with inline images
- Fixed type error when creating outlines by utilizing the isString() test

## **51.21 Version 1.25, 2015-07-07**

BUGFIXES:

- Added Python 3 algorithm for ASCII85Decode. Fixes issue when reading reportlab-generated files with Py 3 (jerickbixly)
- Recognize more escape sequence which would otherwise throw an exception (manuelzs, robertsoakes)
- Fixed overflow error in generic.py. Occurred when reading a too-large int in Python 2 (by Raja Jamwal)
- Allow access to files which were encrypted with an empty password. Previously threw a "File has not been decrypted" exception (Elena Williams)
- Do not attempt to decode an empty data stream. Previously would cause an error in decode algorithms (vladir)
- Fixed some type issues specific to Py 2 or Py 3
- Fix issue when stream data begins with whitespace (soloma83)
- Recognize abbreviated filter names (AlmightyOatmeal and Matthew Weiss)
- Copy decryption key from PdfFileReader to PdfFileMerger. Allows usage of PdfFileMerger with encrypted files (twolfson)
- Fixed bug which occurred when a NameObject is present at end of a file stream. Threw a "Stream has ended unexpectedly" exception (speedplane)

FEATURES:

- Initial work on a test suite; to be expanded in future. Tests and Resources directory added, README updated (robertsoakes)



- Added document cloning methods to PdfFileWriter: `appendPagesFromReader`, `cloneReaderDocumentRoot`, and `cloneDocumentFromReader`. See official documentation (robertsoakes)
- Added method for writing to form fields: `updatePageFormFieldValues`. This will be enhanced in the future. See official documentation (robertsoakes)
- New `addAttachment` method. See documentation. Support for adding and extracting embedded files to be enhanced in the future (moshekaplan)
- Added methods to get page number of given PageObject or Destination: `getPageNumber` and `getDestinationPageNumber`. See documentation (mozbugbox)

#### OTHER ENHANCEMENTS:

- Enhanced type handling (Brent Amrhein)
- Enhanced exception handling in NameObject (sbywater)
- Enhanced `extractText` method output (peircej)
- Better exception handling
- Enhanced regex usage in NameObject class (speedplane)

## 51.22 Version 1.24, 2014-12-31

- Bugfixes for reading files in Python 3 (by Anthony Tuininga and pqqp)
- Appropriate errors are now raised instead of infinite loops (by naure and Cyrus Vafadari)
- Bugfix for parsing number tokens with leading spaces (by Maxim Kamenkov)
- Don't crash on bad /Outlines reference (by eshellman)
- Conform tabs/spaces and blank lines to PEP 8 standards
- Utilize the `readUntilRegex` method when reading Number Objects (by Brendan Jurd)
- More bugfixes for Python 3 and clearer exception handling
- Fixed encoding issue in merger (with eshellman)
- Created separate folder for scripts

## 51.23 Version 1.23, 2014-08-11

- Documentation now available at [pythonhosted.org](http://pythonhosted.org)
- Bugfix in `pagerange.py` for when `__init__.__doc__` has no value (by Vladir Cruz)
- Fix typos in `OutlinesObject().add()` (by shilluc)
- Re-added a missing return statement in a `utils.py` method
- Corrected viewing mode names (by Jason Scheirer)
- New PdfFileWriter method: `addJS()` (by vfigueiro)
- New bookmark features: color, boldness, italics, and page fit (by Joshua Arnott)
- New PdfFileReader method: `getFields()`. Used to extract field information from PDFs with interactive forms. See documentation for details
- Converted README file to markdown format (by Stephen Bussard)
- Several improvements to overall performance and efficiency (by mozbugbox)

- Fixed a bug where geospatial information was not scaling along with its page
- Fixed a type issue and a Python 3 issue in the decryption algorithms (with Francisco Vieira and koba-ninkigumi)
- Fixed a bug causing an infinite loop in the ASCII 85 decoding algorithm (by madmaardigan)
- Annotations (links, comment windows, etc.) are now preserved when pages are merged together
- Used the Destination class in addLink() and addBookmark() so that the page fit option could be properly customized

## **51.24 Version 1.22, 2014-05-29**

- Added .DS\_Store to .gitignore (for Mac users) (by Steve Witham)
- Removed \_\_init\_\_() implementation in NameObject (by Steve Witham)
- Fixed bug (inf. loop) when merging pages in Python 3 (by commx)
- Corrected error when calculating height in scaleTo()
- Removed unnecessary code from DictionaryObject (by Georges Dubus)
- Fixed bug where an exception was thrown upon reading a NULL string (by speedplane)
- Allow string literals (non-unicode strings in Python 2) to be passed to PdfFileReader
- Allow ConvertFunctionsToVirtualList to be indexed with slices and longs (in Python 2) (by Matt Gilson)
- Major improvements and bugfixes to addLink() method (see documentation in source code) (by Henry Keiter)
- General code clean-up and improvements (with Steve Witham and Henry Keiter)
- Fixed bug that caused crash when comments are present at end of dictionary

## **51.25 Version 1.21, 2014-04-21**

- Fix for when /Type isn't present in the Pages dictionary (by Rob1080)
- More tolerance for extra whitespace in Indirect Objects
- Improved Exception handling
- Fixed error in getHeight() method (by Simon Kaempflein)
- implement use of utils.string\_type to resolve Py2-3 compatibility issues
- Prevent exception for multiple definitions in a dictionary (with carlosfunk) (only when strict = False)
- Fixed errors when parsing a slice using pdfcat on command line (by Steve Witham)
- Tolerance for EOF markers within 1024 bytes of the actual end of the file (with David Wolever)
- Added overwriteWarnings parameter to PdfFileReader constructor, if False PyPDF2 will NOT overwrite methods from Python's warnings.py module with a custom implementation.
- Fix NumberObject and NameObject constructors for compatibility with PyPy (Rüdiger Jungbeck, Xavier Dupré, shezadkhan137, Steven Witham)
- Utilize utils.Str in pdf.py and pagerange.py to resolve type issues (by egbutter)
- Improvements in implementing StringIO for Python 2 and BytesIO for Python 3 (by Xavier Dupré)
- Added /x00 to Whitespaces, defined utils.WHITESPACES to clarify code (by Maxim Kamenkov)
- Bugfix for merging 3 or more resources with the same name (by lucky-user)

- Improvements to Xref parsing algorithm (by speedplane)

## 51.26 Version 1.20, 2014-01-27

- Official Python 3+ support (with contributions from TWAC and cgmamans) Support for Python versions 2.6 and 2.7 will be maintained
- Command line concatenation (see pdfcat in sample code) (by Steve Witham)
- New FAQ; link included in README
- Allow more (although unnecessary) escape sequences
- Prevent exception when reading a null object in decoding parameters
- Corrected error in reading destination types (added a slash since they are name objects)
- Corrected TypeError in scaleTo() method
- addBookmark() method in PdfFileMerger now returns bookmark (so nested bookmarks can be created)
- Additions to Sample Code and Sample PDFs
- changes to allow 2up script to work (see sample code) (by Dylan McNamee)
- changes to metadata encoding (by Chris Hiestand)
- New methods for links: addLink() (by Enrico Lambertini) and removeLinks()
- Bugfix to handle nested bookmarks correctly (by Jamie Lentin)
- New methods removeImages() and removeText() available for PdfFileWriter (by Tien Hai)
- Exception handling for illegal characters in Name Objects

## 51.27 Version 1.19, 2013-10-08

### BUGFIXES:

- Removed pop in sweepIndirectReferences to prevent infinite loop (provided by ian-su-sirca)
- Fixed bug caused by whitespace when parsing PDFs generated by AutoCad
- Fixed a bug caused by reading a 'null' ASCII value in a dictionary object (primarily in PDFs generated by AutoCad).

### FEATURES:

- Added new folders for PyPDF2 sample code and example PDFs; see README for each folder
- Added a method for debugging purposes to show current location while parsing
- Ability to create custom metadata (by jamma313)
- Ability to access and customize document layout and view mode (by Joshua Arnott)

### OTHER:

- Added and corrected some documentation
- Added some more warnings and exception messages
- Removed old test/debugging code

### UPCOMING:

- More bugfixes (We have received many problematic PDFs via email, we will work with them)

- Documentation - It's time for PyPDF2 to get its own documentation since it has grown much since the original pyPdf
- A FAQ to answer common questions

## **51.28 Version 1.18, 2013-08-19**

- Fixed a bug where older versions of objects were incorrectly added to the cache, resulting in outdated or missing pages, images, and other objects (from speedplane)
- Fixed a bug in parsing the xref table where new xref values were overwritten; also cleaned up code (from speedplane)
- New method mergeRotatedAroundPointPage which merges a page while rotating it around a point (from speedplane)
- Updated Destination syntax to respect PDF 1.6 specifications (from jamma313)
- Prevented infinite loop when a PdfFileReader object was instantiated with an empty file (from Jerome Nexedi)

Other Changes:

- Downloads now available via PyPI
- Installation through pip library is fixed

## **51.29 Version 1.17, 2013-07-25**

- Removed one (from pdf.py) of the two Destination classes. Both classes had the same name, but were slightly different in content, causing some errors. (from Janne Vanhala)
- Corrected and Expanded README file to demonstrate PdfFileMerger
- Added filter for LZW encoded streams (from Michal Horejsek)
- PyPDF2 issue tracker enabled on Github to allow community discussion and collaboration

## **51.30 Versions -1.16, -2013-06-30**

- Note: This ChangeLog has not been kept up-to-date for a while. Hopefully we can keep better track of it from now on. Some of the changes listed here come from previous versions 1.14 and 1.15; they were only vaguely defined. With the new \_version.py file we should have more structured and better documented versioning from now on.
- Defined PyPDF2.\_\_version\_\_
- Fixed encrypt() method (from Martijn The)
- Improved error handling on PDFs with truncated streams (from cecilkorik)
- Python 3 support (from kushal-kumaran)
- Fixed example code in README (from Jeremy Bethmont)
- Fixed an bug caused by DecimalError Exception (from Adam Morris)
- Many other bug fixes and features by:

jeansch Anton Vlasenko Joseph Walton Jan Oliver Oelerich Fabian Henze And any others I missed. Thanks for contributing!

## 51.31 Version 1.13, 2010-12-04

- Fixed a typo in code for reading a “\b” escape character in strings.
- Improved `__repr__` in `FloatObject`.
- Fixed a bug in reading octal escape sequences in strings.
- Added `getWidth` and `getHeight` methods to the `RectangleObject` class.
- Fixed compatibility warnings with Python 2.4 and 2.5.
- Added `addBlankPage` and `insertBlankPage` methods on `PdfFileWriter` class.
- Fixed a bug with circular references in page’s object trees (typically annotations) that prevented correctly writing out a copy of those pages.
- New merge page functions allow application of a transformation matrix.
- To all patch contributors: I did a poor job of keeping this ChangeLog up-to-date for this release, so I am missing attributions here for any changes you submitted. Sorry! I’ll do better in the future.

## 51.32 Version 1.12, 2008-09-02

- Added support for XMP metadata.
- Fix reading files with xref streams with multiple `/Index` values.
- Fix extracting content streams that use graphics operators longer than 2 characters. Affects merging PDF files.

## 51.33 Version 1.11, 2008-05-09

- Patch from Hartmut Goebel to permit `RectangleObjects` to accept `NumberObject` or `FloatObject` values.
- PDF compatibility fixes.
- Fix to read object xref stream in correct order.
- Fix for comments inside content streams.

## 51.34 Version 1.10, 2007-10-04

- Text strings from PDF files are returned as Unicode string objects when pyPdf determines that they can be decoded (as UTF-16 strings, or as `PDFDocEncoding` strings). Unicode objects are also written out when necessary. This means that string objects in pyPdf can be either `generic.ByteStringObject` instances, or `generic.TextStringObject` instances.
- The `extractText` method now returns a unicode string object.
- All document information properties now return unicode string objects. In the event that a document provides `docinfo` properties that are not decoded by pyPdf, the raw byte strings can be accessed with an “`_raw`” property (ie. `title_raw` rather than `title`)
- `generic.DictionaryObject` instances have been enhanced to be easier to use. Values coming out of dictionary objects will automatically be de-referenced (`.getObject` will be called on them), unless accessed by the new “`raw_get`” method. `DictionaryObjects` can now only contain `PdfObject` instances (as keys and values), making it easier to debug where non-`PdfObject` values (which cannot be written out) are entering dictionaries.
- Support for reading named destinations and outlines in PDF files. Original patch by Ashish Kulkarni.
- Stream compatibility reading enhancements for malformed PDF files.

- Cross reference table reading enhancements for malformed PDF files.
- Encryption documentation.
- Replace some “assert” statements with error raising.
- Minor optimizations to FlateDecode algorithm increase speed when using PNG predictors.

### **51.35 Version 1.9, 2006-12-15**

- Fix several serious bugs introduced in version 1.8, caused by a failure to run through our PDF test suite before releasing that version.
- Fix bug in NullObject reading and writing.

### **51.36 Version 1.8, 2006-12-14**

- Add support for decryption with the standard PDF security handler. This allows for decrypting PDF files given the proper user or owner password.
- Add support for encryption with the standard PDF security handler.
- Add new pythondoc documentation.
- Fix bug in ASCII85 decode that occurs when whitespace exists inside the two terminating characters of the stream.

### **51.37 Version 1.7, 2006-12-10**

- Fix a bug when using a single page object in two PdfFileWriter objects.
- Adjust PyPDF to be tolerant of whitespace characters that don't belong during a stream object.
- Add documentInfo property to PdfFileReader.
- Add numPages property to PdfFileReader.
- Add pages property to PdfFileReader.
- Add extractText function to PdfFileReader.

### **51.38 Version 1.6, 2006-06-06**

- Add basic support for comments in PDF files. This allows us to read some ReportLab PDFs that could not be read before.
- Add “auto-repair” for finding xref table at slightly bad locations.
- New StreamObject backend, cleaner and more powerful. Allows the use of stream filters more easily, including compressed streams.
- Add a graphics state push/pop around page merges. Improves quality of page merges when one page's content stream leaves the graphics in an abnormal state.
- Add PageObject.compressContentStreams function, which filters all content streams and compresses them. This will reduce the size of PDF pages, especially after they could have been decompressed in a mergePage operation.
- Support inline images in PDF content streams.

- Add support for using .NET framework compression when zlib is not available. This does not make pyPdf compatible with IronPython, but it is a first step.
- Add support for reading the document information dictionary, and extracting title, author, subject, producer and creator tags.
- Add patch to support NullObject and multiple xref streams, from Bradley Lawrence.

### 51.39 Version 1.5, 2006-01-28

- Fix a bug where merging pages did not work in “no-rename” cases when the second page has an array of content streams.
- Remove some debugging output that should not have been present.

### 51.40 Version 1.4, 2006-01-27

- Add capability to merge pages from multiple PDF files into a single page using the PageObject.mergePage function. See example code (README or web site) for more information.
- Add ability to modify a page’s MediaBox, CropBox, BleedBox, TrimBox, and ArtBox properties through PageObject. See example code (README or web site) for more information.
- Refactor pdf.py into multiple files: generic.py (contains objects like NameObject, DictionaryObject), filters.py (contains filter code), utils.py (various). This does not affect importing PdfFileReader or PdfFileWriter.
- Add new decoding functions for standard PDF filters ASCIIHexDecode and ASCII85Decode.
- Change url and download\_url to refer to new pybrary.net web site.

### 51.41 Version 1.3, 2006-01-23

- Fix new bug introduced in 1.2 where PDF files with \r line endings did not work properly anymore. A new test suite developed with various PDF files should prevent regression bugs from now on.
- Fix a bug where inheriting attributes from page nodes did not work.

### 51.42 Version 1.2, 2006-01-23

- Improved support for files with CRLF-based line endings, fixing a common reported problem stating “assertion error: assert line == “%%EOF””.
- Software author/maintainer is now officially a proud married person, which is sure to result in better software... somehow.

### 51.43 Version 1.1, 2006-01-18

- Add capability to rotate pages.
- Improved PDF reading support to properly manage inherited attributes from /Type=/Pages nodes. This means that page groups that are rotated or have different media boxes or whatever will now work properly.
- Added PDF 1.5 support. Namely cross-reference streams and object streams. This release can mangle Adobe’s PDFReference16.pdf successfully.

## **51.44 Version 1.0, 2006-01-17**

- First distutils-capable true public release. Supports a wide variety of PDF files that I found sitting around on my system.
- Does not support some PDF 1.5 features, such as object streams, cross-reference streams.



## MIGRATION GUIDE: 1.X TO 2.X

PyPDF2<2.0.0 ([docs](#)) is very different from PyPDF2>=2.0.0 ([docs](#)).

Luckily, most changes are simple naming adjustments. This guide helps you to make the step from PyPDF2 1.x (or even the original PyPdf) to PyPDF2>=2.0.0.

You can execute your code with the updated version and show deprecation warnings by running `python -W all your_code.py`.

### 52.1 Imports and Modules

- `PyPDF2.utils` no longer exists
- `PyPDF2.pdf` no longer exists. You can import from `PyPDF2` directly or from `PyPDF2.generic`

### 52.2 Naming Adjustments

#### 52.2.1 Classes

The base classes were renamed as they also allow operating with BytesIO streams instead of files. Also, the `strict` parameter changed the default value from `strict=True` to `strict=False`.

- `PdfFileReader PdfReader`
- `PdfFileWriter PdfWriter`
- `PdfFileMerger PdfMerger`

`PdfFileReader` and `PdfFileMerger` no longer have the `overwriteWarnings` parameter. The new behavior is `overwriteWarnings=False`.

#### 52.2.2 Function, Method, and Property Names

In `PyPDF2.xmp.XmpInformation`:

- `rdfRoot rdf_root`
- `xmp_createDate xmp_create_date`
- `xmp_creatorTool xmp_creator_tool`
- `xmp_metadataDate xmp_metadata_date`
- `xmp_modifyDate xmp_modify_date`
- `xmpMetadata xmp_metadata`
- `xmpmm_documentId xmpmm_document_id`

- `xmpmm_instanceId` `xmpmm_instance_id`

In `PyPDF2.generic`:

- `readObject` `read_object`
- `convertToInt` `convert_to_int`
- `DocumentInformation.getText` `DocumentInformation._get_text` : This method should typically not be used; please let me know if you need it.
- `readHexStringFromStream` `read_hex_string_from_stream`
- `initializeFromDictionary` `initialize_from_dictionary`
- `createStringObject` `create_string_object`
- `TreeObject.hasChildren` `TreeObject.has_children`
- `TreeObject.emptyTree` `TreeObject.empty_tree`

In many places:

- `getObject` `get_object`
- `writeToStream` `write_to_stream`
- `readFromStream` `read_from_stream`

`PdfReader` class:

- `reader.getPage(pageNumber)` `reader.pages[page_number]`
- `reader.getNumPages()` / `reader.numPages` `len(reader.pages)`
- `getDocumentInfo` `metadata`
- `flattenedPages` attribute `flattened_pages`
- `resolvedObjects` attribute `resolved_objects`
- `xrefIndex` attribute `xref_index`
- `getNamedDestinations` / `namedDestinations` attribute `named_destinations`
- `getPageLayout` / `pageLayout` `page_layout` attribute
- `getPageMode` / `pageMode` `page_mode` attribute
- `getIsEncrypted` / `isEncrypted` `is_encrypted` attribute
- `getOutlines` `get_outlines`
- `readObjectHeader` `read_object_header`
- `cacheGetIndirectObject` `cache_get_indirect_object`
- `cacheIndirectObject` `cache_indirect_object`
- `getDestinationPageNumber` `get_destination_page_number`
- `readNextEndLine` `read_next_end_line`
- `_zeroXref` `_zero_xref`
- `_authenticateUserPassword` `_authenticate_user_password`
- `_pageId2Num` attribute `_page_id2num`
- `_buildDestination` `_build_destination`

- `_buildOutline` `_build_outline`
- `_getPageNumberByIndirect(indirectRef)` `_get_page_number_by_indirect(indirect_ref)`
- `_getObjectFromStream` `_get_object_from_stream`
- `_decryptObject` `_decrypt_object`
- `_flatten(..., indirectRef)` `_flatten(..., indirect_ref)`
- `_buildField` `_build_field`
- `_checkKids` `_check_kids`
- `_writeField` `_write_field`
- `_write_field(..., fieldAttributes)` `_write_field(..., field_attributes)`
- `_read_xref_subsections(..., getEntry, ...)` `_read_xref_subsections(..., get_entry, ...)`

PdfWriter class:

- `writer.getPage(pageNumber)` `writer.pages[page_number]`
- `writer.getNumPages()` `len(writer.pages)`
- `addMetadata` `add_metadata`
- `addPage` `add_page`
- `addBlankPage` `add_blank_page`
- `addAttachment(fname, fdata)` `add_attachment(filename, data)`
- `insertPage` `insert_page`
- `insertBlankPage` `insert_blank_page`
- `appendPagesFromReader` `append_pages_from_reader`
- `updatePageFormFieldValues` `update_page_form_field_values`
- `cloneReaderDocumentRoot` `clone_reader_document_root`
- `cloneDocumentFromReader` `clone_document_from_reader`
- `getReference` `get_reference`
- `getOutlineRoot` `get_outline_root`
- `getNamedDestRoot` `get_named_dest_root`
- `addBookmarkDestination` `add_bookmark_destination`
- `addBookmarkDict` `add_bookmark_dict`
- `addBookmark` `add_bookmark`
- `addNamedDestinationObject` `add_named_destination_object`
- `addNamedDestination` `add_named_destination`
- `removeLinks` `remove_links`
- `removeImages(ignoreByteStringObject)` `remove_images(ignore_byte_string_object)`
- `removeText(ignoreByteStringObject)` `remove_text(ignore_byte_string_object)`
- `addURI` `add_uri`

- `addLink add_link`
- `getPage(pageNumber) get_page(page_number)`
- `getPageLayout / setPageLayout / pageLayout page_layout attribute`
- `getPageMode / setPageMode / pageMode page_mode attribute`
- `_addObject _add_object`
- `_addPage _add_page`
- `_sweepIndirectReferences _sweep_indirect_references`

PdfMerger class

- `__init__` parameter: `strict=True` `strict=False` (the PdfFileMerger still has the old default)
- `addMetadata add_metadata`
- `addNamedDestination add_named_destination`
- `setPageLayout set_page_layout`
- `setPageMode set_page_mode`

Page class:

- `artBox / bleedBox / cropBox / mediaBox / trimBox artbox / bleedbox / cropbox / mediabox / trimbox`
  - `getWidth, getHeight width / height`
  - `getLowerLeft_x / getUpperLeft_x left`
  - `getUpperRight_x / getLowerRight_x right`
  - `getLowerLeft_y / getLowerRight_y bottom`
  - `getUpperRight_y / getUpperLeft_y top`
  - `getLowerLeft / setLowerLeft lower_left property`
  - `upperRight upper_right`
- `mergePage merge_page`
- `rotateClockwise / rotateCounterClockwise rotate_clockwise`
- `_mergeResources _merge_resources`
- `_contentStreamRename _content_stream_rename`
- `_pushPopGS _push_pop_gs`
- `_addTransformationMatrix _add_transformation_matrix`
- `_mergePage _merge_page`

XmpInformation class:

- `getElement(..., aboutUri, ...) get_element(..., about_uri, ...)`
- `getNodesInNamespace(..., aboutUri, ...) get_nodes_in_namespace(..., aboutUri, ...)`
- `_getText _get_text`

utils.py:

- `matrixMultiply`matrix_multiply`
- `RC4_encrypt` is moved to the security module

### 52.2.3 Parameter Names

- PdfWriter.get\_page: pageNumber page\_number
- PyPDF2.filters (all classes): decodeParms decode\_parms
- PyPDF2.filters (all classes): decodeStreamData decode\_stream\_data
- pagenum page\_number
- PdfMerger.merge: position page\_number
- PdfWriter.add\_outline\_item\_destination: dest page\_destination
- PdfWriter.add\_named\_destination\_object: dest page\_destination
- PdfWriter.encrypt: user\_pwd user\_password
- PdfWriter.encrypt: owner\_pwd owner\_password

### 52.2.4 Deprecations

A few classes / functions were deprecated without replacement:

- PyPDF2.utils.ConvertFunctionsToVirtualList
- PyPDF2.utils.formatWarning
- PyPDF2.isInt(obj): Use instance(obj, int) instead
- PyPDF2.u\_(s): Use s directly
- PyPDF2.chr\_(c): Use chr(c) instead
- PyPDF2.barray(b): Use bytearray(b) instead
- PyPDF2.isBytes(b): Use instance(b, type(bytes())) instead
- PyPDF2.xrange\_fn: Use range instead
- PyPDF2.string\_type: Use str instead
- PyPDF2.isString(s): Use instance(s, str) instead
- PyPDF2.\_basestring: Use str instead
- b\_(...) was removed. You should typically be able to use the bytes object directly, otherwise you can [copy this](#)



## PROJECT GOVERNANCE

This document describes how the pypdf project is managed. It describes the different actors, their roles, and the responsibilities they have.

### 53.1 Terminology

- The **project** is pypdf - a free and open-source pure-python PDF library capable of splitting, merging, cropping, and transforming the pages of PDF files. It includes the [code](#), [issues](#), and [discussions on GitHub](#), and the [documentation on ReadTheDocs](#), the [package on PyPI](#), and the [website on GitHub](#).
- A **maintainer** is a person who has technical permissions to change one or more parts of the projects. It is a person driven to keep the project running and improving.
- A **contributor** is a person who contributes to the project. That could be through writing code - in the best case through forking and creating a pull request, but that is up to the maintainer. Other contributors describe issues, help to ask questions on existing issues to make them easier to answer, participate in discussions, and help to improve the documentation. Contributors are similar to maintainers, but without technical permissions.
- A **user** is a person who imports pypdf into their code. All pypdf users are developers, but not developers who know the internals of pypdf. They only use the public interface of pypdf. They will likely have less knowledge about PDF than contributors.
- The **community** is all of that - the users, the contributors, and the maintainers.

### 53.2 Governance, Leadership, and Steering pypdf forward

pypdf is a free and open source project with over 100 contributors and likely (way) more than 1000 users.

As pypdf does not have any formal relationship with any company and no funding, all the work done by the community are voluntary contributions. People don't get paid, but choose to spend their free time to create software of which many more are profiting. This has to be honored and respected.

Despite such a big community, the project was dormant from 2016 to 2022. There were still questions asked, issues reported, and pull requests created. But the maintainer didn't have the time to move pypdf forward. During that time, nobody else stepped up to become the new maintainer.

For this reason, pypdf has the **Benevolent Dictator** governance model. The benevolent dictator is a maintainer with all technical permissions - most importantly the permission to push new pypdf versions on PyPI.

Being benevolent, the benevolent dictator listens for decisions to the community and tries their best to make decisions from which the overall community profits - the current one and the potential future one. Being a dictator, the benevolent dictator always has the power and the right to make decisions on their own - also against some members of the community.

As pypdf is free software, parts of the community can split off (fork the code) and create a new community. This should limit the harm a bad benevolent dictator can do.

## 53.3 Project Language

The project language is (american) English. All documentation and issues must be written in English to ensure that the community can understand it.

We appreciate the fact that large parts of the community don't have English as their native language. We try our best to understand others - [automatic translators](#) might help.

## 53.4 Expectations

The community can expect the following:

- The **benevolent dictator** tries their best to make decisions from which the overall community profits. The benevolent dictator is aware that his/her decisions can shape the overall community. Once the benevolent dictator notices that she/he doesn't have the time to advance pypdf, he/she looks for a new benevolent dictator. As it is expected that the benevolent dictator will step down at some point of their choice (hopefully before their death), it is NOT a benevolent dictator for life (BDFL).
- Every **maintainer** (including the benevolent dictator) is aware of their permissions and the harm they could do. They value security and ensure that the project is not harmed. They give their technical permissions back if they don't need them any longer. Any long-time contributor can become a maintainer. Maintainers can - and should! - step down from their role when they realize that they can no longer commit that time. Their contribution will be honored in the *History of pypdf*.
- Every **contributor** is aware that the time of maintainers and the benevolent dictator is limited. Short pull requests that briefly describe the solved issue and have a unit test have a higher chance to get merged soon - simply because it's easier for maintainers to see that the contribution will not harm the overall project. Their contributions are documented in the git history and in the public issues. [Let us know](#) if you would appreciate something else!
- Every **community member** uses a respectful language. We are all human, we get upset about things we care and other things than what's visible on the internet go on in our live. pypdf does not pay its contributors - keep all of that in mind when you interact with others. We are here because we want to help others.

### 53.4.1 Issues and Discussions

An issue is any technical description that aims at bringing pypdf forward:

- Bugs tickets: Something went wrong because pypdf developers made a mistake.
- Feature requests: pypdf does not support all features of the PDF specifications. There are certainly also convenience methods that would help users a lot.
- Robustness requests: There are many broken PDFs around. In some cases, we can deal with that. It's kind of a mixture between a bug ticket and a feature request.
- Performance tickets: pypdf could be faster - let us know about your specific scenario.

Any comment that is in those technical descriptions which is not helping the discussion can be deleted. This is especially true for "me too" comments on bugs or "bump" comments for desired features. People can express this with / reactions.

[Discussions](#) are open. No comments will be deleted there - except if they are unrelated spam or only try to insult people (luckily, the community was very respectful so far)



### 53.4.2 Releases

The maintainers follow [semantic versioning](#). Most importantly, that means that breaking changes will have a major version bump.

Be aware that unintentional breaking changes might still happen. The pypdf maintainers do their best to fix that in a timely manner - please [report such issues](#)!

## 53.5 People

- [stefan6419846](#) is the benevolent dictator since January 2025
- [Martin Thoma](#) was the benevolent dictator from April 2022 to January 2025. He still has most of the permissions as a fallback.
- Maintainers:
  - Matthew Stamy (mstamy2) was the benevolent dictator for a long time. He still is around on GitHub once in a while and has permissions on PyPI and GitHub.
  - Matthew Peveler (MasterOdin) is a maintainer on GitHub.



## TAKING OWNERSHIP OF PYPDF

pypdf is currently maintained by stefan6419846. We want to avoid that pypdf ever goes unmaintained again. This document serves as a guide to avoid that if I become unavailable, e.g., due to severe health issues.

This currently is just an abstract scenario. I'm fine, and I will likely do this for several more years, but I have seen how projects stand still for many years because of the maintainer becoming inactive.

### 54.1 What belongs to pypdf?

The resources needed for maintaining pypdf are:

- PyPI: [pypdf](#) and [PyPDF2](#)
- GitHub: [pypdf](#) (the repository, not the organization)
- ReadTheDocs: [pypdf](#) and [PyPDF2](#)

### 54.2 When may somebody take ownership?

**No activity in 180 days:** If I don't answer e-mails (see my GitHub profile) and don't make any commits / merges for half a year, you can consider pypdf "not maintained."

### 54.3 Who may take ownership?

Preferably, one of the owners of the GitHub `py-pdf` organization takes care of that.

As of 27th of August 2023, the following people might be candidates:

- [Lucas-C](#): He maintains `fpdf2` and is a `py-pdf` owner
- [pubpub-zz](#): He is one of the most active contributors to pypdf
- [Matthew Peveler](#): Less active, but he is cautious about breaking changes and an experienced software developer.
- [exiledkingcc](#): He has contributed the core changes related to encryption.

### 54.4 How to take ownership?

- PyPI: Follow [PEP 541 – Package Index Name Retention](#)
- GitHub: Talk with one of the other `py-pdf` organization owners
- ReadTheDocs: Follow the [Abandoned projects policy](#)



## HISTORY OF PYPDF

### 55.1 The Origins: pyPdf (2005-2010)

In 2005, [Mathieu Fenniak](#) launched pyPdf “as a PDF toolkit...” focused on

- document manipulation: by-page splitting, concatenation, and merging;
- document introspection;
- page cropping; and
- document encryption and decryption.

The last release of PyPI was [pyPdf 1.13](#) in 2010.

### 55.2 PyPDF2 is born (2011-2016)

At the end of 2011, after consultation with Mathieu and others, Phaseit sponsored PyPDF2 as a fork of pyPdf on GitHub. The initial impetus was to handle a wider range of input PDF instances; Phaseit’s commercial work often encounters PDF instances “in the wild” that it needs to manage (mostly concatenate and paginate), but that deviates so much from PDF standards that pyPdf can’t read them. PyPDF2 reads a considerably wider range of real-world PDF instances.

Neither pyPdf nor PyPDF2 aims to be universal, that is, to provide all possible PDF-related functionality. Note that the similar-appearing [pyfpdf](#) of Mariano Reingart is most comparable to [ReportLab](#), in that both ReportLab and pyfpdf emphasize document generation. Interestingly enough, pyfpdf builds in a basic HTML→PDF converter while PyPDF2 has no knowledge of HTML.

So what is PyPDF2 truly about? Think about popular [pdftk](#) for a moment. PyPDF2 does what pdftk does, and it does so within your current Python process, and it handles a wider range of variant PDF formats [explain]. PyPDF2 has its own FAQ to answer other questions that have arisen.

The Reddit [/r/python crowd](#) chatted obliquely and briefly about PyPDF2 in March 2012.

The core developer / maintainer was Matthew Stamy.

### 55.3 PyPDF3 and PyPDF4 (2018-2022)

Two approaches were made to get PyPDF2 active again: PyPDF3 and PyPDF4.

PyPDF3 had its first release in 2018 and its last one in February 2022. It never got the user base from PyPDF2.

PyPDF4 only had one release in 2018.

## 55.4 PyPDF2: Reborn (2022)

Martin Thoma took over maintenance of PyPDF2 in April 2022. It had over 100 open PRs and 321 open issues.

[pubpub-zz](#) was extremely active, especially for text extraction.

[Matthew Peveler](#) helped a lot with reviews and general project decisions.

[exiledkingcc](#) added support for modern encryption schemes.

## 55.5 pypdf: Back to the Roots (2023-2024)

In order to simplify things for beginners, PyPDF2 was merged back into pypdf. Now all lowercase, without a number. We hope that the folks who develop PyPDF3 and PyPDF4 also join us.

Compared to PyPDF2 `>= 3.0.0`, pypdf `>= 3.1.0` now offers:

- AES reading and writing support. Not only with PyCryptoDome, but also with cryptography.
- Text extraction improvements, e.g., for math content. [pypdf is now comparable with Tika, pypdfium2, and PyMuPDF](#)
- Annotation support
- Performance Improvements and Bugfixes
- Page Label support

[stefan6419846](#) made his [first PR for pypdf](#) in July 2023 and joined the project.

## CONTRIBUTORS

pypdf had a lot of contributors since it started as pyPdf in 2005. We are a free software project without any company affiliation. We cannot pay contributors, but we do value their contributions. A lot of time, effort, and expertise went into this project. With this list, we recognize these awesome people

The list is definitely not complete. You can find more contributors via the git history and [GitHub's 'Contributors' feature](#).

### 56.1 Contributors to the pypdf (formerly pyPdf / PyPDF2) project

- abyesilyurt
- ArkieCoder
- Beers, PJ
- Clauss, Christian
- DL6ER
- Duy, Phan Thanh
- ediamondscience
- Ermeson, Felipe
- Freitag, François
- Gagnon, William G.
- Gillard, James
- Górny, Michał
- Grillo, Miguel
- Gutteridge, David H.
- Hale, Joseph
- harshhes
- Jackowitz, Noah | LinkedIn
- JianzhengLuo
- Karvonen, Harry
- King, Hunter
- Kotler, Mitchell

- KourFrost
- Lightup1
- Majumder, Jonah
- Manini, Lorenzo
- maxbeer99
- McNeil, Karen: Arabic Language Support
- Mérino, Antoine
- Murphy, Kevin
- nalin-udhaar
- Noah-Houghton | LinkedIn
- Paramonov, Alexey
- Paternault, Louis
- Perrensen, Olsen
- pilotandy
- Pinheiro, Arthur
- pmiller66
- Poddar, Arka
- programmarchy
- pubpub-zz: involved in community development
- Ramos, Leodanis Pozo
- RitchieP | LinkedIn | StackOverflow
- robbiebusinessacc
- Roder, Thomas
- Rogmann, Sascha
- Röthenbacher, Thomas
- shartzog
- stefan6419846: Maintainer of pypdf since January 2025
- sietzeberends
- Stober, Marc
- Stüber, Timo
- Thoma, Martin: Maintainer of pypdf from April 2022 to January 2025. I hope to build a great community with many awesome contributors. [LinkedIn](#) | [StackOverflow](#) | [Blog](#)
- Thomas, Reuben
- Tobeabellwether
- van Alst, Ludo
- WevertonGomes



- Wilson, Huon
- ztravis

## 56.2 Adding a new contributor

Contributors are:

- Anybody who has a commit in `main` - no matter how small or how many. Also if it's via *co-authored-by*.
- People who opened helpful issues:
  1. Bugs: with complete MCVE
  2. Well-described feature requests
  3. Potentially some more.

The maintainers of pypdf have the last call on that one.

- Community work: This is exceptional. If the maintainers of pypdf see people being super helpful in answering issues / discussions or being very active on Stackoverflow, we also consider them being contributors to pypdf.

Contributors can add themselves or ask via an GitHub Issue to be added.

Please use the following format:

\* Last name, First name: 140-characters of text; links to LinkedIn / GitHub / other profiles and personal pages are ok

OR

\* GitHub Username: 140-characters of text; links to LinkedIn / GitHub / other profiles and personal pages are ok

and add the entry in the alphabetical order. The 140 characters are everything visible after the Name:.

Please don't use images.



## SCOPE OF PYPDF

What features should pypdf have and which features will it never have?

pypdf aims at simplifying interactions with PDF documents. Core tasks that pypdf can perform are:

- Document manipulation: Splitting, merging, cropping, and transforming the pages of PDF files
- Data Extraction: Extract text and metadata from PDF documents
- Security: Decrypt / encrypt PDF documents

Typical indicators that pypdf should do something:

- The task needs in-depth knowledge of the PDF format
- It currently requires a lot of code or even is impossible to do with pypdf
- It's neither mentioned in “belongs in user code” nor in “out of scope”
- It already is in the issue list with the [is-feature tag](#).

The [moonshot extensions](#) are features we would like to have, but are currently not able to add (PRs are welcome )

### 57.1 Belongs in user code

Here are a few indicators that a feature belongs into users' code (and not into pypdf):

1. The use-case is very specific. Most people will not encounter the same need.
2. It can be done without knowledge of the PDF specification
3. It cannot be done without (non-pdf) domain knowledge. Anything that is specific to your industry.

### 57.2 Out of scope

While this list is infinitely long, there are a few topics that are asked multiple times.

Those topics are out of scope for pypdf. They will never be part of pypdf:

1. **Optical Character Recognition (OCR):** OCR is about extracting text from images. That is very different from the kind of text extraction pypdf is doing. Please note that images can be within PDF documents. In the case of scanned documents, the whole page is an image. Some scanners automatically execute OCR and add a text-layer behind the scanned page. That is something pypdf can use if it's present. As a rule-of-thumb: If you cannot mark/copy the text, it's likely an image. A noteworthy open source OCR project is [tesseract](#).
2. **Format Conversion:** Converting docx / HTML to PDF or PDF to those formats. You might want to have a look at [pdfkit](#) and similar projects.

Out of scope for the moment, but might be added if there are enough contributors:

- **Digital Signature Support** ([reference ticket](#)): Cryptography is complicated. It's important to get it right. pypdf currently doesn't have enough active contributors to properly add digital signature support. For the moment, [pyhanko](#) seems to be the best choice.
- **PDF Generation from Scratch**: pypdf can manipulate existing PDF documents, add annotations, combine / split / crop / transform. It can add blank pages. But if you want to generate invoices, you might want to have a look at [reportlab](#) / [fpdf2](#) or document conversion tools like [pdftkit](#).
- **Replacing words within a PDF**: *Extracting text from PDF is hard*. Replacing text in a reliable way is even harder. For example, one word might be split into multiple tokens. Hence, it's not a simple "search and replace" in some cases.
- **(Not) Extracting headers/footers/page numbers**: While you can apply heuristics, there is no way to always make it work. PDF documents simply don't contain the information what a header/footer/page number is.

### 57.2.1 Library vs. Application

It's also worth pointing out that pypdf is designed to be a library. It is not an application. That has several implications:

- **Execution**: pypdf cannot be executed directly, but only be called from within a program written by a pypdf user. In contrast, an application is executed on its own.
- **Dependencies**: pypdf should have a minimal set of dependencies and only restrict them where it is strictly necessary. In contrast, applications should be installed in environments which are isolated from other applications. They can pin their dependencies.

If you're looking for a way to interact with PDF files via Shell, you should either write a script using pypdf or use [pdfly](#).

## PYPDF VS X

pypdf is a [free](#) and open source pure-python PDF library capable of splitting, merging, cropping, and transforming the pages of PDF files. It can also add custom data, viewing options, and passwords to PDF files. pypdf can retrieve text and metadata from PDFs as well.

### 58.1 PyMuPDF and PikePDF

PyMuPDF is a Python binding to MuPDF and PikePDF is the Python binding to QPDF.

While both are excellent libraries for various use-cases, using them is not always possible even when they support the use-case. Both of them are powered by C libraries which make installation harder and might cause security concerns. For MuPDF, you might also need to buy a commercial license.

A core feature of pypdf is that it's pure Python. That means there is no C dependency. It has been used for over 10 years and for this reason a lot of support via StackOverflow and examples on the internet.

### 58.2 pypdf

PyPDF2 was merged back into pypdf. The development continues at pypdf.

### 58.3 PyPDF3 and PyPDF4

Developing and maintaining open source software is extremely time-intensive and in the case of pypdf not paid at all. Having continuous support is hard.

pypdf was initially released in 2012 on PyPI and received releases until 2016. From 2016 to 2022, there was no update - but people were still using it.

As pypdf is free software, there were attempts to fork it and continue the development. PyPDF3 was first released in 2018 and still receives updates. PyPDF4 has only one release from 2018.

Martin Thoma has worked on bringing the community back to one path of development. He deprecated PyPDF2 in favor of pypdf already, and pypdf has more features and a cleaner interface than PyPDF2 now. See [history of pypdf](#).

### 58.4 pdfminer.six and pdfplumber

pdfminer.six is capable of extracting the [font size](#) / font weight (bold-ness). It has no capabilities for writing PDF files.

pdfplumber is a library focused on extracting data from PDF documents. Since pdfplumber is built on top of pdfminer.six, there are **no capabilities of exporting or modifying a PDF file** (see [#440 \(discussions\)](#)). However, pdfplumber is capable of converting a PDF file into an image, [draw lines and rectangles on the image](#), and save it as an image file. Please note that the image conversion is done via ImageMagick (see [pdfplumber's documentation](#)).

The `pdfplumber` community is active in answering questions and the library is maintained as of May 2023.

## 58.5 `pdfcrow` / `pdfcrow2`

I don't have experience with any of those libraries. Please add a comparison if you know `pypdf` and `pdfcrow`!

Please be aware that there is also `pdfminer` which is not maintained. Then there is `pdfcrow2` which doesn't have a large community behind it.

## 58.6 Document Generation

There are (Python) tools to generate PDF documents. `pypdf` is not one of them.

## 58.7 CLI applications

`pypdf` is a pure Python PDF library. If you're looking for an application which you can use from the terminal, give `pdfcrow` a shot.

## FREQUENTLY ASKED QUESTIONS

### 59.1 How is pypdf related to PyPDF2?

PyPDF2 was a fork from the original pyPdf. After several years, the fork was merged back into pypdf (now all lowercase).

### 59.2 Which Python versions are supported?

pypdf 3.0+ supports Python 3.6 and later. PyPDF2 2.0+ supports Python 3.6 and later. PyPDF2 1.27.10 supported Python 2.7 to 3.10.

### 59.3 Who uses pypdf?

pyPdf is vendored into several projects. That means the code of pyPdf was copied into that project.

Projects that depend on pypdf:

- [Camelot](#): A Python library to extract tabular data from PDFs
- [edi](#): Electronic Data Interchange modules
- [amazon-textract-textractor](#): Analyze documents with Amazon Textract and generate output in multiple formats.
- [maigret](#): Collect a dossier on a person by username from thousands of sites
- [deda](#): tracking Dots Extraction, Decoding and Anonymisation toolkit
- [opencanary](#)
- Document Conversions
  - [rst2pdf](#)
  - [xhtml2pdf](#)
  - [doc2text](#)
- [pdfalyzer](#): A PDF analysis tool for visualizing the inner tree-like data structure of a PDF in spectacularly large and colorful diagrams as well as scanning the binary streams embedded in the PDF for hidden potentially malicious content.

### 59.4 How do I cite pypdf?

In BibTeX format:

```
@misc{pypdf,
  title      = {The {pypdf} library},
  author     = {Mathieu Fenniak and
               Matthew Stamy and
               pubpub-zz and
               Martin Thoma and
               Matthew Peveler and
               exiledkingcc and {pypdf Contributors}},
  year       = {2024},
  url        = {https://pypi.org/project/pypdf/}
  note       = {See https://pypdf.readthedocs.io/en/latest/meta/CONTRIBUTORS.html for
  ↪all contributors}
}
```

## 59.5 Which License does pypdf use?

pypdf uses the [BSD-3-Clause license](#), see the LICENSE file.



## INDICES AND TABLES

- `genindex`
- `modindex`
- `search`



## PYTHON MODULE INDEX

### p

`pypdf.annotations`, [149](#)

`pypdf.errors`, [155](#)

`pypdf.generic`, [157](#)



## A

- A0 (*pypdf.PaperSize* attribute), 139
- A1 (*pypdf.PaperSize* attribute), 139
- A2 (*pypdf.PaperSize* attribute), 139
- A3 (*pypdf.PaperSize* attribute), 139
- A4 (*pypdf.PaperSize* attribute), 139
- A5 (*pypdf.PaperSize* attribute), 139
- A6 (*pypdf.PaperSize* attribute), 139
- A7 (*pypdf.PaperSize* attribute), 139
- A8 (*pypdf.PaperSize* attribute), 139
- add\_annotation() (*pypdf.PdfWriter* method), 116
- add\_attachment() (*pypdf.PdfWriter* method), 107
- add\_blank\_page() (*pypdf.PdfWriter* method), 106
- add\_child() (*pypdf.generic.TreeObject* method), 165
- add\_filtered\_articles() (*pypdf.PdfWriter* method), 117
- add\_form\_topname() (*pypdf.PdfReader* method), 100
- add\_js() (*pypdf.PdfWriter* method), 107
- add\_metadata() (*pypdf.PdfWriter* method), 110
- add\_named\_destination() (*pypdf.PdfWriter* method), 111
- add\_named\_destination\_array() (*pypdf.PdfWriter* method), 111
- add\_named\_destination\_object() (*pypdf.PdfWriter* method), 111
- ADD\_OR\_MODIFY (*pypdf.constants.UserAccessPermissions* attribute), 152
- add\_outline() (*pypdf.PdfWriter* method), 111
- add\_outline\_item() (*pypdf.PdfWriter* method), 111
- add\_outline\_item\_destination() (*pypdf.PdfWriter* method), 111
- add\_outline\_item\_dict() (*pypdf.PdfWriter* method), 111
- add\_page() (*pypdf.PdfWriter* method), 106
- add\_transformation() (*pypdf.\_page.PageObject* method), 132
- add\_uri() (*pypdf.PdfWriter* method), 112
- additional\_actions (*pypdf.generic.Field* property), 125
- ALL (*pypdf.constants.ImageType* attribute), 151
- all() (*pypdf.constants.UserAccessPermissions* class method), 153
- ALL\_ANNOTATIONS (*pypdf.ObjectDeletionFlag* attribute), 119
- alternate\_name (*pypdf.generic.Field* property), 125
- alternative\_name (*pypdf.generic.EmbeddedFile* property), 160
- AnnotationDictionary (class in *pypdf.annotations*), 149
- AnnotationFlag (class in *pypdf.constants*), 151
- annotations (*pypdf.\_page.PageObject* property), 135
- append() (*pypdf.PdfWriter* method), 116
- append\_pages\_from\_reader() (*pypdf.PdfWriter* method), 108
- apply\_on() (*pypdf.Transformation* method), 144
- are\_permissions\_valid (*pypdf.\_doc\_common.PdfDocCommon* property), 172
- are\_permissions\_valid (*pypdf.PdfReader* property), 100
- are\_permissions\_valid (*pypdf.PdfWriter* property), 113
- ArrayObject (class in *pypdf.generic*), 157
- artbox (*pypdf.\_page.PageObject* property), 135
- as\_numeric() (*pypdf.generic.FloatObject* method), 161
- as\_numeric() (*pypdf.generic.NumberObject* method), 162
- ASSEMBLE\_DOC (*pypdf.constants.UserAccessPermissions* attribute), 152
- associated\_file\_relationship (*pypdf.generic.EmbeddedFile* property), 160
- attachment\_list (*pypdf.\_doc\_common.PdfDocCommon* property), 173
- attachment\_list (*pypdf.PdfReader* property), 100
- attachment\_list (*pypdf.PdfWriter* property), 113
- attachments (*pypdf.\_doc\_common.PdfDocCommon* property), 173
- ATTACHMENTS (*pypdf.ObjectDeletionFlag* attribute), 118
- attachments (*pypdf.PdfReader* property), 101
- attachments (*pypdf.PdfWriter* property), 113
- author (*pypdf.DocumentInformation* property), 123
- author\_raw (*pypdf.DocumentInformation* property), 123
- autodetect\_pdfdocencoding

(*pypdf.generic.TextStringObject* attribute), 164  
*autodetect\_utf16* (*pypdf.generic.TextStringObject* attribute), 165

## B

*bleedbox* (*pypdf.\_page.PageObject* property), 134  
*bold* (*pypdf.generic.OutlineFontFlag* attribute), 163  
*BooleanObject* (class in *pypdf.generic*), 157  
*bottom* (*pypdf.generic.Destination* property), 121  
*bottom* (*pypdf.generic.RectangleObject* property), 141  
*ByteStringObject* (class in *pypdf.generic*), 157

## C

*C4* (*pypdf.PaperSize* attribute), 139  
*cache\_get\_indirect\_object*() (*pypdf.PdfReader* method), 100  
*cache\_indirect\_object*() (*pypdf.PdfReader* method), 100  
*CHARSETS* (*pypdf.generic.NameObject* attribute), 162  
*checksum* (*pypdf.generic.EmbeddedFile* property), 160  
*children*() (*pypdf.generic.TreeObject* method), 165  
*clean\_page*() (*pypdf.PdfWriter* method), 116  
*clone*() (*pypdf.\_protocols.PdfObjectProtocol* method), 166  
*clone*() (*pypdf.generic.ArrayObject* method), 157  
*clone*() (*pypdf.generic.BooleanObject* method), 157  
*clone*() (*pypdf.generic.ByteStringObject* method), 158  
*clone*() (*pypdf.generic.ContentStream* method), 158  
*clone*() (*pypdf.generic.DictionaryObject* method), 159  
*clone*() (*pypdf.generic.FloatObject* method), 160  
*clone*() (*pypdf.generic.IndirectObject* method), 161  
*clone*() (*pypdf.generic.NameObject* method), 161  
*clone*() (*pypdf.generic.NullObject* method), 162  
*clone*() (*pypdf.generic.NumberObject* method), 162  
*clone*() (*pypdf.generic.PdfObject* method), 163  
*clone*() (*pypdf.generic.TextStringObject* method), 165  
*clone\_document\_from\_reader*() (*pypdf.PdfWriter* method), 109  
*clone\_reader\_document\_root*() (*pypdf.PdfWriter* method), 109  
*close*() (*pypdf.PdfReader* method), 99  
*close*() (*pypdf.PdfWriter* method), 118  
*color* (*pypdf.generic.Destination* property), 122  
*Comb* (*pypdf.constants.FieldDictionaryAttributes.FfBits* attribute), 154  
*Combo* (*pypdf.constants.FieldDictionaryAttributes.FfBits* attribute), 154  
*CommitOnSelChange* (*pypdf.constants.FieldDictionaryAttributes.FfBits* attribute), 154  
*compress*() (*pypdf.Transformation* static method), 143  
*compress\_content\_streams*() (*pypdf.\_page.PageObject* method), 132

*compress\_identical\_objects*() (*pypdf.PdfWriter* method), 110  
*content* (*pypdf.generic.EmbeddedFile* property), 160  
*ContentStream* (class in *pypdf.generic*), 158  
*create*() (*pypdf.xmp.XmpInformation* class method), 147  
*create\_blank\_page*() (*pypdf.\_page.PageObject* static method), 129  
*create\_string\_object*() (in module *pypdf.generic*), 165  
*create\_viewer\_preferences*() (*pypdf.PdfWriter* method), 106  
*creation\_date* (*pypdf.DocumentInformation* property), 124  
*creation\_date* (*pypdf.generic.EmbeddedFile* property), 160  
*creation\_date\_raw* (*pypdf.DocumentInformation* property), 124  
*creator* (*pypdf.DocumentInformation* property), 123  
*creator\_raw* (*pypdf.DocumentInformation* property), 123  
*cropbox* (*pypdf.\_page.PageObject* property), 134  
*custom\_properties* (*pypdf.xmp.XmpInformation* property), 148

## D

*data* (*pypdf.\_page.ImageFile* attribute), 135  
*dc\_contributor* (*pypdf.xmp.XmpInformation* property), 147  
*dc\_coverage* (*pypdf.xmp.XmpInformation* property), 147  
*dc\_creator* (*pypdf.xmp.XmpInformation* property), 147  
*dc\_date* (*pypdf.xmp.XmpInformation* property), 147  
*dc\_description* (*pypdf.xmp.XmpInformation* property), 147  
*dc\_format* (*pypdf.xmp.XmpInformation* property), 147  
*dc\_identifier* (*pypdf.xmp.XmpInformation* property), 147  
*dc\_language* (*pypdf.xmp.XmpInformation* property), 147  
*dc\_publisher* (*pypdf.xmp.XmpInformation* property), 147  
*dc\_relation* (*pypdf.xmp.XmpInformation* property), 148  
*dc\_rights* (*pypdf.xmp.XmpInformation* property), 148  
*dc\_source* (*pypdf.xmp.XmpInformation* property), 148  
*dc\_subject* (*pypdf.xmp.XmpInformation* property), 148  
*dc\_title* (*pypdf.xmp.XmpInformation* property), 148  
*dc\_type* (*pypdf.xmp.XmpInformation* property), 148  
*DECIMAL* (*pypdf.constants.PageLabelStyle* attribute), 151  
*decode\_as\_image*() (*pypdf.generic.StreamObject* method), 164  
*decode\_pdfdocencoding*() (in module *pypdf.generic*), 166

[decode\\_permissions\(\)](#)  
     (*pypdf.\_doc\_common.PdfDocCommon*  
     *method*), 172  
[decode\\_permissions\(\)](#) (*pypdf.PdfReader* *method*),  
 101  
[decode\\_permissions\(\)](#) (*pypdf.PdfWriter* *method*), 113  
[DecodedStreamObject](#) (*class in pypdf.generic*), 159  
[decrypt\(\)](#) (*pypdf.PdfReader* *method*), 100  
[default\\_value](#) (*pypdf.generic.Field* *property*), 125  
[delete\(\)](#) (*pypdf.generic.EmbeddedFile* *method*), 160  
[delimiter\\_pattern](#) (*pypdf.generic.NameObject*  
*attribute*), 161  
[DependencyError](#), 155  
[DeprecationError](#), 155  
[description](#) (*pypdf.generic.EmbeddedFile* *property*),  
 160  
[dest\\_array](#) (*pypdf.generic.Destination* *property*), 121  
[Destination](#) (*class in pypdf.generic*), 121  
[DictionaryObject](#) (*class in pypdf.generic*), 159  
[DirectReferenceLink](#) (*class in pypdf.generic*), 159  
[DocumentInformation](#) (*class in pypdf*), 123  
[DoNotScroll](#) (*pypdf.constants.FieldDictionaryAttributes.FfBits*  
*attribute*), 154  
[DoNotSpellCheck](#) (*pypdf.constants.FieldDictionaryAttributes.FfBits*  
*attribute*), 154  
[DRAWING\\_IMAGES](#) (*pypdf.constants.ImageType* *attribute*),  
 151  
[DRAWING\\_IMAGES](#) (*pypdf.ObjectDeletionFlag* *attribute*),  
 119  

## E

[Edit](#) (*pypdf.constants.FieldDictionaryAttributes.FfBits*  
*attribute*), 154  
[Ellipse](#) (*class in pypdf.annotations*), 149  
[EmbeddedFile](#) (*class in pypdf.generic*), 160  
[empty\\_tree\(\)](#) (*pypdf.generic.TreeObject* *method*), 165  
[EmptyFileError](#), 156  
[EmptyImageDataError](#), 156  
[encode\\_pdfdocencoding\(\)](#) (*in module pypdf.generic*),  
 166  
[EncodedStreamObject](#) (*class in pypdf.generic*), 160  
[encrypt\(\)](#) (*pypdf.PdfWriter* *method*), 109  
[EXTRACT](#) (*pypdf.constants.UserAccessPermissions*  
*attribute*), 152  
[extract\\_links\(\)](#) (*in module pypdf.generic*), 166  
[extract\\_text\(\)](#) (*pypdf.\_page.PageObject* *method*), 133  
[EXTRACT\\_TEXT\\_AND\\_GRAPHICS](#)  
     (*pypdf.constants.UserAccessPermissions*  
     *attribute*), 152  
[extract\\_xform\\_text\(\)](#) (*pypdf.\_page.PageObject*  
*method*), 134  

## F

[Ff](#) (*pypdf.constants.FieldDictionaryAttributes* *attribute*),  
 153  
[FFBITS\\_NUL](#) (*pypdf.PdfWriter* *attribute*), 108  
[Field](#) (*class in pypdf.generic*), 125  
[field\\_type](#) (*pypdf.generic.Field* *property*), 125  
[FieldDictionaryAttributes](#) (*class in*  
*pypdf.constants*), 153  
[FieldDictionaryAttributes.FfBits](#) (*class in*  
*pypdf.constants*), 153  
[FileNotDecryptedError](#), 155  
[FileSelect](#) (*pypdf.constants.FieldDictionaryAttributes.FfBits*  
*attribute*), 154  
[FILL\\_FORM\\_FIELDS](#) (*pypdf.constants.UserAccessPermissions*  
*attribute*), 152  
[find\\_outline\\_item\(\)](#) (*pypdf.PdfWriter* *method*), 118  
[find\\_referenced\\_page\(\)](#)  
     (*pypdf.generic.DirectReferenceLink* *method*),  
 159  
[find\\_referenced\\_page\(\)](#)  
     (*pypdf.generic.NamedReferenceLink* *method*),  
 162  
[Fit](#) (*class in pypdf.generic*), 127  
[fit\(\)](#) (*pypdf.generic.Fit* *class* *method*), 127  
[fit\\_box\(\)](#) (*pypdf.generic.Fit* *class* *method*), 128  
[fit\\_box\\_horizontally\(\)](#) (*pypdf.generic.Fit* *class*  
*method*), 128  
[fit\\_box\\_vertically\(\)](#) (*pypdf.generic.Fit* *class*  
*method*), 128  
[fit\\_horizontally\(\)](#) (*pypdf.generic.Fit* *class* *method*),  
 127  
[fit\\_rectangle\(\)](#) (*pypdf.generic.Fit* *class* *method*), 127  
[fit\\_vertically\(\)](#) (*pypdf.generic.Fit* *class* *method*),  
 127  
[flags](#) (*pypdf.annotations.AnnotationDictionary* *prop-*  
*erty*), 149  
[flags](#) (*pypdf.generic.Field* *property*), 125  
[flate\\_encode\(\)](#) (*pypdf.generic.StreamObject* *method*),  
 164  
[flattened\\_pages](#) (*pypdf.\_doc\_common.PdfDocCommon*  
*attribute*), 169  
[flattened\\_pages](#) (*pypdf.PdfReader* *attribute*), 99  
[flattened\\_pages](#) (*pypdf.PdfWriter* *attribute*), 105  
[FloatObject](#) (*class in pypdf.generic*), 160  
[font\\_format](#) (*pypdf.generic.Destination* *property*), 122  
[FreeText](#) (*class in pypdf.annotations*), 149  
[from\\_dict\(\)](#) (*pypdf.constants.UserAccessPermissions*  
*class* *method*), 153  

## G

[generate\\_file\\_identifiers\(\)](#) (*pypdf.PdfWriter*  
*method*), 109  
[get\\_contents\(\)](#) (*pypdf.\_page.PageObject* *method*), 130  
[get\\_data\(\)](#) (*pypdf.generic.ContentStream* *method*), 158  
[get\\_data\(\)](#) (*pypdf.generic.EncodedStreamObject*  
*method*), 160

- [get\\_data\(\)](#) (*pypdf.generic.StreamObject* method), 164  
[get\\_destination\\_page\\_number\(\)](#) (*pypdf.\_doc\_common.PdfDocCommon* method), 171  
[get\\_destination\\_page\\_number\(\)](#) (*pypdf.PdfReader* method), 101  
[get\\_destination\\_page\\_number\(\)](#) (*pypdf.PdfWriter* method), 113  
[get\\_element\(\)](#) (*pypdf.xmp.XmpInformation* method), 147  
[get\\_encoded\\_bytes\(\)](#) (*pypdf.generic.TextStringObject* method), 165  
[get\\_fields\(\)](#) (*pypdf.\_doc\_common.PdfDocCommon* method), 170  
[get\\_fields\(\)](#) (*pypdf.PdfReader* method), 101  
[get\\_fields\(\)](#) (*pypdf.PdfWriter* method), 114  
[get\\_form\\_text\\_fields\(\)](#) (*pypdf.\_doc\_common.PdfDocCommon* method), 170  
[get\\_form\\_text\\_fields\(\)](#) (*pypdf.PdfReader* method), 101  
[get\\_form\\_text\\_fields\(\)](#) (*pypdf.PdfWriter* method), 114  
[get\\_inherited\(\)](#) (*pypdf.generic.DictionaryObject* method), 159  
[get\\_named\\_dest\\_root\(\)](#) (*pypdf.\_doc\_common.PdfDocCommon* method), 170  
[get\\_named\\_dest\\_root\(\)](#) (*pypdf.PdfReader* method), 101  
[get\\_named\\_dest\\_root\(\)](#) (*pypdf.PdfWriter* method), 114  
[get\\_nodes\\_in\\_namespace\(\)](#) (*pypdf.xmp.XmpInformation* method), 147  
[get\\_num\\_pages\(\)](#) (*pypdf.\_doc\_common.PdfDocCommon* method), 169  
[get\\_num\\_pages\(\)](#) (*pypdf.PdfReader* method), 101  
[get\\_num\\_pages\(\)](#) (*pypdf.PdfWriter* method), 114  
[get\\_object\(\)](#) (*pypdf.\_doc\_common.PdfDocCommon* method), 169  
[get\\_object\(\)](#) (*pypdf.\_protocols.PdfCommonDocProtocol* method), 167  
[get\\_object\(\)](#) (*pypdf.\_protocols.PdfObjectProtocol* method), 166  
[get\\_object\(\)](#) (*pypdf.generic.IndirectObject* method), 161  
[get\\_object\(\)](#) (*pypdf.generic.PdfObject* method), 164  
[get\\_object\(\)](#) (*pypdf.PdfReader* method), 99  
[get\\_object\(\)](#) (*pypdf.PdfWriter* method), 106  
[get\\_original\\_bytes\(\)](#) (*pypdf.generic.TextStringObject* method), 165  
[get\\_outline\\_root\(\)](#) (*pypdf.PdfWriter* method), 110  
[get\\_page\(\)](#) (*pypdf.\_doc\_common.PdfDocCommon* method), 169  
[get\\_page\(\)](#) (*pypdf.PdfReader* method), 101  
[get\\_page\(\)](#) (*pypdf.PdfWriter* method), 114  
[get\\_page\\_number\(\)](#) (*pypdf.\_doc\_common.PdfDocCommon* method), 171  
[get\\_page\\_number\(\)](#) (*pypdf.PdfReader* method), 102  
[get\\_page\\_number\(\)](#) (*pypdf.PdfWriter* method), 114  
[get\\_pages\\_showing\\_field\(\)](#) (*pypdf.\_doc\_common.PdfDocCommon* method), 170  
[get\\_pages\\_showing\\_field\(\)](#) (*pypdf.PdfReader* method), 102  
[get\\_pages\\_showing\\_field\(\)](#) (*pypdf.PdfWriter* method), 115  
[get\\_reference\(\)](#) (*pypdf.PdfWriter* method), 110  
[get\\_threads\\_root\(\)](#) (*pypdf.PdfWriter* method), 110
- ## H
- [has\\_children\(\)](#) (*pypdf.generic.TreeObject* method), 165  
[hash\\_bin\(\)](#) (*pypdf.\_page.PageObject* method), 129  
[hash\\_bin\(\)](#) (*pypdf.generic.ArrayObject* method), 157  
[hash\\_bin\(\)](#) (*pypdf.generic.BooleanObject* method), 157  
[hash\\_bin\(\)](#) (*pypdf.generic.ByteStringObject* method), 158  
[hash\\_bin\(\)](#) (*pypdf.generic.DictionaryObject* method), 159  
[hash\\_bin\(\)](#) (*pypdf.generic.FloatObject* method), 160  
[hash\\_bin\(\)](#) (*pypdf.generic.IndirectObject* method), 161  
[hash\\_bin\(\)](#) (*pypdf.generic.NameObject* method), 161  
[hash\\_bin\(\)](#) (*pypdf.generic.NullObject* method), 162  
[hash\\_bin\(\)](#) (*pypdf.generic.NumberObject* method), 162  
[hash\\_bin\(\)](#) (*pypdf.generic.PdfObject* method), 163  
[hash\\_bin\(\)](#) (*pypdf.generic.StreamObject* method), 164  
[hash\\_bin\(\)](#) (*pypdf.generic.TextStringObject* method), 165  
[hash\\_func\(\)](#) (*pypdf.generic.PdfObject* method), 163  
[hash\\_value\(\)](#) (*pypdf.\_protocols.PdfObjectProtocol* method), 166  
[hash\\_value\(\)](#) (*pypdf.generic.PdfObject* method), 163  
[hash\\_value\\_data\(\)](#) (*pypdf.\_page.PageObject* method), 129  
[hash\\_value\\_data\(\)](#) (*pypdf.generic.PdfObject* method), 163  
[hash\\_value\\_data\(\)](#) (*pypdf.generic.StreamObject* method), 164  
[height](#) (*pypdf.generic.RectangleObject* property), 141  
[hex\\_to\\_rgb\(\)](#) (in module *pypdf.generic*), 166  
[HIDDEN](#) (*pypdf.constants.AnnotationFlag* attribute), 151  
[Highlight](#) (class in *pypdf.annotations*), 149
- ## I
- [image](#) (*pypdf.\_page.ImageFile* attribute), 135  
[ImageFile](#) (class in *pypdf.\_page*), 135



- `images` (*pypdf.\_page.PageObject* property), 130
  - `IMAGES` (*pypdf.constants.ImageType* attribute), 151
  - `IMAGES` (*pypdf.ObjectDeletionFlag* attribute), 119
  - `ImageType` (class in *pypdf.constants*), 151
  - `inc_parent_counter_default()`  
(*pypdf.generic.TreeObject* method), 165
  - `inc_parent_counter_outline()`  
(*pypdf.generic.TreeObject* method), 165
  - `incremental` (*pypdf.\_protocols.PdfWriterProtocol* attribute), 167
  - `incremental` (*pypdf.PdfWriter* attribute), 105
  - `indices()` (*pypdf.PageRange* method), 137
  - `indirect_reference` (*pypdf.\_page.ImageFile* attribute), 135
  - `indirect_reference` (*pypdf.\_protocols.PdfObjectProtocol* attribute), 166
  - `indirect_reference` (*pypdf.generic.IndirectObject* property), 161
  - `indirect_reference` (*pypdf.generic.PdfObject* attribute), 163
  - `IndirectObject` (class in *pypdf.generic*), 161
  - `initialize_from_dictionary()`  
(*pypdf.generic.StreamObject* static method), 164
  - `INLINE_IMAGES` (*pypdf.constants.ImageType* attribute), 151
  - `INLINE_IMAGES` (*pypdf.ObjectDeletionFlag* attribute), 119
  - `insert_blank_page()` (*pypdf.PdfWriter* method), 107
  - `insert_child()` (*pypdf.generic.TreeObject* method), 165
  - `insert_page()` (*pypdf.PdfWriter* method), 106
  - `INVISIBLE` (*pypdf.constants.AnnotationFlag* attribute), 151
  - `is_encrypted` (*pypdf.\_doc\_common.PdfDocCommon* property), 173
  - `is_encrypted` (*pypdf.PdfReader* property), 100
  - `is_encrypted` (*pypdf.PdfWriter* property), 105
  - `is_null_or_none()` (in module *pypdf.generic*), 166
  - `isolate_graphics_state()`  
(*pypdf.generic.ContentStream* method), 159
  - `italic` (*pypdf.generic.OutlineFontFlag* attribute), 163
  - `items()` (*pypdf.\_page.VirtualListImages* method), 135
  - `items()` (*pypdf.generic.ArrayObject* method), 157
- ## K
- `keys()` (*pypdf.\_page.VirtualListImages* method), 135
  - `keywords` (*pypdf.DocumentInformation* property), 124
  - `keywords_raw` (*pypdf.DocumentInformation* property), 124
  - `kids` (*pypdf.generic.Field* property), 125
- ## L
- `left` (*pypdf.generic.Destination* property), 121
  - `left` (*pypdf.generic.RectangleObject* property), 141
  - `LimitReachedError`, 156
  - `Line` (class in *pypdf.annotations*), 149
  - `Link` (class in *pypdf.annotations*), 149
  - `LINKS` (*pypdf.ObjectDeletionFlag* attribute), 118
  - `list_objects_in_increment()` (*pypdf.PdfWriter* method), 110
  - `LOCKED` (*pypdf.constants.AnnotationFlag* attribute), 151
  - `LOCKED_CONTENTS` (*pypdf.constants.AnnotationFlag* attribute), 151
  - `lower_left` (*pypdf.generic.RectangleObject* property), 141
  - `lower_right` (*pypdf.generic.RectangleObject* property), 141
  - `LOWERCASE_LETTER` (*pypdf.constants.PageLabelStyle* attribute), 152
  - `LOWERCASE_ROMAN` (*pypdf.constants.PageLabelStyle* attribute), 152
- ## M
- `mapping_name` (*pypdf.generic.Field* property), 125
  - `MarkupAnnotation` (class in *pypdf.annotations*), 149
  - `matrix` (*pypdf.Transformation* property), 143
  - `mediabox` (*pypdf.\_page.PageObject* property), 134
  - `merge()` (*pypdf.PdfWriter* method), 117
  - `merge_page()` (*pypdf.\_page.PageObject* method), 131
  - `merge_rotated_page()` (*pypdf.\_page.PageObject* method), 131
  - `merge_scaled_page()` (*pypdf.\_page.PageObject* method), 131
  - `merge_transformed_page()` (*pypdf.\_page.PageObject* method), 131
  - `merge_translated_page()` (*pypdf.\_page.PageObject* method), 132
  - `metadata` (*pypdf.\_doc\_common.PdfDocCommon* property), 169
  - `metadata` (*pypdf.PdfReader* property), 102
  - `metadata` (*pypdf.PdfWriter* property), 110
  - `modification_date` (*pypdf.DocumentInformation* property), 124
  - `modification_date` (*pypdf.generic.EmbeddedFile* property), 160
  - `modification_date_raw` (*pypdf.DocumentInformation* property), 124
  - `MODIFY` (*pypdf.constants.UserAccessPermissions* attribute), 152
  - module
    - pypdf.annotations*, 149
    - pypdf.errors*, 155
    - pypdf.generic*, 157
  - `mult()` (in module *pypdf*), 136
  - `Multiline` (*pypdf.constants.FieldDictionaryAttributes.FfBits* attribute), 153

MultiSelect (*pypdf.constants.FieldDictionaryAttributes.FfBits* attribute), 154

myrepr() (*pypdf.generic.FloatObject* method), 161

## N

name (*pypdf.\_page.ImageFile* attribute), 135

name (*pypdf.generic.EmbeddedFile* property), 160

name (*pypdf.generic.Field* property), 125

named\_destinations (*pypdf.\_doc\_common.PdfDocCommon* property), 170

named\_destinations (*pypdf.PdfReader* property), 102

named\_destinations (*pypdf.PdfWriter* property), 115

NamedReferenceLink (class in *pypdf.generic*), 162

NameObject (class in *pypdf.generic*), 161

NO\_ROTATE (*pypdf.constants.AnnotationFlag* attribute), 151

NO\_VIEW (*pypdf.constants.AnnotationFlag* attribute), 151

NO\_ZOOM (*pypdf.constants.AnnotationFlag* attribute), 151

node (*pypdf.generic.Destination* attribute), 121

NoExport (*pypdf.constants.FieldDictionaryAttributes.FfBits* attribute), 153

NONE (*pypdf.constants.ImageType* attribute), 151

NONE (*pypdf.ObjectDeletionFlag* attribute), 118

NOT\_DECRYPTED (*pypdf.PasswordType* attribute), 104

NoToggleToOff (*pypdf.constants.FieldDictionaryAttributes.FfBits* attribute), 153

NullObject (class in *pypdf.generic*), 162

NumberObject (class in *pypdf.generic*), 162

NumberPattern (*pypdf.generic.NumberObject* attribute), 162

## O

ObjectDeletionFlag (class in *pypdf*), 118

OBJECTS\_3D (*pypdf.ObjectDeletionFlag* attribute), 119

open\_destination (*pypdf.\_doc\_common.PdfDocCommon* property), 170

open\_destination (*pypdf.PdfReader* property), 102

open\_destination (*pypdf.PdfWriter* property), 107

operations (*pypdf.generic.ContentStream* property), 158

original\_bytes (*pypdf.generic.ByteStringObject* property), 158

original\_bytes (*pypdf.generic.TextStringObject* property), 165

original\_page (*pypdf.\_page.PageObject* attribute), 129

outline (*pypdf.\_doc\_common.PdfDocCommon* property), 171

outline (*pypdf.PdfReader* property), 102

outline (*pypdf.PdfWriter* property), 115

outline\_count (*pypdf.generic.Destination* property), 122

OutlineFontFlag (class in *pypdf.generic*), 163

OutlineItem (class in *pypdf.generic*), 163

OWNER\_PASSWORD (*pypdf.PasswordType* attribute), 104

page (*pypdf.generic.Destination* property), 121

page\_labels (*pypdf.\_doc\_common.PdfDocCommon* property), 171

page\_labels (*pypdf.PdfReader* property), 102

page\_labels (*pypdf.PdfWriter* property), 115

page\_layout (*pypdf.\_doc\_common.PdfDocCommon* property), 171

page\_layout (*pypdf.PdfReader* property), 102

page\_layout (*pypdf.PdfWriter* property), 113

page\_mode (*pypdf.\_doc\_common.PdfDocCommon* property), 172

page\_mode (*pypdf.PdfReader* property), 103

page\_mode (*pypdf.PdfWriter* property), 116

page\_number (*pypdf.\_page.PageObject* property), 133

PageLabelStyle (class in *pypdf.constants*), 151

PageObject (class in *pypdf.\_page*), 129

PageRange (class in *pypdf*), 137

pages (*pypdf.\_doc\_common.PdfDocCommon* property), 171

pages (*pypdf.\_protocols.PdfCommonDocProtocol* property), 166

pages (*pypdf.PdfReader* property), 103

pages (*pypdf.PdfWriter* property), 115

PagesizeNotDefinedError, 155

PaperSize (class in *pypdf*), 139

parent (*pypdf.generic.Field* property), 125

ParseError, 155

Password (*pypdf.constants.FieldDictionaryAttributes.FfBits* attribute), 153

PasswordType (class in *pypdf*), 104

patch\_reference() (*pypdf.generic.DirectReferenceLink* method), 159

patch\_reference() (*pypdf.generic.NamedReferenceLink* method), 162

pdf\_header (*pypdf.\_doc\_common.PdfDocCommon* property), 169

pdf\_header (*pypdf.\_protocols.PdfCommonDocProtocol* property), 166

pdf\_header (*pypdf.PdfReader* property), 99

pdf\_header (*pypdf.PdfWriter* property), 106

pdf\_keywords (*pypdf.xmp.XmpInformation* property), 148

pdf\_pdfversion (*pypdf.xmp.XmpInformation* property), 148

pdf\_producer (*pypdf.xmp.XmpInformation* property), 148

pdfaid\_conformance (*pypdf.xmp.XmpInformation* property), 148

pdfaid\_part (*pypdf.xmp.XmpInformation* property), 148

PdfCommonDocProtocol (class in *pypdf.\_protocols*), 166

PdfDocCommon (class in *pypdf.\_doc\_common*), 169

PdfObject (class in *pypdf.generic*), 163  
 PdfObjectProtocol (class in *pypdf.\_protocols*), 166  
 PdfReader (class in *pypdf*), 99  
 PdfReaderProtocol (class in *pypdf.\_protocols*), 167  
 PdfReadError, 155  
 PdfReadWarning, 155  
 PdfStreamError, 155  
 PdfWriter (class in *pypdf*), 105  
 PdfWriterProtocol (class in *pypdf.\_protocols*), 167  
 Polygon (class in *pypdf.annotations*), 150  
 PolyLine (class in *pypdf.annotations*), 150  
 Popup (class in *pypdf.annotations*), 150  
 prefix (*pypdf.generic.NameObject* attribute), 161  
 PRINT (*pypdf.constants.AnnotationFlag* attribute), 151  
 PRINT (*pypdf.constants.UserAccessPermissions* attribute), 152  
 PRINT\_SCALING (*pypdf.generic.ViewerPreferences* property), 165  
 PRINT\_TO\_REPRESENTATION (*pypdf.constants.UserAccessPermissions* attribute), 152  
 producer (*pypdf.DocumentInformation* property), 123  
 producer\_raw (*pypdf.DocumentInformation* property), 124  
 Pushbutton (*pypdf.constants.FieldDictionaryAttributes.FfBits* attribute), 154  
 pypdf.annotations  
     module, 149  
 pypdf.errors  
     module, 155  
 pypdf.generic  
     module, 157  
 PyPdfError, 155

## R

R1 (*pypdf.constants.UserAccessPermissions* attribute), 152  
 R13 (*pypdf.constants.UserAccessPermissions* attribute), 152  
 R14 (*pypdf.constants.UserAccessPermissions* attribute), 152  
 R15 (*pypdf.constants.UserAccessPermissions* attribute), 152  
 R16 (*pypdf.constants.UserAccessPermissions* attribute), 152  
 R17 (*pypdf.constants.UserAccessPermissions* attribute), 152  
 R18 (*pypdf.constants.UserAccessPermissions* attribute), 152  
 R19 (*pypdf.constants.UserAccessPermissions* attribute), 152  
 R2 (*pypdf.constants.UserAccessPermissions* attribute), 152  
 R20 (*pypdf.constants.UserAccessPermissions* attribute), 152  
 R21 (*pypdf.constants.UserAccessPermissions* attribute), 152  
 R22 (*pypdf.constants.UserAccessPermissions* attribute), 152  
 R23 (*pypdf.constants.UserAccessPermissions* attribute), 152  
 R24 (*pypdf.constants.UserAccessPermissions* attribute), 152  
 R25 (*pypdf.constants.UserAccessPermissions* attribute), 152  
 R26 (*pypdf.constants.UserAccessPermissions* attribute), 153  
 R27 (*pypdf.constants.UserAccessPermissions* attribute), 153  
 R28 (*pypdf.constants.UserAccessPermissions* attribute), 153  
 R29 (*pypdf.constants.UserAccessPermissions* attribute), 153  
 R30 (*pypdf.constants.UserAccessPermissions* attribute), 153  
 R31 (*pypdf.constants.UserAccessPermissions* attribute), 153  
 R32 (*pypdf.constants.UserAccessPermissions* attribute), 153  
 R7 (*pypdf.constants.UserAccessPermissions* attribute), 152  
 R8 (*pypdf.constants.UserAccessPermissions* attribute), 152  
 Radio (*pypdf.constants.FieldDictionaryAttributes.FfBits* attribute), 153  
 RadiosInUnison (*pypdf.constants.FieldDictionaryAttributes.FfBits* attribute), 154  
 raw\_get() (*pypdf.generic.DictionaryObject* method), 159  
 read() (*pypdf.PdfReader* method), 100  
 read\_from\_stream() (*pypdf.generic.ArrayObject* static method), 157  
 read\_from\_stream() (*pypdf.generic.BooleanObject* static method), 157  
 read\_from\_stream() (*pypdf.generic.DictionaryObject* static method), 159  
 read\_from\_stream() (*pypdf.generic.IndirectObject* static method), 161  
 read\_from\_stream() (*pypdf.generic.NameObject* static method), 162  
 read\_from\_stream() (*pypdf.generic.NullObject* static method), 162  
 read\_from\_stream() (*pypdf.generic.NumberObject* static method), 163  
 read\_hex\_string\_from\_stream() (in module *pypdf.generic*), 166  
 read\_object() (in module *pypdf.generic*), 166

- read\_object\_header() (pypdf.PdfReader method), 99  
 READ\_ONLY (pypdf.constants.AnnotationFlag attribute), 151  
 read\_string\_from\_stream() (in module pypdf.generic), 166  
 ReadOnly (pypdf.constants.FieldDictionaryAttributes.FfBits attribute), 153  
 reattach\_fields() (pypdf.PdfWriter method), 108  
 Rectangle (class in pypdf.annotations), 150  
 RectangleObject (class in pypdf.generic), 141  
 remove\_annotations() (pypdf.PdfWriter method), 111  
 remove\_child() (pypdf.generic.TreeObject method), 165  
 remove\_from\_tree() (pypdf.generic.TreeObject method), 165  
 remove\_images() (pypdf.PdfWriter method), 112  
 remove\_links() (pypdf.PdfWriter method), 111  
 remove\_objects\_from\_page() (pypdf.PdfWriter method), 112  
 remove\_page() (pypdf.\_doc\_common.PdfDocCommon method), 172  
 remove\_page() (pypdf.PdfReader method), 103  
 remove\_page() (pypdf.PdfWriter method), 115  
 remove\_text() (pypdf.PdfWriter method), 112  
 rename\_form\_topname() (pypdf.PdfReader method), 100  
 renumber() (pypdf.generic.NameObject method), 162  
 renumber\_table (pypdf.generic.NameObject attribute), 161  
 replace() (pypdf.\_page.ImageFile method), 135  
 replace\_contents() (pypdf.\_page.PageObject method), 131  
 replicate() (pypdf.generic.ArrayObject method), 157  
 replicate() (pypdf.generic.ContentStream method), 158  
 replicate() (pypdf.generic.DictionaryObject method), 159  
 replicate() (pypdf.generic.IndirectObject method), 161  
 replicate() (pypdf.generic.PdfObject method), 163  
 replicate() (pypdf.generic.StreamObject method), 164  
 Required (pypdf.constants.FieldDictionaryAttributes.FfBits attribute), 153  
 reset\_translation() (pypdf.PdfWriter method), 118  
 resolved\_objects (pypdf.PdfReader attribute), 99  
 RichText (pypdf.constants.FieldDictionaryAttributes.FfBits attribute), 154  
 right (pypdf.generic.Destination property), 121  
 right (pypdf.generic.RectangleObject property), 141  
 root\_object (pypdf.\_doc\_common.PdfDocCommon property), 169  
 root\_object (pypdf.\_protocols.PdfCommonDocProtocol property), 167  
 root\_object (pypdf.PdfReader property), 99  
 root\_object (pypdf.PdfWriter property), 105  
 rotate() (pypdf.\_page.PageObject method), 130  
 rotate() (pypdf.Transformation method), 144  
 rotation (pypdf.\_page.PageObject property), 130
- ## S
- scale() (pypdf.\_page.PageObject method), 132  
 scale() (pypdf.generic.RectangleObject method), 141  
 scale() (pypdf.Transformation method), 144  
 scale\_by() (pypdf.\_page.PageObject method), 132  
 scale\_to() (pypdf.\_page.PageObject method), 132  
 set\_data() (pypdf.generic.ContentStream method), 158  
 set\_data() (pypdf.generic.EncodedStreamObject method), 160  
 set\_data() (pypdf.generic.StreamObject method), 164  
 set\_need\_appearances\_writer() (pypdf.PdfWriter method), 106  
 set\_page\_label() (pypdf.PdfWriter method), 118  
 set\_page\_layout() (pypdf.PdfWriter method), 112  
 setdefault() (pypdf.generic.DictionaryObject method), 159  
 size (pypdf.generic.EmbeddedFile property), 160  
 Sort (pypdf.constants.FieldDictionaryAttributes.FfBits attribute), 154  
 StreamObject (class in pypdf.generic), 164  
 strict (pypdf.\_doc\_common.PdfDocCommon attribute), 169  
 strict (pypdf.\_protocols.PdfCommonDocProtocol property), 167  
 strict (pypdf.PdfReader attribute), 99  
 strict (pypdf.PdfWriter attribute), 105  
 subject (pypdf.DocumentInformation property), 123  
 subject\_raw (pypdf.DocumentInformation property), 123  
 subtype (pypdf.generic.EmbeddedFile property), 160  
 suffix (pypdf.generic.NameObject attribute), 162
- ## T
- Text (class in pypdf.annotations), 150  
 TEXT (pypdf.ObjectDeletionFlag attribute), 118  
 TextStringObject (class in pypdf.generic), 164  
 threads (pypdf.\_doc\_common.PdfDocCommon property), 171  
 threads (pypdf.PdfReader property), 103  
 threads (pypdf.PdfWriter property), 111  
 title (pypdf.DocumentInformation property), 123  
 title (pypdf.generic.Destination property), 121  
 title\_raw (pypdf.DocumentInformation property), 123  
 to\_dict() (pypdf.constants.UserAccessPermissions method), 153  
 to\_slice() (pypdf.PageRange method), 137  
 TOGGLE\_NO\_VIEW (pypdf.constants.AnnotationFlag attribute), 151  
 top (pypdf.generic.Destination property), 121



top (*pypdf.generic.RectangleObject* property), 141  
 trailer (*pypdf.\_protocols.PdfReaderProtocol* property), 167  
 transfer\_rotation\_to\_content()  
     (*pypdf.\_page.PageObject* method), 130  
 transform() (*pypdf.Transformation* method), 143  
 Transformation (class in *pypdf*), 143  
 translate() (*pypdf.Transformation* method), 144  
 TreeObject (class in *pypdf.generic*), 165  
 trimbox (*pypdf.\_page.PageObject* property), 135  
 typ (*pypdf.generic.Destination* property), 121

## U

unnumber() (*pypdf.generic.NameObject* static method), 162  
 update\_page\_form\_field\_values()  
     (*pypdf.PdfWriter* method), 108  
 upper\_left (*pypdf.generic.RectangleObject* property), 141  
 upper\_right (*pypdf.generic.RectangleObject* property), 141  
 UPPERCASE\_LETTER (*pypdf.constants.PageLabelStyle* attribute), 152  
 UPPERCASE\_ROMAN (*pypdf.constants.PageLabelStyle* attribute), 151  
 user\_access\_permissions  
     (*pypdf.\_doc\_common.PdfDocCommon* property), 172  
 user\_access\_permissions (*pypdf.PdfReader* property), 104  
 user\_access\_permissions (*pypdf.PdfWriter* property), 116  
 USER\_PASSWORD (*pypdf.PasswordType* attribute), 104  
 user\_unit (*pypdf.\_page.PageObject* property), 129  
 UserAccessPermissions (class in *pypdf.constants*), 152  
 utf16\_bom (*pypdf.generic.TextStringObject* attribute), 165

## V

valid() (*pypdf.PageRange* static method), 137  
 value (*pypdf.generic.Field* property), 125  
 viewer\_preferences (*pypdf.\_doc\_common.PdfDocCommon* property), 169  
 viewer\_preferences (*pypdf.PdfReader* property), 104  
 viewer\_preferences (*pypdf.PdfWriter* property), 116  
 ViewerPreferences (class in *pypdf.generic*), 165  
 VirtualListImages (class in *pypdf.\_page*), 135

## W

width (*pypdf.generic.RectangleObject* property), 141  
 with\_as\_usage (*pypdf.PdfWriter* property), 106  
 write() (*pypdf.\_protocols.PdfWriterProtocol* method), 167

write() (*pypdf.PdfWriter* method), 110  
 write\_stream() (*pypdf.PdfWriter* method), 109  
 write\_to\_stream() (*pypdf.\_protocols.PdfObjectProtocol* method), 166  
 write\_to\_stream() (*pypdf.generic.ArrayObject* method), 157  
 write\_to\_stream() (*pypdf.generic.BooleanObject* method), 157  
 write\_to\_stream() (*pypdf.generic.ByteStringObject* method), 158  
 write\_to\_stream() (*pypdf.generic.ContentStream* method), 159  
 write\_to\_stream() (*pypdf.generic.Destination* method), 121  
 write\_to\_stream() (*pypdf.generic.DictionaryObject* method), 159  
 write\_to\_stream() (*pypdf.generic.FloatObject* method), 161  
 write\_to\_stream() (*pypdf.generic.IndirectObject* method), 161  
 write\_to\_stream() (*pypdf.generic.NameObject* method), 162  
 write\_to\_stream() (*pypdf.generic.NullObject* method), 162  
 write\_to\_stream() (*pypdf.generic.NumberObject* method), 163  
 write\_to\_stream() (*pypdf.generic.OutlineItem* method), 163  
 write\_to\_stream() (*pypdf.generic.PdfObject* method), 164  
 write\_to\_stream() (*pypdf.generic.StreamObject* method), 164  
 write\_to\_stream() (*pypdf.generic.TextStringObject* method), 165  
 write\_to\_stream() (*pypdf.xmp.XmpInformation* method), 147  
 WrongPasswordError, 156

## X

xfa (*pypdf.\_doc\_common.PdfDocCommon* property), 173  
 xfa (*pypdf.PdfReader* property), 104  
 xfa (*pypdf.PdfWriter* property), 116  
 xmp\_create\_date (*pypdf.xmp.XmpInformation* property), 148  
 xmp\_creator\_tool (*pypdf.xmp.XmpInformation* property), 148  
 xmp\_metadata (*pypdf.\_doc\_common.PdfDocCommon* property), 169  
 xmp\_metadata (*pypdf.generic.DictionaryObject* property), 159  
 xmp\_metadata (*pypdf.PdfReader* property), 99  
 xmp\_metadata (*pypdf.PdfWriter* property), 105

xmp\_metadata\_date (*pypdf.xmp.XmpInformation* property), 148  
xmp\_modify\_date (*pypdf.xmp.XmpInformation* property), 148  
XmpDocumentError, 156  
XmpInformation (*class in pypdf.xmp*), 147  
XmpInformationProtocol (*class in pypdf.\_protocols*), 166  
xmppmm\_document\_id (*pypdf.xmp.XmpInformation* property), 148  
xmppmm\_instance\_id (*pypdf.xmp.XmpInformation* property), 148  
XOBJECT\_IMAGES (*pypdf.constants.ImageType* attribute), 151  
XOBJECT\_IMAGES (*pypdf.ObjectDeletionFlag* attribute), 119  
xref (*pypdf.\_protocols.PdfReaderProtocol* property), 167  
xyz() (*pypdf.generic.Fit* class method), 127

## **Z**

zoom (*pypdf.generic.Destination* property), 121